Pi Developer Architecture Guide

A source-grounded path from first contribution to extension authoring

Pi maintainers

2026-07-23

Table of contents

1 Welcome to the Pi Developer Guide

This guide is for TypeScript-literate Pi users who want to build extensions. It starts with a hook or command and progresses through tools, durable state, custom UI, provider-adjacent hooks, and packages - without requiring a fork or a source-code tour that does not serve that goal.

The index is the authoritative orientation hub. The rendered parts and routes below are the sequence to follow.

1.1 Choose a route

You do not need to read every chapter. Choose the outcome that matches your work:

Intent	Route
Build a first extension	Setup → resources, settings, and trust → your first extension → the lifecycle map → extension patterns
Contribute to core	quick Pi TypeScript → architecture → prompt lifecycle → agent core → built-in tools → testing and contribution
Embed or automate Pi	architecture → models and authentication → modes and SDK → failure and recovery
Customize without code	sessions and compaction → settings, skills, prompts, and themes
Learn TypeScript first	optional foundations workbook → quick Pi TypeScript → architecture

The first route is for extension authors; the second follows ownership into the core runtime; the third emphasizes host boundaries; and the fourth is a complete beginner route that returns to the shorter Pi chapter after the foundations.

First success in 10–15 minutes

Use the staged first extension rather than copying a second example into this index.

1. Create the discovered location and save the stage-1 `hello-pi.ts` listing: `mkdir -p ~/.pi/agent/extensions`.
2. Start `pi` in any project, run `/hello-pi Ada`, and see the greeting notification.
3. Work through stages 2-4 in the chapter, running `/reload` after each edit. Reload tears down and recreates the extension runtime; the stage-4 counter proves durable state survives it.
4. **Cleanup:** remove `~/.pi/agent/extensions/hello-pi.ts` when finished.

Extensions run with the permissions of the Pi process. Read the file before running it, and do not install untrusted extension packages.

1.2 Prerequisites and development setup

Before diving into the codebase, make sure your environment is ready.

Requirement	Minimum version	Why
Node.js	>= 22.19.0	Pi runs erasable TypeScript directly with Node's native type stripping and uses modern ESM features.
npm	Bundled with Node.js	Workspace management and package installs. The repo uses `npm workspaces`.
git	Any recent version	To clone and navigate repository history.
TypeScript knowledge	Reading level	The quick chapter teaches the Pi patterns; the workbook supplies beginner foundations.

Erasable TypeScript and the repository checks

Erasable TypeScript is type syntax that can be removed without changing the resulting JavaScript. Native Node stripping does not type-check the file or transform syntax that needs JavaScript emit, so Pi avoids constructor parameter properties, `enum`, and `namespace` in checked source. Use explicit fields and literal unions instead.
Runtime and checking are separate. `tsx` is a development runner, `jiti` loads extensions, and `tsgo` is the TypeScript-native checker/build tool. Use `npx tsgo --noEmit` for a focused no-emit check; it does not run the program. For repository code changes, `npm run check` is the broader quality gate.

1.2.1 Quick start

```
git clone https://github.com/earendil-works/pi.git
cd pi
npm install --ignore-scripts  # Install dependencies without lifecycle scripts
npm run check                 # Check the repository
./pi-test.sh                  # Run Pi from this checkout
```

Invoke `./pi-test.sh` by that path from the Pi checkout: `cd` into the cloned repository first. It is not a globally installed command. From another working directory, use the checkout's absolute path, for example `/path/to/pi/pi-test.sh`.

`npm install --ignore-scripts` avoids install lifecycle scripts. Use `npm ci --ignore-scripts` only for a clean CI-style install when the lockfile matches `package.json`. The test launcher runs the repository's Pi implementation directly rather than a globally installed or published build.

If you only want to write extensions, you do not need to clone this repository. Install Pi globally and start writing `.ts` files under `~/.pi/agent/extensions/`; Your first extension walks through discovery and reload.

1.2.2 Editor setup

Any editor with TypeScript support works. VS Code users get inline type hints, autocomplete for `ExtensionAPI` methods, and schema validation on tool parameters when TypeScript resolves the `@earendil-works/*` package types. If you are writing extensions outside the monorepo, install the published packages as dev dependencies:

```
npm install --save-dev @earendil-works/pi-ai @earendil-works/pi-coding-agent
```

1.2.3 TypeScript configuration for extension authors

Extensions are loaded by Pi at runtime with `jiti`, so a single-file extension does not need a `tsconfig.json` or build step. A multi-file extension or Pi package should use a minimal configuration that resolves the Pi package types:

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "skipLibCheck": true
  }
}
```

Type-check an extension with `npx tsc --noEmit`. Loading through `jiti` and checking through TypeScript are separate checks.

1.3 What Pi is

Pi is a TypeScript monorepo for an **agent harness** (the runtime that coordinates model requests, tools, and application state) and a self-extensible coding agent. The root `package.json` declares npm workspaces. The interactive `pi` command is implemented by `@earendil-works/pi-coding-agent`; it composes a terminal UI, an agent runtime, and a provider-neutral AI API.

1.3.1 The workspace in brief

The root `package.json` declares npm workspaces for the five direct packages under `packages/*`, plus nested storage and extension-example workspaces, and `npm install` links them for imports such as `@earendil-works/pi-ai`. Inside the repo, cross-package imports use published package names; relative `.ts` imports stay within one package. Extensions written outside the repo import the published packages and declare them as peer dependencies.

Architecture: packages, ownership, and entry points is the authoritative map: what each package owns, the import rules, and the startup construction path. This index keeps only the mental model below so the two do not drift.

1.4 A mental model in one diagram

The vertical arrows show the usual prompt path. Extensions are loaded by the coding agent and hook into defined `AgentSession` and agent-loop boundaries—for example to inspect input, observe session events, add or wrap tools, customize UI, or alter requests. These hooks participate in the normal runtime; they do not bypass the agent loop.

1.5 Reading order and outcomes

The rendered book makes its roles visible with parts. The route below matches the rendered order recorded in `developer-guide/_quarto.yml` and in `developer-guide/README.md` for filesystem browsers.

1.5.1 Start here

1. TypeScript used by Pi is the short route; the optional foundations workbook is at the end as an appendix.

1.5.2 Using Pi effectively

2. Using Pi day to day: first session, login and models, slash commands, keybindings, and CLI flags.
3. Sessions, trees, and compaction: durable context, branching, and where session files live.
4. Customizing Pi: settings, skills, prompts, themes, and trust - the no-code mechanisms to try before writing code.

1.5.3 How Pi works

5. Architecture: packages, ownership, and entry points gives each package one clear job.
6. Prompt lifecycle starts with a unified agent-loop, tool, and system-prompt assembly model, then traces Enter to settlement and maps every extension hook point.

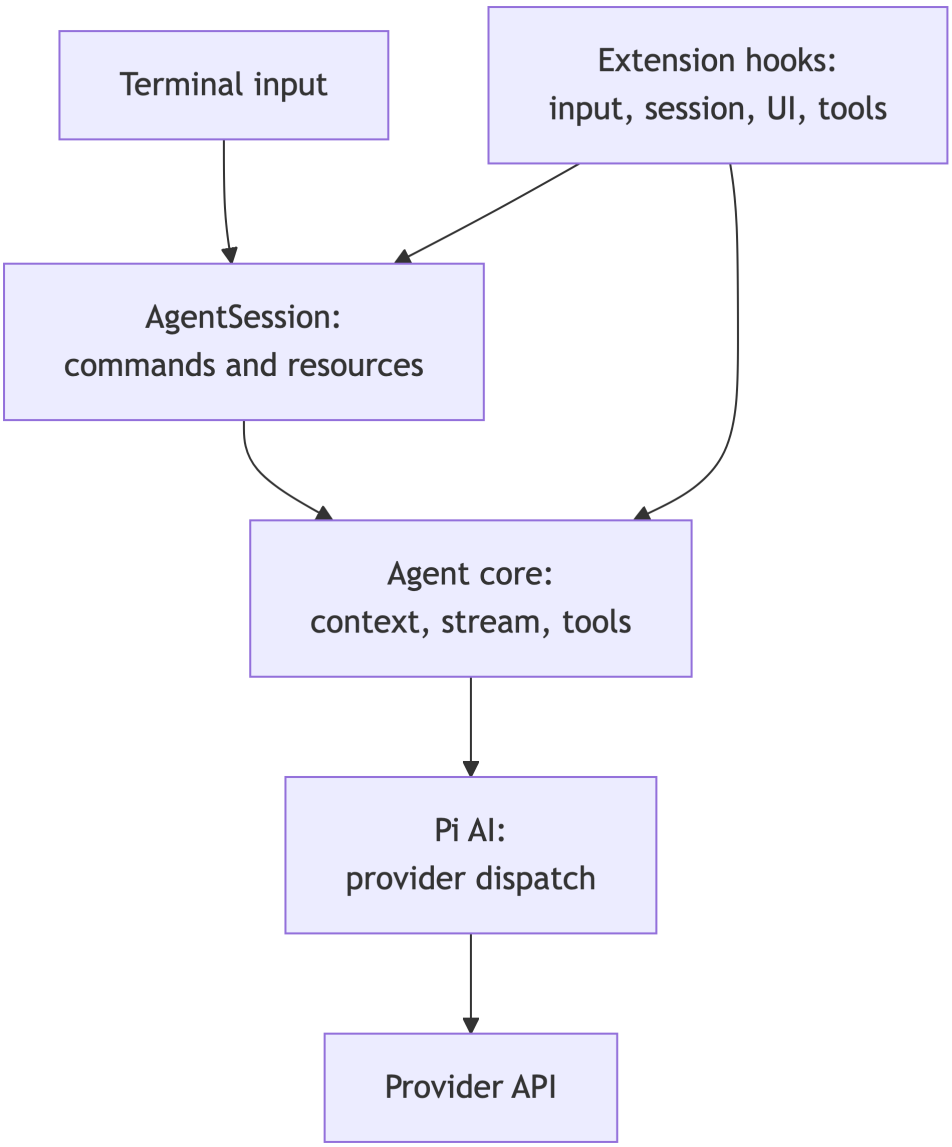

Figure 1.1: Pi's prompt path and the extension surfaces along it.

1.5.4 Building extensions

7. Your first extension is the canonical runnable example.
8. Extension patterns turns that demo into a production extension.
9. Packaging and distribution ships it as a Pi package.

1.5.5 Subsystem deep dives

10. Built-in tools: the seven tools and their contracts.
11. Models, providers, and authentication.
12. Agent core: implementation-level loop, events, queues, and tool scheduling detail.
13. The terminal UI.
14. Run modes and SDK embedding.

1.5.6 Practice and reference

15. Testing, debugging, and contribution.
16. Guided source tours.
17. Extension event and API reference.
18. Failure and recovery.
19. Glossary.

1.6 Source map used for this guide

The guide is checked against package manifests, extension examples, coding-agent documentation, and these implementation seams:

- CLI composition: `packages/coding-agent/src/main.ts` and `src/core/agent-session-services.ts`.
- Session construction and prompt preprocessing: `packages/coding-agent/src/core/sdk.ts` and `src/core/agent-session.ts`.
- Low-level loop and tool execution: `packages/agent/src/agent.ts`, `src/agent-loop.ts`, and `src/types.ts`.
- Provider dispatch: `packages/coding-agent/src/core/sdk.ts`, `src/core/model-runtime.ts`, and the composed providers; the compatibility dispatch in `packages/ai/src/compat.ts` is a legacy/default fallback.
- Extension contract and loading: `packages/coding-agent/src/core/extensions/` plus `packages/coding-agent/docs/extensions.md`.

The repository evolves. If a symbol named in this book moves, search from the repository root with `rg "symbolName" packages`, then update your mental model from the current implementation rather than preserving an old path.

Part I

Start here

2 Reading and changing Pi's TypeScript

Pi is written in TypeScript, but a contributor does not need the whole language before opening the source. This chapter is the short, Pi-oriented route: learn the syntax that lets you follow package boundaries, events, tools, and cancellation, then continue directly to architecture.

> **i** Choose your route
>
> - **Comfortable with TypeScript:** continue in this chapter, then read architecture.
> - **JavaScript-only or new to TypeScript:** read the optional TypeScript foundations workbook first, then return here for Pi's patterns.
> - **Need one concept:** jump to the workbook's Everyday Types, modules, unions, or TypeBox practice. It keeps the longer explanations, exercises, beginner mistakes, and reference tables that were moved out of this route.

2.1 Words we use

Six terms appear throughout this chapter and the workbook. Read these once now; each is explained fully where it is used.

- **Static / compile-time:** what the TypeScript checker can prove by reading the source, before the program runs.
- **Runtime:** what actually happens when Node executes the JavaScript.
- **Erasable syntax:** TypeScript syntax (annotations, interfaces, `type` aliases) that can be removed without changing behavior. Pi restricts itself to erasable syntax so Node can run its `.ts` files directly.
- **Boundary:** a place where data enters from outside the checked program — JSON, configuration files, the network, a model response, an extension.
- **Discriminant:** a shared literal property (usually `type`) that identifies which variant of a union a value is.
- **Narrowing:** a runtime check (`typeof`, `===`, `in`, `Array.isArray`) that tells the checker which union variant it is looking at.
- **Schema:** a runtime description of allowed data that a program can pass to a validator. Pi uses TypeBox schemas.

2.2 What this route teaches

The goal is recognition, not memorization. By the end you should be able to:

- distinguish install, runtime execution, and type checking;
- read modules and public package imports;

- follow a discriminated union and a callback stored for later;
- treat external data as `unknown` until evidence makes it safe;
- distinguish a TypeBox runtime schema from its `Static` compile-time type;
- follow a Promise, an async stream, and an `AbortSignal` into the operation;
- identify the source files and call sites to inspect before changing Pi.

If a primitive, optional property, assertion, or nullish operator is unfamiliar, use the Everyday Types foundations in the workbook rather than stopping the Pi route.

2.3 Setup and runtime orientation

2.3.1 The toolchain in one paragraph

The repository is an npm workspace. `package.json` lists the dependencies and the **scripts** — named commands such as `check` that you run as `npm run check`. `npm install` downloads the dependencies into `node_modules/`. The repository convention is `npm install --ignore-scripts`, which skips package lifecycle scripts for supply-chain safety; the dependencies work without them. `npm run check` is the repository quality gate: it type-checks, lints, and runs related static checks, but not the test suite. Testing, debugging, and contribution covers the full contribution workflow; here you only need the three verbs below.

2.3.2 Install, run, and check are different actions

Job	What it does	Example
Install	Gets the repository's packages	`npm install --ignore-scripts`
Run	Executes JavaScript or erasable TypeScript	`node --experimental-strip-types example.ts`
Check	Finds type errors without running the program	`npx tsgo --noEmit`

Running code does not automatically check its types. Node's native type stripping removes erasable type syntax; it does not type-check the file or transform TypeScript constructs that need JavaScript emit. `tsx` is a development runner and `jiti` loads extensions. `tsgo` is the repository's TypeScript-native checker/build tool: `tsgo --noEmit` checks without writing output.

The normal loop is:

```
install -> write a small example -> run it -> check it -> inspect the error
```

For a guide-only change, render the book. For code changes, the repository quality gate is `npm run check`.

2.3.3 Your first ten minutes: run and check one file

Do this once, now. It makes every later example concrete. Create a scratch directory outside the repository and one file in it:

```
// hello.ts
type User = { name: string };

const user: User = { name: "Ada" };
console.log(`Hello, ${user.name}`);
```

Run it — Node strips the type syntax and executes the JavaScript:

```
$ node --experimental-strip-types hello.ts
Hello, Ada
```

Check it — `tsgo` reads the types and reports nothing, because the file is valid:

```
$ npx tsgo --noEmit hello.ts
(no output: the check passed)
```

Now introduce a mistake: change `user.name` to `user.nmae` and check again:

```
$ npx tsgo --noEmit hello.ts
hello.ts:5:28 - error TS2339: Property 'nmae' does not exist on type 'User'.
```

Read the diagnostic left to right: the file and position (`hello.ts`, line 5, column 28), the error code (`TS2339`), the value you have (`User`), and the mismatch (a property that does not exist). Note two things:

1. `node --experimental-strip-types hello.ts` still runs the misspelled file and prints `Hello, undefined` — running is not checking.
2. Every TypeScript error answers the same three questions: *what value do I have, what did the code require, and where did they diverge?*

The workbook's common TypeScript errors reference decodes the diagnostics you will meet most often in Pi.

2.3.4 Pi's erasable TypeScript boundary

Pi source avoids TypeScript syntax that needs a JavaScript transform, including constructor parameter properties, `enum`, and `namespace`. Prefer explicit fields, literal unions, and ESM imports. This matters when a file is run by native Node stripping rather than compiled first.

`declare` is a type-checking-only declaration: it tells TypeScript that a value exists but emits no JavaScript. It is useful in small type-reading examples, not as a way to create a runtime value:

```
declare const raw: unknown;
const claimed = raw as string; // assertion only; no runtime validation
```

2.3.5 Readiness check

Before continuing, try to answer these questions. The answers at the end of the chapter point to the workbook section that explains each one — peeking is fine; being unable to find the answer is the signal to read that section first.

1. Which command installs packages, which runs a file, and which checks types?
2. What is the difference between a value at runtime and a type before runtime?
3. When a property is written `name?: string`, what value can a read produce?
4. What evidence narrows `string | number` to one branch?
5. What does `as User` change, and what does it *not* validate?

2.4 Everyday Types at a glance

Use this compact table while reading Pi. The workbook gives the full lesson, worked dashboard, practice lab, and common-error explanations.

Syntax	What the checker sees	Reading question
`const name = "Ada"`	a literal string value	Can this binding change?
`let name = "Ada"`	generally `string`	Could another string be assigned?
`const ids: number[] = []`	an array of numbers	Can an index be missing?
`last?: string`	an optional property	What happens when it is absent?
`string | number`	one of two alternatives	Which check narrows it?
`"idle" | "running"`	a closed literal vocabulary	Which values are allowed?
`value as User`	an assertion to the checker	What runtime evidence supports it?
`Static<typeof Schema>`	a derived compile-time type	Where does runtime checking happen?

The central distinction is simple: TypeScript descriptions help the checker before execution; JavaScript values and checks determine what happens at runtime. An interface and an assertion disappear, while a schema object and a validator remain available to inspect data.

2.5 Modules and imports

A **module** is a file that exports names for other files and imports names it uses. Runtime values and compile-time-only types can be imported separately:

```ts
// math.ts
export function add(left: number, right: number): number {
  return left + right;
}

export type Pair = [number, number];
```

```ts
// main.ts
import { add, type Pair } from "./math.ts";

const pair: Pair = [2, 3];
console.log(add(...pair));
```

When reading Pi, recognize three import shapes: relative imports with a `.ts` suffix inside one package, public package names across packages, and `import type` for checker-only names:

```ts
import { Type } from "@earendil-works/pi-ai";
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";
```

Prefer public entry points over deep imports into another package's source. The public boundary is the contract used by published consumers. Default exports, re-exports, and module-resolution troubleshooting are covered in the workbook's modules section.

2.6 Discriminated unions

A **discriminant** is a shared literal property that identifies one variant of a union. Read the tag first, then inspect the fields available in that branch:

```ts
type AgentNotice =
  | { type: "started"; prompt: string }
  | { type: "finished"; stopReason: "stop" | "error" }
  | { type: "failed"; message: string };

function assertNever(value: never): never {
  throw new Error(`Unexpected variant: ${String(value)}`);
}

function label(notice: AgentNotice): string {
  switch (notice.type) {
    case "started":
      return `started: ${notice.prompt}`;
    case "finished":
      return `finished: ${notice.stopReason}`;
    case "failed":
      return `failed: ${notice.message}`;
    default:
```

```
      return assertNever(notice);
  }
}
```

The `type` check is evidence. Before narrowing, code may use only operations shared by every variant; after `notice.type === "failed"`, `message` is safe. The `default` branch looks unreachable, and it is: `assertNever` accepts only `never`, so the line compiles precisely because every variant is already handled. When Pi later adds a fourth variant, that line stops compiling until you handle it — the checker becomes your to-do list. This pattern matters in practice: Pi uses discriminated unions for agent events, message roles, content blocks, and session entries, all of which grow over time. Start at the union definition, then find each `if` or `switch` that consumes it. The workbook's discriminated unions section adds user-defined predicates and more narrowing forms.

2.7 Callbacks and closures

A **callback** is a function passed to another function to be called now or later. Registration stores the function; invocation calls it:

```
type Listener = (message: string) => void;
let listener: Listener | undefined;

function register(next: Listener): void {
  listener = next;
}

function emit(message: string): void {
  listener?.(message);
}

register((message) => console.log(message));
emit("ready");
```

The common error is `register(next())`: that calls `next` immediately and passes its return value, rather than passing the function. A **closure** is a function that remembers variables from where it was created. Look for the cleanup API when a closure is stored in a long-lived registry.

Pi handlers often receive a full contract even when they use only one argument, and names such as `_event`, `_signal`, and `_ctx` mark parameters that are part of the contract but unused by that handler:

```
function onMessage(handler: (text: string, source: string) => void): void {
  handler("ready", "agent");
}

onMessage((text, _source) => console.log(text));
```

The workbook's callbacks section covers optional callback parameters, overloads, and the `this` pitfall when a class method is registered as a callback.

2.8 `unknown` versus `any`

At a JSON, configuration, network, model, or extension boundary, the checker cannot prove that incoming data has the expected shape. `any` disables that question — nothing is checked, and mistakes surface at runtime:

```
function unsafe(value: any): string {
  return value.name.toUpperCase(); // compiles; may crash
}
```

`unknown` requires evidence before use:

```
function readName(value: unknown): string | undefined {
  if (!value || typeof value !== "object" || !("name" in value)) {
    return undefined;
  }

  const name = value.name;
  return typeof name === "string" ? name : undefined;
}
```

An assertion such as `value as User` changes only the checker's view. It does not convert, inspect, or repair the runtime value. Prefer `unknown` plus a narrowing guard, or a runtime schema when the boundary is shared. The workbook walks one JSON value through four treatments and covers error handling at the same boundaries in errors and exceptions.

2.9 TypeBox: one schema, two views

TypeBox is Pi's runtime validation bridge. `Type.Object(...)` creates a schema value that exists while the program runs. `Static<typeof Schema>` derives a compile-time type for the checker. The static type does not validate a value by itself:

```
import { Type, type Static } from "@earendil-works/pi-ai";
import { Value } from "typebox/value";

const UserSchema = Type.Object({
  name: Type.String(),
});

type User = Static<typeof UserSchema>;

const raw: unknown = { name: "Ada" };
if (Value.Check(UserSchema, raw)) {
  // The runtime check supplies the evidence used here.
  const user = raw as User;
  console.log(user.name);
}
```

The schema is runtime data; `User` is erased. `Value.Check` answers whether an actual value matches the schema. TypeBox does not add defaults, perform side effects, or enforce application rules such as "the path exists."

2.9.1 A Pi tool preview

This is intentionally a **preview** of extension registration. The canonical, runnable staged example is in Your first extension; read that chapter for discovery, reload, permissions, and the complete API.

```
import { Type } from "@earendil-works/pi-ai";
import {
  defineTool,
  type ExtensionAPI,
} from "@earendil-works/pi-coding-agent";

const helloTool = defineTool({
  name: "hello",
  label: "Hello",
  description: "Greet one person.",
  parameters: Type.Object({ name: Type.String() }),
  async execute(_toolCallId, params, signal, _onUpdate, _ctx) {
    if (signal?.aborted) {
      return { content: [{ type: "text", text: "cancelled" }] };
    }
    return { content: [{ type: "text", text: `Hello, ${params.name}!` }] };
  },
});

export default function register(pi: ExtensionAPI): void {
  pi.registerTool(helloTool);
}
```

Read the example from the outside in: `Type.Object` is a runtime parameter schema; `defineTool` connects that schema to `params`; Pi validates model arguments before `execute`; and the returned `content` becomes the tool result. Pi passes five arguments to `execute`; the `_`-prefixed ones are part of the contract that this tool does not use. The optional signal should be forwarded to cancellable work. The workbook breaks this same example down field by field in TypeBox in a Pi tool.

2.10 Promises, streams, and cancellation

An `async` function returns one eventual result as a `Promise<T>`; `await` unwraps it. An async iterable produces multiple values over time, and `for await` consumes each one as it arrives:

```
async function loadStatus(): Promise<string> {
  return "working";
}
```

```ts
async function* tokens(): AsyncIterable<string> {
  yield "Hel";
  yield "lo";
}

const status = await loadStatus();
for await (const token of tokens()) {
  console.log(token);
}
```

An `AbortSignal` carries a cancellation request from its owner to the work:

```ts
async function load(url: string, signal: AbortSignal): Promise<string> {
  const response = await fetch(url, { signal });
  return response.text();
}
```

When reading Pi, follow the signal into the file, process, network, or provider operation. The owner creates or receives the `AbortController`; the operation must decide how to stop and clean up. Rejection, `Promise.all`, timeouts, and error behavior are covered in the workbook's async section; retry and parallel behavior in concrete contexts belong to the agent and tool chapters.

2.11 Pi workflow and source-reading checklist

Pi uses an agent harness and a coding-agent application. The normal path crosses these packages:

Package	Responsibility	First source to inspect	Do this now
`packages/tui`	reusable terminal rendering primitives	`src/index.ts`, `src/tui.ts`	open `src/tui.ts` and find one interface with a literal-union property
`packages/ai`	models, provider dispatch, and streaming contracts	`src/index.ts`, `src/types.ts`	open `src/types.ts` and list three literal `type` tags
`packages/agent`	provider-independent loop, messages, and tools	`src/agent.ts`, `src/agent-loop.ts`	find the `AgentEvent` union in `src/types.ts`, then one consumer switch in `src/agent.ts`
`packages/coding-agent`	CLI, sessions, built-in tools, resources, extensions	`src/main.ts`, `src/core/sdk.ts`	open `examples/extensions/hello.ts` and identify the schema, the callback, and the signal

For a change, use this order:

1. classify the behavior by owner: TUI, AI/provider, generic agent, or coding-agent application;
2. find the declaration and its public export;
3. find construction sites and call sites;
4. identify literal tags, optional fields, and external data boundaries;
5. follow callbacks, streams, signals, and cleanup to their owner;
6. make the smallest change that preserves the contract;
7. update union consumers when a variant changes;
8. run the relevant check and inspect the diff.

Continue with architecture: packages, ownership, and entry points. Use the workbook's guided source tour if you want a slower exercise before opening the repository.

2.12 Closing checkpoint

You are ready for architecture when you can explain: which package owns a behavior, which literal selects a union branch, which callback runs later, why an external value is `unknown`, where TypeBox performs runtime validation, and which signal or cleanup method owns cancellation. Check yourself against the answers below.

2.13 Answers to the readiness check

1. `npm install --ignore-scripts` installs, `node --experimental-strip-types` (or `tsx`) runs, `tsgo --noEmit` checks. See install, run, and check.
2. A type is a compile-time description the checker erases; a value is what exists while JavaScript runs. See the workbook's what TypeScript checks—and what it erases.
3. `string | undefined` — the property may be absent. See object types and optional properties.
4. A runtime check such as `typeof value === "string"`. See union types and basic narrowing.
5. `as User` changes only the checker's view; it validates nothing at runtime. See type assertions.

2.14 Answers to the closing checkpoint

- **Package ownership:** TUI rendering → `packages/tui`; models/providers → `packages/ai`; the provider-independent loop → `packages/agent`; CLI, sessions, tools, extensions → `packages/coding-agent`.
- **Union branch:** the literal value of the shared discriminant property (`type`, `role`, …) compared with `===` or matched in a `switch`.
- **Callback timing:** registration stores the function; it runs when the owner invokes it — later for event handlers and tool `execute`.
- **External values:** the checker cannot prove the shape of JSON, config, network, or model data, so it is typed `unknown` until a guard or schema check supplies evidence.
- **TypeBox validation:** in `Value.Check(schema, value)` (and inside Pi where tool arguments are validated), never in `Static<typeof ...>`.
- **Cancellation ownership:** the object that created the `AbortController` decides when to abort; the operation that received the `signal` decides how to stop and clean up.

Part II

Using Pi effectively

3 Using Pi day to day

In plain terms: Pi is a terminal program you run inside a project directory, sign in to once, and talk to. It reads your code, edits files, and runs commands on your behalf. This chapter teaches the daily-driver surface: starting a session, signing in, the slash commands and keys you will actually press, the CLI flags worth remembering, and how to get project context into the conversation.

It is grounded in `packages/coding-agent/README.md`, `packages/coding-agent/src/cli/args.ts`, `packages/coding-agent/src/core/slash-commands.ts`, and `packages/coding-agent/docs/keybindings.md`.

3.1 Prerequisites

Install Pi globally from npm:

```
npm install -g --ignore-scripts @earendil-works/pi-coding-agent
```

The `--ignore-scripts` flag disables dependency lifecycle scripts; Pi does not need install scripts for a normal npm install. An installer script is the alternative:

```
curl -fsSL https://pi.dev/install.sh | sh
```

Either way you get a `pi` binary on your PATH. Verify with `pi --version`.

3.2 Your first session

Change into a project directory and run:

```
pi
```

The interactive interface (the **TUI**, terminal user interface) has four regions, top to bottom:

- **Startup header** — loaded `AGENTS.md` files, prompt templates, skills, and extensions.
- **Messages** — your prompts, assistant responses, tool calls and results.
- **Editor** — where you type. Its border color indicates the thinking level.
- **Footer** — working directory, session name, token and cache usage, cost, context usage, and the current model.

Type a prompt in the editor and press **Enter** to submit. By default the model has four tools — `read`, `write`, `edit`, and `bash` — and uses them to fulfill your request. You watch its tool calls stream by in the message region.

Quit with `/quit`, press **Ctrl+C** twice, or press **Ctrl+D** when the editor is empty.

A **session** is the durable record of that conversation. Pi auto-saves sessions as JSONL files under `~/.pi/agent/sessions/`, organized by working directory — so a session belongs to the directory you started it in, and running `pi` in two different projects keeps two separate histories. Session files, branching, and what happens when a conversation gets long are covered in Sessions, trees, and compaction.

3.3 Signing in and picking a model

Pi needs credentials for at least one provider before the model can answer. There are two routes.

Subscription login. Run `pi`, then type `/login` and pick a provider. Pi supports subscription logins for Anthropic Claude Pro/Max, OpenAI ChatGPT Plus/Pro (Codex), and GitHub Copilot. `/logout` removes credentials.

API keys. Set the provider's environment variable before starting Pi. The most common examples:

Environment variable	Provider
`ANTHROPIC_API_KEY`	Anthropic
`OPENAI_API_KEY`	OpenAI
`GEMINI_API_KEY`	Google Gemini
`OPENROUTER_API_KEY`	OpenRouter

Many more providers are supported (DeepSeek, Groq, Mistral, xAI, Amazon Bedrock, and others); the full list of environment variables is in `pi --help` and `packages/coding-agent/README.md`. The `--api-key <key>` flag overrides environment variables for one run.

Once authenticated, switch models any time with `/model` (or **Ctrl+L**), which opens a selector over every model your credentials can reach. Cycle through your enabled models with **Ctrl+P**; `/scoped-models` controls which models participate in that cycle. Providers, model catalogs, custom providers, and the authentication model get a full treatment in Models, providers, and authentication; this chapter only gets you talking to a model.

3.4 Slash commands you will actually use

Type `/` in the editor to open the command list. The built-in commands are defined in `src/core/slash-commands.ts`; the ones a new user needs first:

Command	What it does
`/login`, `/logout`	Configure or remove provider credentials
`/model`	Switch models (opens a selector)
`/new`	Start a new session

Command	What it does
`/resume`	Pick a previous session to resume
`/name <name>`	Set the session's display name
`/session`	Show session info: file, ID, messages, tokens, cost
`/compact [prompt]`	Manually compact context, optionally with custom instructions
`/settings`	Thinking level, theme, message delivery, transport
`/trust`	Save the project trust decision (takes effect after restart)
`/reload`	Reload keybindings, extensions, skills, prompts, themes, context files
`/hotkeys`	Show every keyboard shortcut
`/quit`	Quit Pi

There is no built-in `/help`; `pi --help` covers the CLI and `/hotkeys` covers the keys. Extensions, skills, and prompt templates add their own commands on top of this list — see Customizing Pi.

3.5 The keys that matter

The full keybinding reference lives in `docs/keybindings.md` and `/hotkeys`. Learn these first:

Key	Action
Enter	Submit the prompt; while the agent is working, queues a *steering* message delivered after the current turn
Shift+Enter	Insert a newline (Ctrl+Enter on Windows Terminal)
Escape	Cancel/abort the running agent; restores queued messages to the editor
Escape twice	Open `/tree` session navigation
Ctrl+C	Clear the editor; twice to quit
Ctrl+L	Open the model selector
Shift+Tab	Cycle the thinking level
Ctrl+V	Paste an image or text from the clipboard (Alt+V on Windows)
Ctrl+G	Open the prompt in an external editor

Two message-queue subtleties are worth knowing early. **Alt+Enter** queues a *follow-up* message that is delivered only after the agent finishes all its work, unlike Enter's steering message. **Alt+Up** pulls queued messages back into the editor. Steering while streaming is a first-class feature: you never have to wait for a run to finish before correcting course.

Every keybinding is configurable in `~/.pi/agent/keybindings.json`; run `/reload` to apply edits without restarting. The mechanism is covered in Customizing Pi.

3.6 CLI flags for daily use

`pi [options] [@files...] [messages...]` — flags are parsed in `src/cli/args.ts`. The useful daily subset:

Flag	Description
`-p`, `--print`	Non-interactive: process the prompt, print the response, exit
`--model <pattern>`	Model pattern or ID; accepts `provider/id` and a `:<thinking>` suffix (e.g. `sonnet:high`)
`--provider <name>`	Provider name when `--model` has no prefix
`-c`, `--continue`	Continue the most recent session in this directory
`-r`, `--resume`	Browse and select a past session
`--session <path|id>`	Use a specific session file or partial UUID
`--name <name>`, `-n`	Set the session display name at startup
`--no-session`	Ephemeral mode; nothing is saved
`--list-models [search]`	List available models, optionally filtered
`-e`, `--extension <path>`	Load an extension (repeatable)
`--tools <list>`, `-t`	Allowlist tools, e.g. `--tools read,grep,find,ls` for read-only runs
`-h`, `--help`	Full CLI help, including every provider environment variable
`-v`, `--version`	Print the version

Passing messages as arguments works in both modes: `pi "List all .ts files"` opens the TUI with that first prompt already submitted, while `pi -p "Summarize this codebase"` runs headless and exits. Print mode also reads piped stdin and merges it into the prompt:

```
cat README.md | pi -p "Summarize this text"
```

Session-selection flags (`-c`, `-r`, `--session`) map directly onto the session machinery in Sessions, trees, and compaction.

3.7 Getting context into Pi

Pi is only as useful as the context you hand it. Three everyday mechanisms:

@-file mentions. Type `@` in the editor to fuzzy-search project files and attach one to your message. The same syntax works on the command line: `pi @code.ts @test.ts "Review these files"`. You can also paste text or images directly into the editor with Ctrl+V, or drag images onto the terminal.

`!` for shell output. Prefix a line with `!command` to run a command and send its output to the model with your message; `!!command` runs it without sending the output.

Context files. Pi loads `AGENTS.md` (or `CLAUDE.md`) at startup from `~/.pi/agent/`, from every ancestor directory down to your cwd, and from the cwd itself. Use these for project conventions, build commands, and rules the model should always follow. The exact discovery order and precedence rules are in Customizing Pi.

3.8 Where to go next

- Sessions, trees, and compaction — how sessions persist, branch with `/tree` and `/fork`, and compact when context fills.
- Customizing Pi — settings, keybindings, skills, prompt templates, themes, and trust.
- Models, providers, and authentication — the full provider and auth picture behind `/login` and `/model`.
- Architecture: packages, ownership, and entry points — how the TUI, coding-agent, and agent-core packages fit together, for when you start extending Pi.

4 Sessions, trees, and compaction

In plain terms: a session is Pi's durable record of a conversation. It survives restarts, supports `/resume` and `/new`, and keeps branches when you explore an earlier point with `/tree` (move within one session file), `/fork` (continue from an earlier message in a new file), or `/clone` (copy the current active root-to-leaf path into a new file). When context fills, Pi can compact older work into a summary instead of discarding the session.

This chapter first explains those user-visible behaviors. The final sections map them to `ctx.sessionManager`, `appendEntry`, and `session_*` events for extension authors. `sessionDir` and related configuration are covered under Files, headers, and locations below; see Customizing Pi for settings in general.

It is grounded in four files:

- `packages/coding-agent/src/core/session-manager.ts` for persistence and context reconstruction.
- `packages/coding-agent/src/core/messages.ts` for message conversion.
- `packages/coding-agent/src/core/agent-session-runtime.ts` for replacement and cleanup.
- `packages/coding-agent/docs/session-format.md` for the durable file format.

4.1 Prerequisites

Read Using Pi day to day first and complete its first session: you should know what a slash command is and have used `/resume` at least once. Hands-on familiarity with `/tree` helps but is not required; the "Try it" checkpoint below walks through it.

4.2 What a session is

A **session** is Pi's durable record of a conversation. An **entry** is one durable record in that session. A **branch** is one path through entries that share a common history. The **leaf** is the current last entry on that path. **Context** is the selected representation sent to the model; it is not automatically the whole file on disk.

This distinction is the starting point for extension work. The session file keeps durable history, including branches and extension state. Before a model request, Pi derives a model-facing view from the selected branch, then converts that view to provider messages. A record can therefore remain available to `/resume` or `/tree` without being sent on the next request.

A concrete timeline fixes the terms. You run `pi` in `~/proj` and exchange ten messages: that conversation is one **session**, persisted to one JSONL file (JSON Lines: one complete JSON object per line), and each message (plus tool calls and tool results) is one **entry**. You quit. The next day you run

`/resume`, pick the session, and Pi reloads the same file and keeps appending entries to it. Midway through, you go back to your fourth message and ask something different: the conversation now has two versions of everything after message four, and each version is a **branch**. The **leaf** is the last entry of whichever branch is active. When you send the next prompt, the **context** is the entry chain from that leaf back to the root — not necessarily every entry the file has ever stored.

Start by imagining two answers to the same user question: answer A and answer B share the earlier messages but diverge later. A flat array cannot keep both answers without duplication. The small tree below shows why Pi stores parents and children before the chapter introduces JSONL details.

4.3 Where sessions live on disk

Each project directory — the cwd where you run `pi` — gets its own session storage. One project directory accumulates many session files over time: one file per conversation, plus more as you resume, fork, or start over with `/new`.

By default the files live under `~/.pi/agent/sessions/` in a subdirectory whose name encodes the cwd. `getDefaultSessionDirPath()` in `packages/coding-agent/src/core/session-manager.ts` computes it: resolve the cwd, strip one leading slash or backslash, replace every `/`, `\`, or `:` with `-`, and wrap the result in `--`. Running `pi` in `/Users/alice/proj` therefore stores its sessions in:

```
~/.pi/agent/sessions/--Users-alice-proj--/<timestamp>_<uuid>.jsonl
```

The practical consequence: `/resume` opens with only the sessions created in the current directory (it lists them via `SessionManager.list(cwd, ...)`). Run `pi` in a different project and the selector shows nothing from the first project until you press Tab to switch the selector scope to all projects.

The `sessionDir` setting relocates session storage away from this cwd-derived layout; see Files, headers, and locations below for precedence and examples, and Customizing Pi for settings in general.

4.4 Branches: why persistence is a tree

`/tree` changes the leaf to a different entry. Work on the old branch may still matter for the new branch even though it is no longer on the parent chain. Pi can summarize that abandoned path and append a `branch_summary` at the navigation point so the new path keeps a concise account without replaying all old messages.

Configure this under `branchSummary` in the global or project settings files (see Files, headers, and locations):

```
{
  "branchSummary": {
    "reserveTokens": 16384,
    "skipPrompt": false
  }
}
```

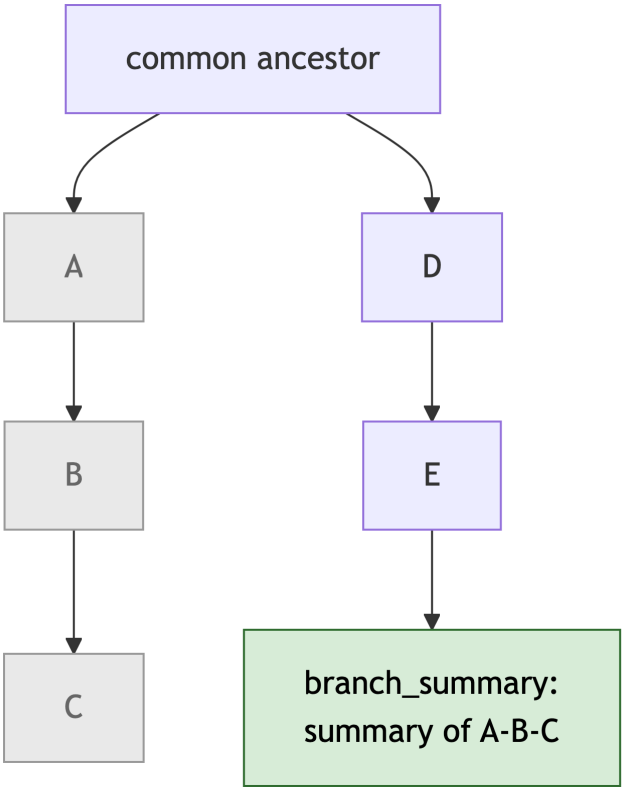

Figure 4.1: Tree navigation with a branch summary. The abandoned path (grey) is no longer on the parent chain; Pi can append a branch_summary entry (green) near the navigation point so the new path keeps a concise account of it.

`reserveTokens` reserves response space for the summarizer. With the default `skipPrompt: false`, `/tree` asks whether to summarize; `true` suppresses that question and defaults to no summary. Extensions can provide or cancel a summary with `session_before_tree` and observe the result with `session_tree`.

Suppose a user asks for an implementation, then navigates back to an earlier message and explores a different plan. A linear transcript either loses one path or duplicates a large prefix into another file. Pi instead stores entries with an `id` and `parentId`. One file can represent both paths.

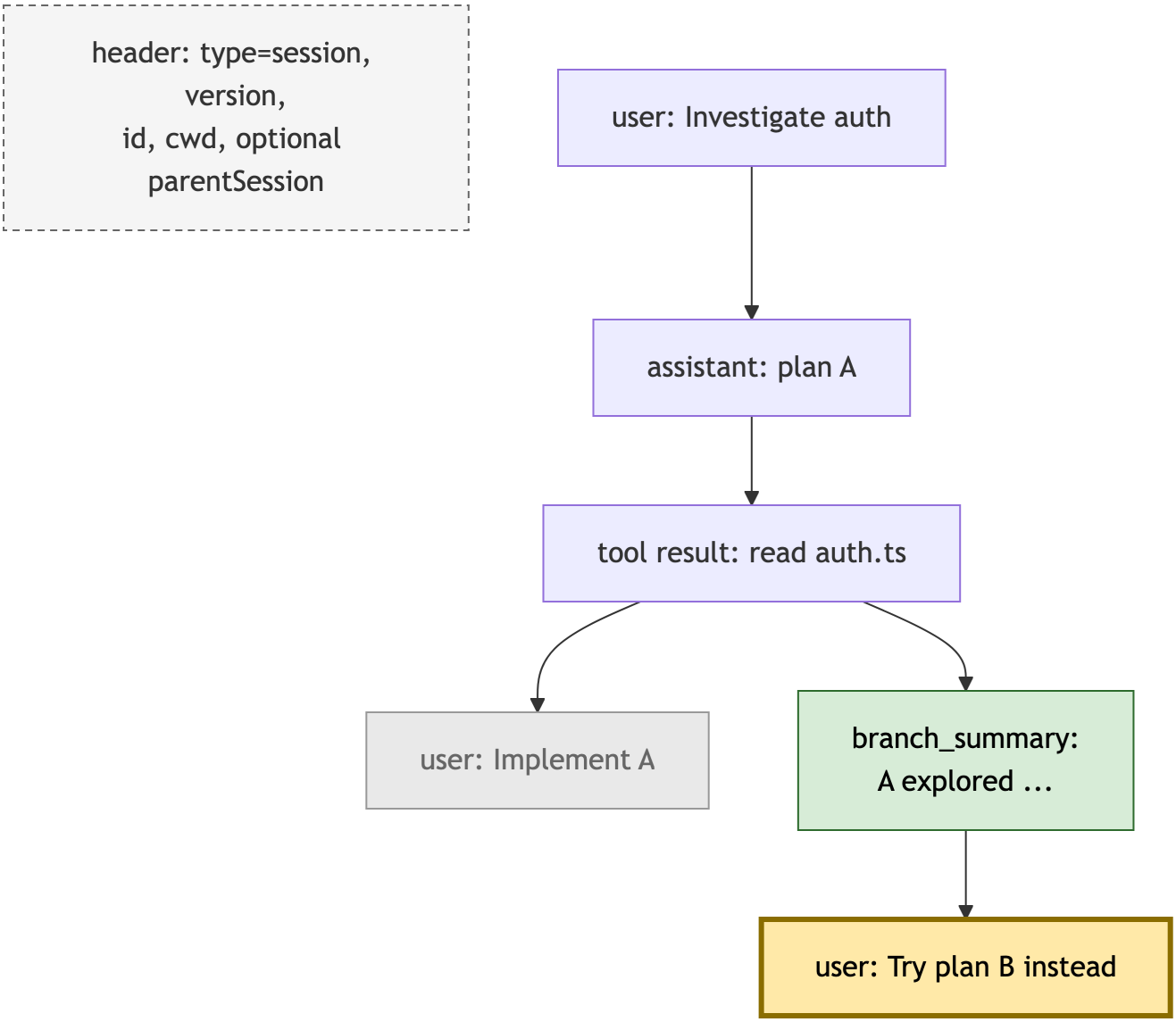

Figure 4.2: One session file, two branches. Entries link by parentId; the header (dashed) is metadata, not a tree node. The old leaf (grey) stays on disk; the current leaf (amber) defines the active branch.

`parentSession` on the header records fork provenance across session files. It is not a `parentId` edge inside this entry tree.

The active branch is the chain from the current leaf to the root, read in chronological order. Other

children are not provider context — the messages actually sent to the model provider — merely because they exist on disk. Conceptually, the next request is built by following parent links from the selected leaf, reversing that chain into chronological order, applying any compaction checkpoint, and converting the selected entries into provider messages. When debugging a surprising model response, inspect that active path rather than the whole session file (the debugging checklist below walks through this).

4.4.1 A worked example: file state and leaf after each step

Step (a): chat a few messages. One file, one path, leaf at the end.

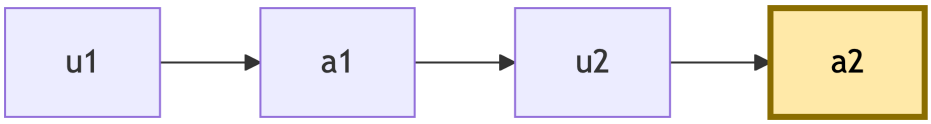

Figure 4.3: Step (a): a single linear path; the leaf (amber) sits at the last entry.

Step (b): `/tree` back to `u2` and continue with a different request. The new messages append as children of `u2`, so the file now holds two branches. The leaf moves to the new path; the old path is untouched on disk.

Step (c): `/tree` again and select `a2`. Only the leaf pointer changes. Both branches remain in the file, and context for the next prompt is the chain `u1 -> a1 -> u2 -> a2`.

`/fork` performs the same jump to an earlier message but creates a **new session file** instead of an in-file branch: the new file contains only the path from the root to the chosen entry, and its header records the source file as `parentSession`. To move between the fork and the original you `/resume` the other file; `/tree` navigates branches within one file.

The navigation commands differ in which file they touch and where the leaf ends up:

Command	Session file	Leaf	Effect
`/resume`	Opens an existing file	Moves to that file's saved leaf	Switches to another session
`/new`	Creates a new file	Starts empty	Starts a fresh conversation
`/tree`	Same file	Moves to a selected entry	Switches branches within one session
`/fork`	Creates a new file	Starts at the chosen earlier user message	Copies the root-to-message path into a new session
`/clone`	Creates a new file	Same position as the current leaf	Copies the current active root-to-leaf path into a new session

4.4.2 Try it: watch a branch appear on disk

1. Run `pi` in a scratch directory and send two short messages.
2. Run `/tree`, select your first message, and send a different follow-up.

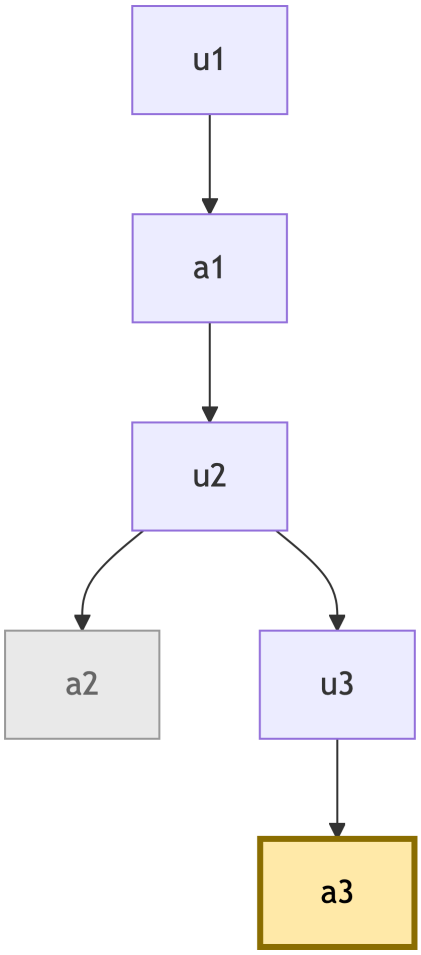

Figure 4.4: Step (b): continuing from u2 appends a second branch. The old branch (grey) is unchanged on disk; the leaf moves to a3.

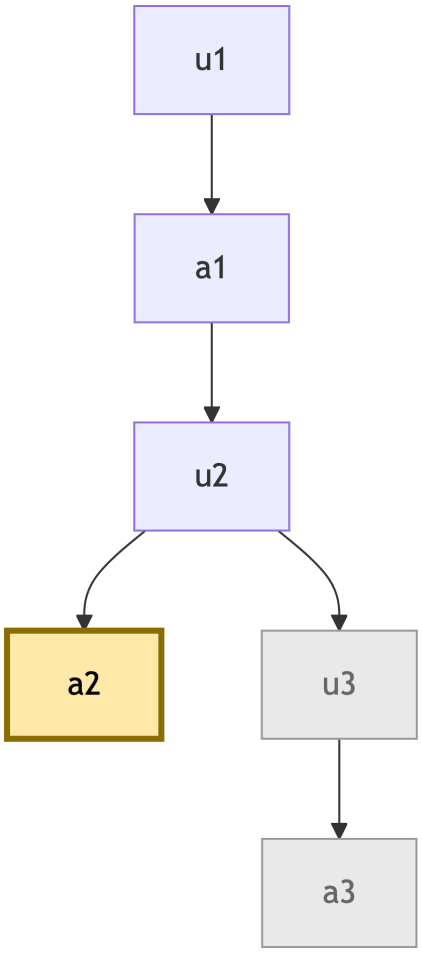

Figure 4.5: Step (c): navigating back only moves the leaf. Both branches stay in the file; the next prompt's context is the chain from a2 to the root.

3. Quit, then list the session directory from Where sessions live on disk (`~/.pi/agent/sessions/--<encoded-cwd`
one JSONL file holds both branches.
4. `cat` that file and look for two entries sharing the same `parentId` — that shared parent is the fork point you created in step 2.

4.5 Files, headers, and locations

By default, coding-agent sessions live under the agent directory, arranged by working directory:

```
~/.pi/agent/sessions/--<encoded-cwd>--/<timestamp>_<uuid>.jsonl
```

The `<encoded-cwd>` name comes from `getDefaultSessionDirPath()`, not URL encoding. Pi resolves the cwd, strips one leading slash or backslash, replaces every slash, backslash, or colon with `-`, and wraps the result in `--`. For example, `/work/project` becomes `--work-project--`; the separators are only a safe filesystem-directory name, not a reversible URL representation. A custom `sessionDir` bypasses this cwd-derived directory name.

Each persisted session is a `.jsonl` file. Its first line is a file header, for example:

```
{
  "type": "session",
  "version": 3,
  "id": "<id>",
  "timestamp": "<timestamp>",
  "cwd": "/work/project"
}
```

The header's `id` identifies the session, not a node. Unlike every subsequent entry, the header has no `parentId` and cannot be selected as the active leaf. `version` controls format migration, `timestamp` and `cwd` describe where the session began, and optional `parentSession` records fork provenance only. Pi migrates older supported session versions when it loads them; extension authors normally use `SessionManager` rather than depending on a particular on-disk version.

> **Official session-format reference**
>
> This chapter explains how session persistence affects a model request. For the complete JSONL schema, all entry kinds, version details, and the public `SessionManager` API, see the official Session format documentation.

Every line after the header is one entry. A user message and the assistant's reply look like this (abridged from `packages/coding-agent/docs/session-format.md`):

```
{
  "type":"message", "id":"a1b2", "parentId":null,
  "timestamp":"...", "message":{"role":"user",...}
}
{
```

```
  "type":"message", "id":"b2c3", "parentId":"a1b2",
  "timestamp":"...", "message":{"role":"assistant",...}
}
```

Note the chain: the first entry's `parentId` is `null`, and each later entry points at its parent's `id`. Those two fields per line are all the tree structure the file needs.

Session storage and other options are ordinary nested keys in settings JSON, not filesystem path conventions. Global settings live in `~/.pi/agent/settings.json`; project settings live in `.pi/settings.json`. When both files set the same keys, project wins — but how two values combine depends on their type:

Value type	Merge behavior	Example
Scalar	Project value replaces the global one	Project `"defaultModel"` wins outright
Object	Merged at the top level only; nested objects are replaced, not recursively merged	Project can set `compaction.reserveTokens` while keeping the global `compaction.keepRecentTokens`, but a nested object field would be replaced whole
Array	Effective settings value is replaced by the project value; runtime resource/package resolution may still combine raw global and project sources	The effective `"packages"` setting is replaced, while package/resource loading can still consider global and project sources

Override the session directory with:

```
--session-dir  >  PI_CODING_AGENT_SESSION_DIR  >  settings.json sessionDir
```

`sessionDir` accepts absolute or relative paths and expands `~`. Example:

```
{ "sessionDir": ".pi/sessions" }
```

Use `SessionManager.inMemory(cwd?)` when embedding Pi where persistence is not desired. An in-memory manager still has branches and context construction; it simply has no session-file path.

4.6 Compaction: what happens when context fills

When a long session approaches the model context limit, Pi can compact it.

Compaction in one sentence: Pi summarizes older context into a durable entry and continues with the summary plus recent entries. The split-turn details below are primarily for debugging and extension interception; they use the word **turn** in the session sense — one user message plus the assistant's response and any tool calls and results that follow it, up to the next user message.

The model never sees the session file. On every turn it sees only the **context**: the entries on the active branch, converted to provider messages. The file on disk is the full history; the context is the extract of it that must fit inside the model's context window. As a conversation grows, that extract eventually stops fitting. Compaction resolves this by replacing the model-visible context with a summary of the older span plus the recent entries. The summarized entries stay in the session file and remain visible to `/tree` and `/resume`; only what gets sent to the model changes.

Context windows are finite. Auto-compaction triggers when:

```
estimated context tokens > model context window - reserveTokens
```

Defaults are `compaction.reserveTokens: 16384` and `compaction.keepRecentTokens: 20000`, configurable under the `compaction` object in global or project settings. Exact token calculation and cut-point behavior live in `packages/coding-agent/src/core/compaction/compaction.ts` (for example `findCutPoint()`); coding-agent coordinates the surrounding session operation, extension events, UI, and retry behavior.

With concrete numbers: against a 200,000-token context window and the default `reserveTokens: 16384`, auto-compaction triggers once the estimated context passes roughly 183,600 tokens. After it runs with the default `keepRecentTokens: 20000`, the model sees approximately:

```
[compaction summary of the older span] + [the most recent ~20,000 tokens of entries]
```

The full pipeline is:

Pi chooses a valid cut near the recent-history target, summarizes the older span (including a prior summary when present), appends a `compaction` entry, and rebuilds context from that summary plus the retained entries. Tool-call and tool-result relationships constrain where the cut may land. Raw history stays on disk; only provider context is replaced.

For an extension author, the important contrast is simple: compaction changes what the next model request sees, not what the session remembers. It can run automatically at the threshold, recover from a context overflow before retrying a turn, or run when the user invokes `/compact`. Repeated compactions keep summarizing older model-visible work while retaining the recent working suffix. `/tree` has a related but separate branch-summary flow: it can carry a concise account of the branch being left into the newly selected branch.

An extension can intercept `session_before_compact` to cancel compaction or provide a custom summary, then observe the result through `session_compact`. `session_before_tree` and `session_tree` provide the corresponding branch-summary boundaries. See the extension event reference for the complete event shapes, and the official Compaction documentation for cut-point rules, split turns, summary structure, and custom-summary fields.

4.6.1 Split turns and tool continuity

Normally a compaction cut sits at a turn boundary, using the session sense of turn defined above. Pi avoids cutting between an assistant tool call and its tool result because that would make the remaining context internally incoherent for the provider.

If one turn exceeds `compaction.keepRecentTokens`, Pi may split it at an assistant message, never at a tool result. For example:

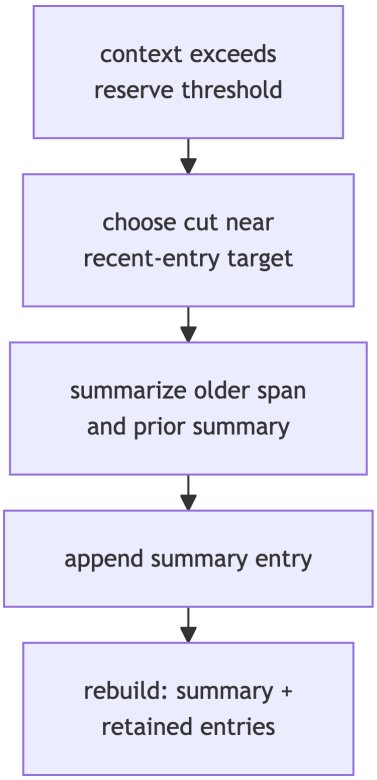

Figure 4.6: The auto-compaction pipeline: trigger check, cut selection, summarization, then context rebuild from the summary plus retained entries.

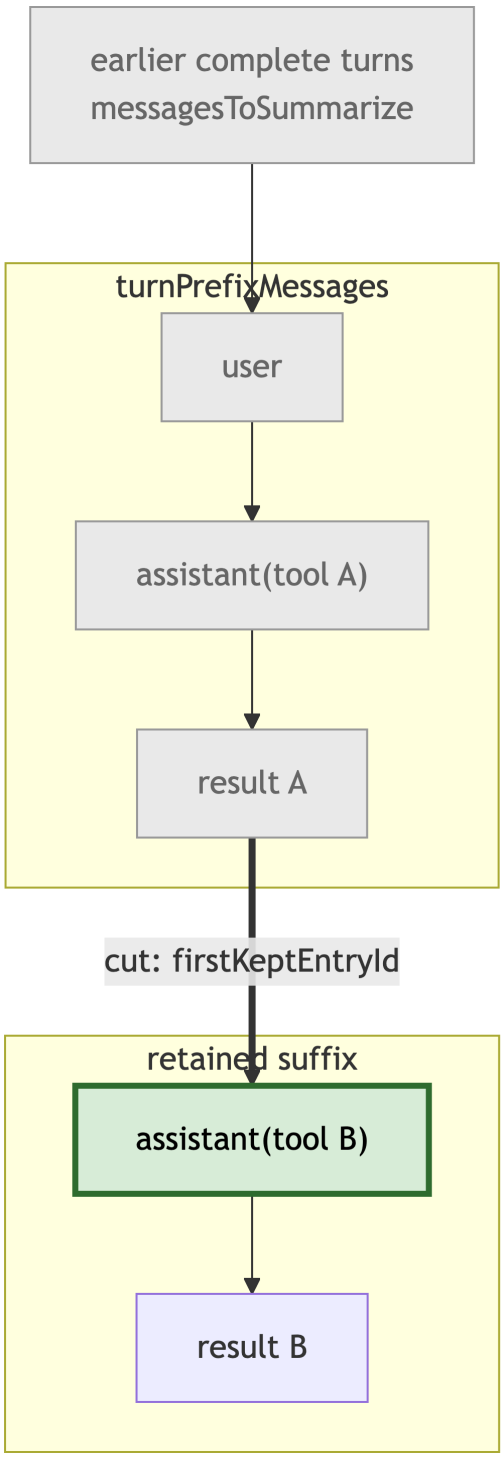

Figure 4.7: A split turn. Older complete turns and the summarized turn prefix (grey) become turnPrefixMessages/messagesToSummarize; the retained suffix (green) starts at an assistant message - firstKeptEntryId - so tool call B stays beside result B.

The heavy edge is the cut: `firstKeptEntryId` names `assistant(tool B)` and `isSplitTurn` is `true`. Pi separately summarizes any older complete turns and the early prefix of this large turn, then merges those summaries. Here the retained suffix begins at `assistant(tool B)`, so tool call B remains beside result B. Extensions must handle both `turnPrefixMessages` and `messagesToSummarize` rather than assuming that compaction removes only whole user turns.

4.7 Session names, labels, and human navigation

`/name` and `pi.setSessionName()` append `session_info`. The selector then shows the stored name rather than deriving a name from the first user message. Labels are per-entry bookmarks, added by `pi.setLabel(entryId, label)`, and appear in tree navigation. They are lightweight metadata: setting a label does not edit a historic message or change model context.

Use labels for checkpoints, handoff points, and experiment names. Use a session name for the overall task. That division makes a large session tree easier to scan without placing extra prose in provider context.

4.8 Session replacement tears down the old runtime

New, resume, and fork flows go through `AgentSessionRuntime`. That object owns the current `AgentSession` and cwd-bound services such as settings, resources, models, and auth. Here, **cwd-bound** means scoped to the session's current working directory. Replacement deliberately tears the old runtime down and builds a new one for the destination session; it does not mutate the previous session object into a new identity.

Extension commands call `ctx.newSession`, `ctx.switchSession`, or `ctx.fork`. Each accepts a `withSession` callback option (declared in `packages/coding-agent/src/core/extensions/types.ts`) that runs once the replacement finishes and receives a fresh replacement context. Captured pre-switch `pi`, command context, or `SessionManager` objects are stale after replacement and must not be reused. Capture plain strings or IDs before the switch, then use only the callback context for session-bound work:

```
// Wrong: sm was captured before the switch and is stale after replacement.
const sm = ctx.sessionManager;
await ctx.switchSession(sessionFile);
sm.getBranch();

// Right: capture plain values, then use the fresh callback context.
const file = sessionFile;
await ctx.switchSession(file, {
  withSession: async (freshCtx) => {
    freshCtx.sessionManager.getBranch();
  },
});
```

Everything so far describes what you see as a user. The rest of the chapter is for extension authors — and it is vocabulary-first for a reason: before you can react to an event like `session_compact`, you need

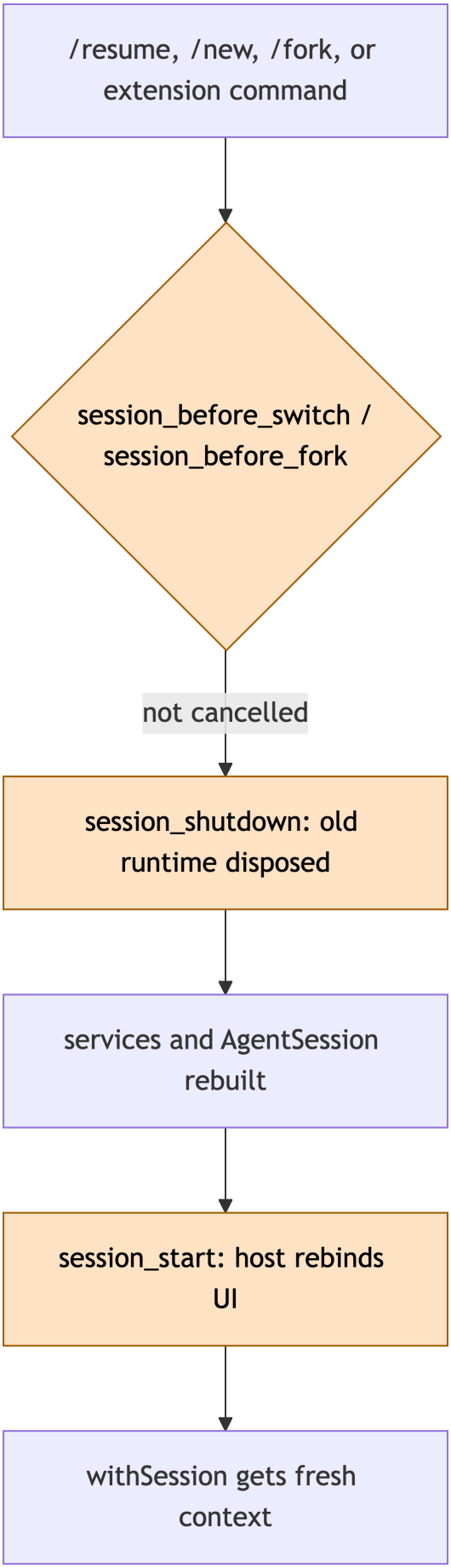

Figure 4.8: Session replacement: hooks (amber) bracket runtime teardown and rebuild.

to know what a compaction entry *is* and how context is rebuilt from the tree. The user-facing tree has a public extension surface. `ctx.sessionManager` exposes the active session; `pi.appendEntry()` stores extension-private state; and `session_*` events let an extension observe or participate in session changes. The sections below explain the corresponding persistence and context mechanics. For implementation patterns, continue to Your first extension and Extension patterns.

4.9 The entry vocabulary

Every non-header entry has a `type`, an ID, a parent ID, and a timestamp. The following table shows why each entry exists and whether it becomes model context through `buildSessionContext()`.

Entry type	Meaning	Becomes LLM context?
`message`	A standard `AgentMessage`: user, assistant, tool result, or supported coding-agent message	Yes, subject to branch/compaction selection
`model_change`	Selected provider/model at that point in history	Used to restore state, not a chat message
`thinking_level_change`	Selected thinking level (how much extended reasoning the model does)	Used to restore state, not a chat message
`compaction`	Summary plus the first entry retained after summarization	Yes, as a compaction-summary message
`branch_summary`	Summary of the path left during tree navigation	Yes, as a branch-summary message
`custom`	Extension-specific durable data	No
`custom_message`	Extension message with display/details	Yes
`label`	Named marker attached to an entry	No
`session_info`	User-visible session name	No

Extension authors should follow one rule from this table: use `custom` for state that Pi or the user needs, and use `custom_message`/`pi.sendMessage()` only when the model should receive it.

4.10 Message shapes at the provider boundary

The `message` entry wraps an `AgentMessage`. Core roles are:

```
user          user text and optional images
assistant     streamed text, thinking, and/or toolCall blocks
toolResult    result for a specific assistant tool call
```

Coding-agent adds message kinds for local interaction and synthesized context, including user bash execution, custom messages, compaction summaries, and branch summaries. `convertToLlm` and the agent loop decide how these richer messages become the provider's normalized message types.

An assistant's content is a list of typed blocks. A tool call is one such block, containing an ID, name, and argument object. The corresponding `toolResult` refers to the call ID. Retaining that link is essential: a model provider expects a tool result to answer a particular tool call, not merely a similarly named tool.

4.11 `SessionManager` as the persistence boundary

The class exposes both static constructors/listing methods (called on the class, such as `SessionManager.create(...)`) and instance methods (called on an open manager, such as `ctx.sessionManager.getBranch()`). Useful API groupings:

Intent	Methods
Start/open (static)	`create`, `open`, `continueRecent`, `inMemory`, `forkFrom`
Discover (static)	`list`, `listAll`
Append transcript (instance)	`appendMessage`, `appendModelChange`, `appendThinkingLevelChange`
Append summaries/state (instance)	`appendCompaction`, `appendCustomEntry`, `appendCustomMessageEntry`, `appendLabelChange`, `appendSessionInfo`
Navigate tree (instance)	`getLeafId`, `getBranch`, `getTree`, `getChildren`, `branch`, `resetLeaf`, `branchWithSummary`
Build context	`buildContextEntries()` on the manager; standalone `buildSessionContext(entries, leafId?)` helper
Inspect metadata (instance)	`getHeader`, `getSessionName`, `getCwd`, `getSessionFile`, `isPersisted`

The manager owns the JSONL representation. Do not have an extension write a second, unrelated copy of the session file: it will not participate in branch or compaction semantics and may become inconsistent with the transcript.

`buildContextEntries()` is available on `ctx.sessionManager`. The richer `buildSessionContext()` helper is standalone: import it and pass the manager's entries and leaf ID, or use the exposed manager methods to assemble the pieces.

4.12 Building context from a selected leaf

`buildContextEntries()` is the intermediate view used by the interactive transcript and session context construction. At a high level:

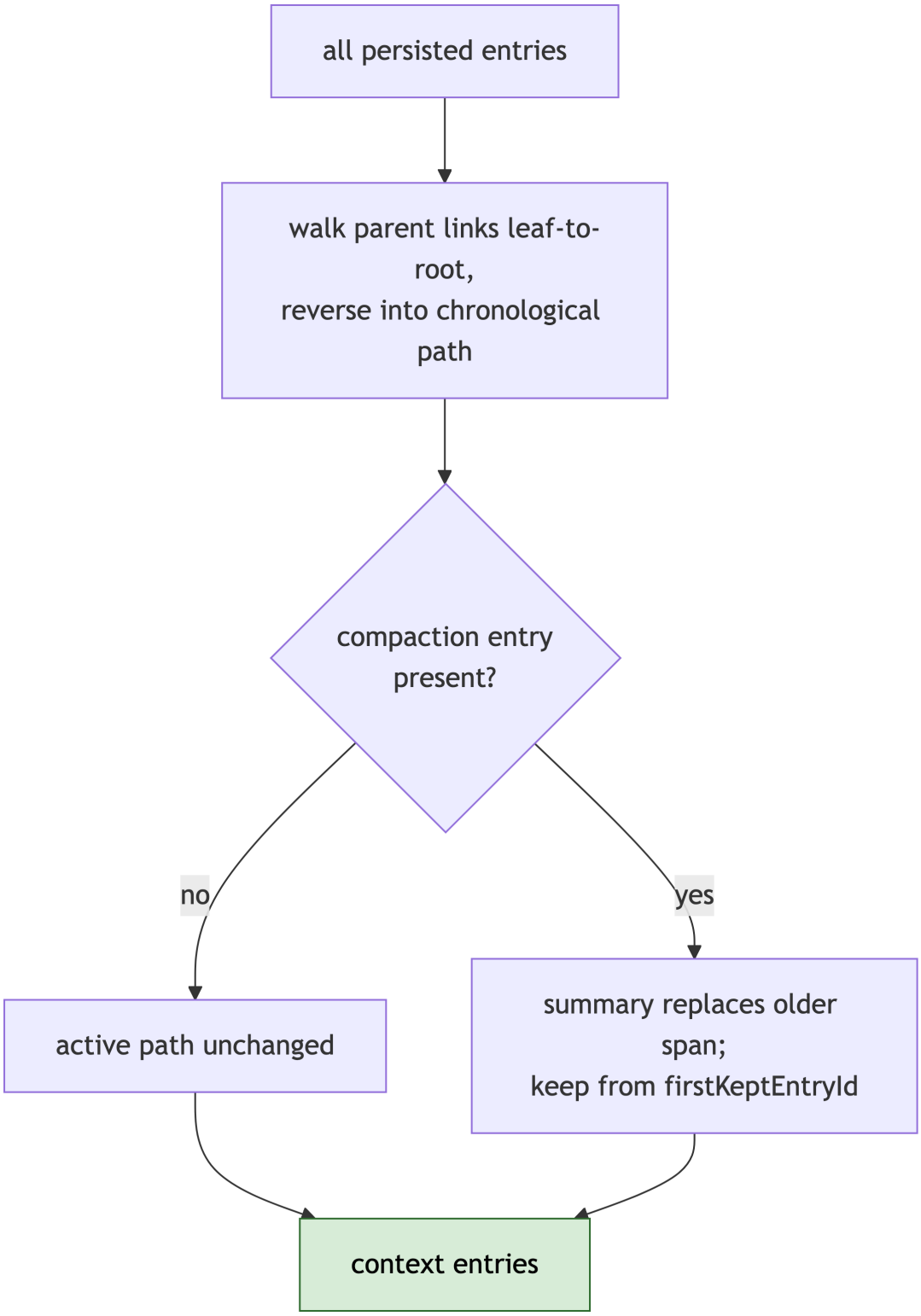

Figure 4.9: Building context from a selected leaf: the compaction boundary substitutes the summary and keeps entries from firstKeptEntryId.

When no compaction entry exists, that active path passes through unchanged and becomes the selected context. After compaction, Pi substitutes the summary for the older span and keeps entries beginning at `firstKeptEntryId`. This field stores the ID of the first retained entry, not an array index: the context builder follows the active parent chain, inserts the compaction summary, then retains that entry and the later entries on the chain. Replaced records remain in JSONL for history but are not sent to the LLM.

`buildSessionContext()` also scans the full path to restore the current model and thinking level, then converts selected entries to messages. It omits `custom` entries, so private extension state added with `pi.appendEntry()` does not consume provider context.

The exported `convertToLlm()` transformer in `packages/coding-agent/src/core/messages.ts` performs the provider-boundary projection. It passes ordinary `user`, `assistant`, and `toolResult` messages through; turns bash-execution, custom, branch-summary, and compaction-summary messages into user messages; and drops bash executions marked `excludeFromContext`. The result is the normalized `Message[]` shape expected by the provider layer (covered in Models, providers, and authentication), while the original session entries remain richer and persisted.

4.13 Extension persistence patterns

4.13.1 Stateful tool results

When state is the consequence of a tool call, return a serializable snapshot in `details` and reconstruct from current-branch tool results in `session_start`. This keeps state branch-aware. The handler lives in an extension file under `~/.pi/agent/extensions/` (or a project `.pi/extensions/`) and runs on every session start:

```
pi.on("session_start", (_event, ctx) => {
  for (const entry of ctx.sessionManager.getBranch()) {
    if (entry.type !== "message" || entry.message.role !== "toolResult") continue;
    if (entry.message.toolName !== "project_notes") continue;
    // Rebuild extension state from entry.message.details after validating it.
  }
});
```

4.13.2 Private durable data

Use `pi.appendEntry("my-extension", data)` when the model does not need the state. That API delegates to the active session's `SessionManager.appendCustomEntry(customType, data)`, which appends a `custom` node as a child of the current leaf and advances the leaf. On the next `session_start`, scan `getEntries()` or the active branch for entries with that `customType`:

```
// In an extension file under ~/.pi/agent/extensions/, loaded at startup.
pi.appendEntry("my-extension", { lastAudit: "2026-07-01", passed: true });

pi.on("session_start", (_event, ctx) => {
  for (const entry of ctx.sessionManager.getEntries()) {
```

```
    if (entry.type !== "custom" || entry.customType !== "my-extension") continue;
    // Rebuild extension state from entry.data after validating it.
  }
});
```

Register an entry renderer (see Terminal UI) if users should see the data in the transcript.

4.13.3 Persistent model context

Use `pi.sendMessage({ customType, content, display })` when the message should be retained *and* given to the model. Register a message renderer if the default transcript presentation is insufficient. Do not encode secrets or verbose debug data in a custom message because it becomes conversation context.

4.14 Session debugging checklist

When a resumed session behaves unexpectedly, answer these in order. The methods below are called on `ctx.sessionManager` inside an extension handler — a small temporary debug extension is enough; see Your first extension for how to write and load one.

1. Which JSONL file is active (`getSessionFile()`)?
2. Which leaf is selected (`getLeafId()`)?
3. Which entries are in `buildContextEntries()`?
4. Is a compaction or branch summary affecting context?
5. Which model/thinking-level entry is last on the full path?
6. Is the extension reading the current branch or every historic entry?
7. Did a replacement flow leave a stale object captured in a closure?

This approach is faster than treating session persistence as an opaque feature: the entire model context is explainable as a deterministic view of the active JSONL tree.

4.15 Try it

1. In any session, run `/name fix-auth-retry` and reopen `/resume`: the selector now shows your name instead of a truncated first message.
2. Add `"compaction": { "reserveTokens": 8192 }` to `.pi/settings.json` in a scratch project and predict, using the trigger formula above, when auto-compaction will fire relative to the global default.
3. Write the debug extension from the checklist above, load it from `~/.pi/agent/extensions/`, and answer checklist steps 1–3 for a session you have forked at least once.

The next chapter separates settings, skills, prompts, themes, and extensions so you can choose the least powerful customization that solves the problem.

5 Customizing Pi

In plain terms: Pi offers settings, reusable instructions, themes, packages, and extensions. Start with the smallest mechanism that fits: configuration for values, prose resources for model guidance, themes for presentation, and code only when behavior must run at a lifecycle boundary (a named point in the prompt lifecycle where extension code can run; see The prompt lifecycle).

This chapter covers the no-code and low-code end of that spectrum. It is grounded in `packages/coding-agent/src/core/settings-manager.ts`, `src/core/resource-loader.ts`, and `src/core/trust-manager.ts`; the user-facing reference is `packages/coding-agent/docs/settings.md`.

5.1 Prerequisites

You need a working Pi install with a signed-in provider (Using Pi day to day; verify with `pi --version`). This chapter reuses the slash commands introduced there—`/settings`, `/trust`, `/reload`—and refers to compaction and tree navigation from Sessions, trees, and compaction. To follow the examples, create a scratch project directory and run `pi` inside it.

5.2 A day-one scenario

You want three things from Pi in one repository: use a cheaper model here, load your team's review skill, and run a lint gate before the agent edits files. Each maps to a different mechanism:

- Cheaper model → a **setting** (`defaultModel` in `.pi/settings.json`).
- Review skill → a **resource** (a `SKILL.md` file Pi discovers).
- Lint gate → an **extension** (code that runs at a lifecycle boundary, covered from Architecture onward).

The rest of this chapter defines these terms precisely and shows where each file lives.

5.3 The four words to keep separate

A **resource** is something Pi discovers at startup, such as the review skill above, or a prompt, theme, or extension. A **setting** is configuration that changes how Pi runs, such as `defaultModel`. **Trust** is the decision that permits project-local code or settings to load. A **scope** is where a value applies, usually global or project-local.

In the scenario, the skill supplies instructions, the lint-gate extension executes code, and `.pi/settings.json` enables both. They may arrive from the same package directory, but they do not have the same power or trust requirements.

5.4 Global and project settings

Pi reads JSON settings from two conventional scopes:

File	Scope	Relative resource paths resolve from
`~/.pi/agent/settings.json`	All projects for the current user	`~/.pi/agent`
`.pi/settings.json`	The current project	`.pi`

The global file holds your personal defaults: they follow you into every project. The project file belongs to the repository and can be committed, so teammates get the same model choice, resources, and extensions on checkout. Because project-local settings and code come from the repository author rather than from you, Pi gates them behind a trust decision; see Trust is an execution policy, not a cosmetic prompt below.

Project values take precedence over global values. For a top-level object-valued setting, `deepMergeSettings()` in `src/core/settings-manager.ts` combines the global and project objects one level deep: project properties replace same-named global properties, while other global properties remain. This is a shallow merge at the setting's top level, not a recursive merge of nested objects.

Arrays have two related but distinct behaviors. In the merged settings value, a project array replaces the global array; omitting the key leaves the global array in that effective settings object. Runtime resource and package resolution then reads raw global and project sources independently and can combine them. Thus an explicit `[]` is not a denylist for globally configured or auto-discovered resources. This distinction matters for `extensions`, `skills`, `prompts`, `themes`, and `packages`; do not infer runtime loading solely from the merged array value.

For example:

```jsonc
// ~/.pi/agent/settings.json (global)
{
  "compaction": {
    "enabled": true,
    "reserveTokens": 16384
  },
  "extensions": [
    "~/.pi/agent/extensions/shared.ts",
    "~/.pi/agent/extensions/audit.ts"
  ],
  "skills": ["~/.pi/agent/skills/review/SKILL.md"],
  "themes": ["~/.pi/agent/themes/dark.json"]
}

// .pi/settings.json (project)
{
  "compaction": { "reserveTokens": 8192 },
  "extensions": ["extensions/project.ts"],
```

```
  "skills": []
}
```

The effective `compaction` is `{ "enabled": true, "reserveTokens": 8192 }`. The effective settings arrays are `extensions: ["extensions/project.ts"]` and `skills: []`; because `themes` is omitted from the project file, the global theme array remains. However, runtime resource resolution still considers raw global and project resource sources, including global explicit paths and auto-discovered resources. Use omission when you want the merged settings value to inherit; use an explicit array to replace that value, not to disable global resource discovery.

> **Warning**
>
> A project `"extensions": []` (or any explicit array) replaces the effective settings array, but it does not disable global explicit or auto-discovered extensions during runtime resource resolution. Do not use an empty project array as a global-resource denylist; change the global sources or use the package/resource filters intended for that scope.

The one-level merge also has a floor: nested objects inside an object-valued setting replace wholesale. Only the setting's top-level properties merge. For example, `retry.provider` is an object inside the `retry` object:

```
// ~/.pi/agent/settings.json (global)
{ "retry": { "maxRetries": 5, "provider": { "maxRetries": 2, "baseDelayMs": 500 } } }

// .pi/settings.json (project)
{ "retry": { "provider": { "maxRetries": 0 } } }
```

The effective `retry` is `{ "maxRetries": 5, "provider": { "maxRetries": 0 } }`: `maxRetries` merges at the top level of `retry`, but the project `provider` object replaces the global one, so `baseDelayMs` is lost. Restate nested fields you still need.

When a merged value surprises you, work from the effective value back to its source in three steps:

1. Run `/settings` in interactive mode to see the current effective values for common settings.
2. Open `.pi/settings.json` and check whether the project scope sets the value.
3. Open `~/.pi/agent/settings.json`: a value absent from the project file may still be the global one winning by omission.

5.5 Settings that alter runtime behavior

Settings are more than UI preferences. The following groups affect runtime semantics and should be considered part of a reproducible project setup.

5.5.1 Model and thinking

`defaultProvider`, `defaultModel`, `defaultThinkingLevel`, and `enabledModels` influence startup selection and model cycling. `thinkingBudgets` tunes token budgets and provider options; `defaultThinkingLevel` selects the thinking level shown in the UI. A saved session model takes precedence during a resume if it is still configured and authenticated; defaults are a fallback.

```
{
  "defaultProvider": "anthropic",
  "defaultModel": "claude-sonnet-4-20250514",
  "defaultThinkingLevel": "high",
  "enabledModels": ["claude-*", "gpt-4o"]
}
```

5.5.2 Compaction and branch summary

`compaction.enabled`, `reserveTokens`, and `keepRecentTokens` determine when and how much conversation is summarized (the mechanics are in Sessions, trees, and compaction). `branchSummary.reserveTokens` and `skipPrompt` affect tree navigation. These settings change what reaches a provider, so they are not merely transcript retention preferences.

```
{
  "compaction": {
    "enabled": true,
    "reserveTokens": 16384,
    "keepRecentTokens": 20000
  },
  "branchSummary": { "skipPrompt": true }
}
```

5.5.3 Retry and delivery

These settings control how long Pi waits after a failure and how queued messages are ordered—both matter when reproducing a session. Pi retries at two levels (see Retry): its own agent-level loop and the provider/SDK's per-request retries, configured separately under `retry` and `retry.provider`. Provider retries default to zero, so a long SDK delay does not silently take control away from Pi's session-level retry logic; the delay and backoff you observe are Pi's own.

```
{
  "retry": {
    "enabled": true,
    "maxRetries": 3,
    "baseDelayMs": 2000,
    "provider": { "maxRetries": 0 }
  }
}
```

`steeringMode` and `followUpMode` determine whether the steering and follow-up queues drain all messages together or one at a time. `transport` selects the wire transport for providers that support more than one (`"sse"`, `"websocket"`, `"websocket-cached"`, or `"auto"`); `httpIdleTimeoutMs` and `websocketConnectTimeoutMs` bound provider connection and idle behavior.

5.5.4 Shell, images, and terminal

`shellPath`, `shellCommandPrefix`, and `npmCommand` affect process execution and package management. Image settings control resize/block behavior before a provider request. Terminal image and rendering settings affect display but do not make an image model-capable; model input capabilities still control whether an image can be sent.

```
{
  "shellPath": "/bin/zsh",
  "shellCommandPrefix": "shopt -s expand_aliases",
  "images": { "autoResize": true }
}
```

5.6 Skills and prompt templates on the input path

`AgentSession.prompt()` gives extension commands first chance to handle slash input. It then emits the extension `input` hook (hooks and extension events are covered in The prompt lifecycle). If input is not handled, Pi expands skill commands and prompt templates. That means a raw `/skill:name` is not necessarily the message sent to the model; it can become the skill's content plus user arguments.

For example, using the review skill authored later in this chapter, you type:

```
/skill:review focus on the retry logic
```

`_expandSkillCommand()` in `src/core/agent-session.ts` replaces the command with the skill file's body wrapped in a `<skill>` block, and appends your typed text. For readability, the example below shortens paths; runtime `filePath` and `baseDir` values are normally absolute. The model receives:

```
<skill name="review" location=".pi/skills/review/SKILL.md">
References are relative to .pi/skills/review.

Review the staged changes (`git diff --cached`).
Report bugs, security issues, and error-handling gaps.
Do not modify files; report findings only.
</skill>

focus on the retry logic
```

`enableSkillCommands` (default `true`) controls interactive registration of skills as `/skill:name` slash commands and autocomplete entries. Setting it to `false` removes that interactive command surface;

it does not unload skills from system-prompt construction, and `AgentSession` still expands a submitted `/skill:name` prompt into skill content. Here and below, `ctx` means the extension context passed to an extension hook, tool, or command. When debugging prompt content, inspect both `ctx.getSystemPromptOptions()` and the actual context event rather than searching only the typed text.

5.7 Context files and system-prompt construction

Context discovery first checks the agent directory (normally `~/.pi/agent`) for one context file. It then walks from the session cwd through every ancestor to the filesystem root and checks each directory. In each location it loads only the first existing candidate in this exact order: `AGENTS.md`, `AGENTS.MD`, `CLAUDE.md`, then `CLAUDE.MD`. Ancestor files are ordered from the filesystem root down to the cwd after the global file. Thus a cwd-level file does not stop discovery in parent directories, but an `AGENTS.md` and `CLAUDE.md` in the same directory are not both loaded. `--no-context-files/-nc` disables this discovery.

Concretely, for a session started in `/home/alice/team-app`:

```
~/.pi/agent/AGENTS.md           <- global file, loaded first if present
/home/alice/AGENTS.md           <- loaded: ancestors are checked root down to cwd
/home/alice/team-app/AGENTS.md  <- loaded: first candidate in this directory
/home/alice/team-app/CLAUDE.md  <- skipped: loses to AGENTS.md in the same directory
```

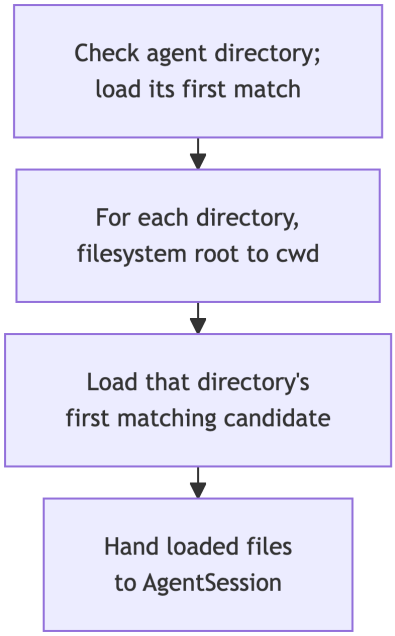

Figure 5.1: Context-file discovery: check the agent directory first, then each ancestor directory from the filesystem root down to the session cwd, taking the first matching candidate per directory.

The loader provides loaded context files and skills to `AgentSession`. `_rebuildSystemPrompt()` is an internal helper; extensions should use `before_agent_start` to inspect or modify the prepared prompt. Internally, `_rebuildSystemPrompt()` creates structured `BuildSystemPromptOptions` from:

- cwd;
- selected tool names, snippets, and guidelines;
- custom/append system-prompt CLI options;
- loaded skills;
- loaded context files.

`buildSystemPrompt()` turns those options into provider-independent system prompt text. The system prompt is rebuilt when the active tool set changes and when session resources reload. For how this string, active tool schemas, and converted messages become a provider request—and how tool results continue the loop—see The agent loop in one picture. An extension that handles `before_agent_start` receives this already-built prompt; to change it, read the prompt you are given, modify or append to it, and return the result. Discarding the built prompt and writing a replacement from scratch throws away the inputs listed above—usually unintentionally.

5.8 Themes

Theme paths load JSON themes. Themes influence colors/formatting used by interactive components and extension renderers; they do not modify keybindings or component logic. An extension can query/change themes through `ctx.ui`, but should use semantic theme keys (`accent`, `warning`, `toolTitle`, and so on) instead of hardcoded ANSI escape codes (raw terminal color sequences) so the user's theme continues to work.

5.9 Resource categories

The resource loader assembles several independently useful kinds of content:

Resource	What it changes	Typical discovery location
Extension	Runtime behavior, tools, commands, UI, providers	`extensions/`, configured paths
Skill	Reusable instruction bundle and optional `/skill:name` command	`skills/`, `.agents/skills`
Prompt template	User-invoked prompt expansion	`prompts/`, configured paths
Theme	Terminal styling data	`themes/`, configured paths
Context file	Instructions added to system-prompt construction	`AGENTS.md` and related loader results
Package	Bundle containing the above	npm/git/local source

These categories are not interchangeable. A theme changes presentation, while skills, prompts, and context files shape model behavior. When instructions are enough, do not replace them with an extension that receives the Pi process's permissions.

5.10 Authoring skills, prompts, and themes

Skills, prompt templates, and themes are plain files you can write yourself; no build step or registration call is required. Drop the file in a discovery directory and Pi picks it up at startup (global: `~/.pi/agent/…`; project: `.pi/…`, loaded only after the project is trusted).

A **skill** is a Markdown file with frontmatter. A directory containing `SKILL.md` is a skill root; direct `.md` files under `~/.pi/agent/skills/` or `.pi/skills/` also load. The `description` field is required; `name` defaults to the parent directory name.

```
---
description: Review staged changes for bugs and security issues
---
Review the staged changes (`git diff --cached`).
Report bugs, security issues, and error-handling gaps.
Do not modify files; report findings only.
```

Save as `.pi/skills/review/SKILL.md` (or use a directory `~/.pi/agent/skills/review/SKILL.md`), restart Pi or run `/reload`, and it becomes available as `/skill:review` and in system-prompt skill listings. A standalone file directly under `skills/` derives its default name from the parent directory (`skills`); to use a file such as `~/.pi/agent/skills/review.md`, add `name: review` to its frontmatter. The startup header lists loaded `AGENTS.md` files, prompt templates, skills, and extensions, so you can confirm discovery there.

A **prompt template** is a Markdown file in `~/.pi/agent/prompts/` or `.pi/prompts/`; the filename becomes the `/name` command, and `$1`, `$@`, or `$ARGUMENTS` placeholders receive typed arguments.

```
---
description: Summarize this week's merged PRs
argument-hint: "[author]"
---
Summarize the PRs merged this week. Filter by author: $1.
List each PR's intent and risk in one line.
```

Save as `.pi/prompts/week.md`, restart Pi or run `/reload`, and invoke as `/week alice`.

A **theme** is a JSON file in `~/.pi/agent/themes/` or `.pi/themes/`. It has a `name`, optional `vars` aliases, and a `colors` object. Every color token in `theme-schema.json` is required, so the practical starting point is a copy of the built-in `dark.json` with values changed:

```
{
  "$schema": "theme-schema.json",
  "name": "team-dark",
  "vars": { "brand": "#00aaff" },
  "colors": {
    "accent": "brand",
    "border": "brand"
  }
}
```

The example is truncated for space; a real file must define all required `colors` tokens or validation rejects it. Restart Pi or run `/reload` so the loader discovers the file, then select it with `"theme": "team-dark"` or via `/settings`.

5.11 Trust is an execution policy, not a cosmetic prompt

This section covers what happens behind the trust decision: where it is stored, how noninteractive modes decide without a dialog, and how extensions participate. Repository-owned settings can select extensions and packages; skills can direct the model to run commands. Pi therefore asks for trust before it loads project configuration, installs project packages, or executes project extensions.

At interactive startup, Pi looks for trust-requiring inputs and for a saved decision covering the current folder or one of its parents. If neither settles the question, Pi can prompt. In the TUI, this appears as an interactive trust dialog asking whether the current project should be trusted. Approval enables `.pi/settings.json`, local resources under `.pi`, missing package installation, `.agents/skills`, and extension code from those sources. Context instructions such as `AGENTS.md` and `CLAUDE.md` are the exception: they load regardless of trust unless context loading is disabled.

Pi's noninteractive modes are:

- **Print mode** (`-p` or `--print`) processes the initial prompt and exits.
- **JSON mode** (`--mode json`) writes session events as JSON lines, with one complete JSON object per line.
- **RPC mode** (`--mode rpc`) accepts commands and returns results over standard input and output for programmatic clients.

None can display the startup trust prompt. Without an applicable saved decision, each follows the global `defaultProjectTrust` setting:

Value	Noninteractive behavior without saved decision
`"ask"`	Ignore project resources because there is no UI prompt
`"never"`	Ignore project resources
`"always"`	Trust project resources

The CLI flags `--approve`/`-a` and `--no-approve`/`-na` override the decision for one run only. In CI, prefer `--approve` for a known checkout or configure a specific noninteractive policy rather than relying on an interactive prompt.

> **Important:** `/trust` immediately writes the selected decision to `~/.pi/agent/trust.json`. The command changes persisted state for future Pi processes; it does not change the trust decision already resolved by the current process. A live `/reload` also keeps that already-resolved trust state, so a newly saved `/trust` answer does not start loading (or stop ignoring) project resources until Pi itself is restarted. A same-cwd session replacement such as `/new` likewise reuses the process-level decision rather than rereading `trust.json`.

A saved decision is a map from normalized absolute folder paths to a boolean (`true` trusted, `false` refused). The nearest entry for the current folder or one of its parents applies:

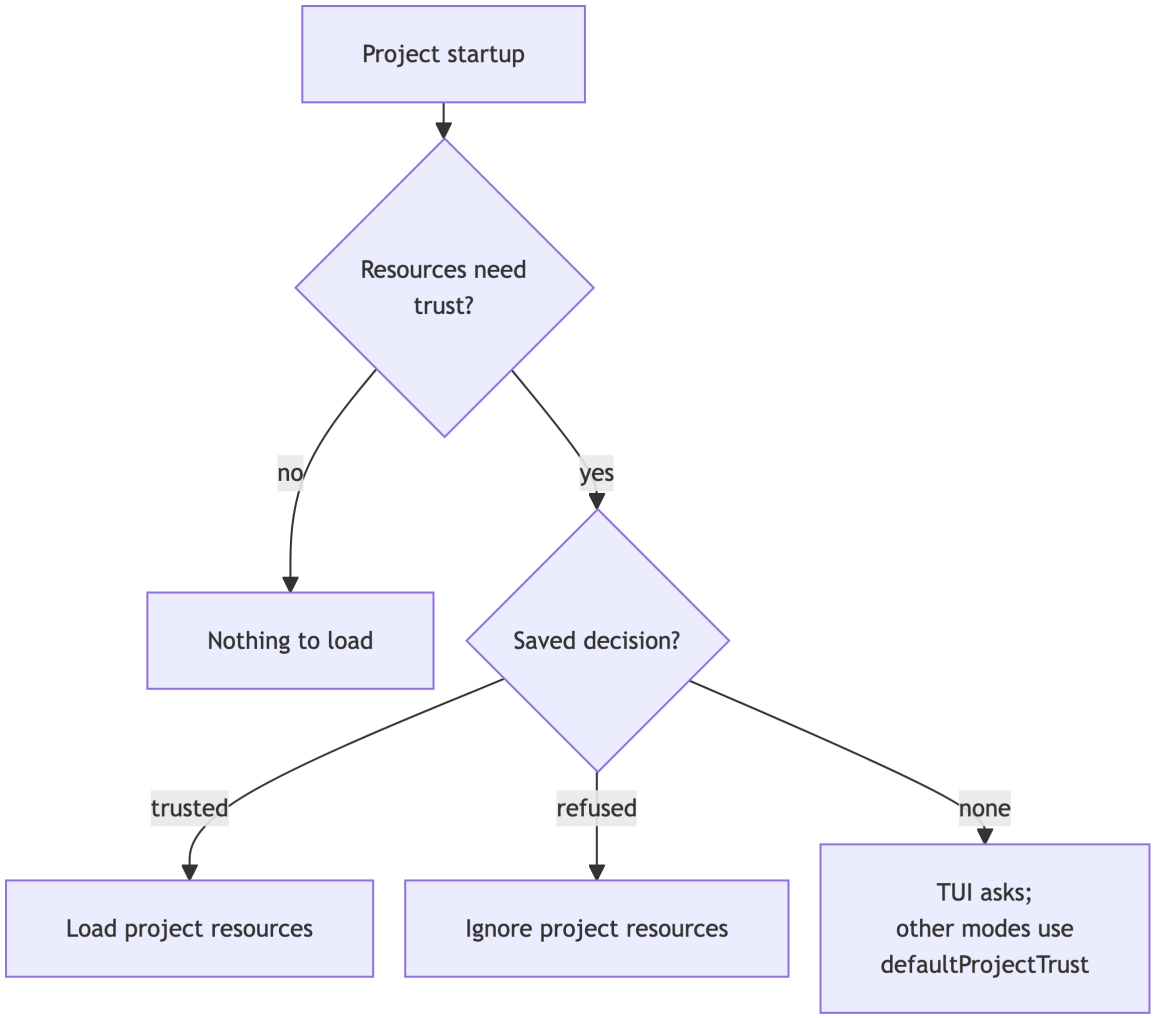

Figure 5.2: How Pi resolves project trust: a saved decision wins; otherwise interactive mode asks and noninteractive modes fall back to `defaultProjectTrust`.

```
{
  "/Users/alice/work/team-app": true,
  "/Users/alice/tmp/scratch-project": false
}
```

Extensions can also participate in the decision, which lets an organization enforce its own policy (for example, "trust only repositories under `~/work`"). Before loading repository extensions, Pi emits the `project_trust` event—but only to user/global and CLI-provided extensions, because project extensions are the very thing being approved. A handler returns `{ trusted: "yes" | "no" | "undecided" }`; the first yes/no response decides the outcome. Code awaiting approval therefore cannot approve itself.

5.12 Project configuration that an extension should respect

The extension context, conventionally named `ctx`, is the object Pi passes to extension hooks, tools, and commands. It provides the current `cwd`, trust status, session manager, model services, UI/mode information, and run-state helpers. Define and use that context before resolving project configuration.

An extension with its own project-level configuration should:

1. Use `CONFIG_DIR_NAME` from coding-agent rather than hardcoding `.pi`.
2. Resolve the path relative to `ctx.cwd`.
3. Call `ctx.isProjectTrusted()` before reading that configuration.
4. Validate parsed JSON as untrusted data.
5. Define which state is global, project-local, and session-local.

A minimal global extension following steps 1–4:

```
// ~/.pi/agent/extensions/team-policy.ts
import { existsSync, readFileSync } from "node:fs";
import { join } from "node:path";
import { CONFIG_DIR_NAME, type ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function (pi: ExtensionAPI) {
    pi.on("session_start", async (_event, ctx) => {
        // Step 3: never read project configuration without trust
        if (!ctx.isProjectTrusted()) return;
        // Steps 1-2: resolve the config directory name against the session cwd
        const configPath = join(ctx.cwd, CONFIG_DIR_NAME, "team-policy.json");
        if (!existsSync(configPath)) return;
        // Step 4: parsed JSON is untrusted data; validate before using it
        const config: unknown = JSON.parse(readFileSync(configPath, "utf-8"));
        // ... validate config, then apply it
    });
}
```

Writing and loading extensions like this is covered end to end in Your first extension.

For example, a global extension may store user preferences in the agent directory but read a project allowlist only after trust. That matches Pi's own security model and avoids creating an unreviewed bypass.

5.13 Pre-commit security and reproducibility checklist

Use this checklist to review whether a repository configuration has the intended security boundary and will behave consistently for teammates and CI:

1. Does this setting execute code or fetch/install dependencies?
2. Does the team intend to trust the repository for that behavior?
3. Are environment references used for secrets instead of literal credentials?
4. Are relative paths intentional for this scope?
5. Does a nested override preserve the global fields you expect?
6. Will noninteractive CI receive the required trust decision?
7. Can the change be represented as a skill/theme/template instead of code?

These questions make configuration review an architectural step rather than an afterthought.

5.14 Which mechanism do you need?

Use the smallest mechanism that owns the behavior:

Need	Prefer
Persist an instruction for an AI model or reusable task procedure	`AGENTS.md`, a skill, or a prompt template
Change colors and visual style	Theme
Change AI model/provider selection or static behavior	Settings or a custom provider
Add a command, tool, lifecycle gate, state, custom UI, or request hook	Extension
Reuse Pi from another Node program	Coding-agent SDK (`createAgentSession`; see Run modes, SDK embedding, and operational boundaries)
Change generic loop/provider/TUI behavior for every consumer	Core contribution in the owning package

When none of the first four mechanisms suffice, you are writing an extension. The next two chapters show the runtime it lives in and the lifecycle points where it can act.

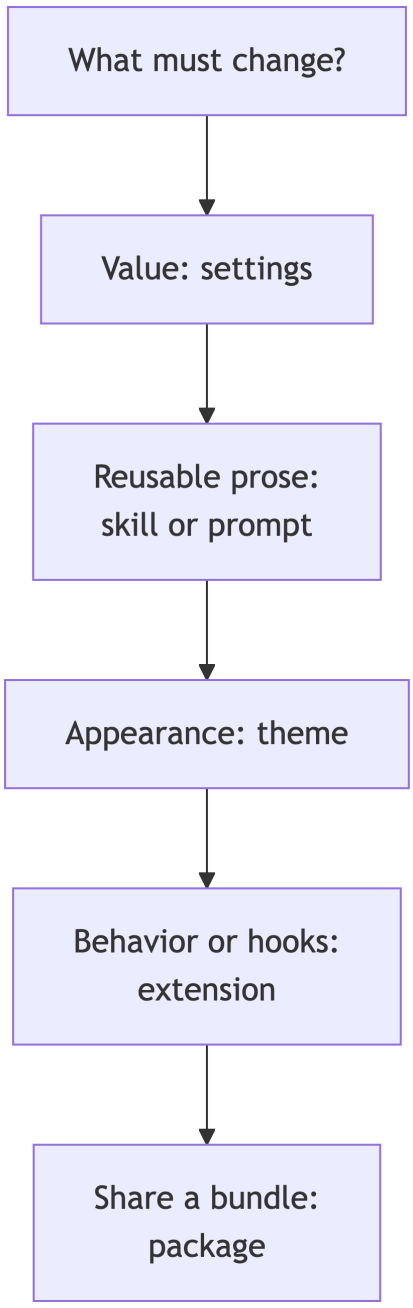

Figure 5.3: Choose the smallest Pi customization mechanism that owns the change.

5.15 Try it

Close the day-one scenario from the top of this chapter:

1. In a scratch project, create `.pi/settings.json` containing `{ "defaultModel": "claude-sonnet-4-2025051` `}` (or another configured model ID), restart `pi`, approve the trust prompt, and confirm the current model in the footer changed.
2. Write the review skill from the authoring section above to `.pi/skills/review/SKILL.md`, run `/reload`, and confirm it appears in the startup header and as a `/skill:review` autocomplete entry. Invoke it with an argument and predict the expanded message the model receives.
3. With at least one extension in `~/.pi/agent/extensions/`, add `"extensions": []` to the project `.pi/settings.json` and predict whether the global extension still loads. The empty project array replaces the effective setting but does not disable global explicit or auto-discovered resources at runtime. Restart `pi` and check the startup header; then remove the key and compare.

Part III

How Pi works

6 Architecture: packages, ownership, and entry points

An extension runs inside the coding-agent package. This chapter shows what that package owns, what the lower layers provide, and why an extension can use public APIs instead of forking Pi. If needed, start with the TypeScript route; terminology is available in the glossary.

6.1 Prerequisites

Read Using Pi day to day and Customizing Pi first: you should have run `pi`, used a slash command, and know what a setting, skill, prompt template, and theme are.

This chapter constantly references files in Pi's source tree, so clone the monorepo (a single repository containing several separately published packages) and open it in an editor to follow along:

```
git clone https://github.com/earendil-works/pi.git
```

Every path in this chapter is relative to that repository root.

A useful rule is: move a feature down only when it is genuinely useful outside the coding agent. The coding-agent package owns command-line behavior, local filesystem tools, project resources, session UX, and the extension API; the `agent` package should not learn those policies.

6.2 Pi's minimal defaults

Pi is deliberately a minimal coding harness (the small fixed core the model runs inside, with everything else bolted on from outside): the design goal is the smallest core that still makes a viable agent, with everything else arriving through extensions, skills, prompts, and themes (see Customizing Pi for those four resource kinds). Understanding that baseline explains both the package map and why the extension API is so prominent.

> **Skippable on first read**
>
> This section explains Pi's design philosophy and system-prompt internals. If you would rather see the package map first, jump ahead to How to read this chapter and come back later.

The default tool set is four tools. A fresh session activates only `read`, `bash`, `edit`, and `write` (`defaultActiveToolNames` in `core/sdk.ts`). Three more definitions — `grep`, `find`, and `ls` — ship but stay inactive until selected with `--tools`, an extension's `pi.setActiveTools()`, or a setting. The

reasoning: a model with `bash` can already run `rg`, `find`, and `ls` itself, so Pi does not spend context-window budget on dedicated tools for what the shell does. Built-in tools covers the full registry and selection mechanics.

The base system prompt is intentionally compact. `buildSystemPrompt()` in `core/system-prompt.ts` assembles a fixed base, then appends per-session context on top:

The base opens with two role sentences: "You are an expert coding assistant operating inside pi, a coding agent harness" and "You help users by reading files, executing commands, editing code, and writing new files." It then has three short sections:

- `Available tools:` — built from the active tools' prompt snippets, so changing the active set changes the prompt.
- `Guidelines:` — encodes the minimalism directly. When `bash` is active but `grep`/`find`/`ls` are not, the prompt says "Use bash for file operations like ls, rg, find". The only unconditional rules are "Be concise in your responses" and "Show file paths clearly when working with files".
- A pointer block to Pi's own README/docs/examples, read only when the user asks about Pi itself.

There is no date line and no other behavioral boilerplate. The appended parts are `--append-system-prompt` text, `AGENTS.md` context files wrapped in a `<project_context>` tag (an XML-like marker that keeps file contents visibly separate from Pi's own instructions), skills, and a final cwd line; Customizing Pi covers those inputs. A `--system-prompt` replacement swaps the base out entirely while keeping the appended parts. For the complete model-facing assembly—why tool schemas, tool guidance, and tool results are separate—and the rule that continues or ends a run, read The agent loop in one picture.

What Pi deliberately does not build in (see the philosophy section of `packages/coding-agent/README.md`):

Not built in	Use instead
MCP integration	An extension, or CLI tools plus their READMEs
Sub-agents	`examples/extensions/subagent/`
Plan mode	`examples/extensions/plan-mode/`
Built-in to-do lists	A `TODO.md` file (`examples/extensions/todo.ts` as code)
Permission popups	`examples/extensions/permission-gate.ts`
Background shell	tmux

Each is a considered "no" — the core stays small, and the extension surface carries the rest. The example extensions are the pattern in action: reference implementations you can adopt, not built-in behavior you must disable.

6.3 How to read this chapter

Start with the package map, then follow the startup diagram, and only then read the package walk-through. A **package** is a deployable unit with its own public exports. A **dependency** is a package another package imports. An **entry point** is the file where a feature or application begins. A **boundary** is a place where one representation becomes another, such as a provider (an external model API such as Anthropic or OpenAI) message becoming an agent event.

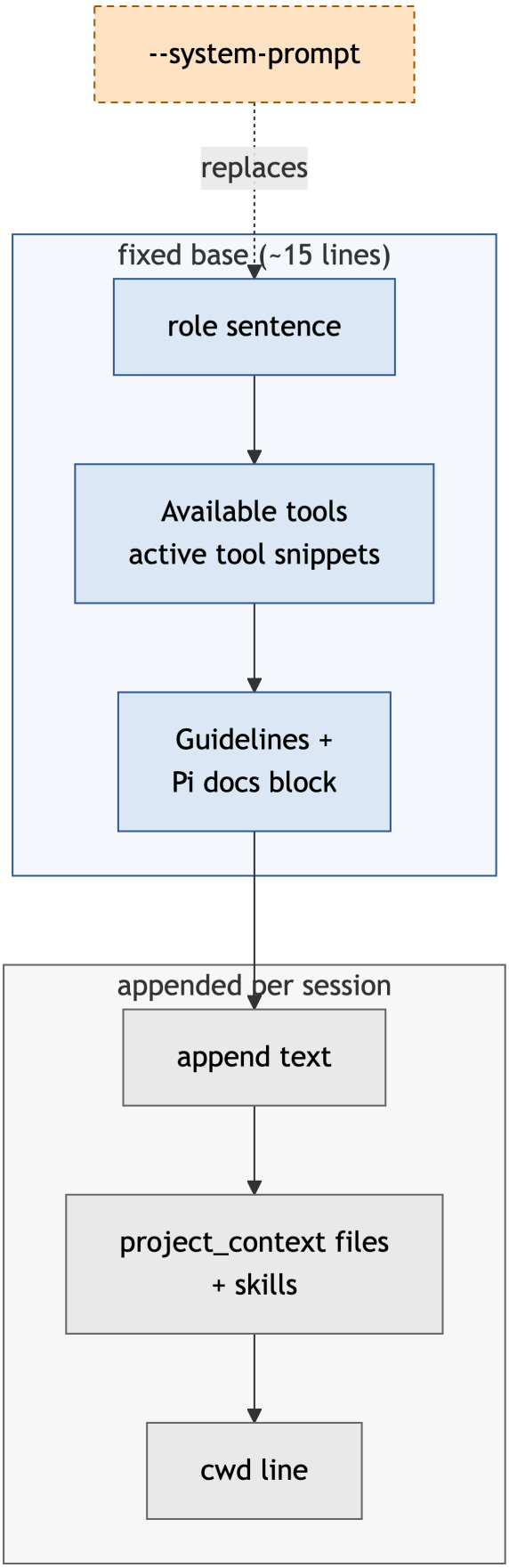

Figure 6.1: Anatomy of the default system prompt: a short fixed base (blue), then per-session additions (grey). –system-prompt swaps the base; –append-system-prompt only adds.

Five direct packages under `packages/*` provide reusable terminal, loop, and provider layers; nested storage and example workspaces are separate. The layers can be reused and tested without the others: `tui` knows nothing about models, `agent` knows nothing about the CLI, and `ai` knows nothing about sessions. As an extension author you mainly need three of them: `coding-agent` is your home, `agent` is the loop your prompts run through, and `ai` provides provider contracts. `tui` and the experimental `server` package can wait.

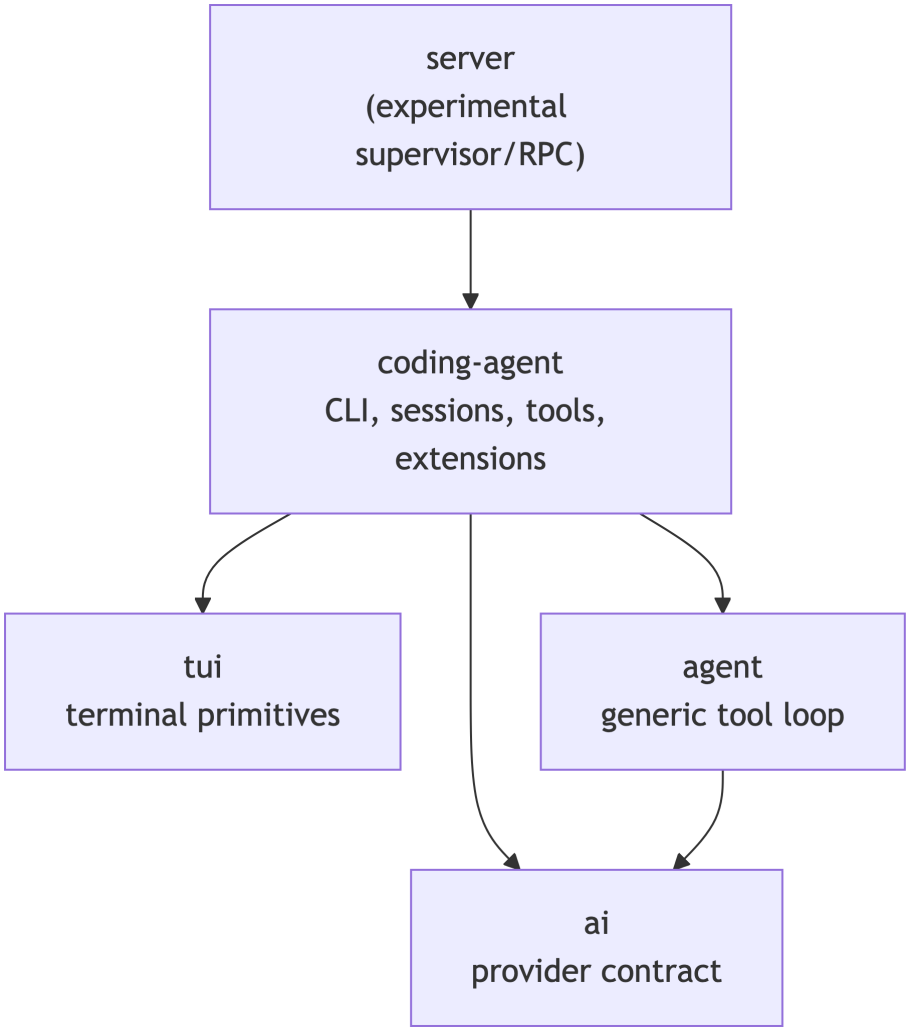

Figure 6.2: The package map. An arrow points from a package to the package it imports; lower layers never import higher ones.

Each arrow points from the package that imports and depends on another package to that dependency; the map shows the normal architectural direction, not every development-tool dependency. `coding-agent` integrates all lower layers and depends directly on each of them; `agent` also depends on the AI message/model contracts. `tui` has no dependencies on other packages in this monorepo (a single repository containing several separately published packages), allowing it to be reused by other terminal applications.

For a beginner-readable boundary example, follow a `/hello-pi` slash command from keystroke to provider request. The command is hypothetical — Pi does not ship a `hello-pi` command — but every step names the real file that handles it. The closest shipped example is

`packages/coding-agent/examples/extensions/commands.ts`, which registers a `/commands` command; run any example extension from the repository root with the repeatable `-e` flag, for example:

```
pi -e packages/coding-agent/examples/extensions/commands.ts
```

1. **Input.** You type `/hello-pi world` in the interactive mode (`modes/interactive/interactive-mode.ts` in coding-agent), which passes the text to `AgentSession.prompt()`.
2. **Command dispatch.** `AgentSession.prompt()` (`packages/coding-agent/src/core/agent-session.ts`) sees the leading `/` and calls `_tryExecuteExtensionCommand()`, which asks the `ExtensionRunner` for a command named `hello-pi`. Extensions register slash commands with `pi.registerCommand()` (see `packages/coding-agent/examples/extensions/commands.ts`); the command's `handler(args, ctx)` runs with a session context.
3. **Session API.** The handler calls session APIs — for example `pi.sendUserMessage("Say hello to world")` (see `packages/coding-agent/examples/extensions/send-user-message.ts`), which re-enters `AgentSession.prompt()` and starts a model turn.
4. **Agent loop.** The turn runs in `packages/agent/src/agent-loop.ts`: agent messages are converted to LLM messages via `convertToLlm`, a provider `Context` is built, and the loop calls its configured stream function. Generic `Agent` uses an explicit `streamFn` or the process-wide default; coding-agent wires `ModelRuntime.streamSimple()`.
5. **Provider dispatch.** `core/sdk.ts` calls `ModelRuntime.streamSimple()`, which prepares auth and headers and streams through the composed provider selected for the model's API.
6. **Provider request.** The resolved module — for example `packages/ai/src/api/anthropic-messages.ts` — serializes the `Context` into that provider's JSON, sends the HTTP request, and translates the response stream back into provider-neutral Pi events.

The same six steps apply to any extension command that triggers a model turn; only the handler body changes. The lifecycle chapter gives the full 10-stage version of this path, including hooks, tool execution, and rendering. The direction tells you where a fix belongs: a terminal component does not belong in `packages/agent`, and provider JSON does not belong in the TUI.

6.4 Repository-level conventions

The root `package.json` defines npm workspaces (npm's mechanism for linking the packages of one repository together). Packages are ESM, and public package names map back to source during development. An extension should import published package names for public types and APIs; use relative imports only inside the same package. This preserves the boundary installed users see.

Concretely, an extension imports from the package root:

```
// Correct: the published package name, resolved to its public exports.
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

// Wrong: a relative path into another package's internals. It breaks the
// moment that file moves, and it compiles only inside the monorepo.
import type { ExtensionAPI } from "../../coding-agent/src/core/extensions/types.ts";
```

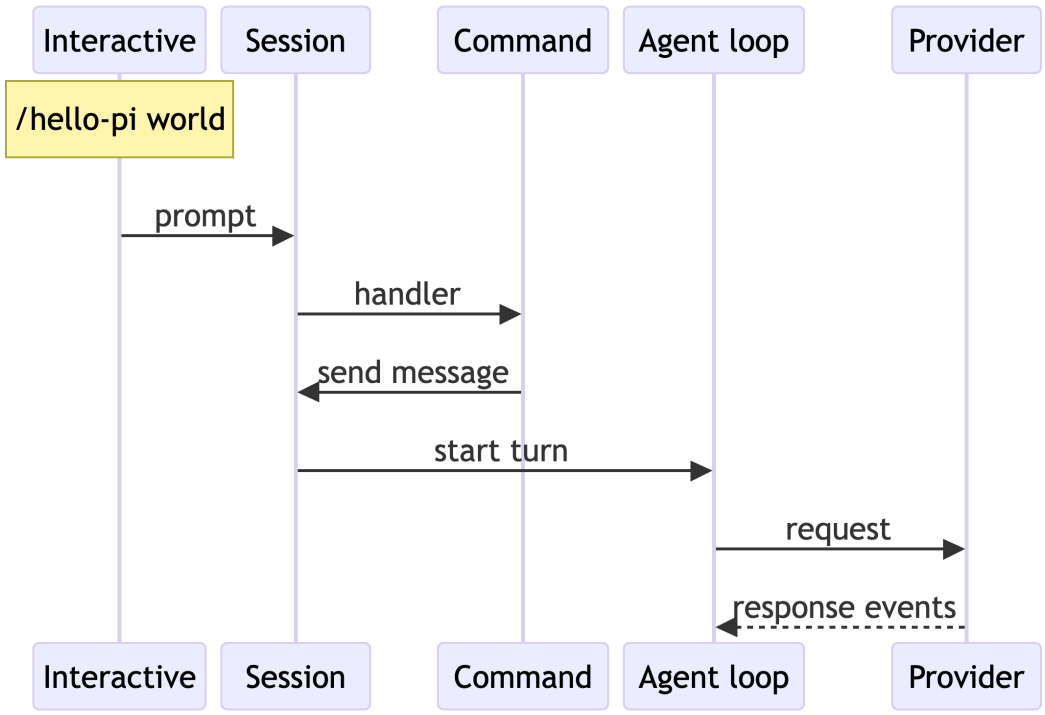

Figure 6.3: The /hello-pi trace as a message sequence, one lifeline per package. Messages 1–3 show the subtle part: the command handler re-enters AgentSession.prompt(), which is how a slash command starts a model turn. Only the last hop leaves the process.

6.5 Component walkthrough

Five packages, five reasons to edit. The table is the one-paragraph version; the subsections below add detail per package.

Package	You work here when	Key entry file
`packages/tui`	A terminal primitive any TUI could reuse	`packages/tui/src/tui.ts`
`packages/ai`	Provider protocol JSON or streaming	`packages/ai/src/index.ts`
`packages/agent`	A loop rule for every consumer	`packages/agent/src/agent-loop.ts`
`packages/coding-agent`	Pi application behavior	`packages/coding-agent/src/main.ts`
`packages/server`	Supervisor/RPC process management (experimental)	`packages/server/src/supervisor.ts`

6.5.1 `packages/tui`: terminal rendering primitives

You work here when a change is a general terminal interaction primitive — layout, focus, key parsing, editor components — that any terminal app could reuse. For a Pi-specific chat row, command, or tool renderer, start in coding-agent instead.

`packages/tui/src/index.ts` is the public barrel (a file that re-exports symbols from other modules). It exports the `TUI` class, the `Component` interface, containers, text/editor/list components, terminal abstractions, key parsing, keybindings, autocomplete, and ANSI-aware (terminal escape-code-aware) text utilities.

`packages/tui/src/tui.ts` owns the component tree, focus, layout, overlays, input routing, and differential rendering (repainting only the terminal cells that changed since the last frame, instead of redrawing the whole screen). `src/terminal.ts` provides the process-terminal boundary. The components under `src/components/` render specific UI pieces; `src/keybindings.ts`, `src/keys.ts`, and the editor components handle keyboard semantics. This package does not know what a model, tool call, session, or extension is.

6.5.2 `packages/ai`: a provider-neutral streaming contract

You work here when a change is protocol-specific JSON, streaming parsing, or a provider quirk. UI decisions and built-in filesystem tools do not belong in this package.

The small public entry point `packages/ai/src/index.ts` exports core types, models, authentication utilities, event-stream helpers, TypeBox (the runtime schema library Pi uses for tool parameter validation; see TypeScript basics), and selected API option types. It intentionally avoids eagerly importing provider catalogs and provider factories. Provider-specific modules live under `src/providers/` and streaming implementations under `src/api/`.

The core concepts are defined in `src/types.ts`:

- A `Model` carries provider/API metadata and capabilities.
- A `Context` combines the system prompt, normalized messages, and tool definitions.
- An `AssistantMessageEventStream` yields provider-independent start, text, thinking, tool-call, completion, and error events.

A `Context` is exactly three fields (`src/types.ts`):

```
interface Context {
  systemPrompt?: string;  // the assembled system prompt
  messages: Message[];    // normalized user/assistant/toolResult messages
  tools?: Tool[];         // tool definitions with TypeBox parameter schemas
}
```

In plain terms, an `AssistantMessageEventStream` is one async stream of events — `start`, `text_delta` chunks, `thinking_delta` chunks, `toolcall_end`, then `done` or `error` — with the same shape no matter which provider produced the response. Callers never parse provider JSON; the provider module under `src/api/` already translated it.

`src/compat.ts` supplies the compat layer, a legacy/default compatibility `streamSimple()` dispatch for callers that do not provide a stream function. The coding-agent request path instead uses `core/sdk.ts` → `ModelRuntime.streamSimple()` → the composed provider. The runtime composes built-in catalogs, configuration, extension registrations, and auth before the provider adapter handles the request. Individual files such as `src/api/openai-responses.ts` and `src/api/anthropic-messages.ts` serialize the normalized context into a provider request and translate that provider's stream back into Pi events.

6.5.3 `packages/agent`: generic conversation and tool loop

You work here when a change is a loop rule that must hold for every consumer of the agent runtime — event ordering, tool scheduling, queue semantics — independent of Pi's CLI or sessions.

`packages/agent/src/types.ts` declares the generic agent contract: agent messages, tools, events, loop configuration, hooks, queue modes, and tool execution mode. `src/agent.ts` wraps that loop in the stateful `Agent` class. It owns current model, system prompt, tools, messages, active runs, abort controllers, and the steering and follow-up queues (messages held back until the current run reaches the right point to receive them).

`src/agent-loop.ts` is the important execution core. It:

1. Emits agent/turn/message events.
2. Lets a caller transform agent messages, then converts them to LLM messages.
3. Builds a provider `Context` and consumes an async response stream.
4. Validates model-produced tool arguments.
5. Runs tools sequentially or with sequential preflight — checking each tool call through `beforeToolCall` hooks one at a time before any of them run — plus parallel execution of the allowed calls.
6. Appends tool results and repeats until there are no tool calls or queued messages.

For the corresponding coding-agent assembly path and the distinction between a low-level run ending and a session settling, see The agent loop in one picture.

The generic loop deliberately receives callbacks such as `convertToLlm`, `transformContext`, `beforeToolCall`, and `afterToolCall` instead of implementing those decisions itself. The reason is reuse and testability: because every Pi-specific decision arrives as a callback, the same loop runs under unit tests, the SDK, and other harnesses without importing a single coding-agent file. For example, coding-agent passes a `convertToLlm` that translates session message types into LLM messages, dropping durable-only entry data the provider should never see — the persistence half of that split is covered in Persistence and context are different things below.

6.5.4 `packages/coding-agent`: the Pi application

You work here when a change is Pi application behavior: a command, session policy, built-in tool, setting, or the extension API itself.

This is where the generic pieces become the `pi` command. Its package manifest defines the `pi` binary as `dist/cli.js`. In source, `src/cli.ts` sets process-level behavior and calls `main(process.argv.slice(2))` from `src/main.ts`.

`src/main.ts` parses command-line arguments, determines the run mode — interactive, RPC (remote procedure call: LF-delimited JSON over stdin/stdout, so another process can drive Pi without a terminal), or print (one prompt in, one response out) — resolves trust (the per-folder decision that permits project-local code and settings to load) and settings, creates the runtime, selects the model, and starts `InteractiveMode`, RPC mode, or print mode. It is the composition root — the one place that wires all the subsystems together — plus application policy, not the agent loop.

The major coding-agent subsystems are listed below. Every path in the **Important files** column is relative to the coding-agent source directory: `packages/coding-agent/src/`. For example, `core/sdk.ts` means the file at that relative path.

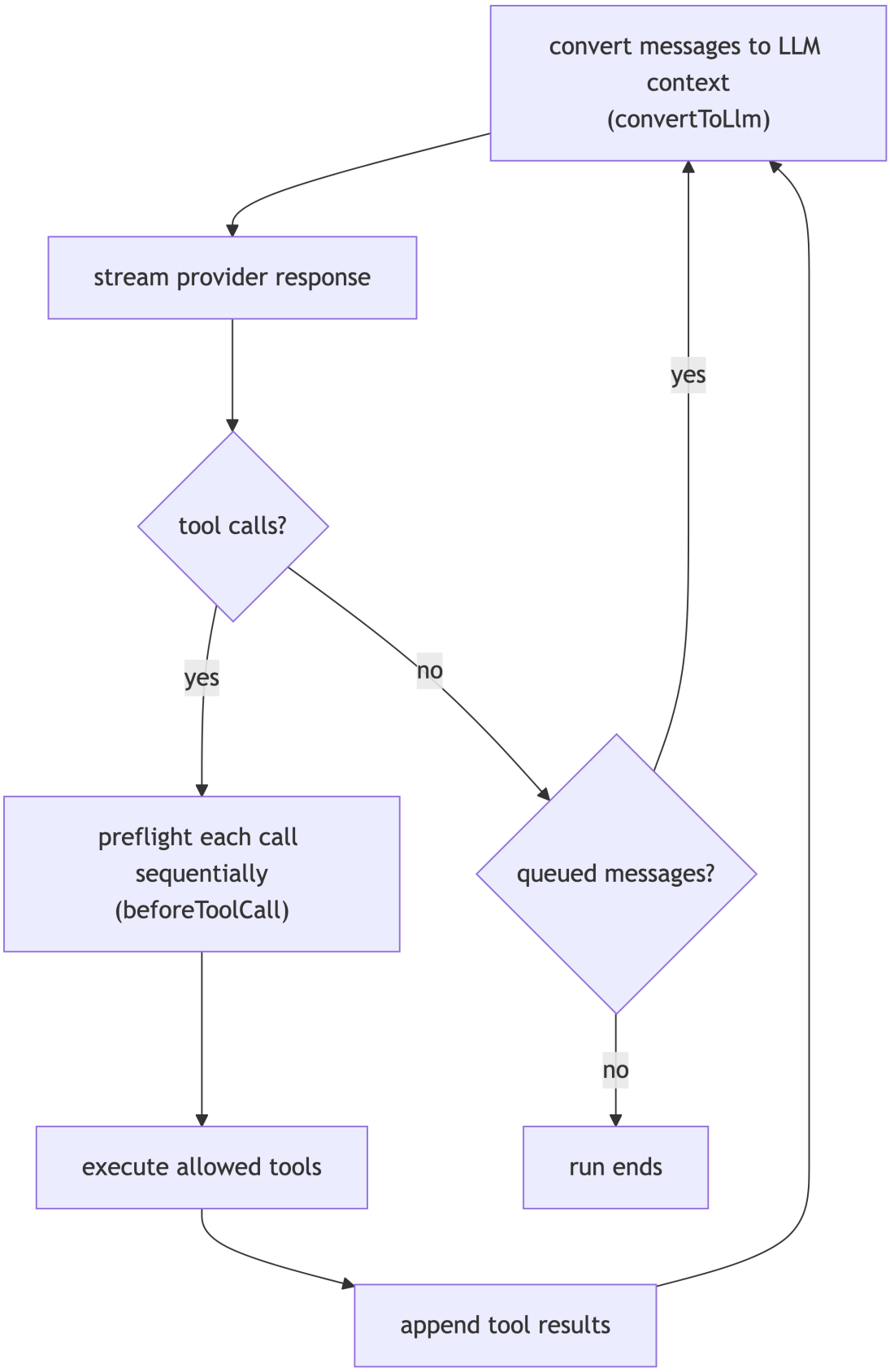

Figure 6.4: Agent loop: tool calls are preflighted, executed, and appended before the loop repeats.

Subsystem	Responsibility	Important files
Session composition	Create settings, auth/model runtime, resources, agent session	`core/sdk.ts`, `core/agent-session-services.ts`
Runtime replacement	Own a session plus cwd-bound services across new/resume/fork	`core/agent-session-runtime.ts`
Session policy	Prompt preprocessing, resource/system prompt assembly, retries, compaction, extension event bridge	`core/agent-session.ts`
Durable transcript	JSONL session entries, tree branches, context reconstruction	`core/session-manager.ts`, `core/messages.ts`
Resources	Load extensions, skills, prompts, themes, and context files	`core/resource-loader.ts`, `core/extensions/loader.ts`
Configuration	Global/project settings and defaults	`core/settings-manager.ts`, `core/defaults.ts`
Model/auth resolution	Canonical model catalogs, composed providers, credential lookup	`core/model-runtime.ts`, `core/model-registry.ts` (extension-facing facade), `core/auth-storage.ts`
Built-in tools	Read, bash, edit, write, search, file mutation coordination	`core/tools/`, `utils/tools-manager.ts`
Interactive app	Chat transcript, editor, dialogs, commands, renderer adapters (translation layers between session data and TUI components)	`modes/interactive/interactive-mode.ts`, `modes/interactive/components/`
Extensions	Types, discovery/loading, event dispatch, context wrappers	`core/extensions/types.ts`, `core/extensions/loader.ts`, `core/extensions/runner.ts`, `core/extensions/wrapper.ts`

The table is an orientation map, not an exhaustive API list. On a first read, look at the first column and the row matching your task; skip the file paths until you need them. To navigate it, start with the subsystem matching the behavior you are changing, then follow its named entry file before reading neighboring implementation files. Public coding-agent exports come from `src/index.ts`; extension authors should import from the package root rather than internal paths unless an API is explicitly documented for that purpose.

6.5.5 `packages/server`: experimental process orchestration

You work here when a change concerns supervisor or RPC process management, not normal prompt processing. This package is experimental and contains the server-side process-management surface.

It is outside the normal extension path. Treat it as a separate surface for supervisor/RPC orchestration, not as a hidden prompt stage.

6.6 Constructing a running coding-agent session

The startup path is intentionally split so the runtime can be rebuilt when a session changes its working directory:

`createAgentSessionServices()` reloads the resource loader and applies provider registrations that extensions made during their factories (an extension's default-exported setup function; Your first extension covers factories). `createAgentSession()` then resolves/restores a model and thinking level, creates an `Agent`, and wraps it in `AgentSession`. The `AgentSessionRuntime` class tears down the old extension/session runtime before applying a newly created one for operations such as session resume or fork.

The same chain as code, reduced to the calls `main.ts` actually makes (`src/main.ts`, with session construction in `core/sdk.ts`):

```
// 1. Build the cwd-bound services: settings, auth/model runtime,
//    resource loader (which loads extensions, skills, prompts, themes).
const services = await createAgentSessionServices({ cwd, agentDir, ... });

// 2. Create the session from those services; internally this calls
//    createAgentSession() in core/sdk.ts, which builds Agent + AgentSession.
const { session } = await createAgentSessionFromServices({ services, ... });

// 3. Hand the session to one run mode:
//    new InteractiveMode(...), runRpcMode(...), or runPrintMode(...).
```

6.7 Persistence and context are different things

Architecture matters here because the session file and the model context are different representations owned by different layers: `SessionManager` in coding-agent owns the durable JSONL tree, while the context sent to the provider is rebuilt from it on every turn.

	Durable session file	Provider context
Owned by	`SessionManager` (coding-agent)	The agent loop, rebuilt per turn
Lifetime	Survives restarts and resumes	One request; recomputed next turn
Contains	Every entry on every branch	Converted messages of the selected branch only
Written by	`pi.appendEntry()`, messages, compaction	Never written — it is derived

The two extension calls sit on opposite sides of this line (`core/extensions/types.ts`):

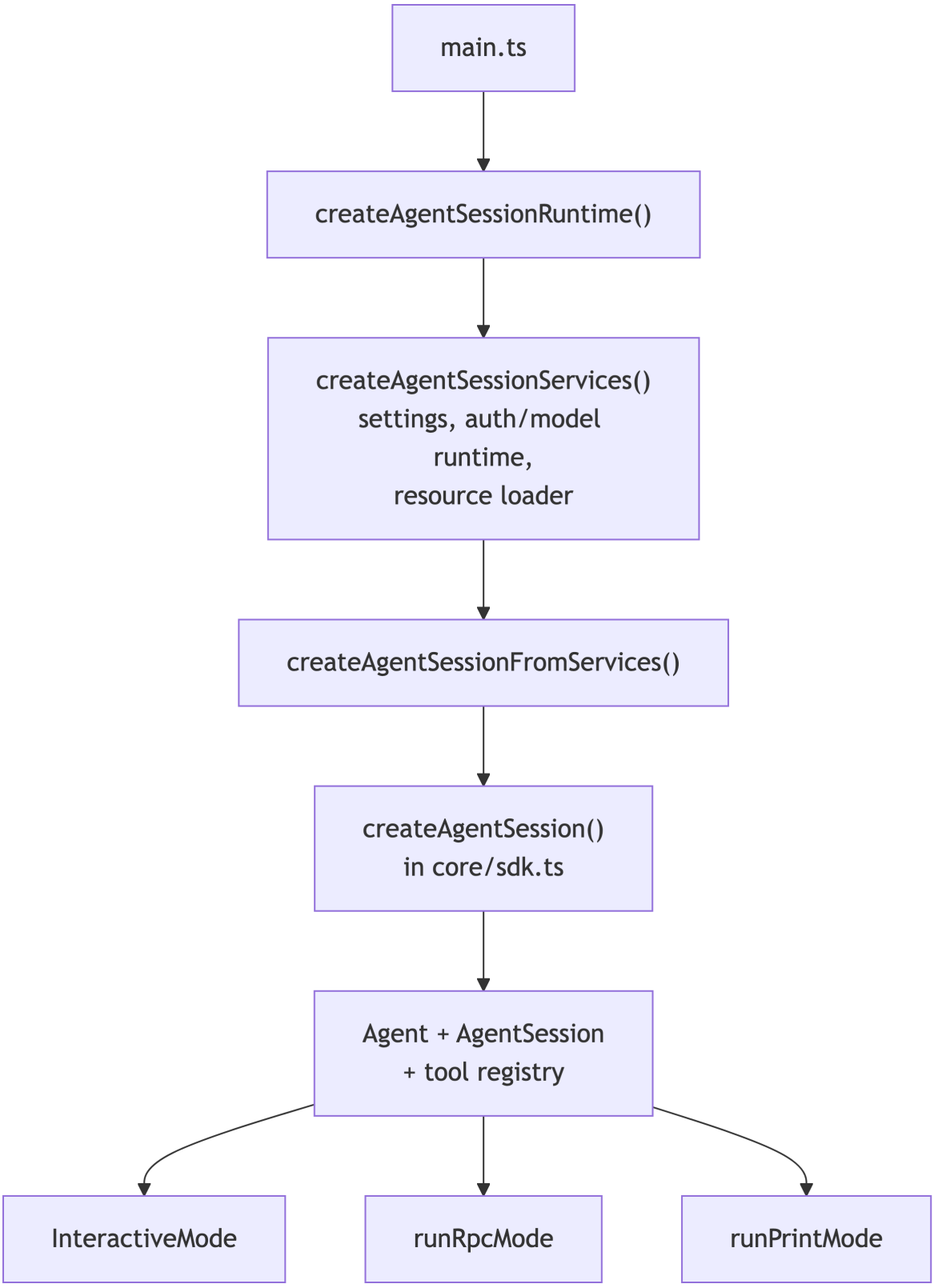

Figure 6.5: Startup construction. Services are built first, the session wraps an Agent around them, then one of the three run modes takes over.

```
// Durable only: appended to the session file, never sent to the model.
pi.appendEntry("my-state", { lastCheck: "2026-07-20" });

// Conversational: enters the session and becomes a message the model
// sees on a later turn.
pi.sendMessage({ customType: "my-notice", content: "check finished", display: true });
```

The sessions chapter covers this boundary in detail — tree structure, `buildContextEntries()`, and the rule that `pi.appendEntry()` writes durable state the model never sees while `pi.sendMessage()` enters the conversation; see the sessions chapter.

What belongs in this chapter is the ownership split for compaction. In plain terms: compaction rewrites what the model will see next; it deletes nothing from the file. Compaction is a context transformation, not deletion: Pi writes a summary entry and reconstructs a compact context from the summary and newer entries. The generic algorithms live one layer down in the `agent` harness (the reusable runtime code `agent` provides for consumers such as coding-agent), exported from `packages/agent/src/harness/compaction/`; coding-agent coordinates the user-facing policy and session entries.

6.8 Where to make a change

In plain terms: find the narrowest layer that truthfully owns the behavior. The decision tree answers the common cases; the table below it is the reference.

The full reference table:

Desired change	Discriminator	First place to investigate
Add a provider/API serializer	Provider protocol or request conversion	`packages/ai/src/api/` and `src/providers/`
Change generic tool scheduling or streaming semantics	Same loop rule for every consumer	`packages/agent/src/agent-loop.ts`
Add keyboard/editor/component capability reusable by TUIs	Terminal primitive, not Pi policy	`packages/tui/src/`
Change Pi commands, session behavior, built-in tool policy	Application behavior or configuration	`packages/coding-agent/src/`
Set values without adding executable behavior	**Configuration** in `settings.json`	Settings and resource-loading code
Add a command, tool, hook, state, or request behavior	**Extension** loaded at runtime	Public `ExtensionAPI` and Your first extension
Give the model reusable procedural guidance	**Skill** loaded as contextual instructions	Resource loader and `skills/`
Change colors and visual style	**Theme** consumed by renderers	Theme resources and TUI renderers
Reuse a written user/model request	**Prompt** template or slash command	Prompt resources and command registration

Desired change	Discriminator	First place to investigate
Share a tested set of resources or behavior	**Package** with its own manifest/version	Package/resource distribution code

Worked examples of the decision in practice:

- **"I want a new slash command."** No package change. An extension calling `pi.registerCommand()` (defined in `packages/coding-agent/src/core/extensions/types.ts`, demonstrated in `packages/coding-agent/examples/extensions/commands.ts`) adds the command at runtime; Pi's own code stays untouched.
- **"I want to support a new model provider."** First try `pi.registerProvider()` from an extension (`types.ts` in the same directory), which points an existing API family such as `openai-responses` or `anthropic-messages` at a new base URL and model list. Touch `packages/ai` only when the provider speaks a genuinely new wire protocol that no module under `packages/ai/src/api/` implements.
- **"I want to change how the transcript renders a tool."** Override the renderer from an extension, not in `tui`. Tools carry optional `renderCall`/`renderResult` functions (`types.ts`), and re-registering a built-in tool with the same name replaces it — including its rendering — as shown in `packages/coding-agent/examples/extensions/built-in-tool-renderer.ts`. `packages/tui` is the wrong layer: it owns generic components, not Pi's chat rows.

Continue to Prompt lifecycle for a source-level trace of how these package boundaries cooperate from interactive input through model streaming, tool execution, persistence, and rendering.

6.9 Beginner checkpoint

Before continuing, name the owner for each change:

1. A reusable terminal primitive.
2. A provider serializer.
3. A generic tool-loop rule.
4. A Pi session rule.
5. A user-specific behavior.

Solution

1. `packages/tui`
2. `packages/ai`
3. `packages/agent`
4. `packages/coding-agent`
5. An extension — no package change at all.

If you placed each one correctly, the package map is working.

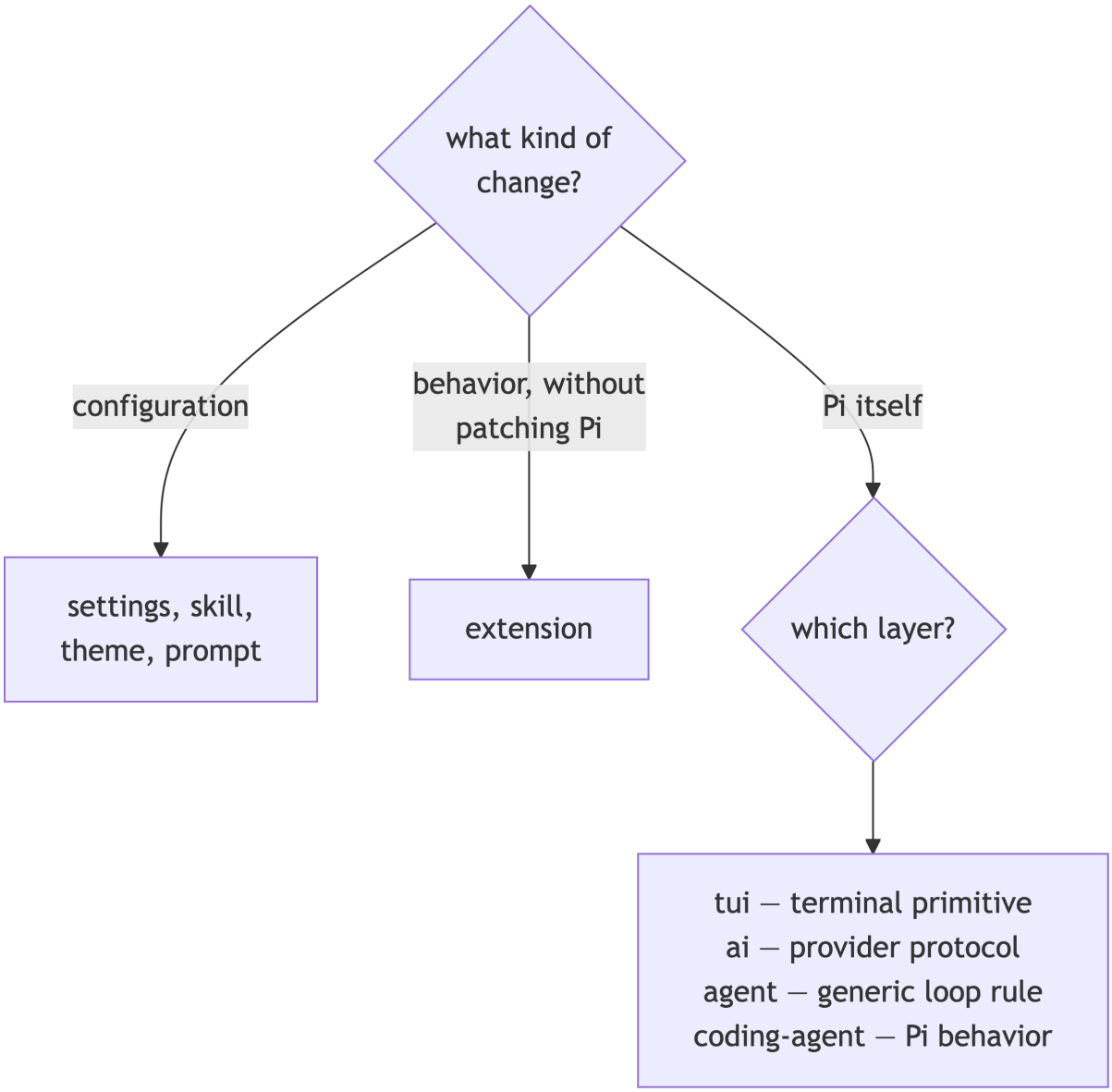

Figure 6.6: Where a change belongs. Code changes go to the owning layer; everything else is a resource or an extension.

7 The prompt lifecycle: where extensions hook in

The lifecycle is the map of extension hook points. This chapter answers one question — **when Pi runs your code, and what that code can influence** — in two passes: the whole lifecycle in one picture first, then a stage-by-stage trace of one prompt from startup to settlement. Chapters 7-8 assume you can read this map.

Before reading on, finish TypeScript used by Pi and Architecture. The chapter stays at the extension-facing boundary: exact event payloads and return values live in the Extension event reference, and the loop's internal implementation lives in Agent core. A final Under the hood section collects optional internals for readers who want them; everything a beginner needs is in the main path.

> **i** Scope and sources
>
> This chapter is source-grounded in the interactive mode and session files, plus `packages/agent/src/agent.ts`, `packages/agent/src/agent-loop.ts`, and the `packages/ai` dispatch layer. The maps summarize the current source and omit helper calls that do not alter the data/control boundary being explained. You can skim details on a first read and treat the maps as the territory.

7.1 A small vocabulary for the trace

An **event** is a record saying that something happened. A **hook** is the extension handler registered to observe or affect an event. A **turn** is one model request and the tool work that follows it. A **run** begins with `agent_start` and can contain several turns. A **settled** run is one with no retry, compaction, or queued continuation left to perform. A Pi message is composed of typed **content blocks** such as `text`, `thinking`, and `toolCall`, rather than one plain string.

Use two prompts to keep the distinction concrete: `"What files changed?"` usually creates one turn, while `"Read the config and fix the failing test"` can create several turns because the model may call `read`, then `bash`, then answer. The path below explains what stays the same in both examples.

7.2 How to read the map

Every hook is one of three kinds. **Notification** hooks observe; their return values are ignored. **Transform** hooks return or mutate a value Pi then uses. **Gate** hooks can cancel, block, or take over the operation entirely. Handlers for the same event run in extension load order, and where a value chains — the system prompt, input transforms, `tool_call` input mutations, `tool_result` patches — each handler sees the previous handler's output. The exact payload and return contract of every event is in the Extension event reference and the official Extensions documentation.

Two distinctions to carry through the whole chapter:

- **`agent_end` is not the end.** Retry, compaction-and-retry, or a queued follow-up can start another low-level run. `agent_settled` is the true-idle signal; Stage 9 makes this concrete.
- **Message events have three lanes.** User, assistant, and finalized toolResult messages each get their own `message_start`/`message_end` pair at different points in a turn; only assistant messages get `message_update` streaming deltas.

7.3 The lifecycle in one picture

You are looking at the complete life of a prompt, from Pi waking up to the run going idle. Each box is a stage; the rest of this chapter walks through the stages one by one. Inside each box are the extension **events** that fire there — the points where your code can run, colored by the vocabulary above. The blue stages handle your input, the amber stages are the model's working turns, and the gray stages keep the record. As a running example, picture the prompt `"Read the config and fix the failing test"`: it produces several turns because the model calls `read`, then `bash`, then answers. The table below lists every event with its firing condition and what a handler can do, and Figure 7.1 gives the same lifecycle in one picture.

Every extension event, grouped by the spine segment where it fires and listed in firing order. **Kind** uses the vocabulary from How to read the map: gates can cancel or block, transforms return or mutate a value Pi then uses, notifications are observe-only; **Both** means the row mixes notification and transform behavior. Exact payloads and return shapes are in the Extension event reference.

Event	When it fires	Kind	What a handler can do
Startup			
`project_trust`	Before project resources load	Gate	Return `yes`/`no`/`undecided`; first yes/no wins
`session_start`	Session opened, reloaded, or replaced	Notify	Initialize resources; rebuild state
`resources_discover`	After `session_start`	Transform	Return skill/prompt/theme paths
Prompt preparation			
`input`	Raw text, before skill/template expansion	Gate	`continue`, `transform`, or `handled`
`before_agent_start`	After input, before the loop	Transform	Inject a message; chain the system prompt
`agent_start`	Low-level run begins	Notify	
Each turn			
`turn_start` / `turn_end`	Bracket one model request + tool batch	Notify	Measure turns; `turn_end` carries `toolResults`
`message_start/message_end`	User message submitted	Notify	
`context`	Before every provider request	Transform	Return a replacement message list
`before_provider_headers`	Headers assembled; once per request	Transform	Mutate headers in place (`null` deletes)

Event	When it fires	Kind	What a handler can do
`before_provider_request`	Payload serialized, before send	Transform	Return a replacement payload
`after_provider_response`	Response received, before stream consumed	Notify	Observe status and headers
`message_start/update/end`	Assistant streaming output	Both	Observe deltas; `message_end` may replace the message
`tool_execution_start`	Tool call validated	Notify	
`tool_call`	Before execution	Gate	`{ block: true, reason }`; mutate `event.input`
`tool_execution_update`	Tool progress	Notify	Render progress
`tool_result`	After execution, before append	Transform	Return partial patches (chained)
`tool_execution_end`	Tool finalized (completion order)	Notify	
Run end			
`agent_end`	Low-level loop stopped	Notify	`event.messages` from this run
`agent_settled`	No retry, compaction, or queued work left	Notify	True-idle signal
Other paths			
`session_before_switch` / `_fork` / `_compact` / `_tree`	Before the named session operation	Gate	`{ cancel: true }`; compact/tree accept custom summaries
`session_shutdown`	Before teardown (exit, reload, replace)	Notify	Clean up session-bound resources
`session_compact` / `session_tree`	Compaction or navigation completed	Notify	Rebuild branch-dependent state
`session_info_changed`	`/name` or `setSessionName()`	Notify	Update UI
`model_select` / `thinking_level_select`	Model or thinking level changed	Notify	Update per-model UI/state
`user_bash`	`!` / `!!` shell command	Gate	Supply a backend, wrap, or answer directly

Session replacement (`/new`, `/resume`, `/fork`, and also `/reload`) tears the old extension instance down completely: `session_shutdown` fires, extensions are reloaded and rebound, and the *new* instance receives `session_start`. That is why durable-state patterns always pair cleanup in `session_shutdown` with reconstruction in `session_start`. Session replacement can also re-enter `project_trust` when it lands in a directory whose trust has not been resolved in the current process.

7.4 Stage 1: Startup, trust, and resource discovery

In plain terms: Pi wakes up. Before any prompt exists, it loads every extension's registration code, decides whether the current folder is allowed to contribute its own code and settings, opens a session

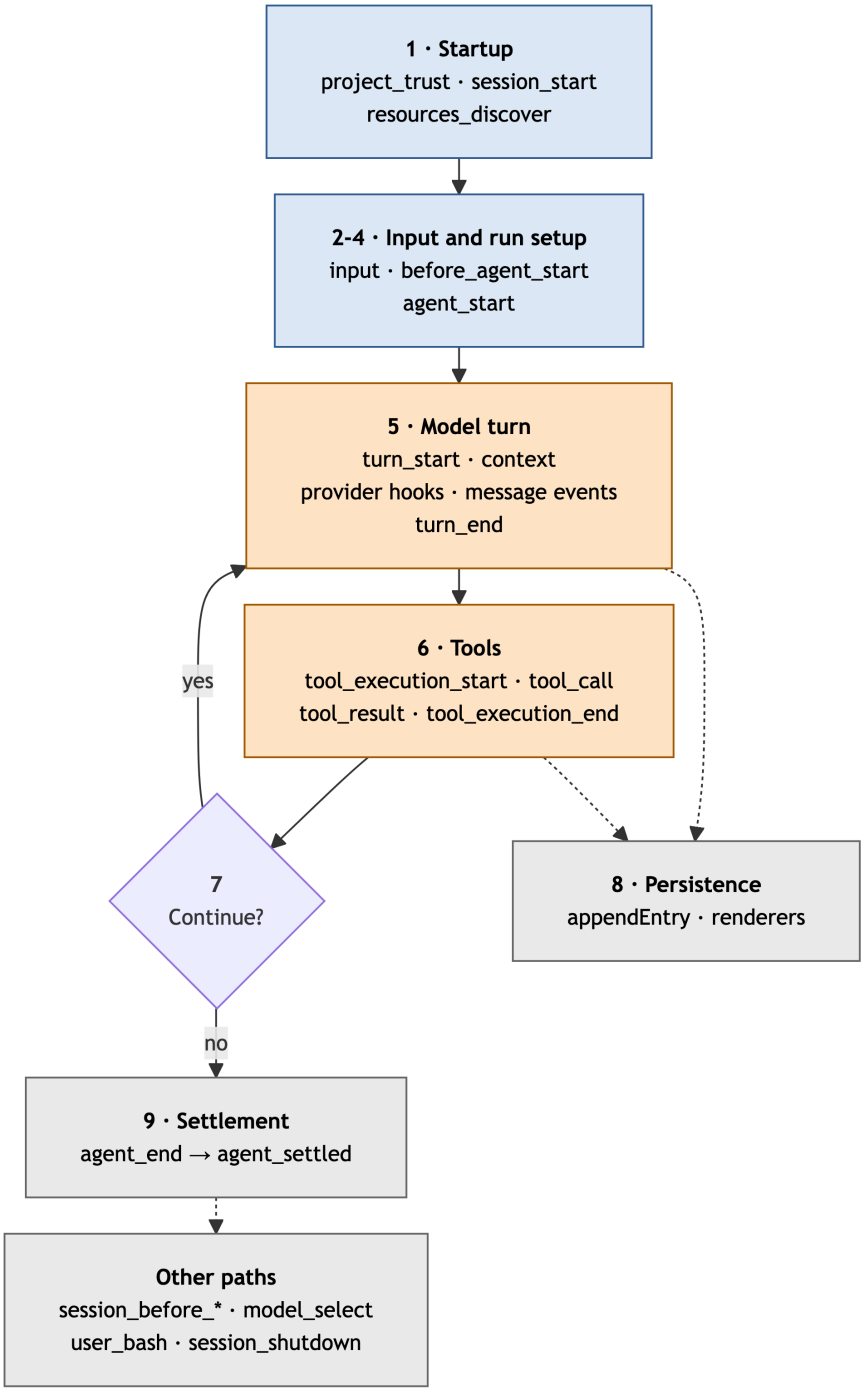

Figure 7.1: The lifecycle in one picture: the nine stages of a prompt with the key extension events that fire in each. Blue handles input, amber is the model's working turn, gray keeps the record. The table above lists every event on each segment.

— the notebook the conversation will live in — and asks extensions whether they have extra skills, prompts, or themes to add. Only then is Pi ready to listen. The mechanics, in order:

At startup the resource loader loads each extension's factory function. An async factory is awaited before startup continues, so its registrations are complete before `session_start` and `resources_discover` fire. The factory also runs for invocations that never start a session (for example `pi --list-models`), so it must not start timers, watchers, processes, or sockets; defer those to `session_start`. Your first extension covers discovery locations and loading in detail.

Before project-local resources load, Pi resolves **project trust**. The `project_trust` event fires here, and only user/global extensions and CLI `-e` extensions participate — project-local extensions are not loaded until trust is resolved. A handler can own the decision or defer:

```
pi.on("project_trust", async (event, ctx) => {
  // No UI host (print/JSON modes) cannot answer a dialog; defer instead.
  if (!ctx.hasUI) return { trusted: "undecided" };
  if (await ctx.ui.confirm("Trust project?", event.cwd)) {
    return { trusted: "yes", remember: true }; // persist the decision
  }
  return { trusted: "undecided" }; // let later handlers or Pi's flow decide
});
```

The first `"yes"` or `"no"` across handlers wins and suppresses Pi's built-in trust prompt; if every handler returns `"undecided"`, saved decisions and the `defaultProjectTrust` setting apply.

After `session_start`, Pi emits `resources_discover` so extensions can contribute additional resource paths. It fires again with `reason: "reload"` on `/reload`, and with `reason: "startup"` after session replacement:

```
pi.on("resources_discover", async (event) => {
  return { skillPaths: ["/path/to/skills"], promptPaths: ["/path/to/prompts"] };
});
```

Why an extension author cares: the factory is for registration and cheap one-time setup only; `project_trust` is the one hook that runs *before* project code loads; and `resources_discover` is how a package-like extension exposes its skills and prompts without settings changes.

7.5 Stage 2: Terminal input enters the TUI

In plain terms: you type something and press Enter, and your keystrokes become a submission to the session. If Pi is idle, the submission is picked up immediately. If Pi is still working, the submission does not interrupt — it waits in a queue for the right moment to be heard. Pressing Esc is the opposite: it stops the current work instead of queueing. How the editor decides which of those happens:

`InteractiveMode` owns the coding-agent's interactive screen. Its editor submission handler recognizes built-in slash commands and `!`/`!!` user-shell commands before normal agent prompting. User-shell commands fire the `user_bash` event, where an extension can supply a custom execution backend (for example SSH or a sandbox), wrap Pi's local backend, or return a result directly; the official documentation has the three return forms. A normal idle submission is queued

or resolved through `getUserInput()`. The `run()` loop awaits that method, then calls `await this.session.prompt(userInput)`.

When Pi is already streaming, regular Enter calls `session.prompt(text, { streamingBehavior: "steer" })`; Alt+Enter uses `"followUp"` (the editor keys are covered in Using Pi day to day). **Steering** waits for the current assistant response and its complete tool batch, then injects the message before the next provider request—even if the tool results already require that request. A **follow-up** is held until there are no more automatic tool continuations or steering messages and the run would otherwise stop, then starts another turn. Neither starts a second concurrent agent run; each call queues a message at that exact loop boundary. When Pi is idle, Alt+Enter acts like Enter because there is no active run to defer. Esc takes the opposite path: it aborts the active run rather than queuing input — Stage 9 covers that. **Compaction** summarizes older context into a durable entry so the model's context window is not exceeded; Sessions and context covers it in detail. During compaction, non-command input is held by interactive mode until the compaction flow completes.

The initial prompt supplied by command-line arguments travels through the same `session.prompt()` API. Print and RPC modes do not instantiate `InteractiveMode`, but they converge on `AgentSession` rather than reimplementing the provider path.

Why an extension author cares: steering and follow-up are the same queue modes you select with `sendUserMessage(text, { deliverAs: "steer" | "followUp" })` — input you inject rides the boundaries described here.

7.6 Stage 3: Commands and input interception

In plain terms: not everything you submit is meant for the model. Before your text becomes a prompt, Pi sorts it: is it a built-in command like `/model`? A command some extension registered? An instruction for the shell? Only what no one claims continues on as ordinary text — and even that gets one last chance to be rewritten by an extension before it counts as a prompt. The sorting order:

There are two distinct command systems:

- Built-in interactive commands such as `/model`, `/settings`, `/new`, and `/compact` are handled in `InteractiveMode` before a normal prompt reaches the session.
- Extension commands registered with `pi.registerCommand()` are detected first inside `AgentSession.prompt()`. They execute immediately, including while an agent is streaming, and do not enter the model context unless their handler explicitly sends a message.

For ordinary text, `AgentSession.prompt()` asks the extension runner to emit an `input` event. The event sees raw text after extension-command lookup and before skill/template expansion. An extension can continue, transform, or handle it. If it continues, Pi expands `/skill:name ...`; skills are reusable model instructions and prompt templates are parameterized prompt snippets. Pi then continues with the expanded text.

Hook selection comes first: built-in commands have already been handled by `InteractiveMode`, and a matching registered extension command has already returned before `input` is emitted. Therefore `input` is the interception point for ordinary text and unclaimed slash text. An extension that wants to intercept a skill invocation must use `input`; one that wants a durable custom command should use `registerCommand`, not parse slash text manually.

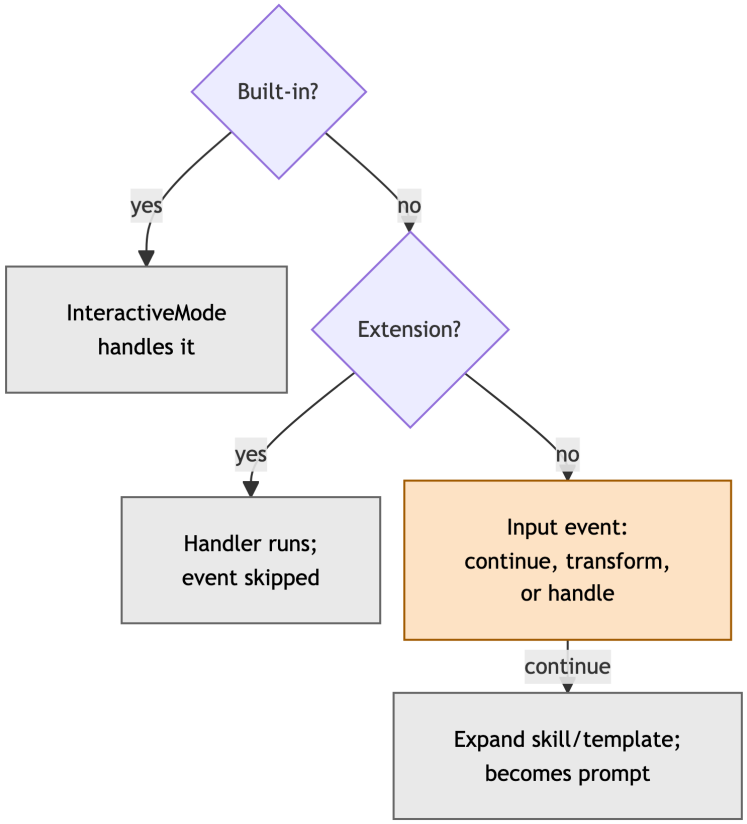

Figure 7.2: Stage 3 input routing. Built-in and extension commands are claimed before the input event fires; only unclaimed text reaches skill and template expansion and becomes a prompt.

A minimal extension that uses both interception points (save it, then load with `pi -e ./input-routing.ts` or copy it to `~/.pi/agent/extensions/` and restart or `/reload`):

```
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function (pi: ExtensionAPI) {
  // A durable custom command: /my-cmd runs immediately, even mid-stream.
  pi.registerCommand("my-cmd", {
    description: "Show a notification",
    handler: async (_args, ctx) => ctx.ui.notify("ran /my-cmd", "info"),
  });
  // Ordinary text: rewrite it before skill/template expansion.
  pi.on("input", async (event) => {
    if (event.text.startsWith("?quick "))
      return {
        action: "transform",
        text: `Respond briefly: ${event.text.slice(7)}`,
      };
  });
}
```

Why an extension author cares: this stage decides which interception point is yours — `registerCommand()` for a new slash command, the `input` event for transforming or handling ordinary text.

7.7 Stage 4: Run construction and delivery modes

In plain terms: once your text is accepted as a prompt, Pi packs the request. It checks that a model and credentials are ready, creates your message, folds in anything extensions asked to attach to this prompt, gives extensions one last chance to adjust the instructions the model will run under, and then starts the engine that will actually talk to the model. What gets packed, and in what order:

`AgentMessage` is the type the session passes around: it is either a provider-ready LLM `Message` or a coding-agent-specific custom message type. That union lets the session keep rich application messages (custom UI entries, tool details) alongside the plain messages the model will eventually see, even when the later conversion step maps them to a different LLM representation.

For a fresh run, `AgentSession.prompt()` validates the selected model and credentials, checks whether compaction is needed, and creates a user message. It then appends pending `nextTurn` messages: custom messages buffered by `pi.sendMessage(..., { deliverAs: "nextTurn" })`. Buffering for `nextTurn` lets an extension attach context to the *next* fresh user prompt — for example, notes a command just computed — instead of injecting into a run that is already active.

The three delivery modes that appear in this chapter:

Delivery mode	Buffered when	Delivered when	Selected with
`steer`	While a run is streaming	After the current assistant response and its tool work finish, before the next provider request	Enter while streaming; `sendUserMessage(text, { deliverAs: "steer" })`
`followUp`	While a run is streaming	Only when the run has no more tool calls or steering messages and would otherwise stop	Alt+Enter while streaming; `{ deliverAs: "followUp" }`
`nextTurn`	Any time, by an extension	Appended to the next fresh user prompt; never interrupts or triggers a run	`pi.sendMessage(msg, { deliverAs: "nextTurn" })`

Steer and follow-up take the same input path: both run extension input interception and skill/template expansion before Pi queues them. Only their delivery boundary differs. For example, while Pi is continuing an edit through several tool calls, Enter on `also inspect the log` injects that instruction before the next provider request; Alt+Enter on `then update the changelog` waits until that tool chain and any steering work have finished.

Next the session emits `before_agent_start`, whose handlers can add custom messages and chain a system-prompt override, then calls `_runAgentPrompt(messages)`. A minimal handler:

```
pi.on("before_agent_start", async (event) => {
  return {
    // Extra message the model will see (persisted in the session):
    message: { customType: "my-ext", content: "Extra context", display: true },
    // Chained system-prompt override for this turn:
    systemPrompt: event.systemPrompt + "\n\nPrefer small, reversible edits.",
  };
});
```

`_runAgentPrompt()` calls `Agent.prompt()`; the reusable Agent API is detailed in Agent core. It can later call `Agent.continue()` for automatic retry, automatic compaction retry, or work queued by an `agent_end` handler. On settlement it clears the per-run prompt override, flushes pending user-bash messages, and emits `agent_settled`.

Why an extension author cares: `before_agent_start` is your last chance to add messages or override the system prompt before the loop takes over; `agent_settled` is the signal that all follow-up work is done.

7.8 Stage 5: One model turn

In plain terms: this is the moment Pi actually talks to the model. It translates the conversation so far into the exact format the provider's API expects, sends the request, and then listens as the answer

arrives chunk by chunk — a few tokens at a time. Each chunk is republished as a Pi event, which is why you see the transcript typing out live. The three translations, and the stream that follows:

Immediately before *each* model request, `streamAssistantResponse()` in `agent-loop.ts` crosses three construction boundaries in sequence:

First, the coding agent builds a base system prompt in `AgentSession._rebuildSystemPrompt()` — working directory, context files, skills, custom and appended prompts, and active-tool guidance. The `transformContext` callback then dispatches the extension `context` event. A handler receives a deep copy of the message list and returns a replacement, so it can reshape what the model is about to see per request — for example, pruning stale scratchpad messages:

```
pi.on("context", async (event) => {
  // event.messages: deep copy of the AgentMessage[] list; safe to modify.
  return { messages: event.messages.filter((m) => !isStaleScratchpad(m)) };
});
```

The distinction that matters: `transformContext` works on Pi's rich `AgentMessage[]` list, while `convertToLlm()` converts that list into the plain, provider-compatible `Message[]` the provider API accepts. A `context` handler changes neither the JSONL history nor the compaction record; use session APIs or session hooks when durable state is the goal.

The resulting `Context` has exactly `systemPrompt`, `messages`, and `tools`. That shape lets the session persist rich UI-only entries while only a selected representation reaches the LLM.

The three provider hooks then bracket the request itself, in this order:

1. `before_provider_headers` — after credentials and headers are assembled; handlers mutate the map in place (set a key to add or override, set it to `null` to delete). Fires once per request; retries reuse the same headers.
2. `before_provider_request` — after the provider-specific payload is serialized, right before it is sent; returning a value replaces the payload. Changes here are not reflected by `ctx.getSystemPrompt()`, which reports Pi's assembled prompt text, not the wire payload.
3. `after_provider_response` — after the HTTP response arrives and before its stream body is consumed; handlers observe status and headers.

These are debugging and tracing boundaries, not substitutes for a custom provider; use `pi.registerProvider()` when you must declare a model catalog, endpoint, auth, or streaming implementation (see Models, providers, and authentication).

`streamAssistantResponse()` then consumes the provider's `AssistantMessageEventStream` with `for await`: this is the async-iteration boundary where provider deltas — the small partial chunks a provider emits, such as a few tokens of text at a time — become agent events. It creates a partial assistant message on `start`, replaces that partial message as text/thinking/tool-call deltas arrive, and emits `message_start`, `message_update`, and `message_end` events. `InteractiveMode` renders those events into the visible transcript.

Why an extension author cares: the `context` event reshapes the message list the model is about to see — per request, not just at run start — and `message_start`/`message_update`/`message_end` are how you observe model output live, for example to stream progress into custom UI.

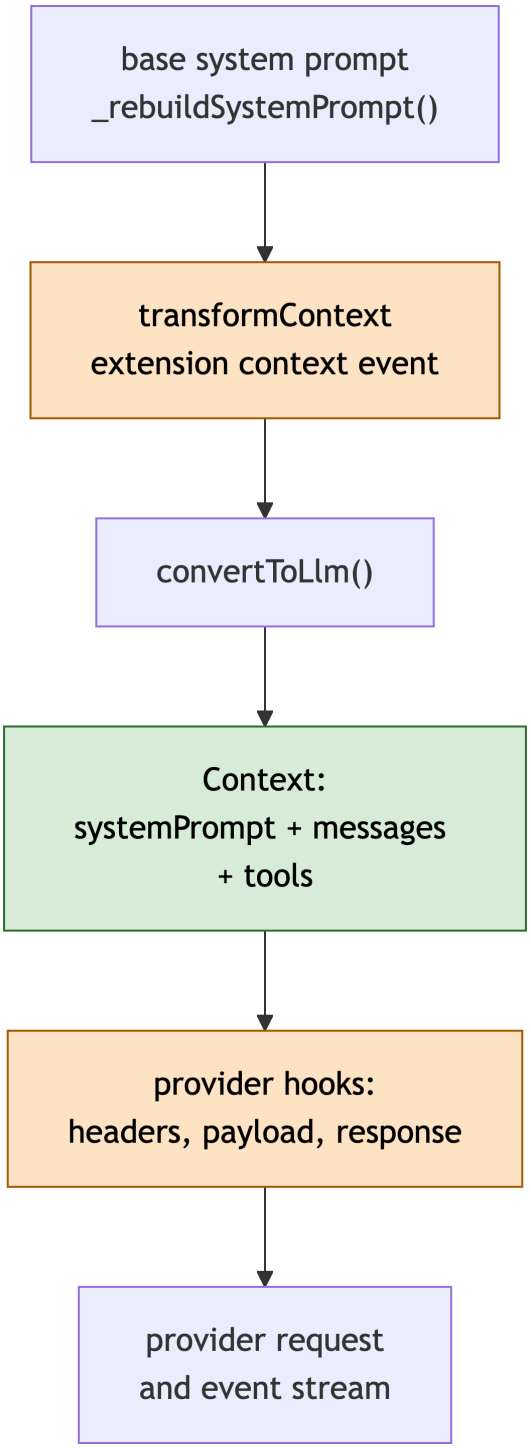

Figure 7.3: Stage 5's construction boundaries. The extension context event (amber) sits between base system-prompt assembly and provider conversion; the provider hooks (amber) bracket the request itself.

7.9 Stage 6: Tool calls are validated, gated, and executed

In plain terms: the model often answers with a request to *do* something — read this file, run this command. Pi does not obey blindly. It checks each request against the tool's declared input shape, gives extensions a chance to veto or adjust it, runs it, and files the result where the model will see it on the next turn. In the running example, this is where the model's `read` call on the config file is checked and executed. The pipeline every tool call passes through:

After a final assistant message, `agent-loop.ts` filters content blocks whose `type` is `"toolCall"`. A `"length"` stop means the token limit cut off the assistant output, so any tool call may contain truncated JSON arguments — for example `{"path": "src/comp` — even if its parsed block appears complete. The loop therefore fails every tool call in that message rather than risk side effects from incomplete arguments.

For each normal tool call, the loop's short pipeline is:

1. Emit `tool_execution_start`, look up the active tool, and prepare and validate its arguments against the TypeBox schema (a runtime validation schema; see TypeBox).
2. Run the `tool_call` gate, then call `tool.execute(id, args, signal, onUpdate)`.
3. Turn `onUpdate` values into `tool_execution_update` events, apply the `tool_result` patch, emit `tool_execution_end`, and append a `toolResult` message (which fires its own `message_start`/`message_end` pair).

The `tool_call` gate runs after schema validation and before anything executes. A handler can return `{ block: true, reason }` — the call never runs, and the reason reaches the model as an error `toolResult` it can respond to — or mutate `event.input` in place to rewrite the arguments (Pi does not revalidate after a mutation, so keep mutations narrow). The runnable safety-gate recipe, including how to ask the user and what to do when no UI exists, is in Extension patterns.

Tool execution defaults to parallel. Agent core sets `toolExecution` to `"parallel"` (see Agent core for the ordering contract), and coding-agent does not override it. A tool declaring `executionMode: "sequential"` makes its containing batch sequential.

In parallel mode the ordering guarantees are: preflight (`tool_execution_start` and the `tool_call` gate) happens in assistant source order; tool executions may run concurrently, so `tool_execution_update` events interleave and `tool_execution_end` fires in completion order; final `toolResult` messages are still emitted in assistant source order.

> **Warning**
>
> Because `tool_execution_end` fires in completion order, a `tool_call` or `tool_result` handler is not guaranteed to see sibling tool results from the same assistant message. See the extension documentation before using sibling tool results as state.

Built-in tools are ordinary tools at this boundary, which is why an extension can add a tool or override a built-in one. Custom filesystem-mutating tools must participate in `withFileMutationQueue()` — a per-path async queue that serializes writes to the same file — to avoid lost writes when sibling tools run in parallel; Extension patterns has the runnable queue recipe.

Why an extension author cares: the `tool_call` event is your safety gate (veto or rewrite arguments before execution) and `tool_result` lets you patch what the model sees afterwards.

7.10 Stage 7: Results create the next agent action

In plain terms: after a turn finishes, Pi looks at what it has and decides what happens next. Tool results go back to the model so it can continue working. If you queued an instruction mid-run, it is heard now. Only when the model has nothing more to do and no one has anything more to say does the run wind down. In the running example, the `read` result returns here and starts the turn where the model calls `bash` to run the test; when the model finally answers with plain text, this is where the run decides to stop. The decision order:

Each final tool result is appended to the in-memory agent context and becomes the model-facing material for the next turn. This includes schema failures, blocked calls, and thrown tool errors: they become error `toolResult` messages so the model can correct or explain the failure. At every successful turn boundary, the loop first checks steering. It then continues when either steering is present or an assistant tool batch is not explicitly terminating. Only when neither condition applies does it check follow-up:

A tool can return `terminate: true` to hint that the next model call should be skipped, but the batch terminates only when every finalized tool result sets that flag. A tool signals failure by throwing; merely returning a field named `isError` does not mark it failed.

Why an extension author cares: messages you queued with `deliverAs: "steer"` or `"followUp"` are drained here, in that priority order.

7.11 Stage 8: Persistence is a side lane, not a stage

In plain terms: while all of this runs, a quiet side lane keeps the record. Every message that finalizes — yours, the model's, each tool result — is appended to the session file on disk, and in parallel to the transcript on screen. Nothing waits for the run to finish, and nothing about how things look changes what the model sees. What exactly gets recorded, and what does not:

Persistence does not wait for the run to finish. As each message finalizes — the user message, each assistant message, each toolResult — `AgentSession` records it through its `SessionManager`. The session is a JSONL tree, so the persisted form can later rebuild a branch and compact context. Model and thinking-level changes are also session entries. What is *not* persisted: streaming deltas between `message_start` and `message_end`, provider headers and payloads, and transient UI state — only finalized messages and explicit entries are durable. Extension state added with `appendEntry()` is persisted as a custom entry — durable across restarts and reloads — but remains outside LLM context.

`InteractiveMode.handleEvent()` responds to the same event stream. Assistant updates request transcript updates for the streaming message; tool lifecycle events manage pending tool rows; finalized messages become permanent transcript components. Renderer registration lets an extension replace the display of its custom messages, entries, or tool calls/results without changing how the model loop works.

> **Scope note**
>
> This chapter stops at the event-to-component boundary: how events request transcript updates, not how the terminal paints them. Terminal rendering internals live in `packages/tui` and Ter-

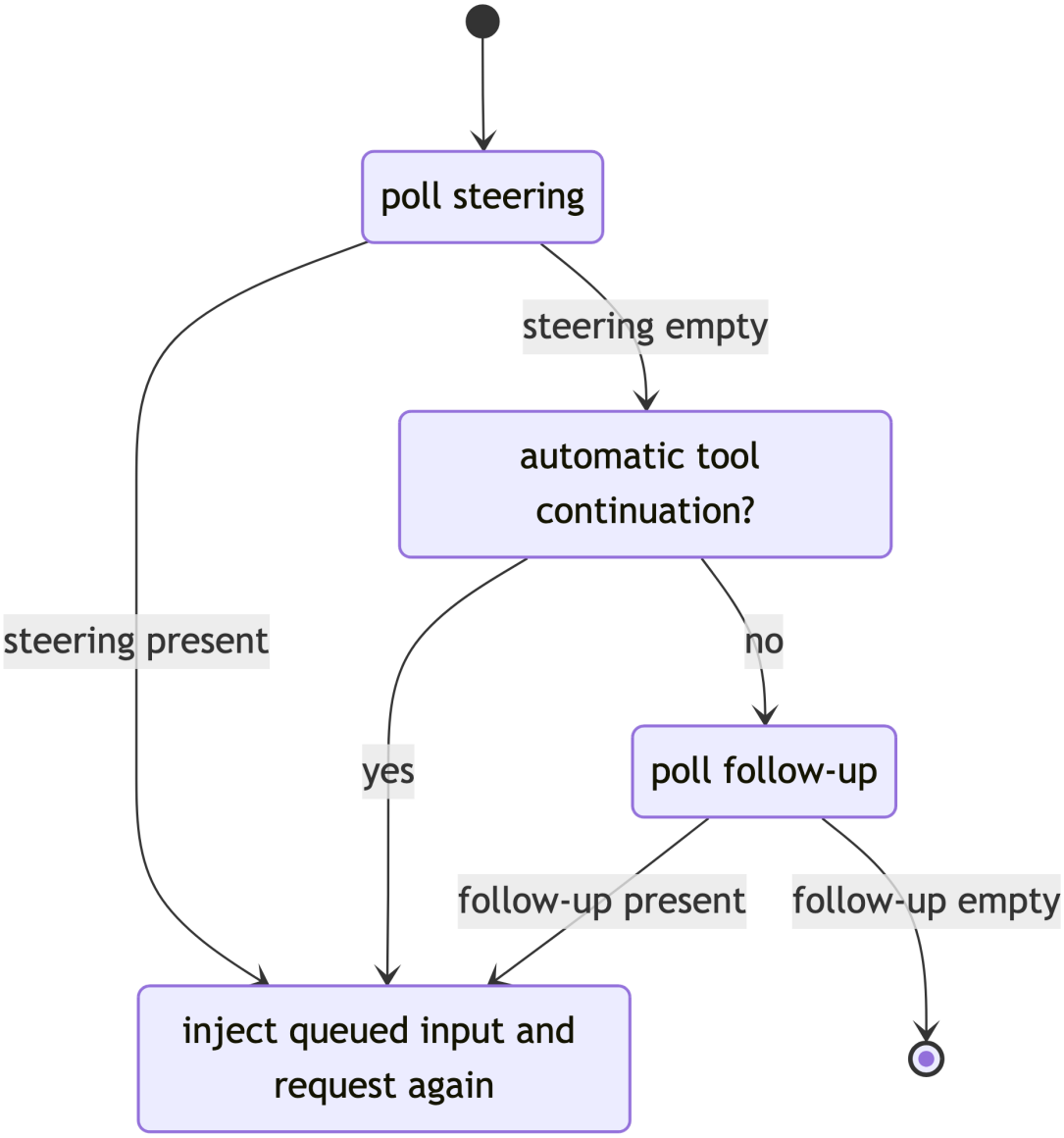

Figure 7.4: The queue boundary after a turn. Steering is polled before automatic tool continuation; follow-up is polled only after both are exhausted.

minal UI.

Why an extension author cares: `appendEntry()` persists your state without polluting model context, and renderer registration controls how your custom messages and entries look on screen.

7.12 Stage 9: Errors, retries, and true settlement

In plain terms: things go wrong, and Pi is built to absorb that. A provider error can be retried. A conversation that grew too large can be summarized and continued. You can cancel at any time. Each of these paths ends the current attempt, but the *run* is only truly over when there is nothing automatic left to do — no pending retry, no queued message. How the pieces fit:

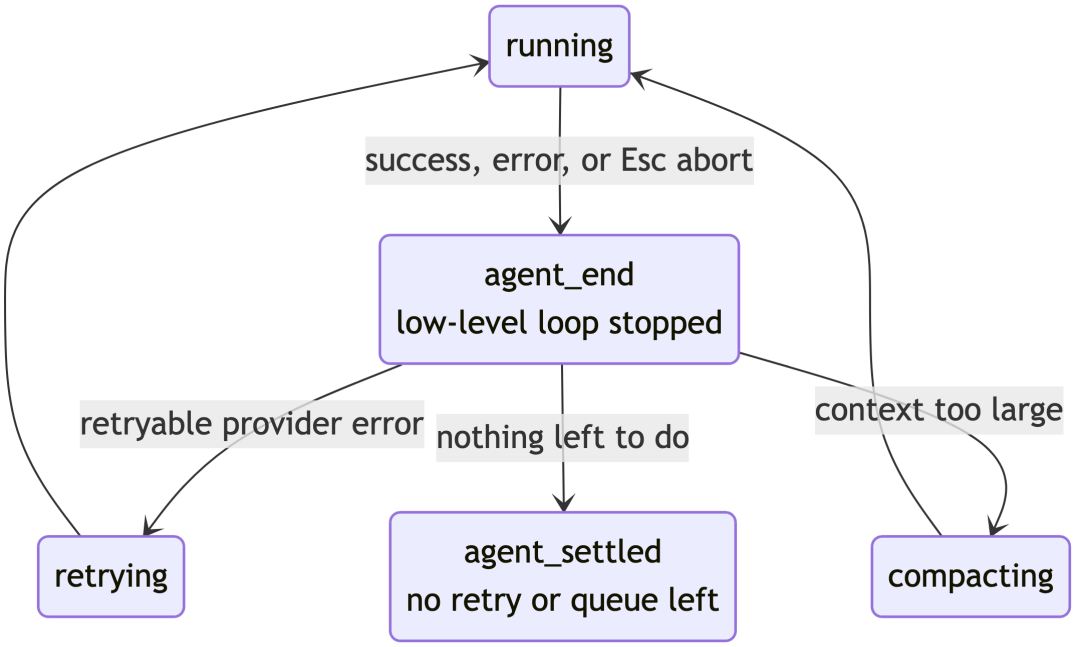

Figure 7.5: Run lifecycle. Every low-level run ends with agent_end — success, failure, and abort alike. agent_settled fires once, when no retry, compaction, or queued continuation remains.

The low-level loop deliberately separates provider failures from tool failures:

Failure type	What the model sees	Recovery path
Retryable provider error	An assistant message with `stopReason "error"`	The low-level run ends with `agent_end`; `AgentSession` may retry automatically
Context overflow	An assistant message with `stopReason "error"`	`AgentSession` compacts older context and retries
User abort (Esc)	An assistant message with `stopReason "aborted"`	No automatic retry; the run settles unless new work is queued
Tool schema error, blocked call, or thrown execution	An error `toolResult` message	The model sees the failure and can recover in a later turn

Esc aborts the active `AbortController`; provider, tool, and extension work must honor the signal. For the complete recovery matrix, see Failure and recovery.

Above this boundary, `AgentSession` may retry, compact older context, rebuild it, and continue; Sessions and context explains compaction and links to the official Compaction reference for its detailed rules. Automatic compaction is part of this recovery path — it fires between `agent_end` and the retry, not from the settled state.

One automatic retry makes the event counts concrete: the provider errors, so `agent_end` fires for the first low-level run; `AgentSession` retries and the second run completes, so `agent_end` fires again; only then does `agent_settled` fire — twice for `agent_end`, once for `agent_settled`. Status integrations and cleanup that require true idleness should use `agent_settled` or `ctx.waitForIdle()` in a command context.

Why an extension author cares: honor the abort signal in your own async work, clean up session-bound resources on `session_shutdown`, and use `agent_settled` (not `agent_end`) when you need true idleness.

7.13 Other lifecycle paths

The prompt trace above is the main road. These paths branch off between prompt loops; each has its own gate and completion event.

7.13.1 Session replacement

`/new`, `/resume`, `/fork`, `/clone`, and `/reload` all tear the current extension instance down and build a new one:

Operation	Gate (can cancel)	Then, in order
`/new`, `/resume`	`session_before_switch`	`session_shutdown` → `session_start { reason: "new" | "resume", previousSessionFile? }` → `resources_discover { reason: "startup" }`
`/fork`, `/clone`	`session_before_fork`	`session_shutdown` → `session_start { reason: "fork", previousSessionFile }` → `resources_discover { reason: "startup" }`
`/reload`	—	`session_shutdown { reason: "reload" }` → `session_start { reason: "reload" }` → `resources_discover { reason: "reload" }`

Cleanup belongs in `session_shutdown`; state reconstruction belongs in `session_start`. If your state depends on the active branch, rebuild again on `session_tree` — State that survives restart and branching has the pattern.

7.13.2 Compaction and tree navigation

`/compact` fires `session_before_compact` (cancel, or supply a custom summary) then `session_compact`. `/tree` navigation fires `session_before_tree` (cancel, or supply a custom summary for the abandoned branch) then `session_tree`. Both `reason` fields distinguish manual from automatic triggers; automatic compaction instead rides the Stage 9 recovery path.

7.13.3 Model and session metadata

`model_select` fires when the model changes via `/model`, cycling, or session restore; when the change clamps the thinking level, `thinking_level_select` fires first. Thinking-level changes on their own also fire `thinking_level_select`. `/name` or `pi.setSessionName()` fires `session_info_changed`. All three are notification-only; use them to update status UI or per-model state.

7.14 Under the hood

Optional on first read. The main path above is everything you need to choose hooks; this section collects the internals that explain why *the boundaries sit where they do.*

7.14.1 What the model can see

Every model-visible value must become one of the three `Context` fields — `systemPrompt`, `messages`, or `tools`. The assembly has three timings: resource discovery and input expansion happen before a run; messages are transformed and converted before every provider request.

Input	How Pi discovers or creates it	Model route	When it is visible
Pi default instructions, custom prompt, appended prompt, cwd	CLI options and `buildSystemPrompt()`	`systemPrompt`	Every request after assembly or refresh
`AGENTS.md` and other context files	Context-file discovery	Appended to `systemPrompt`	Every request
Available skills	Resource loader discovers configured/global/project skill files	A skill catalog—name, description, and location—inside `systemPrompt`	Every request when `read` is active, except skills marked `disable-model-invocation`
`/skill:name` invocation	Input expansion before a run	Full skill body plus typed arguments in the user message	The submitted message and later conversation history

Input	How Pi discovers or creates it	Model route	When it is visible
Prompt template invocation	Input expansion before a run	Expanded text in the user message	The submitted message and later conversation history
Active built-in and extension tools	Tool registry and active-tool selection	Name, description, and parameter schema in `Context.tools`; optional guidance in `systemPrompt`	Every request after activation or refresh
Assistant text, tool calls, and tool results	Provider stream and tool execution	Converted assistant and `toolResult` messages	Later requests in the same active context
Extension custom messages	`before_agent_start`, `sendMessage`, or queued delivery	Converted message, if its type is model-visible	At its configured delivery point
Durable custom entries and renderers	`appendEntry()` and UI registration	No route by default	Never, unless an extension also sends a message
MCP connection/configuration	Pi has no built-in MCP subsystem	No route by default	Never, unless an extension exposes model-visible tools or messages

A discovered skill is not copied wholesale into every request. Instead, `formatSkillsForPrompt()` adds an `<available_skills>` entry containing its name, description, and file location, plus instructions to use `read` when a task matches. Skills with `disable-model-invocation: true` are omitted from that catalog and can only be invoked explicitly. An explicit `/skill:name arguments` follows a different path: before a run starts, Pi reads the skill file and expands its full body, location, and the user's arguments into the user message. Customizing Pi documents discovery locations and input expansion rules.

Pi does not provide MCP integration as a built-in subsystem. An MCP extension or another bridge can connect to an MCP server, but that connection is model-invisible by itself. The model sees an MCP-backed capability only when the integration registers and activates a Pi tool, or deliberately injects a model-visible message.

7.14.2 The agent loop in one picture

Pi repeatedly gives a model three things — the assembled system prompt, the conversation so far, and the active tool schemas. The model then either answers or requests tools. Tool results become part of the conversation for the next request. A run ends only when there are no tool calls that require a model follow-up and no queued user messages.

The actual implementation emits events, supports parallel tools, and refreshes next-turn state. It uses an inner loop for automatic tool continuation and steering, wrapped by an outer loop for follow-up. Its control flow can be read as this deliberately simplified version:

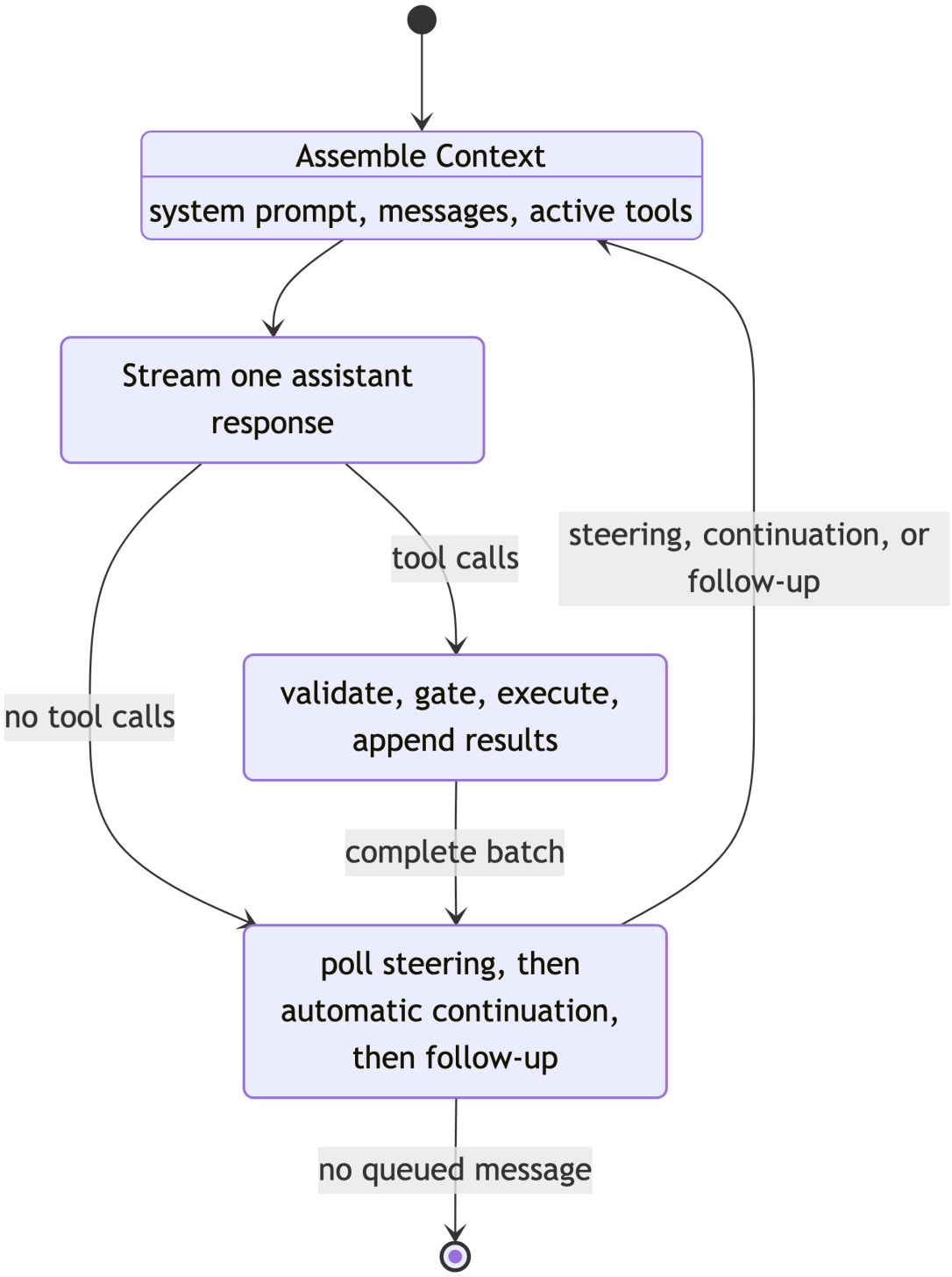

Figure 7.6: The model-facing loop. Tool schemas and tool guidance have different roles: schemas are sent in Context; guidance helps construct the system prompt.

```
let pending = takeSteeringMessages(); // Also catches input queued just before the run starts.

while (true) { // Outer loop: only follow-up re-enters here.
  let automaticContinuation = true;

  while (automaticContinuation || pending.length > 0) {
    context.messages.push(...pending); // Steering or a follow-up being delivered.
    pending = [];

    const response = await streamModel(context);
    if (response.stopReason === "error" ||
        response.stopReason === "aborted") return;

    const calls = response.toolCalls;
    const results = await validateGateAndExecute(calls);
    context.messages.push(response, ...results);

    await prepareNextTurn();
    if (await shouldStopAfterTurn()) return;

    automaticContinuation =
      calls.length > 0 && !results.every((r) => r.terminate);
    pending = takeSteeringMessages();
  }

  // Only after both tool continuation and steering are exhausted:
  pending = takeFollowUpMessages();
  if (pending.length === 0) break;
}
```

This sketch preserves the rules that matter most: a failed tool becomes an error result the model can inspect, rather than ending the run; one terminating tool does not stop a batch unless **every** finalized result terminates; and steering is checked after every successful turn—even when tool results require another provider request. Follow-up is checked only after that tool/steering chain would otherwise become quiescent. A complete sibling tool batch remains atomic: a steering message never interrupts one tool or prevents a later sibling from starting.

By default, each queue drains **one-at-a-time**, so one queued message is injected before a provider request. The configurable `"all"` mode injects all currently queued messages as one batch. The real loop also calls `prepareNextTurn` after each turn, so a later request can see changed active tools, a rebuilt system prompt, model, or thinking level without changing the request already in progress. An embedding can also supply `shouldStopAfterTurn`; when it returns true, the generic loop ends before it polls steering or follow-up queues.

7.14.3 Provider dispatch

Coding-agent installs a custom `streamFn` on `Agent` in `core/sdk.ts`: `streamFn` is the function the agent loop calls for every model request. Its coding-agent path is `core/sdk.ts` →

`ModelRuntime.streamSimple()` → the composed provider and its adapter. Request preparation resolves credentials and headers (this is where `before_provider_headers` fires), the composed provider selects the adapter for `model.api` and serializes the normalized context (`before_provider_request`), and the adapter returns a provider-independent assistant event stream (`after_provider_response`, when the adapter exposes HTTP metadata). `ModelRuntime` is the canonical internal owner of model catalogs, auth, OAuth, and composed-provider request preparation. Vendor wire formats, transport negotiation, custom provider registration, and auth-storage details live in Models, providers, and authentication.

7.15 Trace this yourself

All source paths in this chapter are relative to the repository root. For a live trace, start at these symbols in order:

1. `InteractiveMode.run()` and `getUserInput()` in `packages/coding-agent/src/modes/interactive/interactive-mode.ts`.
2. `AgentSession.prompt()` and `_runAgentPrompt()` in `packages/coding-agent/src/core/agent-session.ts`.
3. `Agent.prompt()`, `createLoopConfig()`, and `processEvents()` in `packages/agent/src/agent.ts`.
4. `runAgentLoop()`, `streamAssistantResponse()`, and `executeToolCalls()` in `packages/agent/src/agent-loop.ts`.
5. `streamFn` in `packages/coding-agent/src/core/sdk.ts`, which calls `ModelRuntime.streamSimple()`.
6. The `streamSimple` function in the selected composed provider/API adapter.

Use the extension event timeline in `packages/coding-agent/docs/extensions.md` as a second view of the same lifecycle. It documents what extension authors can rely on; the source symbols above show which layer makes each guarantee.

7.16 Where to go next

- Your first extension — build a working extension against the hooks you just mapped.
- Extension patterns — production recipes for the gates and state patterns named here.
- Extension event reference — exact payloads, return values, and middleware ordering for every event on the lifecycle map.
- Agent core and Models, providers, and authentication — the internals summarized in Under the hood.

7.17 Try it

1. Write an extension that records every tool gate with a TUI notification: `pi.on("tool_call", async (event, ctx) => ctx.ui.notify(event.toolName, "info"))`. Load it with `pi -e ./log-tools.ts`, ask Pi to read a file, and answer: which stage does `tool_call` fire in, and which events fire immediately before and after it? (Check your answer against the lifecycle map and the timeline in `packages/coding-agent/docs/extensions.md`.) Avoid `console.log` here — in print, JSON, and RPC modes, stdout carries machine-readable output.

2. Using the delivery-mode table in Stage 4, predict what happens when you type a message and press Enter while Pi is mid-stream, then do it and confirm the message is injected before the next provider request, not appended at the end.
3. Register a `before_agent_start` handler that appends one line to the system prompt, then a `context` handler that drops the oldest user message. Ask a question and use the `context` handler's effect to explain, in your own words, the difference between the two boundaries in Stage 5.

Part IV

Building extensions

8 Your first extension

In plain terms: an extension is a file of TypeScript or JavaScript that Pi loads at startup and that adds behavior to every session. This chapter takes you from an empty file to a working extension in four runnable stages: a `/hello-pi` command you can run immediately, a `hello` tool the AI model can call, a startup notification, and a greeting counter that survives `/reload` and restart. Each stage ends with something you can run and check.

Extensions execute with the permissions of the Pi process. They are trusted code, not a sandbox. Review a package before installing it and treat an extension's filesystem, process, network, and credential access as you would a normal local program.

8.1 Prerequisites

Finish Customizing Pi first — this chapter assumes you know how settings, trust, and scopes interact. You also need:

- A working `pi` install, signed in to a provider (required to exercise the tool in stage 2; see Using Pi day to day).
- Basic TypeScript familiarity: functions, `async`/`await`, and imports are enough (see Reading and changing Pi's TypeScript).
- Optionally, a checkout of the `pi` repository so you can follow the referenced source files; every file path in this chapter is relative to its root.

8.2 The minimal mental model

Under the hood, Pi loads an extension with `jiti`, a runtime TypeScript loader. Jiti transpiles TypeScript syntax on demand, so an extension needs no build step; it does not type-check the code. The module's default export is a **factory**: a function that receives `ExtensionAPI` and registers the extension's behavior — hooks, tools, commands, shortcuts, flags, renderers, and providers.

A **hook** is a handler subscribed to a named lifecycle **event** — an occurrence such as "the session started" or "a tool is about to run" — where your code can observe or change what Pi does next. The full event map is The prompt lifecycle; this chapter uses only four events and introduces each when the tutorial needs it.

Most extensions register at least one of three things; choosing among them is the first design decision:

Register a…	When you want…	Invoked by
Command (`pi.registerCommand`)	An action the user triggers explicitly, such as `/hello-pi`	The user typing `/name`

Register a…	When you want…	Invoked by
Tool (`pi.registerTool`)	A capability the AI model can call during a turn, with validated arguments	The model, via a tool call
Hook (`pi.on`)	To observe or gate something Pi already does: react at startup, block a tool call, persist state at shutdown	Pi, at the matching lifecycle event

Each registration surface receives a different object. Keep this table beside you while reading the stages:

Surface	Receives	Purpose
Factory	`pi: ExtensionAPI`	Register tools, commands, hooks, and more
Event handler	`event` + `ctx: ExtensionContext`	Observe or affect the current runtime
Command handler	`args` + `ctx: ExtensionCommandContext`	Same as above, plus session-changing methods (`newSession`, `reload`, `waitForIdle`, …)
Tool `execute`	validated `params`, `signal`, `onUpdate`, `ctx`	Perform the work the model requested

The context fields you will use most are `ctx.cwd`, `ctx.ui` (notifications and dialogs), `ctx.sessionManager` (session entries), `ctx.mode`, and `ctx.hasUI`. The full surfaces are in the Extension event reference.

Before writing code, one mechanism check: an extension is powerful, but do not use one to recreate or patch Pi internals as an unreviewed fork, and favor public `ExtensionAPI` methods over importing coding-agent internals. Which mechanism do you need? helps you confirm an extension — rather than configuration or a core change — is the right tool.

8.3 Stage 1: Create and run `/hello-pi`

Goal: run one command from your own extension. Start with a global file so project trust does not come into play yet.

1. Create the directory and the file:

```
mkdir -p ~/.pi/agent/extensions
```

2. Save this as `~/.pi/agent/extensions/hello-pi.ts`:

```ts
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function register(pi: ExtensionAPI) {
  pi.registerCommand("hello-pi", {
    description: "Show a greeting notification",
    handler: async (args, ctx) => {
      ctx.ui.notify(`Hello, ${args || "world"}!`, "info");
    },
  });
}
```

3. Start `pi` in any project and type `/hello-pi Ada`.

Expected result: a notification reading `Hello, Ada!`. If nothing happens, jump to Debugging extensions — the most likely causes are the file location or a missing default export.

That is the whole loop: write a file with a default-exported factory, register behavior, run it. Everything below adds one capability at a time to this same file.

8.4 How Pi found your file

Pi automatically discovers a single-file extension or a directory `index.ts` in these places:

Scope	Files discovered
Global	`~/.pi/agent/extensions/*.ts`, `~/.pi/agent/extensions/*/index.ts`
Project	`.pi/extensions/*.ts`, `.pi/extensions/*/index.ts`

JavaScript (`.js`) files and npm-packaged extensions work too; the table shows the simplest forms, which this chapter uses. Project-local resources load only after the project is trusted — one reason stage 1 used the global path. You can also add local paths in `settings.json`, or load a file for one invocation with `pi -e ./path/to/extension.ts`, which is ideal for a quick test. `/reload` reloads extensions and other resources; test final placement from an auto-discovered location, because that is the only way to prove discovery works.

Two loading rules you will eventually hit:

- When the same extension file is reachable from more than one scope, Pi loads it once, preferring the project copy.
- If two extensions register the same **command** name, both stay loaded and get numeric invocation suffixes in load order (`/review:1`, `/review:2`). Duplicate **tool** or **flag** names instead surface as load diagnostics. Either way: pick distinctive, namespaced names.

An extension factory can be `async`; Pi awaits it before startup continues, so its registrations are complete before the session starts. Use that for cheap one-time setup only — never start timers, watchers, processes, or sockets in the factory, because factories also run for invocations that never start a session (like `pi --list-models`). Start long-lived resources in `session_start` instead (stage

3) and close them in `session_shutdown`; Extension patterns covers the lifecycle-safe version of this rule, including async provider registration.

8.5 Stage 2: Add a tool the model can call

Goal: make `hello` callable by the AI model. A tool needs a unique `name`, a human-facing `label`, a `description` the model reads, a TypeBox `parameters` schema, and an `execute` function.

TypeBox interlude, because the schema is the part newcomers question: Pi must validate whatever JSON the *model* sends before your code runs, so a runtime schema is required even though TypeScript already knows the types. `Type` is the TypeBox builder re-exported from `@earendil-works/pi-ai`; `Type.Object` builds the runtime schema, `defineTool` derives the static type of `params` from it, and the agent loop rejects malformed arguments before `execute` is called.

Replace the contents of `~/.pi/agent/extensions/hello-pi.ts` with:

```
import { Type } from "@earendil-works/pi-ai";
import { defineTool, type ExtensionAPI } from "@earendil-works/pi-coding-agent";

const helloTool = defineTool({
  name: "hello",
  label: "Hello",
  description: "Greet a person by name",
  parameters: Type.Object({
    name: Type.String({ description: "The person to greet" }),
  }),
  async execute(_toolCallId, params, _signal, _onUpdate, _ctx) {
    return {
      content: [{ type: "text", text: `Hello, ${params.name}!` }],
      details: { greeted: params.name },
    };
  },
});

export default function register(pi: ExtensionAPI) {
  pi.registerTool(helloTool);

  pi.registerCommand("hello-pi", {
    description: "Show a greeting notification",
    handler: async (args, ctx) => {
      ctx.ui.notify(`Hello, ${args || "world"}!`, "info");
    },
  });
}
```

Run `/reload` in your Pi session, then type the literal prompt `Use the hello tool to greet Ada`.

Expected result: the model issues a `hello` tool call with `{ "name": "Ada" }`, and the tool result `Hello, Ada!` appears in the transcript. The return contract: `content` is what the model sees; `details`

is structured metadata for renderers and state recovery. To report failure from `execute`, **throw** — Pi catches it and delivers an `isError: true` tool result to the model. Returning a field named `isError` does nothing.

If the model does not pick the tool, check registration yourself instead of guessing: a quick `pi.on("session_start", ...)` handler can log `pi.getActiveTools()` to a notification, or a temporary command can call `pi.getAllTools()` and show whether `hello` is registered and active. Tool descriptions matter — the model chooses tools from names and descriptions, so tighten `description` before debugging anything else.

8.6 Stage 3: React to the session lifecycle

Goal: run code when the session starts. The lifecycle you need for this whole tutorial is five lines long:

```
factory loads → registrations complete → session_start
→ user runs /hello-pi, or the model calls hello
→ session_shutdown (on /reload, /new, exit) → (on reload) session_start again
```

Add a `session_start` handler to the factory:

```
export default function register(pi: ExtensionAPI) {
  pi.registerTool(helloTool);
  pi.registerCommand("hello-pi", { /* ...unchanged... */ });

  pi.on("session_start", async (_event, ctx) => {
    ctx.ui.notify("hello-pi extension is ready", "info");
  });
}
```

Run `/reload`.

Expected result: the notification appears right after reload. What actually happened: `/reload` emitted `session_shutdown` for the old extension instance, reloaded resources, then emitted `session_start` with `reason: "reload"` for the new instance. The same pair fires on exit, `/new`, `/resume`, and `/fork` — which is why cleanup code belongs in `session_shutdown` and setup code in `session_start`, never in the factory body. The prompt lifecycle has the full event map for when you outgrow the five-line version.

One reload rule bites every extension author eventually: if a *command handler* itself calls `ctx.reload()`, end the handler there:

```
// Author responsibility: do not resume work on the pre-reload instance.
await ctx.reload();
return;
```

Code after `await ctx.reload()` still runs, but inside the dead copy of your extension, with the old `ctx` and stale captured state. Put post-reload behavior in the newly loaded instance (for example its `session_start` handler). Extension patterns generalizes this to `newSession`, `fork`, and `switchSession`.

8.7 Stage 4: Persist state across reload and restart

Goal: count greetings, and keep the count. Anything your extension keeps in a module-level variable disappears on `/reload`, restart, `/resume`, and `/fork`. `pi.appendEntry(customType, data)` persists a small JSON record into the session file as a custom entry — durable, and never sent to the model — and a `session_start` handler rebuilds the variable from those entries.

Here is the complete extension with all four stages together:

```
import { Type } from "@earendil-works/pi-ai";
import { defineTool, type ExtensionAPI } from "@earendil-works/pi-coding-agent";

const helloTool = defineTool({
  name: "hello",
  label: "Hello",
  description: "Greet a person by name",
  parameters: Type.Object({
    name: Type.String({ description: "The person to greet" }),
  }),
  async execute(_toolCallId, params, _signal, _onUpdate, _ctx) {
    return {
      content: [{ type: "text", text: `Hello, ${params.name}!` }],
      details: { greeted: params.name },
    };
  },
});

let greetingCount = 0;

export default function register(pi: ExtensionAPI) {
  pi.registerTool(helloTool);

  pi.registerCommand("hello-pi", {
    description: "Show a greeting notification",
    handler: async (args, ctx) => {
      greetingCount += 1;
      pi.appendEntry("hello-pi-count", { count: greetingCount });
      ctx.ui.notify(
        `Hello, ${args || "world"}! (greeting #${greetingCount})`,
        "info",
      );
    },
  });

  pi.on("session_start", async (_event, ctx) => {
    // Rebuild in-memory state from durable session entries.
    for (const entry of ctx.sessionManager.getEntries()) {
      if (entry.type === "custom" && entry.customType === "hello-pi-count") {
        greetingCount = (entry.data as { count: number }).count;
```

```
      }
    }
    ctx.ui.notify("hello-pi extension is ready", "info");
  });
}
```

Test it: run `/hello-pi Ada` twice, then `/reload`, then `/hello-pi Ada` again.

Expected result: the counter shows `#3` after the reload — it was rebuilt from the durable entries, not reset. The count also survives a full process restart **when you resume the same session** (`pi --resume`); starting a brand-new session starts a brand-new counter, because entries are session-scoped.

Note that the rebuild scans `getEntries()` — every entry in the session file — so this counter is intentionally *session-wide*. State that should follow the active conversation branch instead rebuilds from `getBranch()` and also on `session_tree`; State that survives restart and branching has that production pattern, and the shipped `todo.ts` example (`packages/coding-agent/examples/extensions/todo.ts`) implements it end to end.

Cleanup when you are done: remove `~/.pi/agent/extensions/hello-pi.ts`, then `/reload`.

8.8 Host safety: UI calls outside the terminal

Stage 1 worked because you were in the TUI. The same UI call behaves differently per run mode:

Mode	`ctx.mode`	`ctx.hasUI`	`notify()`	Dialogs (`select`, `confirm`, …)
Interactive	`"tui"`	`true`	Fire-and-forget message	Wait for the user's response
RPC	`"rpc"`	`true`	Request the client may display or ignore	Wait for the matching client response
JSON	`"json"`	`false`	No-op	Return cancellation defaults (`false`/`undefined`)
Print (`-p`)	`"print"`	`false`	No-op	Return cancellation defaults (`false`/`undefined`)

One rule covers most of it: **check `ctx.hasUI` before any dialog, and decide the noninteractive answer in advance.** In RPC mode a dialog is a protocol message your handler *awaits* — if the client never answers, the handler waits indefinitely, so set a dialog timeout when you cannot guarantee a response. Require `ctx.mode === "tui"` for anything that needs a terminal (custom components, editor replacement). Extension patterns shows the timed-dialog pattern; the exact no-op behaviors are in the official Extensions documentation.

8.9 Debugging extensions

Test a one-off file with `pi -e ./ext.ts`; use `/reload` for an auto-discovered file. Errors appear in Pi output and logs. See Testing, debugging, and contribution for the wider workflow.

Work from loading toward the failing behavior:

1. **Confirm the module loads.** In TUI mode, add a `session_start` notification. Then load the file directly with `pi -e path/to/extension.ts`; if that works, discovery is the problem. Check the exact filename (`extensions/name.ts` or `extensions/name/index.ts`), the default factory export, and project trust for `.pi/extensions/`. A named export alone is not loaded.
2. **Check the code independently.** Jiti transpiles but does not type-check, so type errors can survive loading and fail later. A bare file under `~/.pi/agent/extensions/` has no `node_modules` to resolve Pi's types against, so `tsc` there reports module-resolution noise, not your bugs. The working setup for anything nontrivial is a small extension package with a `package.json` and `tsconfig.json` — then `npx tsc --noEmit -p tsconfig.json` gives real type-checking. For the one-file tutorial, treat `pi -e` load success plus editor diagnostics as the check.
3. **Isolate one boundary.** Exercise one command or one simple tool call. Add temporary notifications to the relevant hooks in TUI mode, or concise logging when testing print mode. Check command names, tool names, schemas, and descriptions before debugging a multi-turn interaction.
4. **Inspect model-facing state.** To see what the model can call, use `pi.getAllTools()` / `pi.getActiveTools()` (for example from a temporary command that prints a notification). To see the assembled system prompt, use `ctx.getSystemPrompt()`. The built-in `/debug` command writes rendered TUI lines plus `session.messages` to `~/.pi/agent/pi-debug.log` — useful for conversation history, but it does not dump tool schemas or the system prompt.
5. **Exercise lifecycle edges.** Run `/reload`, then test a new session, resume, fork, and exit when relevant. Verify `session_shutdown` cleanup and reconstruction in `session_start`.
6. **Test without a TUI.** Run `pi -e path/to/extension.ts -p "your prompt"` to catch unguarded UI assumptions. Notifications do nothing in print mode, so do not use their absence there as evidence that loading failed.

8.9.1 Common pitfalls

Symptom	Likely cause	Fix
Extension never loads	Wrong path, missing default export, or untrusted project	Check discovery conventions, export the factory as default, and establish project trust
Command or tool not found	Name mismatch, or a duplicate name got a numeric suffix	Check the registered name; invoke `/name:1`-style suffixes if duplicated
Model never calls the tool	Weak description, or tool registered but not active	Tighten `description`; check `pi.getActiveTools()`
State disappears after `/reload`	State exists only in closed-over variables	Rebuild from session entries in `session_start` (stage 4)
Dialog hangs in RPC mode	Client never answers the `extension_ui_request`	Send the matching response or set a dialog timeout

Symptom	Likely cause	Fix
Dialog silently denies in print/JSON mode	`ctx.hasUI` was not checked	Define a noninteractive policy before calling the dialog
Extension works once but misbehaves after reload	Stale references or leaked resources	Clean up in `session_shutdown`; `return` right after `ctx.reload()`

Pi logs ordinary handler errors and keeps running, so a throwing hook usually degrades behavior instead of crashing Pi. One boundary is deliberately fail-safe: if a `tool_call` handler throws, that tool call is blocked and the error reaches the model as an error tool result. The exact sequencing is in the Extension event reference; test policy hooks on both allow and deny paths.

8.10 Try it

1. Add argument completion to `/hello-pi` with `getArgumentCompletions` (its type is in `core/extensions/types.ts` in the coding-agent package), suggesting a few names, and confirm the completions appear when you type `/hello-pi` in the editor.
2. Extend the counter to be per person (a map from name to count, persisted in one entry), then `/reload` and `pi --resume` and verify the per-person counts survive both.
3. Add a `session_shutdown` handler that writes a farewell notification, `/reload`, and explain — using the five-line lifecycle from stage 3 — why you see the farewell from the *old* instance before the greeting from the *new* one.

8.11 Where to continue

The runnable example is a demo, not a production safety review. Before shipping an extension, continue with Extension patterns: basic to advanced for safety gates, noninteractive behavior, durable state reconstruction, same-file mutation queues, concurrency, and the verification ladder. The prompt lifecycle is the canonical event map, and the Extension event reference has exact payloads and return values.

9 Extension patterns: basic to advanced

In plain terms: this chapter is a pattern catalog for extensions that are meant to last. Each pattern is a small, copyable answer to one recurring question — "how do I block a dangerous command?", "where does my state go?", "how do I inject a message mid-run?" — presented in a uniform shape: **the problem, the recipe, the failure mode it prevents, and how to verify it.**

Tiers are ordered by coupling and failure risk, not by line count. **Tier 1** hardens behavior Pi already has — a mistake here breaks one tool call. **Tier 2** gives your extension a capability of its own — a mistake here corrupts state or files. **Tier 3** composes and evolves extensions — mistakes here leak across reloads, sessions, and providers. Stop at the smallest tier that solves your task.

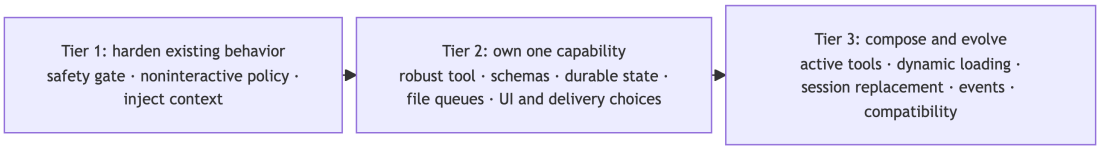

Figure 9.1: Chapter roadmap. Each tier builds on the previous one; stop at the smallest tier that solves your task.

This chapter is the production follow-up to Your first extension. The examples are independent recipes, not a single package to copy wholesale.

9.1 Prerequisites

Finish Your first extension first — this chapter assumes you can write and load a one-file extension with `pi -e path/to/extension.ts`, and that you know what a hook, a session, and a slash command are. Event names below refer to the map in The prompt lifecycle. The snippets reference real repository source files; a checkout of the `pi` repository lets you follow them, but is not required.

One mechanism check before the patterns: use **configuration** for a stable value, an **extension** for behavior that reacts to events or needs Pi APIs, and a **core change** only when every host must share the behavior. Customizing Pi is the canonical guide to that choice; the rest of this chapter assumes you picked the middle row.

9.2 Tier 1 — Harden existing behavior

9.2.1 Gate a dangerous tool call

Problem: the model can propose destructive commands; you want a human in the loop for the ones that match a policy.

Recipe: hook `tool_call`, which fires after schema validation and before execution, and return `{ block: true, reason }` to stop a call. The reason reaches the model as an error tool result it can respond to.

```
// ~/.pi/agent/extensions/block-rm.ts
// Test-drive it with: pi -e ~/.pi/agent/extensions/block-rm.ts
import {
  isToolCallEventType,
  type ExtensionAPI,
} from "@earendil-works/pi-coding-agent";

export default function register(pi: ExtensionAPI) {
  pi.on("tool_call", async (event, ctx) => {
    if (!isToolCallEventType("bash", event)) return;
    if (!event.input.command.includes("rm -rf")) return;

    // Noninteractive hosts cannot show a dialog; choose a safe default first.
    if (!ctx.hasUI) {
      return { block: true,
        reason: "Dangerous command blocked (no UI for confirmation)" };
    }

    const allowed = await ctx.ui.confirm("Dangerous command", "Allow rm -rf?");
    if (!allowed) return { block: true, reason: "Blocked by user" };
  });
}
```

The `isToolCallEventType` guard provides a typed `event.input` for a built-in tool. The shipped `permission-gate.ts` example follows the same order.

Failure modes:

- *The string matcher is illustrative, not a shell policy.* `includes("rm -rf")` misses spacing variants, option ordering, aliases, and scripts. Treat it as a demonstration of the hook, not as command parsing.
- *Blocking is more honest than mutating.* `event.input` is mutable and later handlers see your changes, but Pi does not revalidate after a mutation — reserve mutation for narrow argument patches, not policy.
- *A throwing `tool_call` handler blocks the call* (fail-safe) and the error reaches the model as an error tool result. Test both the allow and deny paths.

Verify: save the snippet, run `pi -e ~/.pi/agent/extensions/block-rm.ts`, and ask Pi to remove a scratch directory with `rm -rf`: the dialog appears, denying stops the command, and the model receives your reason as an error it can respond to. A harmless `ls` never triggers the dialog and runs normally. Repeat with `pi -p` (no UI) to confirm the deny-by-default fallback.

9.2.2 Ask the user without hanging noninteractive hosts

Problem: any gate that asks the user needs an answer even when no user can answer — print and JSON modes return cancellation defaults, and an RPC client that never responds parks your handler forever.

Recipe: decide the noninteractive policy first (deny, or a configured default), then add a timeout or an `AbortSignal` so a silent client cannot wait forever:

```
const confirmed = await ctx.ui.confirm(
  "Timed confirmation",
  "Allow this operation?",
  { timeout: 5000 }, // auto-cancels: confirm() returns false on timeout
);

// Or, to distinguish timeout from user cancel:
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 5000);
const ok = await ctx.ui.confirm(
  "Confirm", "Proceed?", { signal: controller.signal });
clearTimeout(timeoutId);
// ok === false with controller.signal.aborted → timed out, not denied
```

The per-mode behavior of every UI call — TUI, RPC, JSON, print — is in Host safety in the first-extension chapter. Require `ctx.mode === "tui"` for components that need a terminal.

Failure mode: a gate that blocks forever in RPC mode looks like a hung agent; a gate that silently denies in print mode looks like a broken tool. Both are policy bugs, not Pi bugs.

Verify: run the gate under `pi -p` and confirm the noninteractive path executes; run with a short timeout and confirm the deny path fires on time.

9.2.3 Inject focused context

Problem: you want the model to behave differently for some prompts — without rewriting Pi's whole system prompt.

Recipe: `before_agent_start` is the correct point for per-prompt instructions. It can add a custom message (stored in the session, sent to the model) and/or adjust the chained system prompt:

```
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function register(pi: ExtensionAPI) {
  pi.on("before_agent_start", async (event) => {
    if (!event.prompt.startsWith("review ")) return;
    return {
      message: {
        customType: "review-mode",
        content: "Prioritize correctness, regressions, and concise findings.",
```

```
        display: true,
      },
    };
  });
}
```

`display: true` shows the injected message in the transcript; the message reaches the model either way.

Failure mode: *replacing* the system prompt instead of appending can discard skills, tools, context files, and user-provided instructions. Prefer a narrow, conditional append. For a message-level transformation before *every* provider request rather than per prompt, use the `context` event — the prompt lifecycle explains the boundary difference.

Verify: run a prompt with and without the trigger prefix and confirm only the triggered run shows your instruction.

9.3 Tier 2 — Own one tool or stateful capability

9.3.1 A robust custom tool

Problem: your tool works in the demo, then fails in production: huge output floods the context, Esc cannot stop it, and errors confuse the model.

Recipe: Your first extension covers the minimal contract (`name`, `label`, `description`, TypeBox `parameters`, `execute` returning `content`/`details`). The production checklist:

- **Throw from `execute` to report an error** to the model; a returned `isError` field does nothing.
- **Forward `signal`** into `fetch`, process helpers, and long computations so Esc can cancel your work.
- **Use `onUpdate`** for incremental result rendering.
- **Truncate large output yourself** — custom tool output is *not* automatically truncated. The built-in limits are 50 KB or 2,000 lines; use the exported helpers (`truncateHead`, `truncateTail`, `formatSize`, `DEFAULT_MAX_BYTES`, `DEFAULT_MAX_LINES`) and tell the model where the full output went. The shipped `truncated-tool.ts` example wraps `rg` end to end.
- **`terminate: true`** hints that the follow-up model call should be skipped, but it takes effect only when *every* finalized result in the batch terminates — see the prompt lifecycle for the batch rule.
- Add `promptSnippet` for a one-line entry in Pi's default system prompt and `promptGuidelines` for usage bullets. Each guideline must name its tool, because Pi flattens all active tools' guidelines into one section: write `Use hello when the user asks for a greeting,` not `Use this tool when ...`.

Verify: test a malformed call (schema rejection), a cancellation (Esc mid-execution), and an oversized result.

9.3.2 TypeBox schema design

Your first extension introduced the core idea — the schema validates model-supplied arguments at runtime and generates the JSON Schema sent to the provider. The design decisions that remain:

```
import { Type } from "@earendil-works/pi-ai";

// Required string parameter
Type.String({ description: "A description helps the model use it correctly" })

// Optional number
Type.Optional(Type.Number({ description: "Count in seconds" }))

// Required object with typed properties
Type.Object({
  path: Type.String({ description: "File path" }),
  recursive: Type.Optional(Type.Boolean({ default: false })),
})
```

Every parameter deserves a `description`: it is part of what the model reads when deciding arguments. If your tool accepts a path, normalize a leading `@` before resolving it — some models include one, and built-in tools strip it.

9.3.2.1 StringEnum vs Type.Union for string literals

For tools exposed to Google's Gemini API, use `StringEnum` from `@earendil-works/pi-ai` instead of `Type.Union([Type.Literal("a"), Type.Literal("b")])`. The literal union serializes to an `anyOf`/`const` representation that Google's API rejects, so the model may emit values your schema never intended. `StringEnum` emits the plain string `enum` form every provider understands:

```
import { StringEnum } from "@earendil-works/pi-ai";
StringEnum(["small", "medium", "large"], { description: "Size" })
```

9.3.3 State that survives restart and branching

Problem: anything your extension keeps in a variable disappears when Pi reloads, restarts, resumes, or forks.

Recipe: store a snapshot inside something already durable — a tool result's `details` — and rebuild the variable from the session whenever the session (re)starts or the active branch changes. A session is a tree of persisted entries: a branch is one root-to-leaf path, and `getBranch()` returns the active path, excluding sibling branches. Full session semantics are in Sessions, branches, and durable context.

Rebuild on both `session_start` and `session_tree`. `session_tree` fires after `/tree` navigation changes the active branch; fork and resume instead arrive as `session_start` (with `reason: "fork"` / `"resume"`), so the same two handlers cover every path:

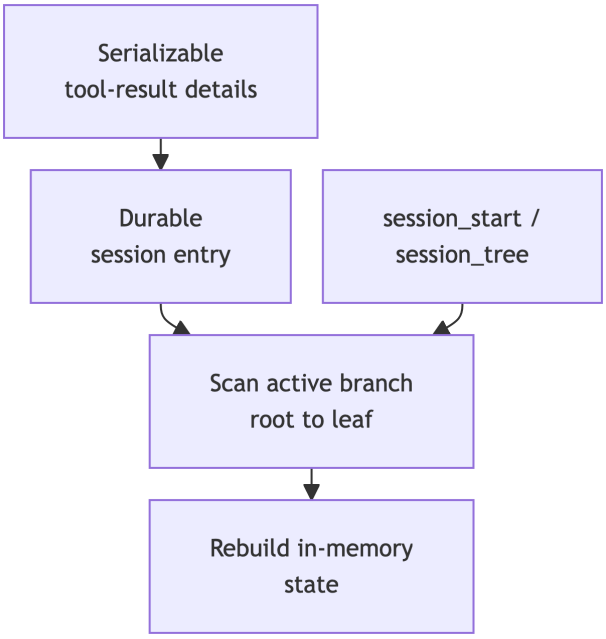

Figure 9.2: Durable state loop. A tool stores a snapshot in its result's details, which Pi persists as a session entry. On session_start and again on session_tree (branch navigation), the extension walks getBranch() and rebuilds its in-memory state from the snapshots.

```ts
// Fragment, inside an existing extension: state, emptyState(), and MyState
// are declared by your module.
function rebuildState(ctx: ExtensionContext) {
  state = emptyState();
  for (const entry of ctx.sessionManager.getBranch()) {
    if (entry.type !== "message") continue;
    const msg = entry.message;
    if (msg.role !== "toolResult" || msg.toolName !== "my_tool") continue;
    const snapshot = msg.details as MyState | undefined;
    if (snapshot) state = snapshot;
  }
}

pi.on("session_start", async (_event, ctx) => rebuildState(ctx));
pi.on("session_tree", async (_event, ctx) => rebuildState(ctx));
```

Reading the scan top to bottom: initialize empty in-memory state, walk the active branch from root to leaf, validate matching tool-result entries, and apply their snapshots in order so the last one becomes the current state. The shipped `todo.ts` extension (`packages/coding-agent/examples/extensions/todo.ts`) implements this loop end to end.

Failure mode: scanning `getEntries()` (the whole session file) instead of `getBranch()` mixes in sibling branches; retaining old state incrementally after navigation mixes in the abandoned branch. Rebuild from the branch, wholesale, every time.

Verify: `/reload`, `/resume`, `/fork`, and `/tree`-navigate; the rebuilt state must match the active branch on every path.

9.3.4 Serialize same-file mutations

Problem: tool calls run in parallel by default. Your custom tool edits `foo.ts` while built-in `edit` also changes `foo.ts` in the same turn; both read the original, and whichever write lands last silently wins.

Recipe: wrap the *entire* read-modify-write window in `withFileMutationQueue()`, the per-path async queue the built-in `edit` and `write` tools already use. Resolve the real target path first — relative to `ctx.cwd` — and use that same absolute path for the queue and every file operation (for existing files the helper canonicalizes through `realpath()`, so symlink aliases share one queue):

```
import { withFileMutationQueue } from "@earendil-works/pi-coding-agent";
import { readFile, writeFile } from "node:fs/promises";
import { resolve } from "node:path";

// Fragment, inside a tool's execute():
const absolutePath = resolve(ctx.cwd, params.path);

await withFileMutationQueue(absolutePath, async () => {
  const current = await readFile(absolutePath, "utf8");
  await writeFile(absolutePath, update(current));
});
```

Verify: ask the model to make two edits to the same file in one turn and confirm both land.

9.3.5 Messages, entries, and delivery modes

Problem: you need to put something into the session — but the four options (model-visible message, transcript-only entry, queued mid-run injection) differ in who sees it and when it lands.

Recipe: choose by visibility first, then by timing:

- `sendMessage()` — a custom message the *model* sees. Never use it for private bookkeeping.
- `appendEntry()` — durable state the model never sees. It renders in the transcript only when paired with `pi.registerEntryRenderer()`; without one it is invisible but still persisted.
- **Message and entry renderers** — control how each looks on screen; the model loop is unaffected either way.

For mid-run injection, `sendMessage()` accepts a `deliverAs` mode — `"steer"` (default), `"followUp"`, or `"nextTurn"`. The exact drain points of the three queues are lifecycle mechanics, covered in Stage 4 of the prompt lifecycle; read that table first. The pattern-level guidance:

- Use `"steer"` to redirect work in progress; `"followUp"` for work that should start only after the current run would otherwise stop; `"nextTurn"` to attach context to the next user-originated prompt without interrupting or starting anything.

- While the agent is idle, `"steer"` and `"followUp"` still only append unless you also pass `triggerTurn: true`:

```
pi.sendMessage(
  { customType: "ci-watch", content: "The CI run failed; investigate.", display: true },
  { deliverAs: "steer", triggerTurn: true },
);
```

`pi.sendUserMessage()` injects a genuine user message and always starts or continues a turn; while streaming you must pass `deliverAs: "steer"` or `"followUp"` — omitting it throws. Do not start a second direct agent run from a hook; use these queues.

Verify: call the `ci-watch` injection from a command handler while idle, then again without `triggerTurn`, and observe the difference.

9.4 Tier 3 — Compose and evolve

9.4.1 Active-tool policy

Problem: you want different tool sets for different working modes (for example read-only review versus full editing).

Recipe: `pi.registerTool()` works after startup, and `pi.setActiveTools()` changes what the model can call next turn. Build on the current selection rather than replacing it blindly:

```
const active = pi.getActiveTools();
pi.setActiveTools([...new Set([...active, "my_tool"])]);
```

Failure mode: `setActiveTools(["read", "bash"])` is a real model-policy change — it rebuilds the system prompt and removes capabilities — not a UI filter. Surface it through a command or shortcut that clearly signals the new mode. Command handlers receive `ExtensionCommandContext`, which adds session-changing methods (`newSession`, `fork`, `navigateTree`, `switchSession`, `reload`, `waitForIdle`); those exist only in commands because calling them from an ordinary event handler can deadlock inside the active agent loop.

9.4.2 Dynamic tool loading: register many, activate few

Problem: your extension ships dozens of tools, but sending every schema on every request wastes context and confuses tool choice.

Recipe: register all tools up front, keep only a small **loader tool** active, and let it activate relevant tools mid-turn. When the change is purely additive, Pi records the added names on the loader's tool result and exposes their definitions before the next model request — with native deferred loading on models that support it, and the normal active-tool list otherwise.

```
// Fragment, inside the factory: SEARCHABLE_TOOL_NAMES is your set of
// initially-inactive tools; loader execution computed `matches`.
pi.on("session_start", () => {
  // Keep searchable tools registered but inactive; preserve built-ins and
  // other extensions' tools; keep the loader itself active.
  const initial = pi.getActiveTools().filter((n) => !SEARCHABLE_TOOL_NAMES.has(n));
  pi.setActiveTools([...new Set([...initial, "search_tools"])]);
});

// Inside the loader tool's execute():
const active = pi.getActiveTools();
const added = matches.filter((name) => !active.includes(name));
pi.setActiveTools([...new Set([...active, ...added])]);
```

The contract: additive changes only (removals work but forfeit deferred loading and can invalidate the provider's cached prompt prefix), and lazily loaded tools should rely on their `description` alone — activating a tool with `promptSnippet`/`promptGuidelines` rebuilds the system prompt, which invalidates the prefix even when the provider supports deferred schemas. The official documentation has a complete `search_tools` example; the shipped `dynamic-tools.ts` covers the simpler command-driven case.

9.4.3 Session replacement without stale contexts

Problem: `ctx.newSession()`, `ctx.fork()`, `ctx.switchSession()`, and `ctx.reload()` tear down the current runtime and spin up a new one. Your handler keeps running in the old call frame, but every captured session-bound object — `ctx.sessionManager`, `pi`, anything your `session_shutdown` handler invalidated — now belongs to a dead runtime and will throw or misbehave.

Recipe: do post-replacement work in the `withSession` callback, which receives a fresh context bound to the replacement session, and capture only plain data (strings, ids, serialized config) into it:

```
pi.registerCommand("handoff", {
  description: "Continue the task in a fresh session",
  handler: async (_args, ctx) => {
    const kickoff = "Continue from the previous session: ..."; // plain data
    await ctx.newSession({
      withSession: async (newCtx) => {
        // Use only newCtx here - never the captured ctx or pi.
        await newCtx.sendUserMessage(kickoff);
      },
    });
  },
});
```

Treat `await ctx.reload()` as terminal for its handler: `return` immediately after it, and put post-reload behavior in the newly loaded instance (for example a `session_start` handler). The official documentation's "Session replacement lifecycle and footguns" section has the full safe/unsafe comparison.

Verify: call the command, then confirm the kickoff message appears in the *new* session and no errors reference the old context.

9.4.4 Inter-extension events with a clean lifecycle

Problem: one extension produces state another consumes, and neither should import the other.

Recipe: `pi.events` is a shared event bus. Namespace your event names so packages from different authors cannot collide — and manage the subscription lifecycle, because the bus survives `/reload` while your extension instance does not:

```
let unsubscribe: (() => void) | undefined;

pi.on("session_start", () => {
  unsubscribe?.(); // defensive: drop any prior subscription
  unsubscribe = pi.events.on("my-ext:indexed", (data) => {
    // Handle the event.
  });
});

pi.on("session_shutdown", () => {
  unsubscribe?.();
  unsubscribe = undefined;
});

// Elsewhere (any extension): pi.events.emit("my-ext:indexed", { count: 17 });
```

Failure mode: `pi.events.on()` returns an unsubscribe function. If you never call it, every `/reload` adds another listener from the old instance, so handlers fire multiple times with stale closures. Events are also ephemeral — not persisted, not replayed — and `emit()` does not await asynchronous listeners, so never use the bus for state that must survive or be acknowledged.

Verify: emit once, `/reload`, emit again, and confirm the handler fires exactly once per event.

9.4.5 Backward-compatible arguments with prepareArguments

Problem: you tightened a tool's schema, and now resuming an old session fails because a stored tool call's arguments no longer validate.

Recipe: `prepareArguments(args)` runs *before* schema validation and before `execute()`. Fold legacy shapes into the modern one there, and strip the legacy fields so what remains is exactly the current schema. Do not add deprecated compatibility fields to `parameters` — that also teaches the model the legacy shape.

```
pi.registerTool({
  name: "edit",
  label: "Edit",
  description: "Edit a single file using exact text replacement",
```

```
  parameters: Type.Object({
    path: Type.String(),
    edits: Type.Array(
      Type.Object({ oldText: Type.String(), newText: Type.String() }),
    ),
  }),
  prepareArguments(args) {
    if (!args || typeof args !== "object") return args;
    const { oldText, newText, ...rest } = args as {
      oldText?: unknown; newText?: unknown;
      edits?: Array<{ oldText: string; newText: string }>;
    };
    if (typeof oldText !== "string" || typeof newText !== "string") {
      return args; // nothing to shim; validation will judge it
    }
    // Modern shape only: legacy fields folded in and removed.
    return { ...rest, edits: [...(rest.edits ?? []), { oldText, newText }] };
  },
  async execute(_id, params) {
    return {
      content: [{ type: "text", text: `Applying ${params.edits.length} edit(s)` }],
      details: {},
    };
  },
});
```

Verify: resume a session containing a legacy-shape call and confirm it validates and executes.

9.4.6 Override a built-in tool

Problem: you need `read`, `bash`, or another built-in to behave differently for every model call — redact secrets from `read` output, route `bash` through a sandbox.

Recipe: register a tool with the same name. Three contracts keep the override invisible to the rest of Pi:

1. **Preserve the built-in result shape**, including the `details` type — the UI and session logic depend on it, or transcripts and renderers break in non-obvious ways.
2. **Renderer inheritance is per slot**: if your override omits `renderCall`, Pi reuses the built-in `renderCall`; same for `renderResult`. Omit both and the built-in UI (syntax highlighting, diffs) keeps working.
3. **Prompt metadata is not inherited** — redefine `promptSnippet` and `promptGuidelines` if the override should keep them.

The built-in implementations and their `*ToolDetails` types are listed in the official documentation, and `tool-override.ts` is a complete `read` override with logging and access control. Built-in tools also accept pluggable `operations` (`ReadOperations`, `BashOperations`, …) for remote execution — the

shipped `ssh.ts` example shows the pattern — and `user_bash` can route the interactive `!`/`!!` shell through the same backends.

Failure mode: prefer a `tool_call` hook (Tier 1) when blocking or observing is enough; override only when the result itself must change.

Verify: confirm the transcript renders exactly as before for overridden calls, then confirm the new behavior.

9.5 Deploy and share

Pi configuration is layered — built-in defaults, global settings under `~/.pi/agent`, project settings under `.pi`, CLI flags and environment, then extension behavior at runtime — and narrower layers win. When a setting "does not apply", check which layer set the value you are actually getting. Use `CONFIG_DIR_NAME` instead of hardcoding `.pi` when constructing project paths (rebranded distributions can rename it), and call `ctx.isProjectTrusted()` before honoring project-local configuration of your own — trust is an input-loading decision, not a sandbox for subsequent tool execution. Resources, settings, and trust is the canonical guide.

A minimal shareable package is a directory with a `package.json` declaring its resources in a `pi` manifest:

```
{
  "name": "@myteam/pi-tools",
  "version": "1.0.0",
  "peerDependencies": {
    "@earendil-works/pi-coding-agent": "*"
  },
  "pi": {
    "extensions": ["./extensions"]
  }
}
```

Manifest fields, dependency placement, install commands, and publishing steps are in Packaging and sharing extensions.

Provider configuration can expose credentials: keep API keys in environment references and never log headers, payloads, or system-prompt options. For `pi.registerProvider()` and provider internals, see Models, providers, and authentication; for custom components, editors, and keybindings, see Terminal UI and interactive mode — never hardcode a raw key check in extension code.

9.5.1 Pattern-specific mistakes

General first-extension pitfalls (loading, naming, discovery) are in Your first extension; exact event contracts are in the Extension event reference. This table keeps only mistakes tied to this chapter's patterns:

Mistake	Why it fails	Better approach
Keep all extension state in a closure	Reload/resume/fork loses it	Rebuild from the active branch (durable state)
Use `sendMessage` for private metadata	The model receives it	`appendEntry` (+ entry renderer if it should render)
Use captured `ctx`/`pi`/`sessionManager` after replacement	They belong to the torn-down runtime	`withSession` with plain captured data (session replacement)
Ignore `AbortSignal` in a tool	Esc cannot stop your nested work	Forward `signal` (robust tool)
Mutate a file without the queue	Parallel siblings lose writes	`withFileMutationQueue` around the whole window (file queue)
Change the built-in result shape in an override	UI/session code depends on it	Preserve the documented `details` contract (override)
Subscribe to `pi.events` without unsubscribing	Every `/reload` adds a stale listener	Unsubscribe in `session_shutdown` (events)
Activate lazy tools with prompt metadata	System-prompt rebuild invalidates the cache prefix	Description-only lazy tools (dynamic loading)
Assume tool hooks observe sibling results	Parallel tools finish independently	Persist/coordinate after the batch or avoid the dependency
Assume `agent_end` means completely idle	Retry, compaction, or queued work may still run	Use `agent_settled` or `waitForIdle()` (prompt lifecycle)
Write to stdout with `console.log`	It corrupts print, JSON, and RPC output	Use notifications or the debug log (run modes)
Test only `pi -e`	It does not prove discovery/reload behavior	Auto-discovery plus `/reload` (verification ladder)

9.5.2 A practical verification ladder

Use evidence appropriate to what you changed:

1. Type-check the extension with the same TypeScript version/API contracts used by Pi, especially tool parameters and event return values. From an extension package with a `tsconfig.json`, run `npx tsc --noEmit -p tsconfig.json`.
2. Load it with `pi -e path/to/extension.ts` to isolate load errors.
3. Move it to an auto-discovered location — for example `~/.pi/agent/extensions/my-extension.ts` — restart Pi, and verify `session_start` ran (add a temporary `ctx.ui.notify` or debug-log line).
4. Exercise the happy path and the relevant failure/deny/cancel path.
5. Run `/reload` in the session, then repeat step 4. Confirm no duplicate resource, stale closure, or leaked status remains.
6. For persistence, exit and start a new process, resume with `pi --resume` (or fork with `/fork` inside the session), and verify the intended branch state is restored.
7. For tools, test a malformed call/schema failure and a cancellation. For mutating tools, test concurrent sibling calls if applicable.

8. If you change repository code, run `npm run check` and the relevant targeted test rather than a real-provider test. The coding-agent suite uses a faux provider and `test/suite/harness.ts` for deterministic tests; as covered in Testing, debugging, and contribution workflow, the faux-provider harness is also the practical way to test an extension's behavior without real API keys.

The repository ships extension examples in `packages/coding-agent/examples/extensions/`. Start with the smallest match:

- `hello.ts`: a tool
- `permission-gate.ts`: a safety gate
- `input-transform.ts`: input routing
- `entry-renderer.ts`: transcript-only state
- `dynamic-tools.ts`: runtime tool changes
- `subagent/`: a larger composition

9.6 Try it

1. **(Tier 1)** Run the safety-gate recipe and test all three paths: dialog approve, dialog deny, and the no-UI fallback under `pi -p`. Then shorten the confirm timeout to 2 seconds and confirm the gate denies on timeout rather than hanging.
2. **(Tier 2)** Take the extension you wrote for Your first extension and move its counter onto the branch-aware rebuild pattern; then `/fork`, `/tree`-navigate back, and verify the state matches the active branch on every path.
3. **(Tier 3)** Write a `ci-watch`-style extension that calls `pi.sendMessage(..., { deliverAs: "steer", triggerTurn: true })` from a command handler, and a second extension that listens for a `ci-watch:fired` event on `pi.events`. `/reload` twice, fire the command, and confirm the listener ran exactly once — if it ran more than once, your unsubscribe lifecycle is missing.

10 Packaging and sharing extensions

In plain terms: an extension can grow from one file into a directory, and then into an npm or git package that bundles extensions, skills, prompts, and themes together. Packaging buys you versioning, reuse across projects, and a way to share a whole setup with a team through one install command. Packages are delivery mechanisms for trusted code and instructions, so review a package's source before installing it.

Related: Customizing Pi (trust, package filters, settings scopes) · Your first extension (the extension this chapter packages)

This chapter is grounded in `packages/coding-agent/src/core/package-manager.ts`; the user-facing reference is `packages/coding-agent/docs/packages.md`.

10.1 Prerequisites

Finish Your first extension first: you should have a working hello-pi extension and know how to load it. You also need `npm` installed (verify with `npm --version`) and, for the publish step, an npm account. All commands in this chapter run in a regular terminal, not inside an interactive Pi session.

10.2 The package lifecycle at a glance

Figure 10.1: Package lifecycle: the author declares resources in a `pi` manifest and publishes; the user installs from a source; the resource loader discovers the package's resources, subject to the same trust gate as other project-local resources.

The rest of this chapter walks each stage: authoring the manifest, installing from each source type, and managing installed packages.

10.3 Distribute an extension as a Pi package

For a reusable package, declare explicit resources under the `pi` key in `package.json`:

```
// package.json
{
  "name": "@acme/pi-team-tools",
  "keywords": ["pi-package"],
  "peerDependencies": {
    "@earendil-works/pi-ai": "*",
    "@earendil-works/pi-agent-core": "*",
    "@earendil-works/pi-coding-agent": "*",
    "@earendil-works/pi-tui": "*",
    "typebox": "*"
  },
  "pi": {
    "extensions": ["./extensions"],
    "skills": ["./skills"],
    "prompts": ["./prompts"],
    "themes": ["./themes"]
  }
}
```

The dependency fields follow three rules:

- **Third-party runtime packages** (anything your code imports that is not Pi itself) belong in `dependencies`, not `devDependencies`. The default npm/git checkout path uses `npm install --omit=dev`, so a package that only declares `devDependencies` breaks at load time; a configured custom `npmCommand` can change the install behavior, but it does not make runtime imports belong in `devDependencies`.
- **Core Pi packages are supplied by the host Pi install.** List each one your code actually imports as a `peerDependency` with a `"*"` range instead of bundling a competing copy. The broad manifest above assumes a full-featured package that imports all five bundled packages; a smaller package lists only what it imports. Two common judgment calls: include `@earendil-works/pi-tui` only when code imports TUI components or TUI types for custom UI/rendering, and include `typebox` only when code imports schemas directly from `typebox` rather than using the `Type` re-export from `@earendil-works/pi-ai`.
- **The `pi` manifest is optional but recommended.** Without it, Pi auto-discovers the conventional `extensions/`, `skills/`, `prompts/`, and `themes/` directories. An explicit manifest is clearer, supports non-standard layouts, and gives package filters a declared baseline to narrow.

The `pi-package` keyword is an npm search and package-gallery convention, not something Pi reads: Pi loads a package from its declared source and manifest, never from keywords. Keep the keyword so others can find the package on npmjs.com and in the community package gallery.

10.4 Install sources

Users install a package with `pi install <source>`, or list the same source string under the `packages` array in settings. Four source forms are accepted:

Source form	Examples	Installed to	`pi update` behavior
npm (`npm:` prefix)	`npm:@foo/bar`, `npm:@foo/bar@1.2.3`	`~/.pi/agent/npm/` (global), `.pi/npm/` (project)	Exact version: pinned, skipped. Range or unpinned: latest allowed version
git shorthand (`git:` prefix)	`git:github.com/user/repo`, `git:git@github.com:user/repo`	`~/.pi/agent/git/<host>/<path>` or `.pi/git/...`	Re-fetches the clone to the configured ref; never moves the ref
git protocol URL	`https://github.com/user/repo`, `ssh://git@github.com/user/repo`	Same as git shorthand	Same as git shorthand
Local path	`/abs/path/to/pkg`, `./relative/path`	Resolved in place, not copied	Never updated

Any source that does not start with `npm:` or `git:` and is not an `http(s)://`, `ssh://`, or `git://` URL is treated as a local path; relative paths resolve against the settings file they appear in. A local-path source can also point at a single `.ts` file, which loads as one extension.

To try a package without installing it, pass it to `pi -e` (for example `pi -e npm:@foo/bar`): Pi installs it to a temporary directory for that run only.

Project-installed packages are trust-gated just like project-local extensions: Pi installs missing project packages on startup only after the project is trusted. The install command adds the source to the `packages` array in settings (global by default, project settings with `-l`), which you can equivalently edit by hand:

```
// .pi/settings.json (project scope)
{
  "packages": [
    "npm:@acme/pi-team-tools@1.2.3",
    "git:github.com/acme/review-pack"
  ]
}
```

The first entry is pinned to an exact version; the second tracks the repository's configured ref. Version pinning is covered under Manage installed packages below.

10.4.1 Step-by-step: from zero to a shareable extension package

The following example creates a minimal distributable package outside the Pi monorepo.

Step 1 – Initialize a package

```
mkdir pi-hello-demo
cd pi-hello-demo
npm init -y
```

Step 2 – Add the extension code

```
mkdir extensions
```

Create `extensions/index.ts` with the hello-pi extension built in Your first extension — the one that registers a `/hello-pi` command and shows a startup notification.

Step 3 – Configure `package.json`

Edit `package.json` to add:

```
{
  "name": "@your-name/pi-hello-demo",
  "keywords": ["pi-package"],
  "peerDependencies": {
    "@earendil-works/pi-ai": "*",
    "@earendil-works/pi-coding-agent": "*"
  },
  "pi": {
    "extensions": ["./extensions"]
  }
}
```

The `pi` manifest declares its resource directories, and `peerDependencies` identifies the Pi runtime packages supplied by the host. This list is intentionally shorter than the earlier generic manifest: the hello extension imports only `@earendil-works/pi-ai` and `@earendil-works/pi-coding-agent`. Add `@earendil-works/pi-agent-core`, `@earendil-works/pi-tui`, or `typebox` only if the package imports them.

> **Note**
>
> Publishing under a scoped name such as `@your-name/...` requires an npm account where the scope matches your username or an organization you belong to; otherwise `npm publish` fails with a permissions error. For a first end-to-end run, skip publishing and use the local-path install in Step 4.

Step 4 – Test locally

Switch to a project directory where you use Pi (the *consumer* project, not the package directory) and install the package from its local path:

```
pi install /absolute/path/to/pi-hello-demo
```

A relative path such as `pi install ../pi-hello-demo` works as well. Local-path packages are resolved in place, not copied, so edits to the source are picked up on the next start or `/reload`.

Start Pi in that project. Your extension should load. Test with `/hello-pi` and verify the startup notification appears — the same behavior the extension had when you loaded it directly in Your first extension, now delivered through a package.

Step 5 – Publish to npm

```
npm publish
```

Users can then install it with:

```
pi install npm:@your-name/pi-hello-demo
```

`pi install` also accepts git sources (see the source table above) for testing before publishing.

10.5 Manage installed packages

Once a package is installed, three commands inspect, remove, and update it.

`pi list` prints the packages configured in user settings and, separately, in project settings, each with its installed path:

```
User packages:
  npm:@acme/pi-team-tools
    /Users/alice/.pi/agent/npm/node_modules/@acme/pi-team-tools

Project packages:
  git:github.com/acme/review-pack (filtered)
    /Users/alice/work/team-app/.pi/git/github.com/acme/review-pack
```

A package is marked `(filtered)` when its settings entry uses the object form to restrict which resources load. A filter layers on top of the package's `pi` manifest and narrows it: omit a resource key to load everything of that type, set it to `[]` to load none, and use `!pattern` exclusions (or `+`/`-` exact paths) for finer control:

```
// .pi/settings.json (project scope)
{
  "packages": [
    {
      "source": "git:github.com/acme/review-pack",
      "skills": ["skills/review/SKILL.md"],
      "extensions": []
    }
  ]
}
```

This entry loads the listed skill, all prompts, and all themes (omitted keys load everything of that type) but no extensions at all — the filter is how a team adopts a package's prose resources without running its code. Settings scopes and array merge behavior for `packages` follow the rules in Customizing Pi.

`pi remove <source>` removes a package and deletes its source from settings (`pi uninstall` is an alias). Pass `-l` to remove from project settings instead of global settings. The source argument must match the configured source; if nothing matches, the command reports `No matching package found` and suggests a similar configured source when one exists.

`pi update` covers three targets:

Command	Target
`pi update` (also `pi update --self`, `pi update pi`)	Pi itself; `--force` reinstalls even when the current version is the latest
`pi update --extensions`	All installed packages
`pi update <source>` or `pi update --extension <source>`	One package
`pi update --models`	Model catalogs only (see Models, providers, and authentication)
`pi update --all`	Pi and all packages

Version pinning controls what an update touches. The short version: an exact version means frozen until you change the setting. Concretely:

- An npm source with an exact version, such as `npm:@foo/bar@1.2.3`, is **pinned**: `pi update` skips it, and resolution reinstalls it only if the installed copy no longer satisfies the configured version.
- A range such as `npm:@foo/bar@^1.2.0` updates to the latest version within that range; an unpinned source tracks the latest release.
- For git sources, an `@ref` is the configured checkout target rather than an update lock: an update resets the clone to exactly that ref ("reconciles" it) instead of moving to newer commits.
- Local-path packages are never updated because they are resolved in place.

Pin an exact version for published packages whose updates you want to review before they land.

10.6 Resource delivery

Packages may ship skills, prompts, and themes alongside extension code. Their resource paths and package filters follow the settings scope; use explicit manifest paths when the layout is non-standard.

> ⚠ Warning
>
> A package is not a sandbox. Its extension code runs with the same privileges as Pi itself, and its skills can instruct the model to run commands. Review a package's source before installing it, and treat a project settings file that adds packages with the same care as contributed code.

10.6.1 Package layout conventions

Directory	Contents
`extensions/`	One or more `.ts`/`.js` extension factories
`skills/`	`SKILL.md` folders (found recursively) and top-level `.md` files
`prompts/`	`.md` prompt template files
`themes/`	`.json` theme files

Directory	Contents

All four directories are discovered automatically when the package has no `pi` manifest. With an explicit manifest, Pi uses only the declared paths, which supports non-standard locations and lets a package exclude conventional directories.

Before sharing, test the installed form, pin or review dependencies as appropriate, and document for your users whether the package expects project trust — the trust model is covered in Trust is an execution policy, not a cosmetic prompt.

10.7 Try it

1. Work through Steps 1–4 above and confirm `/hello-pi` works when delivered from a local-path package install.
2. Add a skill to the demo package: create `skills/demo/SKILL.md` with a `description` frontmatter field (the format from Customizing Pi), add `"skills": ["./skills"]` to the `pi` manifest, run `/reload` in the consumer project, and confirm the skill appears as `/skill:demo`.
3. Run `pi list` and find your package. Then change its settings entry to the object form with `"extensions": []`, run `pi list` again, and confirm it is now marked `(filtered)` and that `/hello-pi` no longer works after `/reload`. Restore the entry when done.

Part V

Subsystem deep dives

11 Built-in tools: filesystem, shell, and discovery

This is reference depth for extension patterns; read it when a recipe needs the built-in tool contract or mutation queue details.

Pi ships seven named tools: `read`, `bash`, `edit`, `write`, `grep`, `find`, and `ls`. Together they cover filesystem access and mutation, shell execution, and bounded repository discovery. The session's **cwd** (current working directory) is the base directory used to resolve relative tool paths; it is not necessarily the process directory of the host embedding Pi. Each built-in is a tool definition with a TypeBox parameter schema that Pi wraps as an `AgentTool`, placing built-ins and extension-provided tools in the same agent loop lifecycle.

Implementation lives in `packages/coding-agent/src/core/tools/`. The public construction helpers are re-exported from the `@earendil-works/pi-coding-agent` package. The chapter focuses on behavior that matters when using, extending, or debugging the tools rather than teaching shell commands.

11.1 Start with the tool contract

Before reading `read` or `bash`, learn the shared shape: a **schema** describes valid inputs, a **description** tells the model when to use the tool, an `execute` function performs the work, and optional **renderers** show the call and result in the TUI. For example, both `read` and a custom `listTodos` tool need validated input and an actionable result, even though one uses the local filesystem and the other may use a database.

The chapter teaches the contract first, then compares read-only work with mutating work and short commands with streaming commands. That order makes the queue, truncation, and cancellation rules easier to recognize in each tool.

11.2 Tool registry and default selection

`core/tools/index.ts` is the tool factory map. It defines the `ToolName` union and dispatches each name to a `create*ToolDefinition` or `create*Tool` helper.

```
tool definition: schema + description + execute + optional renderers
       |
       v
wrapToolDefinition(...)
       |
       v
@earendil-works/pi-agent-core AgentTool: validated arguments + callback
```

In prose, Pi combines a schema, description, execution function, and optional renderers, then wraps that definition as the validated `AgentTool` consumed by the agent loop.

Pi's SDK creates all definitions so extensions can inspect their metadata, but the initial active set contains the four coding tools: `read`, `bash`, `edit`, and `write`. `grep`, `find`, and `ls` remain available definitions outside that default active list. Explicit tool selection, `--no-builtin-tools`, extensions, or `pi.setActiveTools()` can change what the model receives. For example:

```
const active = pi.getActiveTools();
pi.setActiveTools([...new Set([...active, "grep", "find", "ls"])]);
```

This changes the model-facing tool set; it is not merely a UI filter.

11.3 Shared design principles

Every built-in tool follows the same broad contract:

1. A TypeBox schema validates the model's arguments in `agent-loop.ts`.
2. `execute` receives an ID, parsed params, an optional abort signal, and an optional incremental-update callback.
3. Success returns model-facing content plus optional structured `details`.
4. Failure throws; the agent loop converts it into an error tool result.
5. Optional renderers turn call/result data into a compact TUI row.

Tool descriptions and prompt metadata directly shape model behavior. `promptGuidelines` is an optional list of instruction bullets on a tool definition. When rebuilding the system prompt, Pi includes the snippets and `promptGuidelines` from active tools. For example, `read` tells the model to inspect files with `read` rather than `cat` or `sed`; `write` reserves full-file output for new files or complete rewrites. This prose is separate from the tool's parameter schema: the schema is sent in the provider `Context.tools` field, while snippets and guidelines become system-prompt text. See The agent loop in one picture for the complete assembly and continuation path.

11.4 Output limits protect the next provider turn

Large tool output can consume the same context window needed for reasoning and response. `truncate.ts` defines a shared 50 KB / 2,000 line limit, along with head/tail/line helpers and metadata describing why output was truncated.

The correct truncation direction depends on the tool:

Tool use	Useful part	Typical policy
File content or search result	Beginning/ordered matches	Keep head
Long-running command log	Latest state/error	Keep tail
Individual match line	Prefix is usually enough	Truncate line

Built-in tools report truncation to the model and provide an actionable next step. Custom tools should pair every truncated result with the same explicit status and recovery guidance so the model can continue without mistaking partial output for a complete result.

11.5 `read`: text ranges and image attachments

`read.ts` accepts a required `path` plus optional 1-indexed `offset` and `limit`. It resolves the path relative to the session cwd, checks readability, then branches on MIME (media type) detection.

For text files, `read` splits lines, applies the requested range, and then applies shared truncation. The continuation messages are deliberately specific:

```
[Showing lines 1-2000 of 6500. Use offset=2001 to continue.]
```

If a single first line exceeds the byte limit, the `read` result reports that condition and suggests a bounded shell fallback. The explicit notice accurately marks the line as partial.

For supported images (JPEG, PNG, GIF, WebP, BMP), `read` loads the binary data, processes and resizes the image subject to settings, and returns an image content block alongside a text note. The tool checks the current model's input capabilities and adds a note when the model cannot receive images. The global `images.blockImages` setting adds another later defense in the conversion path.

11.5.1 Read operations are pluggable

`ReadOperations` has `readFile`, `access`, and optional MIME detection. The default uses the local filesystem. An extension can build a read tool backed by SSH, a container, or another remote system by providing those operations. The schema and TUI behavior stay the same; only the I/O boundary changes.

Providing `ReadOperations` preserves the existing schema and TUI behavior while redirecting filesystem access, avoiding a duplicate read implementation.

11.6 `write`: create or replace a complete file

`write.ts` accepts `path` and full `content`. It creates parent directories recursively, overwrites existing content, and returns a byte count. Its prompt guideline explicitly reserves it for new files or complete rewrites.

The implementation wraps the complete operation in `withFileMutationQueue`:

```
resolve target path
  -> wait for same-file queue
  -> check abort
  -> mkdir parent
  -> check abort
  -> write file
```

```
  -> check abort
  -> release queue
```

The queue is keyed by a resolved, canonical path. Existing paths are passed through `realpath`, so symlink aliases share a queue; missing target paths use the resolved absolute path. Different files still execute in parallel.

The sequence above checks abort three times—before `mkdir`, after `mkdir`, and after `writeFile`—and holds the same-file queue until each in-flight filesystem operation settles. A custom mutating tool should preserve this ordering so the next writer starts from settled filesystem state.

11.7 `edit`: exact replacement and queued mutation

`edit.ts` is for targeted changes to an existing file. Its schema is built around a path and edit blocks with exact old/new text. The edit first tries an exact match, then a constrained Unicode/whitespace normalization fallback. It detects ambiguous or stale replacements rather than applying an approximate structural patch to the wrong location.

Like `write`, `edit` resolves the target and wraps the full read-modify-write window in `withFileMutationQueue`. If a custom read-modify-write tool queues only its final write, two parallel calls can both read old content, derive conflicting replacements, and lose one change. The lock must cover reading, validation, transformation, and writing.

Use `edit` rather than `write` when preserving unrelated content is important. Use `write` when creating a file or deliberately replacing its entirety. The model prompt guidelines encode this distinction to reduce destructive rewrites.

11.8 `bash`: process execution with streamed tail output

`bash.ts` accepts a command and optional timeout in seconds. It uses `createLocalBashOperations()` by default, which resolves the configured shell, verifies the cwd, and starts a child process with stdout/stderr pipes. On non-Windows systems the child starts as a detached process group. Cancellation and timeout call `killProcessTree`, which invokes `taskkill /F /T` on Windows; on non-Windows systems it sends `SIGKILL` to the process group and falls back to killing the child process.

The tool can apply a configured `shellCommandPrefix` and an extension-provided `spawnHook` that changes the command, cwd, or environment. These are command customization and routing hooks, not security boundaries. For enforced containment, provide `BashOperations` whose `exec` implementation enters the sandbox or remote environment, and replace the filesystem tools' operations so `read`, `write`, `edit`, `find`, and `ls` use the same boundary. `GrepOperations` only replaces grep's path checks and context-file reads; the actual search still spawns the local `rg` executable, so it is not by itself a remote/sandbox search boundary. Apply the boundary to `user_bash` as well. A `tool_call` hook can add policy checks, but operating-system or container controls must enforce isolation.

11.8.1 Streaming output

`OutputAccumulator` collects stdout and stderr. While a process runs, `bash` throttles partial `onUpdate` events to roughly one every 100 ms. The TUI can show a live preview while the model waits for final output.

Final command output keeps the tail. If output was truncated, the tool saves the full output to a temporary file and includes that path in `details` and a model-facing notice. Nonzero exit status, timeout, and abort become thrown errors with accumulated output appended where available. The agent loop then returns that failure as an error tool result to the model.

A failed command therefore remains actionable: the model receives the log and exit status along with the error instead of only an exception string.

11.9 `grep`, `find`, and `ls`: bounded repository discovery

`grep` and `find` use the cwd and apply their search ignore behavior. `ls` directly reads directory entries and includes dotfiles; it does not apply `.gitignore` filtering. Their schemas make the intended scope explicit:

Tool	Important inputs	Default limit	Result shape
`grep`	pattern, optional path/glob, literal/case/context options	100 matches	File path and line-numbered matching lines
`find`	glob pattern, optional path	1,000 results	Matching paths relative to search directory
`ls`	optional directory path	500 entries	Alphabetical listing, directory suffix `/`, includes dotfiles

All three also use the shared byte cap. `grep` additionally limits long output lines. The tools are better suited to agent context than raw `rg`/`find` shell output because their limits and formatting are predictable.

Pi exposes all three as built-in names that an extension can register and activate. When overriding one, retain its result details and renderer contract unless the extension also supplies every affected display and session behavior.

11.10 Paths and remote operations

Tools must distinguish what the model typed from the actual target path. The built-ins resolve relative paths against the session cwd, not an arbitrary process cwd, and format paths for display relative to the session when useful.

Remote execution can preserve structured tool calls. The exported operations interfaces make most tool I/O replaceable; `GrepOperations` is only a partial boundary because search itself remains local:

```
ReadOperations    readFile + access + optional MIME (media type) detection
WriteOperations   mkdir + writeFile
EditOperations    readFile + writeFile + access
BashOperations    exec(command, cwd, { onData, signal, timeout, env })
GrepOperations    isDirectory + readFile (search still uses local rg)
Find/Ls           operation interfaces for filesystem behavior
```

The file-operation lines name focused filesystem methods, while `bash` receives an execution method with streaming and cancellation inputs. For grep, only path checks and context reads are replaceable; the search process still runs the local `rg` executable.

An extension can create a tool with remote operations where the interface covers the full operation, register it under the same name, and preserve Pi's validation and rendering behavior. Do not treat `GrepOperations` alone as a remote search or sandbox boundary.

11.11 User shell commands are separate from model `bash`

The separate `user_bash` path matters when an extension must govern both direct user commands and model-generated shell calls.

In interactive mode, input beginning with `!` runs a user shell command; `!!` marks its result excluded from LLM context. The extension `user_bash` event can provide custom operations or a complete result for the user-command path.

User shell commands and assistant `bash` calls follow distinct paths:

```
user types !command -> user_bash event -> user shell
model requests bash -> tool_call -> tool_execution_start -> bash -> tool result
```

The user directly requests a `!` command, while the model generates a tool call that Pi records in the assistant/tool-result protocol. A sandbox extension should cover both paths by supplying sandboxed operations for built-in tools and `user_bash`.

11.12 Tool rendering is a first-class interface

Built-in definitions include `renderCall` and `renderResult` functions. The interactive tool row receives current arguments, partial/final state, expanded flag, prior component, `executionStarted: boolean`, and an `invalidate()` callback. It does not expose elapsed execution time. Those inputs support syntax-highlighted read/write previews, compact default rendering with an expand hint, streamed shell output, and error-specific presentation.

An extension that overrides built-in execution can omit a renderer and inherit the built-in renderer per slot. Renderer inheritance requires the expected call argument and result-detail shapes. When an override changes either shape, provide compatible renderers or a self-contained rendering shell.

11.13 Tool lifecycle and extension interception

The agent loop emits this sequence for each tool call:

```
tool_execution_start
  -> schema validation and tool_call extension gate
  -> execute / tool_execution_update ...
  -> tool_result extension patch
  -> tool_execution_end
  -> final toolResult message events
```

In prose, Pi validates and gates a call before execution, emits zero or more updates while it runs, allows extensions to patch the result, then emits the end event and final protocol messages.

In default parallel mode, start/preflight is source-ordered, execution/update events can interleave, and final tool-result messages are emitted in assistant source order. Custom tools should coordinate through explicit shared state or lifecycle events instead of expecting an earlier sibling tool result to already appear in the extension context's `ctx.sessionManager` (`ReadonlySessionManager` over the durable session tree) during a concurrent `tool_call` handler.

11.14 A safe custom file-mutation skeleton

The following focused example shows the required same-file mutation ordering. A production tool also needs the schema, result, rendering, and test concerns listed after the code.

```ts
import { withFileMutationQueue } from "@earendil-works/pi-coding-agent";
import { readFile, writeFile } from "node:fs/promises";
import { resolve } from "node:path";

async function replaceOnce(
  cwd: string,
  path: string,
  oldText: string,
  newText: string,
  signal?: AbortSignal,
) {
  const target = resolve(cwd, path);

  // Keep read -> validate -> transform -> write inside the same-file queue.
  return withFileMutationQueue(target, async () => {
    if (signal?.aborted) throw new Error("Operation aborted");

    const current = await readFile(target, "utf8");
    if (signal?.aborted) throw new Error("Operation aborted");

    if (oldText.length === 0) throw new Error("Expected text must not be empty");
    const match = current.indexOf(oldText);
```

```
    if (match === -1) throw new Error("Expected text was not found");
    if (current.indexOf(oldText, match + oldText.length) !== -1) {
      throw new Error("Expected text must occur exactly once");
    }

    const updated =
      current.slice(0, match) + newText + current.slice(match + oldText.length);
    if (signal?.aborted) throw new Error("Operation aborted");

    await writeFile(target, updated, "utf8");
    if (signal?.aborted) throw new Error("Operation aborted");
  });
}
```

In a real tool, add a TypeBox schema, precise description, result content, truncation where relevant, and tests for no-match, abort, and concurrent calls.

11.15 Common tool failures

Symptom	Likely cause	First check
`read` says the file is missing	wrong cwd or path	resolve the path from the tool's cwd
`edit` rejects the operation	old text is not unique or no longer matches	inspect the current file and use a precise replacement
`bash` times out	command is long-running or ignored the signal	pass cancellation and choose an appropriate timeout
`grep`/`find` returns nothing	ignore rules or glob mismatch	verify the pattern and search root

11.16 Tool-author review questions

Before exposing a custom tool to a model, answer:

1. Does the TypeBox schema allow only safe inputs?
2. Is the tool active only when its prompt guidance is appropriate?
3. Does every large output have a bounded, actionable representation?
4. Does cancellation reach all long-running work?
5. Are writes serialized across aliases of the same target?
6. Does a thrown error give the model enough detail to recover?
7. Are `details` durable and renderer-compatible?
8. Does the behavior work in parallel with built-in tools?
9. Is a policy gate better implemented as `tool_call` interception than inside the tool itself?

Treat tools as a public protocol connecting the model, session, UI, and extension ecosystem.

12 Models, providers, and authentication

This is reference depth for provider-adjacent extension patterns; read it when you need models, authentication, or custom providers.

Use this chapter when a model is missing, a request fails auth, custom `models.json` overrides do not apply, thinking levels look wrong, cost/footer numbers surprise you, or you need to understand how Pi turns a selected model into a provider request.

Pi keeps four provider concerns separate:

```
model metadata   -> what a model can do and how much it costs
provider config  -> where/how requests are sent
credentials      -> how a provider is authenticated
API adapter       -> how normalized Pi context becomes provider protocol
```

The separation lets the coding agent present one model selector and one agent loop while supporting OpenAI-compatible servers, Anthropic, Google, and AWS Bedrock (AWS's managed model service), plus OAuth (browser- or device-based authentication), proxies, and extension-defined providers. Read Resources, settings, and trust before changing credentials or project-local config. Streaming is the adapter's delivery path for that protocol, not a separate product surface: adapters emit a normalized event stream that the agent consumes.

Key source files include:

- `packages/ai/src/types.ts`, `packages/ai/src/compat.ts`, and `packages/ai/src/models.ts`;
- `packages/coding-agent/src/core/model-runtime.ts` (canonical internal owner), `packages/coding-agent` (extension-facing facade), and `packages/coding-agent/src/core/auth-storage.ts`.

12.1 Vocabulary and a concrete comparison

A **model** is the named capability record Pi selects. A **provider** is the service or adapter that can answer requests. A **credential** proves access to that provider. An **API adapter** translates Pi's normalized request into one provider's protocol format. In a local setup, `local-model` is a model ID, `local` is a provider name, an environment variable is a credential source, and `openai-completions` is an adapter choice. These are four different values, not four names for the same thing.

Read the chapter in that order: metadata, registry, credentials, dispatch, then custom providers. It prevents an authentication problem from being mistaken for a model-catalog problem.

12.2 The `Model` contract

A model is a data record consumed by the agent and provider layer. Its important fields include provider identifier, model ID, API type, input capabilities, reasoning/thinking support, context window, maximum output tokens, cost, base URL, headers, and compatibility flags.

The model's `api` field chooses an adapter such as `openai-completions`, `openai-responses`, `anthropic-messages`, `google-generative-ai`, or `bedrock-converse-stream`. The provider name is not enough: two provider names can use the same wire protocol, and one provider can expose models with different API requirements.

Use `input: ["text", "image"]` only when a model can actually receive image content. That metadata drives UI/tool behavior and protects against sending an image to a text-only model.

12.3 Model registry assembly

`ModelRuntime` is the canonical internal owner of composed providers and model catalogs. Its composition order is generated built-in models, `models.json` routing and custom-model upserts, extension model application, OAuth `modifyModels()` adjustments, and `models.json` `modelOverrides` last. `ModelRegistry` is an extension-facing compatibility facade over this runtime, not the internal owner of the request path.

```
base built-in models
  -> models.json routing/configuration and custom-model upserts
  -> extension model application
  -> OAuth modifyModels()
  -> models.json modelOverrides (last)
  -> composed provider models
```

> **Failure isolation:** If `models.json` cannot be read, parsed, or validated, the runtime records the error for the interactive application but still loads the generated built-in catalog. The failed load contributes no custom models or catalog overrides. A typo in a local override can therefore disable that file's customizations without making every known built-in model disappear.

`ModelRuntime.reloadConfig()` rebuilds composed providers while retaining the registered extension and OAuth registration maps, then reapplies those registrations. It does not simply reset dynamic registrations.

12.4 Selecting an initial model

At session construction, `createAgentSession()` considers a session non-empty when its reconstructed context contains at least one message. Header and settings entries alone do not count. For such a session, it first tries the provider/model recorded on the current branch; if that model exists and has configured authentication, Pi restores it. Otherwise `findInitialModel()` selects a configured/default available model and Pi reports the fallback. The thinking level is restored from the branch when a thinking entry exists; otherwise construction adds a missing thinking entry using the configured default.

New sessions append both the initial model and thinking-level entries; resumed sessions restore branch state rather than always appending both.

12.5 Thinking levels are Pi vocabulary

Pi's model-facing thinking levels are `off`, `minimal`, `low`, `medium`, `high`, `xhigh`, and `max`. Providers use different APIs and vocabulary. A model's `thinkingLevelMap` is a three-way map from those Pi levels to provider-specific values: each level can be omitted (use the adapter default), a string (send a provider-specific value), or `null` (unsupported):

Value	Support	Selector / clamp	Wire value
omitted key	Standard levels through `high` remain available and use the adapter/provider default; `xhigh` and `max` are unsupported until mapped	Hidden for `xhigh`/`max`; otherwise kept	Adapter default for that Pi level
string	Level is supported	Shown; not clamped away	That string is what adapters send when the level is selected
`null`	Level is unsupported	Hidden/skipped; session clamps requests away from it	Never serialized as a provider thinking value

Maps may contain holes. For example, a model can expose `high` and `max` without `xhigh`:

```
{
  "id": "custom-reasoner",
  "reasoning": true,
  "thinkingLevelMap": {
    "minimal": null,
    "low": null,
    "medium": null,
    "high": "default",
    "xhigh": null,
    "max": "maximum"
  }
}
```

Setting `"off": null` means thinking cannot be disabled. This is more precise than a boolean `reasoning` flag: the selector and clamping path agree on the supported set before any provider-specific serialization runs.

12.6 Custom models in `models.json`

The conventional file is `~/.pi/agent/models.json`. This chapter's source configuration is global; verify current project-local support before assuming that `.pi/models.json` is also loaded. A minimal local OpenAI-compatible provider looks like this:

```
{
  "providers": {
    "local": {
      "baseUrl": "http://localhost:11434/v1",
      "api": "openai-completions",
      "apiKey": "local-placeholder",
      "models": [{ "id": "local-coder" }]
    }
  }
}
```

The placeholder key is deliberate for local services that ignore it. Pi uses configured-auth status when deciding whether to offer a model, so a provider with no auth source would otherwise be unavailable even if its server accepts unauthenticated requests.

A full model definition can set human name, `reasoning`, inputs, context window, maximum tokens, cost rates, and compatibility behavior. Cost rates are per million tokens. Optional `cost.tiers` describe request-wide alternate rate sets, not progressive billing.

```
{
  "cost": {
    "input": 5,
    "output": 30,
    "cacheRead": 0.5,
    "cacheWrite": 6.25,
    "tiers": [
      {
        "inputTokensAbove": 272000,
        "input": 10,
        "output": 45,
        "cacheRead": 1,
        "cacheWrite": 12.5
      }
    ]
  }
}
```

A tier is not a progressive billing bracket: the entire request uses one complete rate set chosen by its highest matching threshold. `calculateCost()` in `packages/ai/src/models.ts` picks rates as follows:

1. Start from the base `cost` rates.
2. Sum input-side usage as `input + cacheRead + cacheWrite`.

3. Among tiers with `inputTokensAbove` strictly below that sum, choose the highest threshold.
4. Apply that complete rate set to ordinary token categories: input, output, and cache reads, plus ordinary (short) cache writes. Tokens are not split across base and tiered rates. A `cacheWrite1h` one-hour cache write is the exception: it is charged as `rates.input * 2`; the selected tier still controls the ordinary categories and short cache writes.

With the rates above, 272,000 input-side tokens still use `$5`/`$30`; 272,001 switches the whole request to `$10`/`$45` for the ordinary categories. Multiple tiers work the same way: only the highest matching threshold is used. This metadata drives Pi's footer and session estimates; it does not determine the provider's invoice.

12.7 Provider overrides and model overrides

Configuration source determines whether a model list merges or replaces:

Goal	Mechanism	Catalog result
Route built-ins through a proxy	`models.json` provider `baseUrl` or `headers`, without `models`	Keeps the built-in catalog and changes request configuration.
Add or replace individual models	`models.json` provider `models` array	Upserts those models into the built-in catalog.
Adjust one known model	`models.json` `modelOverrides`, keyed by model ID	Keeps the catalog and changes only that model's metadata.
Define an extension provider catalog	`pi.registerProvider(name, { models })`	Replaces all current models for that provider.

In particular, `models.json` model definitions merge by model ID, whereas an extension registration with `models` replaces the provider catalog. Choose the source whose catalog semantics match the intended change.

12.8 Config values and secrets

Provider `apiKey` and header values can be literals, `$ENV_VAR`/ `${ENV_VAR}` interpolation, or a leading `!command` whose stdout becomes the value. `$$` escapes a dollar and `$!` escapes a leading bang.

```
{
  "apiKey": "$TEAM_PROVIDER_KEY",
  "headers": {
    "x-tenant": "$TENANT_ID",
    "x-token": "!security find-generic-password -ws team-provider"
  }
}
```

Resolution lives in `packages/coding-agent/src/core/resolve-config-value.ts`. Command handling depends on the credential source:

Source	When resolved	Caching
`models.json` / extension provider `apiKey` and headers	Request time via uncached resolution	No TTL, no stale reuse, no automatic recovery if the command fails
`auth.json` API-key `key`	When that credential is read	Successful or failed command stdout/result is cached for the process lifetime

Availability and auth-status checks deliberately do not execute command-backed values; they report configuration presence rather than proving that the command will succeed. Pi will not infer cache warming, lock contention, or 1Password rate limits for you. If the secret fetch is slow, interactive, rate-limited, or must reuse a previous token for a few minutes, wrap it:

```
{
  "apiKey": "!~/.config/pi/bin/team-provider-token"
}
```

```
#!/usr/bin/env bash
# ~/.config/pi/bin/team-provider-token
set -euo pipefail
cache="${XDG_CACHE_HOME:-$HOME/.cache}/pi/team-provider-token"
max_age_seconds=300
# macOS uses `stat -f %m`; GNU/Linux uses `stat -c %Y` for this timestamp.
if [[ -f "$cache" ]]; then
  now=$(date +%s)
  modified=$(stat -f %m "$cache")
  if (( now - modified < max_age_seconds )); then
    cat "$cache"
    exit 0
  fi
fi
mkdir -p "$(dirname "$cache")"
security find-generic-password -ws team-provider | tee "$cache"
```

Use a path/script you control. Do not put secrets in the script body or logs.

12.9 Credential precedence and storage

`AuthStorage` persists API-key and OAuth credentials in `auth.json` under the agent directory. Its file backend creates parent directories with restrictive permissions and writes the file mode as `0600`. It uses a lock around reads and writes so multiple Pi processes do not race during token refresh.

For the coding agent's `ModelRuntime.getAuth()` request path, credential precedence is:

1. The in-memory runtime override set by `--api-key`.

2. The provider's `auth.json` credential: either an API key or an OAuth token. Expired OAuth credentials are refreshed while holding the storage lock.
3. A provider `apiKey` configured by `models.json` or an extension, resolved from a literal, environment interpolation, or `!command`.
4. If no stored or configured credential supplies the key, the inherited provider resolver may use ambient credentials such as environment variables, AWS profiles, or application-default credentials (ADC).

Provider-scoped `env` values saved with an API-key credential override `process.env` only while expanding that provider's configured key and headers. Runtime overrides are never persisted. The result includes resolved auth, headers, and provider-scoped environment rather than just a string. With `authHeader: true`, it also creates `Authorization: Bearer <key>` and returns an error if no key resolved.

12.9.1 OAuth lifecycle

OAuth lets Pi authenticate with providers that use browser-based or device-code login flows instead of static API keys. Running `/login <provider>` invokes the registered provider's `login()` flow. An extension provider must supply the following shape (the methods are required for the corresponding OAuth integration; this is not a complete login flow):

```
import type { OAuthCredentials, OAuthLoginCallbacks } from "@earendil-works/pi-ai";

const oauthProvider = {
  name: "example-oauth",
  async login(_callbacks: OAuthLoginCallbacks): Promise<OAuthCredentials> {
    return { access: "access-token", refresh: "refresh-token", expires: Date.now() + 3600_000 };
  },
  async refreshToken(credentials: OAuthCredentials): Promise<OAuthCredentials> {
    return credentials;
  },
  getApiKey(credentials: OAuthCredentials): string {
    return credentials.access;
  },
};
```

The full `login()` flow uses callbacks that present an authorization URL or device code, collect prompts and selections, and report progress. Pi stores the returned access/refresh data and expiry in `auth.json`. For a request, `getApiKey()` calls the provider's `getApiKey()` to derive the credential sent in the provider's protocol format. After expiry, one process refreshes through `refreshToken()` under the file lock and persists the replacement credentials; other Pi processes then reread the result. An OAuth provider may also implement `modifyModels()` to derive account-specific endpoints or model metadata from the stored credentials. Thus extension OAuth support requires concrete `login`, `refreshToken`, and `getApiKey` implementations, not just an OAuth label.

12.10 Provider headers and request layering

Header precedence is easiest to read from lowest to highest:

1. Coding-agent session and attribution defaults.
2. Resolved provider auth headers.
3. Model static headers and configured per-model headers.
4. Request-level headers supplied for this call.
5. Extension `before_provider_headers` mutations.
6. Adapter defaults and transport handling for headers not already supplied.

Model static/configured headers override provider auth headers, and request-level headers override those model headers. When `authHeader: true`, the generated bearer header is part of the resolved auth layer; it is not a final layer after request headers. Extension mutations run last before dispatch. Setting a header to `null` asks adapters that support deletion to suppress that header. Never log the merged map: it may contain credentials as well as tracing IDs.

12.11 The unified streaming API

`packages/ai/src/types.ts` defines a normalized, provider-independent `Context`: it contains an optional system prompt, the normalized message list, and optional normalized tool definitions. Adapters convert this `Context` into their own wire format; it is not a raw provider payload. The same module defines `AssistantMessageEventStream`. `StreamOptions` carries abort signal, key, transport preference, cache retention, session ID, headers, timeout, retry, payload/response hooks, metadata, and provider-scoped environment. An `AssistantMessage.stopReason` is one of `"stop"`, `"length"`, `"toolUse"`, `"error"`, or `"aborted"`. A normal `done` stream event carries `stop`, `length`, or `toolUse`; an `error` stream event carries `error` or `aborted`. Keep these normalized values distinct from a provider's raw finish reason.

For coding-agent, `ModelRuntime.streamSimple(model, context, options)` is the canonical dispatch point:

```
model selected by coding-agent
  -> ModelRuntime prepares auth/headers
  -> composed provider and API adapter selected by model.api
  -> provider.streamSimple(model, Context, options)
  -> normalized AssistantMessageEventStream
```

The older `src/compat.ts streamSimple()` remains a compatibility/default fallback for callers that do not supply an explicit stream function.

Individual API adapters serialize messages/tools/system prompt into their wire format, parse streaming chunks, and emit normalized partial/final events. The agent loop therefore does not need a branch for every vendor protocol.

12.12 Payload and response hooks

`StreamOptions` has `onPayload` and `onResponse`. The payload hook observes the serialized request before sending; the response hook observes status and headers after the server replies. Use them for narrowly scoped, sanitized diagnostics such as a correlation ID (for example, an `X-Request-Id` header); do not log credentials, full system prompts, or raw user context. Coding-agent connects these to extension events. `before_provider_request` sees the provider-specific payload after an adapter serializes Pi context. `after_provider_response` sees status/headers after HTTP response receipt and before body consumption when the provider exposes them.

Use these for narrow instrumentation or compatibility experiments. A broad payload rewrite is fragile because it takes ownership of a provider protocol. If the endpoint genuinely differs from supported APIs, use a custom provider stream implementation instead.

12.13 Transport, timeout, and retry

Pi's shared `Transport` values are `sse`, `websocket`, `websocket-cached`, and `auto`. Providers that do not support a choice ignore it. HTTP idle timeout, WebSocket connection timeout, provider SDK retry count, and max retry delay are separate settings because they govern different failures.

The coding-agent stream function applies settings and calls `streamSimple` with an effective timeout. A configured zero HTTP idle timeout is translated to `2147483647` milliseconds in `packages/coding-agent/src/core/sdk.ts`, because the provider SDKs interpret zero as immediate expiry; this is an effective large finite value, not a promise of an unbounded request. An explicit `options.timeoutMs` or provider retry timeout can take precedence. Agent-level retry happens later in `AgentSession`, after it receives a final failed assistant message, and is separate from SDK/provider retries.

12.14 Custom providers from an extension

`pi.registerProvider(name, config)` can override endpoint/headers for an existing provider without model replacement, register a new provider with model definitions, register OAuth callbacks for `/login`, or register a custom `streamSimple` implementation for a nonstandard API.

Provider config validation rejects incoherent registration: models require a base URL and either API-key config or OAuth; each model must resolve an API type. An async extension factory is the correct place for remote model discovery because Pi waits for it before startup/model listing continues.

When unregistering, the registry refreshes and reapplies remaining registered providers so built-in models/protocol behavior previously overridden by that provider are restored.

12.15 Provider debugging workflow

When a model does not appear or a request fails, work in this order:

1. Inspect `ModelRuntime.getError()` for a `models.json` parse/load error.
2. Confirm provider/model ID and API type.

3. Use `getProviderAuthStatus()` as a non-secret configuration diagnostic, not as credential validation: it neither runs `!command` values nor refreshes OAuth tokens, and request-time resolution can still fail.
4. Confirm `baseUrl`, `compat`, input capabilities, and thinking map.
5. Use `before_provider_request` only to inspect a minimal sanitized payload.
6. Check `after_provider_response` status/headers where supported.
7. Read the final assistant message's `stopReason` / `errorMessage`, then separate layers: adapter or HTTP failure ends as `stopReason: "error"` (or `"aborted"`); if the UI then shows another attempt, check session events for `auto_retry_start`/`auto_retry_end` (agent transport retry) versus `compaction_start`/`compaction_end` (context overflow recovery). Provider SDK retries happen earlier inside the transport and will not emit those session events.

This preserves Pi's architecture: the model registry owns configuration and credentials; the adapter owns protocol translation; the agent owns turn flow.

13 Agent core: runs, events, queues, and tool batches

This is reference depth for extension patterns; read it when you need the generic loop, queues, or event ordering.

`packages/agent` is the reusable runtime beneath the coding agent. It owns the model loop, message state, tool execution, streaming, queues, and cancellation. Read Prompt lifecycle before this chapter for the end-to-end path, and use TypeScript used by Pi when a type signature is unfamiliar. It deliberately does not know about JSONL sessions, themes, project trust, extensions, or built-in filesystem tools.

The central source files are `packages/agent/src/agent.ts`, `src/agent-loop.ts`, and `src/types.ts`.

For the coding-agent view of prompt/tool assembly and the high-level rule that continues or ends a run, start with The agent loop in one picture. This chapter supplies the implementation-level event, queue, snapshot, and concurrency detail behind that model.

13.1 The smallest useful mental model

An **agent run** repeats this cycle: send context to a model, receive streamed events, execute any requested tools, append the results, and either stop or send another turn. The `Agent` class owns changing state around that cycle; `agent-loop.ts` performs the cycle; `types.ts` defines the records exchanged.

Keep two cases in mind. A text-only answer finishes after one provider stream. A tool-using answer returns to the provider after each tool batch. The rest of the chapter explains the events and queues that make both cases observable.

13.2 Event stream as the observable protocol

The loop emits events rather than updating a UI directly. Important event families are:

Events	Meaning
`agent_start`, `agent_end`	One low-level run begins/ends
`turn_start`, `turn_end`	A provider response plus any tool batch
`message_start`, `message_end`	User, assistant, and tool-result message lifetime
`message_update`	Assistant streaming updates only
`tool_execution_start`, `tool_execution_update`, `tool_execution_end`	Tool row/process lifetime

`Agent.processEvents()` first updates internal state, then awaits registered listeners. This ordering means a listener sees a coherent agent state through the event it is handling. Coding-agent relies on that property when syncing session entries and calling extension hooks.

The extension event reference is canonical for payloads and ordering guarantees; this chapter uses the event stream to explain the runtime control flow.

13.3 The three layers inside the package

Layer	Main file	Responsibility
Data contracts	`types.ts`	Tool/message/event/loop configuration types
Stateful controller	`agent.ts`	Current state, queues, aborting, listeners, lifecycle
Pure-ish loop orchestration	`agent-loop.ts`	Provider turn, event emission, tool batches, continuation

This split is useful when debugging. If state appears stale, start at `Agent`. If a provider turn or tool batch sequence is wrong, start at `agent-loop`. If the caller cannot supply a valid hook/tool/message shape, start in `types.ts`.

13.4 Agent state and snapshots

The `Agent` holds state including:

- current system prompt, model, **thinking level**, and active tools. A thinking level (`off`, `minimal`, `low`, `medium`, `high`, `xhigh`, or `max`) requests a model's reasoning budget; see Models for capability mapping.
- durable messages accumulated in this runtime
- streaming message and pending tool IDs for UI observers
- active run/abort controller
- a **steering message queue**, whose messages are injected after the current assistant turn and tool batch but before the next provider request
- a **follow-up message queue**, whose messages are considered only after no tool calls or steering messages remain and the run would otherwise stop
- event listeners and optional caller hooks

`Agent.steer()` adds to the first queue; `Agent.followUp()` adds to the second. This distinction is the queue boundary used by the loop, not merely a UI key binding. Coding-agent's `AgentSession` wraps this reusable controller with JSONL persistence, extensions, and session-level retry/compaction behavior; see Prompt lifecycle and Sessions and context.

Before a prompt starts, `createContextSnapshot()` copies the `messages` and `tools` arrays with `slice()` and captures the current system-prompt string. In plain terms, it copies the array containers but not the objects inside them; treat messages and tool definitions as immutable during a run.

This is an array snapshot, not a deep clone: the message objects, their nested content blocks, and the tool definition objects remain shared with agent state.

Array-level changes on either side stay isolated. After the run starts, replacing, pushing into, or reordering the agent state arrays does not alter the current loop context, and the same is true in reverse for the snapshot arrays the loop holds. Mutating an object already inside either array can still affect both views, because those objects are shared by reference. Treat messages and tool definitions as immutable once installed if a run may be active.

The model and thinking level are captured separately in the loop config. The thinking level is copied as a primitive; the model object, like message and tool objects, is not cloned. The loop owns its current context object and appends new messages to its private message array as it emits events; `Agent.processEvents()` appends completed messages to the state transcript. A later `prepareNextTurn` update can explicitly replace the context, model, or thinking level between provider turns.

13.5 Starting a prompt versus continuing a context

When idle, `Agent.prompt()` normalizes input to one or more `AgentMessage` values, then calls `runAgentLoop()` with the snapshot. It does not start a second run when one is active. Because `prompt()` is async, the call returns a rejected promise:

```
Agent is already processing a prompt. Use steer() or followUp() to queue messages, or wait for c
```

The rejection occurs before normalization, event emission, or transcript mutation, and it does not abort or otherwise change the existing run. `Agent.continue()` rejects concurrently in the same way with `Agent is already processing. Wait for completion before continuing.` Callers must `await` the call (or otherwise handle its promise), not expect a synchronous throw. Queue with `steer()` or `followUp()`, or await `waitForIdle()`, instead of retrying concurrently.

When idle, `Agent.continue()` resumes from the current transcript without appending a new caller message. Its default rule is that the last transcript message must not be an assistant message: continuation expects a user/tool-result tail that still needs a model response. There is one exception for queued work that arrived while the previous run was finishing:

1. If the transcript ends in an assistant message, `continue()` first drains the steering queue. Any drained steering messages are run as prompt messages (without an extra initial steering poll).
2. If steering is empty, it drains the follow-up queue and runs those messages the same way.
3. If both queues are empty, it rejects with `Cannot continue from message role: assistant`.

An empty transcript rejects with `No messages to continue from`. A non-assistant tail proceeds through `runContinuation()` and asks the model again from the existing context snapshot.

`prompt()` versus `continue()` exists to prevent accidental replay of a finished turn. Calling `continue()` after a normal assistant stop would re-issue the same trailing context to the provider without a new user or tool-result message. That is usually a bug (double-send / empty re-prompt). Callers that truly need another model call must first change the transcript tail or deliver queued steering or follow-up content.

```
prompt(messages)      adds new user/custom messages and begins a run
continue()             asks the model again from existing context
steer(message)         queues content between assistant turns
followUp(message)      queues content after the agent would otherwise stop
```

In the interactive coding-agent UI, Enter while a run is active maps to steer and Alt+Enter maps to follow-up; details are under Termination, steering, and follow-up below.

Coding-agent uses `continue()` deliberately. It starts with a user message, then may call `continue()` after automatic retry/compaction preparation or after messages were queued by a post-run extension event.

13.6 Active-run lifecycle and cancellation

`runWithLifecycle()` owns one `AbortController` per run. It marks the state as streaming, calls the loop, converts an unexpected thrown failure into an assistant error/aborted message sequence, then clears the active run.

`Agent.abort()` aborts that controller. The signal is passed to the provider stream and tool execution. A tool/extension that starts child work must forward the signal itself; abort does not magically cancel a detached fetch/process whose API was never given the signal.

`waitForIdle()` resolves after the active low-level run and its awaited event listeners finish. In coding-agent, `AgentSession` may then automatically retry, compact and retry, or start a queued continuation. `AgentSession` emits `agent_settled` once after that session-level chain has ended and no such automatic work remains. `agent_settled` is a coding-agent extension/session event, not an event in the `packages/agent` `AgentEvent` union. Extension authors who need Pi as a whole to be idle should observe `agent_settled`; see Extension event reference: `agent_start`, `agent_end`, and `agent_settled`.

13.7 Runnable offline example

This listing uses the current explicit provider-collection APIs. The faux provider is deterministic and local: it needs no API key or network. It prints the final assistant content and records the two event families that matter for an observer. Save it as `agent-faux-example.ts` in the repository root and run it with `npx tsx agent-faux-example.ts`.

```ts
import { createModels, fauxAssistantMessage, fauxProvider } from "@earendil-works/pi-ai";
import { Agent } from "@earendil-works/pi-agent-core";

const faux = fauxProvider();
const models = createModels();
models.setProvider(faux.provider);
faux.setResponses([fauxAssistantMessage("offline hello")]);

const agent = new Agent({
```

```
    initialState: { model: faux.getModel(), systemPrompt: "", messages: [] },
    streamFn: (model, context, options) => models.streamSimple(model, context, options),
});

const observed: string[] = [];
const unsubscribe = agent.subscribe((event) => {
    if (event.type === "message_update") {
        observed.push(event.type);
    }
    if (event.type === "agent_end") {
        observed.push(event.type);
        const finalMessage = event.messages[event.messages.length - 1];
        if (finalMessage?.role === "assistant") {
            console.log(finalMessage.content);
        }
    }
});

try {
    await agent.prompt("Say hello.");
    await agent.waitForIdle();
    console.log({
        sawMessageUpdate: observed.includes("message_update"),
        sawAgentEnd: observed.includes("agent_end"),
    });
} finally {
    unsubscribe();
    models.deleteProvider(faux.provider.id);
}
```

Expected output includes `[{ type: "text", text: "offline hello" }]` and `{ sawMessageUpdate: true, sawAgentEnd: true }`. The final unsubscribe and provider deletion matter in a long-lived host: event listeners and provider registrations are runtime resources, even when this one-shot example exits.

13.8 One provider turn in detail

`streamAssistantResponse()` is the provider boundary in `agent-loop.ts`.

```
AgentMessage[] context
  -> optional transformContext
  -> convertToLlm
  -> provider Context { systemPrompt, messages, tools }
  -> streamFn(model, context, options)
  -> for await normalized provider events
  -> partial/final AssistantMessage plus message events
```

The low-level loop does not assume a provider has the same message shape as an agent message. `convertToLlm` belongs to the caller precisely because a coding agent may have session-only messages while another embedding has a different message representation.

On a provider `start` event, the loop pushes a partial assistant message into its current context and emits `message_start`. Text, thinking, and tool-call deltas replace that partial state and emit `message_update`. A final `done` or `error` resolves the stream result and emits `message_end`.

13.9 Tool argument preparation and validation

After a final assistant message, the loop finds content blocks with `type: "toolCall"`. It resolves the requested tool from active context, calls optional `prepareArguments`, then validates the result against the tool schema.

`prepareArguments` exists for session compatibility. An extension/tool can adapt legacy stored call shapes into its current strict schema before validation without weakening the public parameters declaration. For example, an old call might use `file` while the current schema expects `path`:

```
prepareArguments: (args) => ({
  path: typeof args === "object" && args !== null && "file" in args
    ? String(args.file)
    : "",
})
```

It is not a general way to accept arbitrary invalid input.

An unknown tool, invalid schema, hook failure, or explicit block becomes an immediate error result rather than a crash. The model receives that error in a tool-result message and can decide how to recover.

13.10 Sequential and parallel tool execution

`ToolExecutionMode` has two choices:

Mode	Behavior
`sequential`	Prepare, execute, finalize, and persist each tool before the next
`parallel`	Emit each start event and preflight each call in assistant-content order; after all preflights, run prepared tools concurrently; emit updates and end events as work progresses; after all calls finish, emit tool-result messages in assistant-content order

The `Agent` default is `parallel`. The ordering contract is direct: in a parallel batch, `tool_execution_start` and argument preparation occur in source order; prepared `execute()` functions start only after

the preflight pass; `tool_execution_update` and `tool_execution_end` can interleave and end in completion order; `message_start`/`message_end` for the final `toolResult` messages wait for the whole batch and then occur in source order. An immediate preflight error can emit its end event during the ordered preflight pass, before prepared siblings begin execution.

The effective mode is batch-wide. The batch is sequential if `agent.toolExecution === "sequential"` **or** any tool called by that assistant message has `executionMode: "sequential"` in the active context. It is parallel only when the agent mode is `"parallel"` and none of those called tools requires sequential execution. A tool marked `"parallel"` does not override a global or sibling sequential requirement.

To inspect this behavior, first record `agent.toolExecution` and the `executionMode` fields of the active tool definitions at run start and after any `prepareNextTurn` refresh, then match those definitions to the tool calls in the assistant message. Subscribe to `tool_execution_start`, `_update`, and `_end` to observe lifecycle ordering; `agent.state.pendingToolCalls` contains the IDs between start and end. A start event and pending ID include preflight, so they do not prove that the tool's `execute()` body ran—instrument the tool wrapper or `execute()` entry when that distinction matters.

Parallel execution means a tool hook cannot assume a sibling's result has finished or been persisted. Use explicit coordination or `turn_end` state when tools depend on one another.

13.11 Before and after tool hooks

The generic loop accepts two caller callbacks:

- `beforeToolCall` receives assistant message, tool call, validated args, and current context. It may block with a reason.
- `afterToolCall` receives the executed result/error state and may replace content, details, error flag, or terminate hint.

Coding-agent connects these callbacks to `ExtensionRunner.emitToolCall` and `emitToolResult`. That bridge keeps the reusable agent package free of extension types while allowing extension hooks to participate in execution.

13.12 Termination, steering, and follow-up

When a tool batch completes, the loop creates final `toolResult` messages and appends them to context. Unless every result has `terminate: true`, it performs another provider turn. The all-results rule avoids one tool silently stopping a batch whose other tools expect a model follow-up.

After each completed turn, `prepareNextTurn` can refresh the context, model, or thinking level; if the caller's `shouldStopAfterTurn` hook returns `true`, the loop ends at that boundary before polling steering or follow-up queues. Otherwise it asks for steering messages. Steering messages enter before the next provider request. When there are no more tool calls and no steering messages, it asks for follow-up messages. Follow-ups restart the inner loop only after the agent would otherwise stop.

```
assistant/tool batch complete
  -> steering queue?  yes: inject before next model call
  -> no tools/steering: follow-up queue? yes: begin another turn
  -> neither: agent_end
```

In interactive mode, these queues are selected by the submit key while Pi is working:

- **Enter** submits with `streamingBehavior: "steer"`. The message waits for the current assistant response and its entire tool batch to finish, then enters before the next provider request. It does not interrupt a tool midway.
- **Alt+Enter** submits with `streamingBehavior: "followUp"`. The message is not considered until the inner tool/steering loop would otherwise stop, so already queued steering and the work it causes run first.

When Pi is idle, both keys perform a normal submission; Alt+Enter does not create a deferred message. Queue drain settings can deliver either the oldest message only (`"one-at-a-time"`, the default) or all messages at a drain point (`"all"`). During compaction the interactive layer preserves the same steering-versus-follow-up classification and delivers the queued input when compaction permits.

13.13 Next-turn refresh hook

The loop can call `prepareNextTurn` after `turn_end`. It may replace context, model, or thinking level for the next provider call. Coding-agent uses this to refresh the current system prompt and active tool list from session state.

This matters when an extension dynamically changes tools or model state during a run. The current turn remains internally consistent; the next provider call sees the refreshed session snapshot rather than a stale configuration.

13.14 Error semantics are intentionally normalized

The `StreamFn` contract says ordinary request/model/runtime failures should be represented by a returned assistant event stream whose final message has `stopReason: "error"` or `"aborted"`, rather than a rejected promise. The common stop reasons are `"stop"` (normal completion), `"toolUse"` (the model requested tools), `"length"` (context or output limit), `"error"`, and `"aborted"`. The agent also protects itself by converting unexpected throws into a final assistant error message.

Tool failures are different: execute throws become an error `toolResult`, so a model can read output/error and choose a recovery. This difference is important for tests and UI: a provider failure ends the run; a tool failure usually lets the model continue. Context pressure can trigger compaction above this package; see Sessions and context for the summary-and-retry path.

13.15 Embedding the agent package

If you use `packages/agent` outside coding-agent, you must provide policy that coding-agent normally supplies:

- initial `AgentState` with a valid model/system prompt/tools
- `convertToLlm` for your message representation
- provider `streamFn` and credentials or key resolver
- listeners that persist/render state if needed
- context transformation/queue behavior appropriate to your application
- tool definitions and any permission gate

The package gives a rigorous loop, not a complete coding application. See the guided source tours for a concrete integration trace. That is the reason Pi can support the interactive CLI, SDK examples, and other hosts without coupling all of them to terminal/session decisions.

13.16 Agent-core debugging checklist

When a turn misbehaves, capture these facts before editing code. For example, if a tool never executes, first determine whether the model emitted a tool call, whether validation passed, and whether `execute()` was entered; do not start by changing provider code.

1. What did `convertToLlm` produce?
2. What system prompt and active tools were in the context snapshot?
3. Which normalized provider events arrived, and was a final message emitted?
4. Did the assistant end with tool calls, error, aborted, length, or stop?
5. Which tool arguments passed validation/preparation?
6. Was the batch sequential or parallel, and what event ordering occurred?
7. Did steering/follow-up queues inject another message?
8. Did a `prepareNextTurn` refresh change model/tools/context?

This reduces a complex agent issue to a small number of observable protocol boundaries.

14 Terminal UI and interactive mode

This is reference depth for custom UI patterns; read it when an extension needs interactive-mode or terminal rendering detail.

Pi's interactive application is built from a reusable terminal UI package and a coding-agent-specific composition layer. Before reading this chapter, review Prompt lifecycle and Agent core. Extension authors should also read Extensions. The separation is worth learning: an extension normally customizes the coding-agent UI surface, while a contribution to `packages/tui` changes primitives that could serve another terminal application.

Use this chapter according to the layer you plan to change. Extension authors should focus on editor submission, transcript renderers, and `ctx.ui`; coding-agent contributors should follow `InteractiveMode` and session events; TUI contributors should focus on terminal lifecycle, components, input, and differential rendering.

Relevant files include:

- `packages/tui/src/tui.ts`, `packages/tui/src/terminal.ts`, and `packages/tui/src/keybindings.ts`;
- `packages/tui/src/autocomplete.ts`;
- `packages/coding-agent/src/modes/interactive/interactive-mode.ts`.

14.1 Vocabulary before rendering details

A **terminal UI (TUI)** is an interface drawn inside a terminal window. A **component** is an object that renders lines and handles input. A **frame** is one rendered screen state. **Differential rendering** updates only lines that changed between frames. `InteractiveMode` composes these primitives into Pi's chat; it does not define the generic terminal primitives itself.

Start with a static line, then a component that changes its text, then a streaming assistant row. That progression explains why components invalidate themselves and why the TUI listens to agent events instead of polling a model.

14.2 The TUI package owns terminal mechanics

The public barrel (a re-export file) `packages/tui/src/index.ts` exports the `TUI` class, component/container interfaces, text/markdown/editor/list components, terminal implementations, key parsing/matching, keybindings, autocomplete, image support, and ANSI-aware text utilities.

The `TUI` class is responsible for the component tree, focus, layout, input routing, overlays, terminal lifecycle, and differential screen updates. A component returns lines for an available width and can invalidate itself when its rendered state changes. This is a **retained component model**: components

persist across frames and the TUI updates their state, rather than rebuilding the screen from scratch on every update. It is not immediate `console.log` output.

14.3 Differential rendering in practical terms

Terminal rendering is expensive and visually disruptive if every update clears the whole screen. The TUI tracks prior lines and writes only the changes needed to transform the terminal into the newly rendered view.

```
component tree changes
  -> render visible lines with ANSI styling
  -> compare with previous terminal frame
  -> emit minimal cursor movement/text changes
```

That design enables token-by-token assistant updates, tool-output streaming, and autocomplete without a full redraw for every byte. It also explains why components should invalidate only when necessary and reuse component instances for high-frequency partial updates where the API supports it.

14.4 Components are composition, not application policy

The TUI exports general components such as `Text`, `Box`, `Container`, `Markdown`, `Editor`, `Input`, `SelectList`, `SettingsList`, `Loader`, and `Image`. Their job is layout/presentation/input behavior. They do not know what a session, provider, or tool result means.

Coding-agent builds a chat-specific component tree around those primitives:

```
InteractiveMode
  -> header/status regions
  -> chat transcript container
       -> user/assistant/thinking/tool components
  -> pending messages/tool area
  -> widgets above/below editor
  -> editor or extension replacement editor
  -> footer and overlays/dialogs
```

This means a custom transcript renderer should return a TUI `Component`; it does not need to manage raw terminal escape sequences itself.

14.5 Terminal lifecycle and capabilities

`ProcessTerminal` and terminal support code manage stdin/stdout, raw mode (where the terminal passes each keystroke directly instead of waiting for Enter), size information, terminal capabilities, and cleanup. Image rendering is capability-dependent: Pi can choose Kitty, iTerm2, or fallback behavior based on what the terminal supports.

Interactive mode initializes terminal/theme behavior before starting its event loop and stops/restores terminal state at shutdown. If you add a feature that opens an external editor, subprocess, or custom modal, follow the existing interactive-mode lifecycle rather than leaving raw terminal state altered.

14.6 Keyboard input is parsed before it becomes a binding

`packages/tui/src/keys.ts` parses terminal input into key semantics and `packages/tui/src/keybindings.ts` maps semantic keybinding IDs to user-configurable keys. A binding ID is stable, namespaced policy such as:

```
tui.input.submit
tui.editor.cursorWordLeft
tui.select.confirm
app.interrupt
app.tools.expand
app.model.cycleForward
```

An extension should display hints with `keyHint("app.tools.expand", "to expand")` or inspect the injected `KeybindingsManager`. `keyHint()` returns a styled string: it looks up the current configured key(s) with `keyText()`, styles the key as dim, and appends the styled description. It is presentation only; use `keybindings.matches(key, "app.tools.expand")` to handle input. It is exported from the coding-agent package's interactive components, not from the generic TUI package. Do not hardcode raw `ctrl+x` checks. Hardcoding bypasses user configuration and breaks the mapping between the keybindings file, help text, and actual behavior.

14.7 Editor submission and queue semantics

The submission outcome comes first: when idle, regular Enter resolves `getUserInput()`, whose loop calls `session.prompt(text)`. While the agent is streaming, regular Enter sends a steering prompt for the active run, and Alt+Enter queues a follow-up prompt for after that run finishes.

The interactive editor is therefore more than a text box. Its submission handler also handles:

- built-in slash commands;
- `!`/`!!` user shell commands;
- message holding during compaction, which replaces older conversation context with a shorter summary to free context-window space;
- steering/follow-up delivery while an agent is active;
- normal queued input for the main `run()` loop;
- editor history, multiline text, image paste, autocomplete, and external edit.

The detailed handling stays in `InteractiveMode` so `AgentSession` can serve print/RPC (Remote Procedure Call) clients without importing terminal components.

14.8 Transcript rendering from agent events

`InteractiveMode.subscribeToAgent()` receives coding-agent session events. `handleEvent()` dispatches behavior by type. In a typical tool-calling turn, the first occurrence of each event follows this runtime order:

Event	UI responsibility
`agent_start`	Clear pending tool bookkeeping and begin activity state
`message_start`	Create the row for a user or assistant message
`message_update`	Refresh the partial assistant, thinking, and tool-call display
`message_end`	Finalize the current message component
`tool_execution_start`	Start or recover the tool row created from the assistant's tool call
`tool_execution_update`	Refresh a streaming partial tool result
`tool_execution_end`	Finalize the tool result and remove it from pending bookkeeping
`agent_end`	Clear activity state and any leftover streaming/tool components

After tool execution, `message_start`/`message_end` carry the tool-result message, and another assistant turn may repeat the message lifecycle before `agent_end`. The UI works from events rather than polling the model. This preserves the same visual behavior whether an assistant emits text, thinking, tool calls, errors, or a synthetic retry/compaction message.

14.9 Tools in the transcript

Tool rows are composed by the interactive components around a tool definition's `renderCall` and `renderResult` slots. The public `ToolDefinition` type passes the current `Theme` as the `theme` argument to each renderer; the renderer does not discover a global theme by importing an internal module. Extension UI factories similarly receive `theme`, and `ctx.ui.theme` exposes the current coding-agent theme for code that needs to style outside a renderer. The context passed to those renderers includes current args, a row-local mutable state object, optional last component, partial/final flags, expanded state, show-images preference, cwd, and `executionStarted: boolean`, plus an `invalidate()` method. It does not expose elapsed execution time.

This produces a useful layering:

```
tool definition knows how to render its call/result data
interactive mode knows where the row belongs in chat and when it updates
TUI knows how to layout/diff-render the resulting components
```

For an extension tool, return a compact default component and make details available when expanded. For a high-frequency partial result, reuse `context.lastComponent` when it is already the same component class you would create and can be updated in place (for example a `Text` you can `setText` on).

That reuse is the criterion for "safe": same render slot, compatible instance, mutations only. Do not cast and mutate an unrelated previous component, and do not invent a new large component tree on every stream update when an existing instance can carry the change.

14.10 Extension UI surface

`ctx.ui` is an adapter owned by `InteractiveMode`. It gives extensions a controlled surface rather than direct access to private chat/editor fields:

API family	Examples
Dialogs	`select`, `confirm`, `input`, `editor`
Notices/status	`notify`, `setStatus`, working message/indicator controls
Editor/widgets	`setEditorText`, `pasteToEditor`, `setWidget`, custom editor
Transcript/rendering	message/entry renderers, tool expansion controls
Look and completion	themes, autocomplete provider, custom footer
Complex interaction	`custom(factory)` and experimental overlays

`ctx.mode` identifies the host surface, while `ctx.hasUI` says whether a dialog-capable adapter is actually bound. Guard on `ctx.hasUI` before operations that require a user response, and require `ctx.mode === "tui"` for terminal-only behavior such as custom TUI components; Run modes has the full per-mode table.

14.11 Custom components and focus

`ctx.ui.custom()` temporarily replaces the editor with a factory that receives the TUI instance, current theme, a keybindings manager, and a `done(value)` callback. The component handles input until it calls `done`.

Start with a static component:

```
await ctx.ui.custom(() => new Text("Press Enter to continue", 1, 1));
```

Then add focus, keybindings, and a `done(value)` result:

```
const answer = await ctx.ui.custom<boolean>((_tui, _theme, keybindings, done) => {
  // Text(text, paddingX, paddingY): cell padding left/right then top/bottom
  const component = new Text("Enter confirms; Escape cancels", 1, 1);
  component.handleInput = (data) => {
    if (keybindings.matches(data, "tui.select.confirm")) done(true);
    if (keybindings.matches(data, "tui.select.cancel")) done(false);
  };
  return component;
});
```

The example uses named bindings, so a user who rebinds confirm/cancel receives the expected behavior. More complex overlays have focus/visibility APIs; use them only when an editor replacement/dialog cannot express the interaction.

14.12 Autocomplete composition

The TUI supplies autocomplete interfaces and a combined provider. Pi's interactive editor layers slash-command and path completion. An extension can add a provider by receiving the current provider and delegating when its own trigger syntax does not apply.

The reliable pattern is:

1. Inspect text before the cursor.
2. If extension syntax matches, return an explicit prefix and candidate list.
3. Otherwise call the previous provider.
4. Delegate completion application unless custom insertion is necessary.
5. Preserve existing file-completion decisions unless deliberately changing it.

This prevents an extension's `#issue` or `$variable` completion from disabling normal slash/path completion elsewhere in the line. The public interfaces in `packages/tui/src/autocomplete.ts` are enough for a small provider wrapper:

```
import type {
  AutocompleteItem,
  AutocompleteProvider,
  AutocompleteSuggestions,
} from "@earendil-works/pi-tui";

const addIssueCompletion = (previous: AutocompleteProvider): AutocompleteProvider => ({
  triggerCharacters: ["#"],
  async getSuggestions(
    lines, cursorLine, cursorCol, _options,
  ): Promise<AutocompleteSuggestions | null> {
    const line = lines[cursorLine] ?? "";
    const beforeCursor = line.slice(0, cursorCol);
    const match = /(?:^|\s)(#[\w-]*)$/.exec(beforeCursor);
    if (!match) return previous.getSuggestions(lines, cursorLine, cursorCol, _options);
    const prefix = match[1];
    const items: AutocompleteItem[] = [
      { value: "#bug", label: "#bug", description: "Track a defect" },
      { value: "#docs", label: "#docs", description: "Documentation work" },
    ].filter((item) => item.value.startsWith(prefix));
    return items.length > 0 ? { items, prefix } : null;
  },
  applyCompletion(lines, cursorLine, cursorCol, item, prefix) {
    const nextLines = [...lines];
    const line = nextLines[cursorLine] ?? "";
    const start = cursorCol - prefix.length;
```

```
    nextLines[cursorLine] = `${line.slice(0, start)}${item.value}${line.slice(cursorCol)}`;
    return { lines: nextLines, cursorLine, cursorCol: start + item.value.length };
  },
  shouldTriggerFileCompletion: (lines, cursorLine, cursorCol) =>
    previous.shouldTriggerFileCompletion?.(lines, cursorLine, cursorCol) ?? false,
});

ctx.ui.addAutocompleteProvider((previous) => addIssueCompletion(previous));
```

This skeleton delegates non-`#issue` text, returns the exact text prefix it replaces, and preserves the previous provider's file-completion decision. A real provider should use the supplied `AbortSignal` for asynchronous work and return `null` when there are no candidates.

14.13 Themes and semantic styling

Theme functions provide semantic styles such as `accent`, `muted`, `dim`, `success`, `warning`, `error`, and `toolTitle`. Use `theme.fg(name, text)` plus style helpers like `theme.bold(text)` in renderer code.

Do not embed fixed ANSI colors in an extension renderer. A literal color may be unreadable in a user's light/custom theme and cannot participate in Pi's visual language. Semantic keys let an extension look native across themes.

14.14 Images and rendered width

Terminal width is measured in display cells, not JavaScript string length. Wide CJK characters, ANSI escape sequences, hyperlinks, and image rows make naive `text.length` calculations wrong. The TUI exports utilities such as `visibleWidth`, `truncateToWidth`, and ANSI-aware wrapping. Use them in custom components rather than slicing strings manually.

Likewise, terminal image support is capability- and cell-size-dependent. Use the TUI `Image` component/terminal image helpers rather than trying to print a provider image's base64 data as text.

14.15 Debugging visual issues

When an interactive feature is wrong, isolate its layer:

1. Capture the agent/session event sequence; verify data before rendering.
2. Inspect the tool/message renderer output as a component.
3. Check available width and ANSI-aware visible width.
4. Confirm the component invalidates after its state changes.
5. Test narrow terminal widths and long/CJK/ANSI-rich content.
6. Test focus and cancel paths for custom UI.
7. Use `/debug` for rendered TUI lines and all current session messages sent through the model-facing history dump.

The correct fix is usually at the narrowest layer that owns the defect: a tool renderer for tool presentation, interactive mode for chat composition, or the TUI package for a general terminal layout/input primitive.

For a practical next step, follow Tour 5: Follow one streamed token to the terminal, then capture the event/render trace described there while running `./pi-test.sh`.

15 Run modes, SDK embedding, and operational boundaries

This is reference depth for extension hosts and operations; read it when embedding Pi or choosing a run mode.

The `pi` binary is one host for the coding-agent runtime. The same session construction and event model can also serve print, JSON, RPC, and programmatic SDK use. Read Sessions for persistence and Extensions for tool definitions before embedding. Here, "operations" means the host-level concerns around protocols, stdout, cleanup, security, and deployment. Understanding the modes prevents terminal assumptions from leaking into automation and helps you select an integration surface deliberately.

Start with `packages/coding-agent/src/main.ts`, `packages/coding-agent/src/modes/`, and `packages/coding-agent/src/core/sdk.ts`. For SDK examples, see `packages/coding-agent/examples/sdk/` and `packages/coding-agent/docs/sdk.md`. Canonical CLI docs for the noninteractive hosts are `packages/coding-agent/docs/json.md` and `packages/coding-agent/docs/rpc.md`.

15.1 Start with the host, not the flag

A **host** is the program that drives Pi. A **mode** describes how that host communicates: a TUI has a terminal, print mode returns text once, JSON emits machine-readable events, and RPC keeps a process open behind a protocol. An **SDK embedding** calls the runtime from another program instead of starting the `pi` binary.

The same agent/session runtime serves all four modes, but UI assumptions do not transfer automatically. First choose the host, then check `ctx.mode` and `ctx.hasUI`, and only then design extension behavior.

15.2 How Pi chooses a mode

`packages/coding-agent/src/main.ts` resolves application mode from explicit arguments and terminal state:

Application mode	Typical trigger	Runtime host/path
Interactive	TTY stdin/stdout without print, JSON, or RPC selection	`InteractiveMode`
Print	`-p` or non-TTY stdin/stdout	`runPrintMode(..., { mode: "text" })`
JSON	`--mode json`	`runPrintMode(..., { mode: "json" })`

Application mode	Typical trigger	Runtime host/path
RPC	`--mode rpc`	`runRpcMode(...)`

The distinction is not cosmetic. It changes whether Pi can prompt a user, how output is emitted, and which extension UI methods are meaningful. All modes still rely on the coding-agent session/runtime instead of implementing separate provider/tool loops.

15.3 Extension mode behavior

An extension context exposes `mode` and `hasUI`. `ctx.mode` is a string whose `ExtensionMode` type is the literal union `"tui" | "rpc" | "json" | "print"`. `ctx.hasUI` is not a synonym for "interactive terminal"; it reports whether the extension runner is bound to a real `ExtensionUIContext` rather than the no-op context used when there is no host UI protocol.

Application surface	`ctx.mode`	`ctx.hasUI`	Design implication
Interactive terminal	`"tui"`	true	Full components, dialogs, editor behavior
RPC	`"rpc"`	true	Protocol-backed dialogs/notices exist; custom terminal UI is limited
JSON	`"json"`	false	No UI protocol; do not wait for a user prompt or render components
Print	`"print"`	false	Agent runs, then process exits; UI calls are no-ops/fallbacks

That table is the practical distinction between JSON and RPC for extension authors. JSON and print share the single-run print host and bind a no-op UI, so dialog methods either no-op or resolve without a user. RPC binds a protocol UI so `select`/`confirm`/`input`/`editor` become request/response messages over stdin/stdout, while terminal-only methods such as `custom()` remain unavailable. Guard on both:

- use `ctx.hasUI` before waiting for a user decision;
- use `ctx.mode === "tui"` for terminal components and other TUI-only behavior.

A policy tool that requires confirmation needs an explicit noninteractive decision when `!ctx.hasUI`, such as blocking with a clear reason, rather than awaiting `ctx.ui.confirm()`.

15.4 Print mode as a single-run host

Print mode is appropriate for scripts that want one response rather than a long-lived terminal session. `packages/coding-agent/src/main.ts` prepares initial text/images from arguments, files, and stdin,

then passes that input to `runPrintMode` in `packages/coding-agent/src/modes/print-mode.ts` with text output selected. The normal `AgentSession.prompt()` preprocessing remains active, so extensions, context, tools, provider selection, and session policy can still participate. After the prompts finish, the host writes only the text blocks from the final assistant message and exits.

For extension authors, that output rule means a notification or custom UI value is not part of the script's result. Put required information into model-facing content so it can appear in the final response, or persist it through an explicit durable channel. Do not depend on delayed background work or UI state surviving the response.

15.5 JSON mode and machine-readable automation

JSON mode is not a separate session host. `packages/coding-agent/src/main.ts` calls `runPrintMode` with `mode: "json"`. That path writes the session header, when present, followed by subscribed session events as newline-delimited JSON. It therefore provides a single-run event stream rather than print mode's final assistant text or RPC's persistent command channel. See also `packages/coding-agent/docs/json.md`.

15.5.1 Session header on the JSON stream

JSONL means newline-delimited JSON: each JSON object is serialized on one line. The first JSONL object is the same session-file header that begins a persisted session, when one is available. In source it is `SessionHeader` from `packages/coding-agent/src/core/session-manager.ts`: metadata immediately written by `runPrintMode` before subscribed events, not a conversation entry.

```
{"type":"session","version":3,"id":"<id>","timestamp":"<ts>","cwd":"/proj"}
```

Fields:

- `type`: always `"session"` for the header;
- `version`: optional session format version (current writes use `3`; older v1 headers may omit it, and loaders treat missing `version` as `1`);
- `id`: durable session identifier;
- `timestamp`: session creation time;
- `cwd`: working directory recorded for the session;
- `parentSession`: optional source session path when a session is forked.

This header has no `parentId` and is not part of the message tree. It is the metadata line documented in Sessions, branches, and durable context and `packages/coding-agent/docs/session-format.md`. Subsequent JSON-mode lines are `AgentSessionEvent` objects as the single-run prompt proceeds.

In extension code, avoid stdout. Arbitrary `console.log` or other writes can corrupt the JSONL stream consumed by automation. Prefer documented event/UI mechanisms, or emit controlled diagnostics on stderr when that is acceptable for the embedding host.

Provider payload tracing has a second, separate risk: raw payloads may contain private context, credentials, or user instructions even when they are written on stderr. Build a sanitized, opt-in diagnostic command instead of conflating this privacy risk with stdout protocol corruption.

15.6 RPC mode as a persistent process boundary

RPC mode keeps a process alive for command/request exchange. Start it with:

```
pi --mode rpc [options]
```

Implementation and types live under `packages/coding-agent/src/modes/rpc/`. The host entry is `rpc-mode.ts`, framing is `jsonl.ts`, command/response types are in `rpc-types.ts`, and the typed client is `rpc-client.ts`. The runtime and session still own model/tool state; RPC is a transport around that behavior, not an alternate agent implementation. The full command and event catalog is `packages/coding-agent/docs/rpc.md`.

15.6.1 Framing and object shapes

> **Protocol warning:** JSONL records are separated only by LF (`\n`). Unicode `U+2028` and `U+2029` can appear inside JSON strings. Node's `readline` splits on those characters too, so it is not sufficient for a protocol-compliant JSONL client.

- Transport is strict JSONL over stdin/stdout.
- The only record delimiter is LF (`\n`). Clients may strip a trailing `\r` so CRLF-delimited input is accepted, but must not treat Unicode line separators such as `U+2028`/`U+2029` as record boundaries. Those characters are valid inside JSON strings, so a reader that only splits on LF stays correct; Node `readline` is not protocol-compliant because it also splits on `U+2028`/`U+2029` and can break a single logical JSON record.
- Commands are JSON objects on stdin with a `type` field and an optional `id` for request/response correlation.
- Command replies are objects with `type: "response"`, `command`, `success`, optional `data`/`error`, and the same `id` when one was supplied.
- Session events are streamed to stdout as plain event objects. They do not carry command `id` values.
- Extension UI dialog traffic uses `type: "extension_ui_request"` on stdout and, for dialogs, matching `type: "extension_ui_response"` objects on stdin.

15.6.2 Lifecycle in overview

1. The process starts, takes over stdout for protocol traffic, binds the extension/runtime host with `mode: "rpc"`, and waits for commands on stdin.
2. A client sends commands such as `prompt`, `steer`, `follow_up`, `abort`, `get_state`, `get_messages`, model/thinking controls, compaction/retry controls, session navigation around entries, or `bash`.
3. For `prompt`, the immediate `response` means the prompt was accepted, queued, or handled; later progress appears as events. For streaming agents, `prompt` requires `streamingBehavior` (`"steer"` or `"followUp"`) when the agent is already running, or the command fails before acceptance.
4. While work proceeds, the host streams session events (`agent_start`, turns/messages/tools, compaction/retry updates, and eventually `agent_settled` when no automatic continuation remains).

5. If an extension uses dialog/fire-and-forget UI methods, the host emits `extension_ui_request`. Dialog methods block until the client answers with the matching `extension_ui_response` or a timeout/default resolves them; fire-and-forget methods do not wait.
6. The process remains available for further commands until the client closes the stream or shutdown is requested.

Extensions can use `ctx.hasUI` in RPC mode for operations that map to protocol UI messages. Dialog methods (`select`, `confirm`, `input`, `editor`) and fire-and-forget methods such as `notify`/`setStatus` are functional there. Terminal-only methods are not: `ctx.ui.custom()` returns `undefined`, and other TUI-only surface controls are no-ops or degraded. Design commands/tools with a structured noninteractive fallback when they might also run through JSON/print.

In extension code, avoid stdout. RPC clients parse every stdout line as protocol JSON; incidental writes corrupt the transport. Prefer extension UI methods, session/event channels, or stderr for diagnostics.

15.7 The `packages/ai` boundary

`packages/ai` is the lower-level, provider-neutral layer for model definitions, authentication support, request serialization, and normalized streaming events. For example, a new provider adapter belongs here; a `before_agent_start` hook that adds a system-prompt prefix stays in `packages/coding-agent`. It does not own session persistence, extension loading, built-in tools, compaction/retry policy, system-prompt assembly, or any of the run-mode hosts described above.

Choose that package when the work is about model/protocol behavior. Stay in coding-agent modes/SDK when the work is about host policy around those model calls.

15.8 The public SDK entry point

`createAgentSession(options?)` in `packages/coding-agent/src/core/sdk.ts` is the composition entry point for a programmatic host. It can create defaults or accept caller-provided cwd, agent directory, model, session manager, settings manager, `modelRuntime`, resource loader, scoped models, tool restrictions, custom tools, and a session start event. `ModelRegistry` is constructed as an extension-facing facade over the runtime rather than supplied as an option.

The SDK does several pieces of application policy on purpose:

```
resolve cwd and agent directory
  -> auth/model/settings/session services
  -> resource loader (unless caller supplies one)
  -> restore/choose model and thinking level
  -> create Agent with coding-agent conversion/provider hooks
  -> create AgentSession and extension runtime
```

The result gives an `AgentSession`, extension load result, and possible model fallback message. A host can subscribe to the session, call `prompt`, provide custom tools, inspect sessions, or embed its own UI.

15.9 Why SDK callers should not reimplement `packages/coding-agent/src/main.ts`

It is tempting to instantiate `Agent` directly because its constructor is small. That skips coding-agent semantics including resource loading, session persistence, model restore/default selection, built-in tool definitions, extension runner integration, system-prompt building, retry, and compaction. Those policies are exactly what `createAgentSession` and the CLI mode hosts assemble on top of the `packages/ai` stream/provider primitives.

Use `Agent` directly only when you deliberately own all of those policies. Use `createAgentSession` when you want Pi's coding-agent behavior in a new host. Use the CLI/RPC surface when a separate process boundary is preferable.

15.10 Custom tools supplied by an SDK host

`createAgentSession` accepts custom tool definitions alongside built-in-tool selection. These tools enter the same registry/extension wrapping flow as an extension-registered tool. A minimal host-owned tool looks like:

This example imports `Type` from `@earendil-works/pi-ai`, whose public root entry point re-exports `Type` and `Static`. That is convenient when the host already depends on `pi-ai`. Extension documentation instead imports `Type` from the direct `typebox` package because the extension declares that schema dependency itself. Both are valid choices; `@earendil-works/pi-coding-agent` does **not** re-export TypeBox.

```
import { Type } from "@earendil-works/pi-ai";
import { defineTool } from "@earendil-works/pi-coding-agent";

const customTool = defineTool({
  name: "lookup",
  label: "Lookup",
  description: "Look up one application record.",
  parameters: Type.Object({ id: Type.String() }),
  async execute(_id, params, signal, _onUpdate, _ctx) {
    // validate permissions and forward signal to I/O
    if (signal?.aborted) throw new Error("Operation aborted");
    return { content: [{ type: "text", text: params.id }] };
  },
});
```

Give real tools a clear source identity, schema, durable details shape, and prompt metadata where appropriate. See Extensions for the full `defineTool` contract.

An SDK host can use this for application-specific operations without requiring an extension file on disk. The tradeoff is scope: a tool supplied by one host does not automatically appear in a user's ordinary `pi` invocation.

15.11 Initial messages and attachments

The CLI's initial-message machinery can combine explicit prompt text, piped stdin, file arguments, and supported images. Interactive mode feeds the resulting message/images through the same session prompt path used after the editor submits text. This keeps extension `input`/`before_agent_start` behavior consistent between startup prompts and later user prompts.

When embedding, construct user content as text/image blocks and let the session/agent conversion path handle the provider representation. Do not pre-serialize provider-specific image payloads in a generic SDK host.

15.12 Operational safety boundaries

Pi runs with the operating-system permissions of its process. It does not add a built-in filesystem/process/network permission sandbox. **Containerization** means running Pi inside Docker or a similar OS-level sandbox so filesystem, network, and process access are limited to the container's scope. Treat it or other OS-level sandboxing as the recommended way to create stronger isolation when automation or untrusted tools require it. That recommendation matches the root `README.md` containerization guidance and `packages/coding-agent/docs/containerization.md`.

This has mode-specific consequences:

- A print/CI invocation can execute the same tools as an interactive one.
- Trust gates project extensions/resources, but do not sandbox global code.
- A custom provider can transmit context to its configured endpoint.
- `bash`, `write`, and extension `exec` run with host process authority.

Use OS/container boundaries for security enforcement. Use extension tool gates for workflow policy and user confirmation. Do not confuse a model instruction or a theme with a security control.

15.13 Observability responsibilities for hosts

15.13.1 Pi-provided surfaces

Pi provides runtime surfaces that a host can observe: startup and resource-load diagnostics, `AgentSession` events such as agent/turn/tool lifecycle and retry/compaction outcomes, provider response status/headers when the transport exposes them, and protocol output guards. These are behavior and diagnostics, not a promise of a complete tracing or metrics backend. Print and JSON both call the single-run `runPrintMode` host and exit after their prompt, so their observations naturally end with that run. RPC uses a persistent host to accept further commands and stream later events, so an RPC client must correlate observations across its longer-lived process and request stream.

15.13.2 Host telemetry recommendations

For a production-like integration, capture only the fields needed to diagnose an incident:

- selected mode and cwd;
- selected model/provider/API type, without credentials;
- session ID/path and persistence state;
- extension/resource load diagnostics;
- agent/turn/tool timing and completion reason;
- provider status metadata when safely available;
- cancellation/retry/compaction decisions.

This list is host guidance, not a claim that Pi emits each item as a ready-made telemetry record. For correlation, a host may add a sanitized request ID (for example `X-Request-Id`) through `before_provider_headers`; never collect full prompts, raw provider payloads, credentials, or auth headers by default.

15.14 Choosing an integration approach

Goal	Best starting point
Human coding in a terminal	Interactive CLI
One scripted answer	Print mode
Machine-readable event/result stream	JSON mode
Long-lived external controller	RPC mode
Node application sharing Pi semantics	`createAgentSession` SDK
New model protocol/provider behavior	Provider extension or `packages/ai` contribution
Custom execution policy/UI in existing Pi	Extension

Choosing the boundary early is an architectural decision. It determines who owns terminal state, persistence, authentication, user confirmation, and process lifetime.

15.15 Operational checklist

Before putting Pi into automation or another host:

1. Choose the mode/SDK boundary intentionally.
2. Set a trust policy appropriate for noninteractive projects.
3. Select or restrict models/tools explicitly if policy requires it.
4. Store credentials in supported auth/config mechanisms, not prompt text.
5. Ensure output/logging does not corrupt JSON/RPC protocols.
6. Forward cancellation from the host into Pi's abortable work.
7. Decide session persistence and cleanup behavior.
8. Add an OS/container sandbox if untrusted code or high-risk tools are in scope.

These operational decisions are the final layer of the architecture: Pi's internal boundaries are valuable only when the host uses them deliberately. For concrete recovery behavior at these boundaries, continue with Failure and recovery reference; for the provider layer behind every mode, see Models, providers, and authentication.

Part VI

Practice and reference

16 Testing, debugging, and contribution workflow

This chapter turns the system map from Architecture into a practical workflow for reproducing failures, choosing tests, and preparing a change for review. Before code changes, read Extensions for extension surfaces and Agent core for loop ownership. Pi's repository rules distinguish code checks, targeted tests, interactive testing, and real-provider behavior. Use the narrowest validation path that exercises the behavior. A local state transition does not require a provider call.

Repository-wide contribution rules live in `AGENTS.md`. Coding-agent-specific development and extension guidance lives in the coding-agent docs: `packages/coding-agent/docs/`. For runtime failure ownership and recovery behavior, see Failure and recovery.

16.1 Three kinds of evidence

A **check** asks whether code satisfies static rules, a **test** asks whether a repeatable behavior still works, and a **smoke test** exercises a real host such as the interactive TUI. They answer different questions. For a tool change, start with a focused test for success and failure, then use the TUI only if the rendering or input path also changed.

When debugging, first write the expected boundary in one sentence: "the extension should block this validated `bash` call," or "the model should receive this converted message." Then inspect the first boundary where reality differs. The tables below turn that sentence into a repeatable workflow.

16.2 Localize the failure first

For a report such as "Pi ignored my extension" or "a tool result rendered wrong," identify the first subsystem where observed behavior diverges from the expected path before changing code:

Symptom	Inspect first (in order)
Extension is absent	1) trust decision → 2) resource loader discovery → 3) factory load error
Command runs but prompt is unchanged	1) extension command event → 2) input handling → 3) before-agent path
Model gets wrong messages	1) session context → 2) `context` hook → 3) `convertToLlm` → 4) provider payload
Tool has invalid args	1) schema → 2) `prepareArguments` → 3) model tool-call block
Tool output is lost	1) truncation → 2) tool result → 3) parallel ordering → 4) renderer
Provider call fails	1) model registry/auth → 2) adapter response → 3) retry configuration

Symptom	Inspect first (in order)
UI is stale or clipped	1) session event → 2) renderer component → 3) TUI invalidation/width
Resume/fork loses state	1) session branch/context → 2) extension reconstruction → 3) stale replacement closure

This map prevents wasted work: editing the TUI cannot fix an agent that never emitted a message event, and editing a provider adapter cannot fix an extension blocked by project trust.

16.3 Static checking versus tests

After code changes, repository policy calls for `npm run check`. It formats and checks the workspace, verifies pinned dependencies/import rules/shrinkwrap state, and runs the root TypeScript check. It is a broad code-quality gate, not the test suite.

`npm run build` only compiles packages; it is not a test suite. Tests verify behavioral expectations. The full Vitest (the project's test framework) suite can activate end-to-end tests when provider or authentication environment variables are present. To avoid that and get broad non-LLM coverage, use `./test.sh` from the repository root. For a targeted coding-agent test, run the Vitest CLI from the package root as described by `AGENTS.md`.

16.4 Test at the correct package

Change area	First test location/type
Provider serialization/stream parsing	`packages/ai/test/` unit/regression tests
Agent tool scheduling/message events	`packages/agent` tests or harness tests
TUI width, key, component behavior	`packages/tui/test/`
Coding-agent sessions/config/extensions	`packages/coding-agent/test/`
Agent behavior without a paid provider	`packages/coding-agent/test/suite/` with the faux provider

The **faux provider** is a deterministic test double: it returns scripted model responses without network calls, so agent-behavior tests run offline and repeatably. The coding-agent suite harness is intentional. For tests under `packages/coding-agent/test/suite/`, use `test/suite/harness.ts` and the faux provider. Real model keys introduce cost, nondeterminism, rate limits, and a different failure surface than the behavior being tested.

16.5 Your first faux-provider test

This is a complete offline suite test. From the coding-agent package root, copy it into the test suite as `guide-faux-provider.test.ts`:

```ts
import { fauxAssistantMessage } from "@earendil-works/pi-ai";
import { describe, expect, it } from "vitest";
import { createHarness, getMessageText } from "./harness.ts";

describe("guide faux-provider example", () => {
  it("runs one deterministic response through AgentSession", async () => {
    const harness = await createHarness();
    try {
      harness.setResponses([fauxAssistantMessage("offline response")]);
      await harness.session.prompt("hello from the test");

      expect(
        harness.session.messages.map((message) => message.role),
      ).toEqual(["user", "assistant"]);
      expect(getMessageText(harness.session.messages[0])).toBe("hello from the test");
      expect(getMessageText(harness.session.messages[1])).toBe("offline response");
      expect(harness.eventsOfType("agent_end")).toHaveLength(1);
      expect(harness.getPendingResponseCount()).toBe(0);
    } finally {
      harness.cleanup();
    }
  });
});
```

Run it from `packages/coding-agent` (the package root):

```sh
node ../../node_modules/vitest/dist/cli.js --run \
  test/suite/guide-faux-provider.test.ts
```

The stages are deliberate:

1. **Arrange:** `createHarness()` installs an in-memory session and the faux provider; `setResponses()` scripts exactly one response.
2. **Act:** `AgentSession.prompt()` runs the real prompt preprocessing and agent loop, without a network request or external provider credential.
3. **Assert:** check durable message roles/text, an `agent_end` event, and that no scripted response remains pending.
4. **Clean up:** `finally` disposes the session, unregisters the faux provider, and removes the harness temporary directory.

The test uses no real provider, API key, network call, or paid token (the harness's in-memory `faux-key` is only a local test value). Keep the source-verified harness API in sync with `packages/coding-agent/test/suite/harness.ts`; do not replace it with a guessed SDK shortcut.

16.5.1 A compact regression pattern

For an issue-specific regression, use a file such as `test/suite/regressions/<issue>-<slug>.test.ts` from the coding-agent package root; for example, `regressions/3317-network-connection-lost-retry.test.ts`.

Keep the test focused and assert the observable event, not only a private counter:

```
harness.setResponses([
  fauxAssistantMessage("", { stopReason: "error", errorMessage: "Network connection lost." }),
  fauxAssistantMessage("recovered"),
]);
await harness.session.prompt("test");
expect(harness.eventsOfType("auto_retry_end").map((event) => event.success)).toEqual([true]);
```

16.6 Your first contribution

The following scenario is illustrative: it is a workflow example, not a claim about a current issue. Use it for a small change such as making a tool error more actionable.

1. Classify the intended seam with Architecture: decide whether the behavior belongs to the agent loop, a tool, an extension hook, the session/runtime, or the TUI.
2. Locate the owning symbol and its call sites. Use `rg` (ripgrep, a fast repository search tool), or another search tool, then read the symbol's implementation and a nearby test.
3. Write the smallest focused failing test in the owning package. For coding agent behavior, prefer the faux-provider harness above; for an issue regression, use a file under `test/suite/regressions/`.
4. Make the smallest change that makes the test pass. Preserve public event and message contracts unless the change explicitly updates them.
5. Run the targeted test from the package root with the actual Vitest command, for example: `node ../../node_modules/vitest/dist/cli.js --run test/suite/guide-faux-provider.test.ts`.
6. For code changes, run `npm run check` from the repository root. This is the static/type/dependency check, not a replacement for the focused test.
7. Inspect `git diff --check` and the diff for unrelated files, generated metadata, lockfiles, secrets, and accidental broad behavior changes.
8. Apply the existing pre-review checklist below: reproduce the original behavior, name the owning package/contract, run the narrow proof, check trust/security, cover cancellation/error/reload paths where relevant, and inspect the final diff.

A documentation-only change follows the same source-reading discipline, but its interface is the rendered guide: use `quarto render developer-guide`, check links and warnings, and verify every runnable listing against the checkout. This "documentation as a tested interface" rule is for guide contributors; it does not turn documentation edits into product tests.

16.7 High-value test cases by subsystem

16.7.1 Extensions

- Discovery from global/project/CLI paths and trust gating.
- Async factory completion before provider/model use.
- Correct event ordering and return semantics.
- Tool block/allow behavior for policy hooks.

- Reload cleanup and state reconstruction.
- Non-interactive fallback when the extension context reports `ctx.hasUI === false`.
- Session replacement with a fresh `withSession` context.

16.7.2 Tools

- Schema validation and legacy `prepareArguments` shape.
- Success, thrown error, abort, and timeout.
- Bounded output and continuation/full-output notices.
- Parallel sibling behavior and same-file mutation queue.
- Renderer partial/final/expanded behavior.

16.7.3 Sessions and compaction

- JSONL (newline-delimited JSON) loading/migration and malformed input.
- Tree path selection and label/session-info changes.
- Context rebuild with compaction and branch summaries.
- Model/thinking restoration.
- Resume/fork/new runtime cleanup and extension lifecycle.

16.7.4 Provider/model registry

- Custom models/overrides merge semantics.
- Auth source precedence and command/env configuration.
- Header merging and `authHeader` behavior.
- Dynamic register/unregister restoration.
- Compatibility flags for a protocol-specific regression.

16.8 Interactive TUI smoke testing

`AGENTS.md` documents a controlled tmux (terminal multiplexer) pattern for testing the interactive application. The important principle is to use a detached terminal with known dimensions, capture the pane after startup, send text/keys, then cleanly kill the session. This produces a repeatable artifact for editor, command, streaming, and rendering behavior without relying on the developer's current terminal state.

For a UI reproduction, record dimensions, theme/keybindings, exact input and keys, model/streaming state, and captured lines or a `/debug` log. Screenshots alone often omit what is needed to reproduce a visual bug. A condensed smoke sequence is:

```
tmux new-session -d -s pi-test -x 80 -y 24
tmux send-keys -t pi-test "./pi-test.sh" Enter
sleep 3 && tmux capture-pane -t pi-test -p
tmux send-keys -t pi-test "say exactly: ok" Enter
sleep 3 && tmux capture-pane -t pi-test -p
tmux kill-session -t pi-test
```

Run this from the checkout, where `./pi-test.sh` is available. Always kill the detached session in cleanup.

16.9 `/debug` and diagnostic data

In interactive mode, type `/debug` as a built-in slash command. It writes `~/.pi/agent/pi-debug.log` with rendered TUI lines containing ANSI codes and all current `session.messages` sent to the LLM, including user, assistant, and tool-result history. Comparing those two views can reveal whether a mismatch arose before or during rendering. The log may contain sensitive prompts, file contents, and tool results, so review and redact it before sharing any excerpt publicly.

Extension diagnostics should be equally disciplined. Log an event name, safe IDs, timing, and sanitized state—not API keys, auth headers, complete provider payloads, or full context-file content.

16.10 Error classification before retrying

Pi has several retry-like mechanisms with different ownership:

Failure/condition	Owner	Typical action
Invalid tool input or tool execution error	Agent loop/model	Return error tool result; model may recover
Transport/provider transient error	AgentSession/provider settings	Retry according to configured policy
Context overflow	AgentSession compaction policy	Compact and retry if applicable
User abort	Active run abort controller	Stop abort-aware work; do not auto-retry blindly
Extension hook error	Extension runner	Log; tool-call errors fail safe by blocking
Project not trusted	Trust manager	Do not load project code/resources

Misclassifying these leads to bad fixes. Retrying an invalid schema will not correct model arguments; compacting a missing API key is pointless.

16.11 Source navigation habits

The repository is large enough that broad search results can mislead. A useful sequence is to find a public symbol with `rg`, read its contract and focused call sites, identify which package owns the next step, and then read a nearby test or example. Change one subsystem at a time and add a regression where needed.

Examples are valuable for extensions. Start with `packages/coding-agent/examples/extensions/hello.ts`, then choose a focused example such as `permission-gate.ts`, `dynamic-tools.ts`, `entry-renderer.ts`, or `custom-provider-*` instead of copying a large multi-feature extension.

16.12 Git hygiene in a shared worktree

Multiple concurrent agent sessions may edit different files in the same working tree. Stage only explicit paths you changed in your session; never use broad staging commands. Inspect `git status` before staging/committing, and do not reset, clean, stash, or check out unrelated work. `AGENTS.md` also prohibits commits unless the user asks.

For generated model metadata, change the generator source rather than editing the generated file by hand: `packages/ai/src/models.generated.ts`. That policy preserves the link between catalog data and its generation process.

16.13 Documentation as a tested interface

This guide is source-grounded documentation. For documentation-only changes, render it with `quarto render developer-guide` from the repository root. Keep its code examples subject to the same review questions as implementation:

1. Does every exported symbol exist at the documented package path?
2. Does the example respect actual async/cancellation semantics?
3. Does a lifecycle claim match the runner/agent implementation?
4. Does a configuration example state merge/replace/trust semantics?
5. Does a visual description name the responsible renderer or component?

When an API changes, update its focused chapter and the prompt lifecycle chapter together. The lifecycle is the integration test for the book: it reveals contradictions between isolated component explanations.

16.14 Pre-review checklist

Before requesting review for a code or extension change:

1. Reproduce the original behavior with a minimal scenario. For an extension, this may be a one-file extension and one prompt; for agent behavior, use a focused faux-provider test; for a TUI issue, use a deterministic fixture and terminal size.
2. State the package and public contract that own the fix.
3. Add/run the narrowest regression test or controlled interactive proof.
4. Run the repository-required static check for code changes.
5. Check trust/security implications of new configuration or package loading.
6. Test cancellation, error, and reload/replacement paths where relevant.
7. Inspect diffs for generated/lockfile/dependency changes you did not intend.

This workflow keeps extension changes reviewable: each change has a clear runtime owner and a validation step that produces concrete evidence.

17 Guided source tours and architecture exercises

The earlier chapters describe Pi's architecture. Before starting these tours, read Architecture, Agent core, and Testing and debugging. This chapter turns that map into repeatable reading exercises. Each tour has a question, an entry point, the important control-flow boundary, and a concrete outcome. Work through them with the source open; the purpose is to learn how the modules connect, not to memorize file names. All paths in this chapter are relative to the repository root.

17.1 How to do a source tour

Each tour has four parts: a **question**, a starting file, a short list of boundaries to trace, and an **outcome** you can explain in your own words. Before opening the source, predict the path. Then verify the prediction with one symbol search and one nearby test or example. For instance, before Tour 1 you should predict that `cli.ts` parses arguments and hands control to a mode; the tour shows where that prediction is right and where it is incomplete.

17.2 Tour 1: What launches when `pi` starts?

Question: Which module turns process arguments into an interactive agent?

Start at `packages/coding-agent/src/cli.ts`. It assigns `process.title` to `APP_NAME`, sets `process.env.PI_CODING_AGENT` to `"true"`, configures the HTTP connection manager early, and calls `main(process.argv.slice(2))`. In `packages/coding-agent/src/main.ts`, focus on `main()` and the call to `resolveAppMode()` rather than reading the entire file linearly.

Trace these checkpoints:

1. `parseArgs` turns raw argv into a typed argument object.
2. `resolveAppMode` selects RPC, JSON, print, or interactive behavior from explicit flags and TTY (terminal) state.
3. Startup settings/trust/session inputs are prepared.
4. `createAgentSessionRuntime` creates services and a session.
5. `InteractiveMode`, `runRpcMode`, or `runPrintMode` receives that runtime.

Outcome: You should be able to answer why a piped stdin invocation may use print mode and why interactive mode can keep the caller's cwd while resolving Pi's own assets from its package configuration.

17.3 Tour 2: Why can a session change cwd safely?

Question: What is recreated when `/resume` opens a session from another working directory?

Start at `packages/coding-agent/src/core/agent-session-runtime.ts`. Read `switchSession`, then the shared teardown/apply helpers. Follow the `createAgentSessionRuntime` factory passed from `packages/coding-agent/src/main.ts`.

Before reading implementation, predict which objects must be replaced and write your own sequence. Then compare it with this verified outline:

```
old session + old cwd services
  -> session shutdown/dispose
  -> SessionManager.open(target)
  -> target cwd services
  -> target AgentSession
  -> rebind interactive UI
```

Check which objects are cwd-bound: `SettingsManager`, `DefaultResourceLoader`, and the runtime's session/model configuration all depend on the effective cwd. The shared global auth storage can remain reusable, but its resolution is still consulted by the new model registry.

Outcome: You should understand why extension code must not reuse an old session context after a replacement and why `withSession` exists.

17.4 Tour 3: How do resources reach a system prompt?

Question: Where do `AGENTS.md`, skills, prompts, and tool guidance join?

Start at `packages/coding-agent/src/core/resource-loader.ts`. Find callers of `getSystemPrompt`, `getSkills`, and `getAgentsFiles` in `packages/coding-agent/src/core/agent-session.ts`. Read `_rebuildSystemPrompt` and then `packages/coding-agent/src/core/system-prompt.ts`.

Write the data as a structured object before it becomes text:

```
cwd
context files
skills
custom/append prompt
selected tools
tool snippets
tool guidelines
      -> BuildSystemPromptOptions -> string
```

Now inspect `setActiveToolsByName`. It rebuilds the prompt because a changed tool selection must change the instructions available to the model.

Outcome: You can explain why adding a tool has both a runtime registry effect and a prompt/context effect.

17.5 Tour 4: What happens to a user message before a provider sees it?

Question: Which transformations can change user text?

Read `AgentSession.prompt` in `packages/coding-agent/src/core/agent-session.ts` top to bottom. List each return/branch in order:

1. Extension command resolution.
2. `input` event and possible transform/handled result.
3. Skill/template expansion.
4. Streaming steer/follow-up queue behavior.
5. Auth/model checks and possible compaction.
6. User/custom message construction.
7. `before_agent_start` message/system-prompt modifications.
8. `_runAgentPrompt`.

Then read the `Agent` call path and `streamAssistantResponse` to see the later context transform plus `convertToLlm` boundary.

Outcome: You can identify where `input`, `before_agent_start`, `context`, and `before_provider_request` run, then narrow your investigation to the hook that sees the representation you need to change.

17.6 Tour 5: Follow one streamed token to the terminal

Question: How does a provider delta become a changed terminal row?

This tour is the concrete entry for the Terminal UI and interactive mode. Start in the agent loop at `streamAssistantResponse`. Find how `text_delta`, `thinking_delta`, and `toolcall_delta` replace the partial assistant message and emit `message_update`. Then walk this bridge without inventing extra stages:

1. `Agent.processEvents` in `packages/agent/src/agent.ts` stores the partial assistant message and awaits registered listeners.
2. `AgentSession._handleAgentEvent` in `packages/coding-agent/src/core/agent-session.ts` re-emits a session `message_update` (including extension handlers) and fans it out through `AgentSession.subscribe`.
3. `InteractiveMode.subscribeToAgent` calls `handleEvent` in the interactive mode.
4. The `message_update` branch updates `AssistantMessageComponent` via `updateContent`, creates or refreshes `ToolExecutionComponent` rows when tool-call content appears, and calls `ui.requestRender()` so the TUI package diffs the terminal frame. These components live under `packages/coding-agent/src/modes/interactive/components/`.

Do not try to read `interactive-mode.ts` linearly; search for the event type and the component method it calls.

Outcome: You can locate a rendering bug as one of four categories: provider event normalization, agent partial state, session event bridge, or interactive component update.

17.7 Tour 6: Follow one tool call through validation

Question: Why cannot an extension tool execute arbitrary model JSON?

Start at `executeToolCalls` in `packages/agent/src/agent-loop.ts`. Follow one call through:

```
assistant content block
  -> active tool lookup
  -> prepareArguments
  -> validateToolArguments
  -> beforeToolCall
  -> execute
  -> afterToolCall
  -> ToolResultMessage
```

Next open `packages/coding-agent/examples/extensions/hello.ts` and identify where its TypeBox schema becomes both a static parameter type and a runtime validator. Compare it with `packages/coding-agent/src/core/tools/read.ts`, which adds operations and renderers but retains the same fundamental definition shape.

Outcome: You can explain why a schema belongs in an extension even when TypeScript already knows the desired params type.

17.8 Tour 7: Study parallel tool execution without guessing

Question: What ordering is guaranteed when an assistant asks for two tools?

Read `executeToolCallsParallel` and contrast it with the sequential path. Write a timeline for two calls A and B:

```
start(A), validate/gate(A)
start(B), validate/gate(B)
execute(A) and execute(B) concurrently
end events in completion order
final ToolResultMessage(A), ToolResultMessage(B) in source order
```

Compare that timeline with the documented guarantees in `packages/coding-agent/docs/extensions.md` under the `tool_execution_start` / `tool_execution_update` / `tool_execution_end` events: start events are source-ordered during preflight, update/end events can finish in completion order, and final `toolResult` messages are still emitted in source order. Then inspect the file-mutation queue in `packages/coding-agent/src/core/tools/` and its `withFileMutationQueue` helper to see how same-file writes gain a narrower serialization guarantee without making unrelated tools sequential.

Outcome: You can design an extension tool that works correctly under the default mode rather than accidentally relying on timing.

17.9 Tour 8: Explain a model request without opening a network trace

Question: Where do key, headers, model metadata, and payload come from?

First open `packages/coding-agent/src/core/sdk.ts`. This module wires the coding-agent bridge from model-runtime configuration to the AI package's streaming API. In its stream function, trace how `ModelRuntime.streamSimple()` prepares auth and headers, emits `before_provider_headers`, and selects the composed provider for the model's API.

Follow into `packages/coding-agent/src/core/model-runtime.ts`, then into the composed provider and an adapter such as `packages/ai/src/api/openai-completions.ts` or `packages/ai/src/api/anthropic-mes` identify its serialization and event-stream parser entry points. `packages/ai/src/compat.ts` is a compatibility/default fallback, not the primary coding-agent route.

Outcome: You can distinguish configuration bugs (registry), protocol bugs (adapter), and runtime-loop bugs (agent) without a speculative rewrite.

17.10 Tour 9: Trace a resume-context discrepancy

Question: Why did a resumed model refer to an old branch or summary?

Open `packages/coding-agent/src/core/session-manager.ts` and read `buildContextEntries` plus `buildSessionContext`. Use this small tree before opening the source:

```
root session_start
   user prompt
       compaction summary
           current assistant/tool entries  <- active leaf
```

Then add a branch summary as a sibling of the current path. Trace the active leaf backwards, then identify which entries become context messages and which are metadata only.

Cross-check with `packages/coding-agent/docs/session-format.md`. Use `getBranch` and `buildContextEntries` from an extension command to inspect an actual session instead of parsing JSONL manually at first. The standalone `buildSessionContext(entries, leafId?)` helper can also be imported and called with `ctx.sessionManager.getEntries()` and `ctx.sessionManager.getLeafId()`; it is not an instance method on `ctx.sessionManager`.

Outcome: You can prove whether a context difference is persistence policy, compaction, branch navigation, or an extension's reconstruction mistake.

17.11 Tour 10: Add a narrowly scoped extension feature

Question: How do you add a feature without coupling to internals?

Pick one small behavior such as a status widget, a custom command, or a read-only tool. Start from the closest example. Name the lifecycle event/API that owns it, then write a one-file extension under a global auto-discovery directory. Verify startup, action, `/reload`, and noninteractive behavior. `/reload` reloads keybindings, extensions, skills, prompts, themes, and context files without quitting

the process; treat it as a full runtime/resource refresh (`AgentSession.reload`), so do not reuse a pre-reload extension context.

Before writing code, classify the behavior by the narrowest public seam that owns it. Use this table to make that decision, then confirm the selected API in the extension documentation or nearest example:

Desired result	Narrow public seam
Ask model to use a new operation	`registerTool`
Let user invoke a task	`registerCommand`
Block/mutate a tool call	`on("tool_call")`
Add context per user prompt	`on("before_agent_start")`
Change provider endpoint/models	`registerProvider`
Store private session state	`appendEntry`
Render a custom persistent record	`registerEntryRenderer`
Add a small always-visible indicator	`ctx.ui.setStatus` or widget

Outcome: You practice the extension API as a set of explicit seams rather than as permission to reach into `InteractiveMode` or `AgentSession` private state.

17.12 A weekly reading routine for new contributors

Use Testing and debugging's failure-localization table during sessions 2 and 4, and use Agent core's debugging checklist when a source tour reaches a stalled run. The terminal user interface (TUI) is the interactive layer that turns session events into terminal components and rendered rows. Treat it as its own reading boundary rather than as another name for the entire coding-agent package.

Large codebases become legible through repeated, bounded questions. A practical routine is:

Session	Focus	Deliverable to yourself
1	CLI startup and services	One startup diagram with package boundaries
2	Agent loop and messages	One provider/tool turn timeline
3	Sessions/compaction	One active-branch/context sketch
4	Built-in tools	One tool contract comparison
5	Extensions (Tour 10)	A working one-file extension plus `/reload` and noninteractive checks
6	TUI (Tour 5)	Trace one `message_update` from the agent loop to the assistant renderer
7	Provider config (Tour 8)	A local/custom provider configuration review

The guide is intentionally organized so each exercise has a corresponding chapter. Revisit the prompt lifecycle after every tour: it is the shortest integration model of the entire project.

18 Extension event and API reference

This chapter is a practical reference for choosing extension hooks. Read Extensions first for registration and tool basics, Built-in tools for tool lifecycle, Models for provider hooks, and Run modes for mode-specific behavior. It does not replace the source types in the extension types file, `packages/coding-agent/src/core/extensions/types.ts`, the exhaustive `packages/coding-agent/docs/extensions.md`, or the official Extensions documentation; use those when writing code. Its goal is to make event choice, return value, context, and lifecycle safety explicit before implementation.

18.1 Use this chapter as an index

An **event** is a notification with a payload and a timing guarantee. An event is **observation-only** when a handler can inspect it but cannot change the operation. A **return value** is a handler's supported way to continue, transform, or block work. Start with the first table: choose the latest event that still contains the representation you need, then read that event's subsection for ordering, mode, and error behavior.

For example, use `input` to change submitted text, `context` to change the message view before every provider request, and `tool_call` to gate validated tool arguments. Those three examples look similar from the UI but occur at different boundaries.

18.2 Choose an event by the representation you need

The same user action appears in several representations. Select the latest boundary that still contains the information you need, because later hooks are more specific and less likely to disturb unrelated behavior. The rows below follow the normal prompt/turn pipeline from submitted text to tool output; `user_bash` and durable session operations are separate paths shown last.

Need to inspect/change	Event/API	Representation
Raw submitted text before skills/templates	`input`	Text/images plus source/delivery mode
Prompt-specific system context	`before_agent_start`	Expanded prompt and system-prompt options
Message history before every LLM call	`context`	Deep-copied agent messages
Final HTTP headers	`before_provider_headers`	Mutable header map
Provider wire payload	`before_provider_request`	Adapter-specific payload
Provider status/response headers	`after_provider_response`	HTTP metadata before stream consumption

Need to inspect/change	Event/API	Representation
Validated tool arguments	`tool_call`	Tool name and mutable input
Streaming/final tool execution row	`tool_execution_start` / `_update` / `_end`	Observation-only lifecycle payloads
Tool execution output (patchable)	`tool_result`	Content/details/`isError`/`usage` patch opportunity
Final tool transcript message	`message_start` / `message_end` with `role: "toolResult"`	Durable tool-result message after patching
User typed `!` command	`user_bash`	User command and context-exclusion choice
Cancel or customize compaction	`session_before_compact`	Prep metadata; may cancel or supply summary
Observe completed compaction	`session_compact`	Saved compaction entry and source flags
Cancel or summarize tree navigation	`session_before_tree`	Branch preparation; may cancel or supply summary
Observe tree navigation	`session_tree`	New leaf, old leaf, and summary outcome
Initiate session replacement from an extension	Command context (`newSession`, `fork`, `navigateTree`, `switchSession`, `reload`)	User-triggered only; not ordinary event handlers

The rows describe different capabilities, not aliases. For example, a context filter changes Pi's model-message view, whereas provider hooks operate only at the serialized request boundary. Durable session hooks change neither unless their operation rebuilds the active context.

18.2.1 `user_bash`

`user_bash` is the direct-user shell path for commands beginning with `!`. It is not the model `bash` tool path. A handler receives the command and context and can provide custom operations or a complete result. Its return shape has only `operations?` and `result?`; it has no `block` field or context-inclusion toggle. Test this path separately from `tool_call`/`tool_result`; a sandbox that covers only model tools does not cover commands typed directly by a user.

18.3 Startup and trust events

18.3.1 `project_trust`

This event occurs before Pi decides whether project-local dynamic resources may run. It is visible only to user/global and CLI-provided extensions, not project extensions. The handler must return a trust decision:

```ts
pi.on("project_trust", async (event, ctx) => {
  if (!ctx.hasUI) return { trusted: "undecided" };
  const yes = await ctx.ui.confirm("Trust project?", event.cwd);
  return yes ? { trusted: "yes", remember: true } : { trusted: "undecided" };
});
```

Return `"undecided"` when your extension does not own the policy. A yes/no decision suppresses later handlers and the built-in flow. Only use `remember` when it is appropriate to persist a global trust decision for this user.

18.3.2 `resources_discover`

This fires after `session_start` so an extension can return additional skill, prompt, and theme paths. It receives `reason: "startup" | "reload"`. It is a discovery hook, not an instruction to read arbitrary project paths without checking trust. Use the surrounding extension/session context's trust state for project-local behavior.

18.4 Session events

18.4.1 `session_start` and `session_shutdown`

Use `session_start` to initialize state that requires an active session: restore durable data, add UI status, start a watcher, or open a connection. Its reason identifies startup, reload, new, resume, or fork. Use `session_shutdown` to close that same resource. Cleanup must be idempotent because reload, replacement, and quit may all trigger it.

Do not start long-lived resources in the factory. A factory may run for a command that never creates a session, while session events make lifecycle ownership explicit.

18.4.2 `session_before_switch` and `session_before_fork`

These are cancellation gates. A switch handler sees whether the operation is `new` or `resume` plus an optional target path. A fork handler sees entry ID and position (`before` for fork or `at` for clone). Return `{ cancel: true }` to block a user action after a confirmation or policy check:

```ts
pi.on("session_before_switch", async (event, ctx) => {
  if (!ctx.hasUI) return;
  const ok = await ctx.ui.confirm(
    "Change session?",
    event.reason === "new" ? "Start a new session?" : "Open another session?",
  );
  if (!ok) return { cancel: true };
});
```

Do not call session-changing methods such as `newSession`, `fork`, `navigateTree`, `switchSession`, or `reload` from an ordinary event handler. If an extension must initiate one, use a command handler with the command-context APIs described later in this chapter.

18.4.3 Compaction and tree hooks

Compaction replaces older messages on the active branch in model context with a durable summary while retaining a recent suffix; it does not delete the underlying session entries. `session_before_compact` can cancel that operation or return a custom summary plus the required kept-entry/token metadata. `session_compact` observes the durable result and whether it came from an extension.

```
pi.on("session_before_compact", async (event, ctx) => {
  const { preparation, reason, willRetry } = event;
  // reason: "manual" | "threshold" | "overflow"
  // willRetry is true for overflow recovery that retries the aborted turn.

  if (reason === "manual" && !ctx.hasUI) {
    return { cancel: true };
  }

  return {
    compaction: {
      summary: `Kept from ${preparation.firstKeptEntryId}`,
      firstKeptEntryId: preparation.firstKeptEntryId,
      tokensBefore: preparation.tokensBefore,
    },
  };
});

pi.on("session_compact", (event) => {
  // event.compactionEntry, event.fromExtension, event.reason, event.willRetry
});
```

A session is a tree of durable entries linked by `id` and `parentId`; the current leaf (the selected last entry) selects the root-to-leaf branch used for context. `/tree` navigation moves that leaf within the same session file and may summarize the branch being left. `session_before_tree` can cancel or provide that branch summary; `session_tree` observes the selected leaf and summary outcome.

```
pi.on("session_before_tree", async (event) => {
  const { preparation } = event;
  // Summary is only used when the user requested one for this navigation.
  if (!preparation.userWantsSummary) return;
  return {
    summary: {
      summary: `Leaving ${preparation.oldLeafId} for ${preparation.targetId}`,
      details: {},
```

```
    },
  };
});

pi.on("session_tree", (event) => {
  // event.newLeafId, event.oldLeafId, event.summaryEntry, event.fromExtension
});
```

Use these hooks for policy or summary customization. Use `context` if the goal is only to hide/filter data for one provider call; a context hook does not change durable compaction or tree state. For the underlying session-to-context model, start with Sessions, trees, and compaction; for cut points, split turns, and the complete custom-summary contract, see the official Compaction documentation.

18.4.4 Session information

`session_info_changed` fires after `/name` or `pi.setSessionName`. It is a good place to update a title/status integration. It is not a model message and does not require rebuilding the system prompt.

18.5 Input and agent-run events

18.5.1 `input`

Input handlers run after extension command lookup and before skill/template expansion. The result controls the next step:

Result	Effect
no result / `continue`	Leave input unchanged
`transform` with text/images	Pass modified input into later expansion
`handled`	Stop input processing; no agent run starts

Transforms chain in extension load order. Keep transforms simple and make source-aware decisions with `event.source` (`interactive`, `rpc`, or `extension`). During streaming, `event.streamingBehavior` distinguishes **steering** (injecting a message before the next provider request) from a **follow-up** (queueing a message after the current run settles).

18.5.2 `before_agent_start`

This runs after normal input expansion but before the low-level agent loop. It is the best prompt-scoped seam. A handler returns an optional object containing custom messages and/or chained system-prompt options; returning nothing leaves the prepared prompt unchanged. A result may add a custom persistent message and/or replace the chained system prompt. Later handlers receive system-prompt changes from earlier handlers.

The event exposes `systemPromptOptions`, including custom prompt, selected tools, snippets/guidelines, cwd, context files, and skills. Treat it as sensitive extension-local data. It may contain full instructions that should not be copied into command descriptions or telemetry.

18.5.3 `agent_start`, `agent_end`, and `agent_settled`

A low-level agent run begins at `agent_start`, contains one or more turns, and ends at `agent_end`. That end does not prove application idleness: coding-agent may start another run for an automatic retry, compact-and-retry recovery, or a queued continuation. `agent_settled` fires only when that outer processing cycle has no automatic work left. Use settled for a status integration that must not clear itself between runs.

18.5.4 `turn_start`, `turn_end`, and messages

Within a run, each turn is one LLM request and assistant response plus the tool calls/results produced by that response. If tools run and the agent continues, the next provider call is another turn in the same run. `turn_end` includes the assistant message and final tool results. Message events give fine-grained lifecycle: user/assistant/tool-result messages emit start/end; assistant streaming also emits update events. `message_end` can replace the finalized message while preserving its role.

Message replacement is powerful. Preserve required fields and do not change a role merely to repurpose a transcript; provider conversion and session logic rely on role semantics.

18.6 Provider events

These hooks are protocol boundaries, not portable message APIs. The payload is `unknown`, its shape varies by adapter, and payload rewrites are not reflected by `ctx.getSystemPrompt()`. Guard payload logic by provider/model, test it against that provider's exact wire protocol, and avoid logging bodies or headers without redacting prompts, credentials, and provider metadata.

18.6.1 `before_provider_headers`

Pi assembles outgoing headers, then gives handlers a mutable map. Assign a string to add/override, or `null` to delete. The hook runs once per provider request; retries reuse the same headers rather than invoking it again.

Use it for a stable request correlation header, such as an `X-Request-Id` value chosen by the host. Do not use it for a credential refresh protocol that must run anew on every retry unless you explicitly own that behavior.

18.6.2 `before_provider_request`

This sees the adapter-specific payload after Pi has converted context, system prompt, and tools. Return `undefined` to keep it or another value to replace it for later handlers and the actual request. Use it only for a narrow serialization compatibility patch or a sanitized development trace.

18.6.3 `after_provider_response`

This fires after the HTTP response arrives and before its body stream is consumed. It receives normalized status and headers when available. Some provider transports abstract away headers; absence is a provider limitation, not an extension failure.

18.7 Tool events and parallelism

The tool execution path separates observation, policy, and patching:

Event	Role	Handler may
`tool_execution_start`	Observation: tool is about to run	Read `toolCallId`, `toolName`, `args`
`tool_call`	Policy gate after validation	Mutate `event.input` in place, or return `{ block: true, reason? }`
`tool_execution_update`	Observation: partial/streaming output	Read `partialResult`
`tool_result`	Result-patching middleware	Return partial `{ content?, details?, isError?, usage? }`
`tool_execution_end`	Observation: finalized execute result	Read `result` and `isError`

`tool_execution_*` handlers are observation-only: they do not cancel execution or rewrite output. Keep the spelling and layer distinction exact: `tool_result` (snake case) is an extension event, while `toolResult` (camel case) is a message role. The `tool_result` hook can patch the result before it is committed (`content`, `details`, `isError`, and/or `usage`; omitted fields keep their current values). A `toolResult` message is the durable transcript entry (`role: "toolResult"`) that appears later as `message_start`/`message_end` and in the session file. A hook can therefore affect the message that follows, but it is not itself that message.

Not every tool-call outcome reaches `tool_result`: unknown tools, argument validation failures, blocked calls, and early abort/length paths can produce a direct error result during preflight and bypass the post-execution hook. For calls that do execute, the default parallel execution order is subtle:

```
source-order tool_execution_start and tool_call preflight
  -> concurrent execute / tool_execution_update
  -> completion-order tool_result then tool_execution_end
  -> source-order final toolResult messages (message_start / message_end)
```

Therefore, a `tool_call` handler sees session state synchronized through the assistant message but not necessarily sibling tool results. Use a typed helper for built-ins:

```ts
import { isToolCallEventType } from "@earendil-works/pi-coding-agent";

pi.on("tool_call", (event) => {
  if (!isToolCallEventType("read", event)) return;
  // event.input.path is typed for the built-in read tool.
});
```

For a custom tool, derive the TypeBox input type with `Static`. TypeBox is a runtime validation library covered in TypeScript basics. This example imports `Type` and `Static` from the direct `typebox` package; Run modes covers the alternative `pi-ai` re-export path. Supply explicit generic parameters to the guard:

```ts
import { Type, type Static } from "typebox";
import { isToolCallEventType } from "@earendil-works/pi-coding-agent";

const myToolSchema = Type.Object({ action: Type.String() });
export type MyToolInput = Static<typeof myToolSchema>;

pi.on("tool_call", (event) => {
  if (isToolCallEventType<"my_tool", MyToolInput>("my_tool", event)) {
    // event.input.action is typed
  }
});
```

Direct `event.toolName === name` narrowing is not sufficient because arbitrary custom names overlap built-in literals.

18.8 Extension context reference

All handlers receive a context with these recurring values:

Field/method	Practical meaning
`cwd`	Current session working directory
`mode` / `hasUI`	Execution surface and interaction availability
`ui`	Controlled interactive UI adapter
`sessionManager`	Read-only access to durable tree/context state
`modelRegistry` / `model`	Available/current model and authentication view
`signal`	Active agent signal during turn-related work, otherwise usually undefined
`isIdle`, `abort`, `hasPendingMessages`	Flow-control observations/actions
`getContextUsage`	Context-window usage/estimate
`compact`	Trigger compaction with callbacks
`getSystemPrompt`	Current Pi system-prompt string at this lifecycle point

The scope of `signal` is important. A command handler executed while idle does not receive an active turn signal. A `tool_result` handler usually does (`ctx.signal` is the abort signal for that work). Pass it to nested asynchronous work only when it is defined and relevant.

18.9 Command context reference

Command handlers receive `ExtensionCommandContext`, which adds operations that would be unsafe from arbitrary events:

Method	Safe use
`waitForIdle`	Finish automatic work before a session mutation
`newSession`	Create a replacement session, optional setup/handoff
`fork`	Fork/clone from an entry
`navigateTree`	Move to a tree point with optional summary
`switchSession`	Open another session path
`reload`	Run `/reload` lifecycle
`getSystemPromptOptions`	Inspect base prompt inputs in a user-triggered command

Replacement methods accept `withSession`. Use only the callback's context for session-bound work. The old extension instance may have already run shutdown cleanup, and captured old session objects will throw or represent old state.

18.10 Registration API quick decisions

API	Use when	Avoid when
`registerTool`	Model needs a new operation	User-only interaction is enough
`registerCommand`	User needs an explicit action	Input syntax is a natural transform
`registerShortcut`	User needs a configurable keyboard action	You can use a command/menu instead
`registerFlag`	Startup behavior needs a CLI option	State can live in settings
`sendMessage`	Model needs a custom context message	State should remain private
`sendUserMessage`	Agent should receive a genuine user message	You only need a silent state update
`appendEntry`	Durable data should not affect LLM context	You need model-visible instructions
`registerProvider`	Endpoint/models/auth/streaming change	A header-only trace is enough

18.11 Model-facing output truncation

Custom tool output is not truncated automatically. Apply `truncateHead` to file/search-style output or `truncateTail` to logs and command output. Their shared defaults, from `DEFAULT_MAX_LINES` / `DEFAULT_MAX_BYTES` in `packages/coding-agent/src/core/tools/truncate.ts`, are 2,000 lines and 50 KB (`50 * 1024` bytes), whichever is reached first:

```
import { formatSize, truncateHead } from "@earendil-works/pi-coding-agent";

const clipped = truncateHead(output);
let text = clipped.content;
if (clipped.truncated) {
  text += `\n\n[Output truncated: ${clipped.outputLines} of ${clipped.totalLines} lines`;
  text += `; ${formatSize(clipped.outputBytes)} of ${formatSize(clipped.totalBytes)}.`;
  text += " Re-run with a narrower range.]";
}
```

A truncation marker must say what was retained and how to retrieve the rest: provide an offset/narrower-query instruction, or save the full output to a temp file when retaining it is safe and include that path. If individual lines can be enormous, pre-truncate them with `truncateLine` (which limits characters) or provide specialized handling; `truncateHead` returns no partial first line when that line alone exceeds its byte limit. Document limits in the tool description.

18.12 Safety rules for extension authors

1. Start background resources in `session_start`, clean them in `session_shutdown`.
2. Treat project-local config as untrusted until `isProjectTrusted()` says otherwise.
3. Forward cancellation signals.
4. Serialize file mutations with the shared queue.
5. Truncate model-facing output explicitly and make continuation actionable.
6. Use mode/hasUI guards for user prompts and custom components.
7. Reconstruct persistent state from session entries.
8. Use fresh context after session replacement/reload.
9. Prefer public APIs/types over internal imports.

These rules cover the lifecycle errors that are hardest to discover from a happy-path extension demo.

19 Failure and recovery reference

Pi turns many failures into explicit protocol data instead of allowing an exception to escape through every layer. Read Extensions, Resources and trust, and TUI when the lookup below points to those owners. This operational reference is for extension authors and contributors debugging failures or designing recovery; it is a symptom-to-owner lookup, not a step-by-step tutorial.

19.1 How to use the reference

Use these definitions when reading the tables below:

- **Error:** an operation failed and reports failure data.
- **Abort:** work stopped because an explicit cancellation request propagated through an abort signal.
- **Retry:** another provider/agent attempt starts after a failed attempt.
- **Recovery:** a broader safe response, such as an actionable tool result, compaction-and-retry, or a fallback renderer.

Start with the first layer where the symptom appears, then follow that layer's recovery action; do not retry before deciding which kind of failure you have.

For example, malformed tool arguments should become a tool error the model can correct, while a missing API key must be fixed in model/auth configuration. Both may look like "the prompt failed," but they have different owners.

19.2 Failure flow by runtime layer

```
configuration/trust failure
  -> startup diagnostic or ignored project resource

model/provider failure
  -> final assistant message: error or aborted
  -> AgentSession retry/compaction policy may decide next action
     (automatic retry, compaction-and-retry, or final failure)

tool validation/execution failure
  -> error ToolResultMessage
  -> model receives the error and can choose a recovery action

extension hook failure
```

```
  -> runner reports error; tool_call fails safe by blocking

rendering failure
  -> fallback renderer or interactive diagnostic path
```

Each outcome keeps recovery at the layer that can still make a safe decision. Startup rejects or ignores invalid configuration because no reliable runtime exists yet. A provider failure becomes a terminal assistant message because there is no trusted assistant result to continue from, while a tool failure becomes a `ToolResultMessage` because it is useful input to the model's next request. An extension hook fails closed where it controls an operation so a broken policy hook cannot accidentally allow that operation. Rendering alone falls back because presentation failure should not discard otherwise valid result data.

19.3 Startup and configuration failures

Symptom	Owning layer	Expected behavior	First recovery action
Malformed `settings.json` (global `~/.pi/agent/settings.json` or project `.pi/settings.json`) or `models.json`	Settings/model registry	Diagnostic/load error; built-ins remain available where possible	Correct JSON; inspect reported scope/path
Project extension unavailable	Trust/resource loader	Project resources ignored when untrusted	Save/override trust intentionally, then restart/reload
Extension factory fails	Resource loader/application	Startup diagnostics; Pi suggests retry without extensions	Run with `pi -ne -e <path>` to disable discovery while loading one explicit extension; `-e` alone adds a path but does not disable discovery
No model selected	Session setup	Prompt is rejected before agent loop	Configure/choose model and auth
Auth missing	Model registry/session	Prompt is rejected with provider guidance	`/login`, configure key, or select authenticated model

Do not work around a trust failure by making a global extension read an untrusted project's configuration. That moves a deliberate security boundary into code outside Pi's trust boundary, on an uncontrolled path.

19.4 Provider failures

Provider adapters report normal failure/abort outcomes as final assistant messages on the stream rather than as raw exceptions. `Agent.runWithLifecycle()` also converts unexpected throws to a final assistant message whose `stopReason` is `error` or `aborted`, followed by turn/agent end events. The transcript and extensions therefore see a normal terminal event sequence even when the network fails.

`AgentSession` evaluates the final assistant message. Depending on settings and error classification, it may prepare an agent-level retry, auto-compact and retry after an overflow, or finish automatic recovery and emit `agent_settled`. Provider SDK retries and agent-level retries are separate knobs; an excessively long provider retry can delay agent/session-layer retry, compaction, or settle decisions until the provider layer finishes.

For an extension, the correct response to an `agent_end` error is normally to observe/report it, not to immediately create another competing prompt. Use `agent_settled` or a deliberate command action once the built-in policy has finished.

19.5 Tool failures

Tool processing has several failure points:

Point	Result returned to model
Tool name is not active/found	Error result naming the missing tool
`prepareArguments` or schema validation fails	Error result containing the failure reason
`tool_call` policy denies/blocks the call	Error result with the deny/block reason
Tool execute throws	Error result with thrown error text
Abort occurs before/during work	Error result indicating cancellation
`tool_result` hook throws	Diagnostic; original unpatched result is preserved

The loop emits a durable `toolResult` message after execution and the `tool_result` extension hook finishes. If a coding-agent `tool_result` handler throws, the runner catches the hook error, emits a diagnostic, and preserves the original unpatched result; it does not turn that hook failure into an error result. Only an escaping lower-level `afterToolCall` failure becomes an error result. These are different: `tool_result` is the patchable extension event, while `toolResult` is the transcript message role that the model receives. See the extension event reference for ordering and payloads. A model can inspect the resulting message, modify its arguments, call another tool, or explain the failure. Make tool error strings actionable: include an invalid path, missing exact text, exit code, timeout, or next command when safe. Do not put credentials or other secret values in a tool error message; the model and transcript both receive that text.

19.6 Cancellation versus timeout

An abort signal expresses user/application cancellation. A timeout expresses a tool/provider policy deadline. They can lead to similar text in the transcript, but extension code should preserve the distinction:

- Forward abort signals into `fetch`, process execution, and model helpers.
- Clean up partial state deterministically when cancellation happens.
- Use a tool timeout only when the work has a clear bounded deadline.
- Do not convert user cancellation into an automatic retry.

The built-in bash tool kills its process tree for abort and timeout, collects available output, and returns an error with the appropriate status. A custom tool that launches a child process should provide equivalent cleanup rather than leaving a detached process behind.

19.7 Recovery after compaction and tree movement

Compaction/retry is recovery from context pressure, not from every provider error. `firstKeptEntryId` is the ID of the earliest session entry retained verbatim after compaction; entries before it are represented by the summary. The event payloads and cancellation/custom-summary return shape are indexed in the extension event reference. A custom compaction extension that intercepts `session_before_compact` must return a `SessionBeforeCompactResult`. To replace the summary, set `compaction` to an abbreviated `CompactionResult` shaped as `{ summary, firstKeptEntryId, tokensBefore, estimatedTokensAfter?, usage?, details? }`; `estimatedTokensAfter`, `usage`, and `details` are optional. To abort compaction, return `{ cancel: true }`. To keep the default summary from `prepareCompaction`/`compact`, omit `compaction`. Changing or omitting the `firstKeptEntryId` boundary can make context rebuilding retain the wrong entries or omit the intended ones.

Tree navigation changes the active leaf (the selected last entry on the active branch). An extension that reconstructs state from `getEntries()` instead of the active branch may accidentally recover state from an abandoned path. This can look like a failure after `/tree` even though session persistence behaved correctly.

19.8 UI and renderer fallback

A renderer is a function or component that converts tool output or message content into TUI display elements. If a custom tool/message renderer throws, Pi falls back to basic rendering for that slot where supported. Treat a fallback as a diagnostic, not evidence that result data is correct. Inspect the original content/details and renderer context, then make the renderer handle partial, error, and expanded states explicitly.

For a TUI-only issue, replay the same fixture at a narrow width (for example, 40 columns) and a normal width, then test both collapsed and expanded tool output. Narrow width forces wrapping and clipping paths, while toggling output exercises both renderer states. A failure that appears only when ANSI-styled text wraps usually points to a display-column calculation (escape sequences occupy bytes but no visible columns), not to agent or session data.

19.9 Incident checklist

Before filing a bug report or writing a regression test, capture the following:

1. Mode, cwd, trust state, session ID/path, and selected model.
2. The input/event that triggered the problem.
3. Whether it was idle, streaming, compacting, or replacing a session.
4. The relevant final stop reason/tool result error. A `stopReason` records why the final assistant message stopped, such as normal completion, an error, or an abort.
5. Sanitized provider status/headers if applicable.
6. Active tools and extension/resource provenance.
7. Whether Esc/reload/new/resume changes the behavior.
8. A focused test, session tree sketch, or terminal capture proving the result.

This checklist establishes a factual boundary before a fix is proposed. For rendering incidents, pair it with Tour 5: Follow one streamed token to the terminal. For compaction or branch-context incidents, use Tour 9: Trace a resume-context discrepancy.

20 Glossary

Use this page as a lookup table, then follow the linked chapter for examples and source paths. Entries are ordered by concept family so related runtime, session, provider, and extension terms can be compared quickly.

Related: guide index · TypeScript basics · architecture

Agent The stateful runtime in `packages/agent` that owns model context, messages, tools, queues, and one active run. See Agent core.

Agent core The provider-independent package layer that performs model turns, streams, tool batches, queues, and cancellation. See Agent core.

AgentSession The `packages/coding-agent` wrapper around an **Agent**. It coordinates resources, prompts, tools, and extensions. It also manages retries, compaction, and persistence. See Prompt lifecycle and Sessions.

AgentSessionRuntime The lifecycle owner of the current AgentSession and its cwd-bound services, including settings, resources, model registry, and authentication. See Resources, settings, and trust.

Faux provider A deterministic simulated provider that returns scripted model responses without network calls. It is used as a **test double** so agent-behavior tests run offline and repeatably. See Testing, debugging, and contribution.

Host The program that drives Pi, such as the `pi` binary, an RPC client, or an SDK embedding. See Modes, SDK, and operations.

Mode The way a host communicates with Pi: interactive TUI, print, JSON, or RPC. See Modes, SDK, and operations.

RPC mode A run mode where Pi exchanges LF-delimited JSON messages over stdin/stdout, so another process can drive it without a terminal. RPC stands for remote procedure call. When an extension opens a dialog in RPC mode, the client process receives an `extension_ui_request` message and must answer it, otherwise the dialog waits. See Modes, SDK, and operations.

AbortController / AbortSignal An `AbortController` owns cancellation; its `AbortSignal` is passed to work that should stop when the controller is aborted. See TypeScript cancellation and Failure and recovery.

Adapter Code that converts Pi's normalized request and response shapes into one provider's protocol format. See Models, providers, and authentication.

Branch One path through a session tree, represented by linked entries from an ancestor to a selected point. See Sessions and durable context.

Entry One durable record in a session file: a single JSONL line with a `type`, an `id`, a `parentId`, and a timestamp. Messages, tool results, compaction and branch summaries, and extension state are all entries; linked by `parentId` they form the session tree. See Sessions and durable context.

Durable Persisted to disk so it survives restarts, reloads, and session replacement. Session entries are durable; the per-turn model context rebuilt from them is not. See Sessions and durable context.

Leaf The currently selected last entry in a session branch. See Sessions and durable context.

Callback A function passed to another function to be called immediately or later. See TypeScript functions and callbacks.

Compaction Summarizing older context into a durable session entry so recent context fits within a model window. It is not simple transcript deletion. See Sessions and Failure and recovery.

Compaction entry The durable session entry recording a compaction boundary and summary. It is different from the compaction operation that creates it. See Sessions.

Context The provider-facing bundle of system prompt, converted messages, and active tools. It is distinct from the broader persisted session. See Prompt lifecycle.

Context window The model's maximum context budget, measured in tokens, for one request. Pi reserves part of that budget for the response and compacts older session context when the available space is too small. See Sessions and durable context and Models, providers, and authentication.

Barrel A module that only re-exports symbols from other modules, giving a package one public entry point — for example `packages/tui/src/index.ts`. Consumers import from the barrel instead of reaching into internal files. See Architecture.

Compat layer The compatibility API in `packages/ai/src/compat.ts` — `stream()`, `streamSimple()`, and the api-registry — that resolves a provider implementation from `model.api` and streams a response through it. Generic `Agent` uses its explicit `streamFn` or the process-wide default stream function; coding-agent wires `ModelRuntime.streamSimple()` explicitly. See Architecture and Models, providers, and authentication.

Extension A runtime-loaded TypeScript or JavaScript module that receives `ExtensionAPI` and registers hooks, tools, commands, or UI behavior. See Extensions.

Extension command A slash command registered through the extension API, such as:

```
pi.registerCommand("hello-pi", ...)
```

Users invoke it as `/hello-pi`. It is not a `!` user-shell command. See Extensions and Built-in tools.

Extension context The `ctx` passed to hooks, tools, and commands, containing mode, cwd (current working directory), UI, session access, and model services. See Extensions.

Factory The default-exported function of an extension module — `export default function (pi: ExtensionAPI) { ... }`. Pi calls it once per load, before session start, and it registers the extension's hooks, tools, commands, and other behavior. See Extensions.

Hook An extension handler registered with `pi.on()` for a named lifecycle event, so it can observe or affect that occurrence — for example `pi.on("session_start", ...)`. Some hooks can also block or mutate what they observe. See Extensions and Prompt lifecycle.

cwd / current working directory The directory associated with the current Pi session. Relative paths for built-in tools and project resources are resolved against it. See Built-in tools and Resources, settings, and trust.

Follow-up A queued message delivered after the current run has no more tool calls or steering messages. See Prompt lifecycle.

Renderer A function or component that converts tool output or message content into TUI display elements. See Terminal UI.

Settled The state when no automatic agent work remains, including retries, compaction, or queued continuations. See Prompt lifecycle.

Stop reason / `stopReason` A field on the final assistant message that records why the agent stopped, such as normal completion, an output limit, an error, or an abort. See Agent core.

JSONL JSON Lines: one complete JSON value per line. Pi uses it for append-only sessions and JSON/RPC streams. See Sessions and Run modes.

Model A named capability record describing a model's provider, features, limits, and cost. See Models and authentication.

models.json The conventional global model-configuration file at `~/.pi/agent/models.json`. It can override provider settings and add or adjust model definitions; verify supported scope before assuming a project-local copy is loaded. See Models, providers, and authentication.

Model registry The extension-facing compatibility facade for model/provider lookup and registration. The canonical internal owner of composed catalogs, authentication, OAuth, and request preparation is **ModelRuntime**. See Models and authentication.

ModelRuntime The coding-agent service that owns composed model catalogs, authentication, OAuth, and provider request preparation. See Models and authentication.

Provider A service or adapter that can answer model requests. A provider is different from a model, credential, or API adapter. See Models and authentication.

Prompt template A reusable prompt snippet that can be expanded with context before a model request. See Resources, settings, and trust.

Package An installable bundle—from npm, git, or a local path—that contributes extensions, skills, prompts, and themes together as one unit. See Resources, settings, and trust and Packaging and sharing extensions.

Resource loader The coding-agent service that discovers extensions, skills, prompts, themes, context files, and packages according to trust and scope rules. See Resources, settings, and trust.

Schema A runtime description of the data a value is allowed to contain. A schema can validate external input; a TypeScript type or interface alone cannot inspect a runtime value. See TypeScript basics and Built-in tools.

TypeBox The library Pi uses to build runtime schemas and derive TypeScript static types, commonly with `Type.Object(...)` and `Static<typeof Schema>`. The schema still needs a runtime check to validate a value. See TypeScript basics.

settings.json A JSON configuration file for Pi settings. Global settings conventionally live at `~/.pi/agent/settings.json`; project settings live at `.pi/settings.json` and are applied subject to project trust. See Resources, settings, and trust.

Scope Where a setting or resource applies: global (`~/.pi/agent`, all projects for the current user) or project (`.pi`, the current repository). Project values take precedence over global values; project resources load only after **trust**. See Resources, settings, and trust.

Trust The per-folder decision that permits project-local settings, resources, and extension code to load. Saved decisions live in `~/.pi/agent/trust.json`; interactive mode asks, and noninteractive modes fall back to the `defaultProjectTrust` setting. See Resources, settings, and trust.

Retry A later attempt after a failure. Provider/SDK retries and agent-level retries are separate decisions. See Models and authentication and Failure and recovery.

Run One continuous agent loop from submitted prompt through assistant responses, tool calls, and queued continuation until the agent becomes idle. See Agent core.

Preflight The sequential check each tool call passes through before any tool runs: the agent loop's `beforeToolCall` hook, and the extension `tool_call` handlers built on it. In the default parallel tool mode, calls are preflighted one at a time, then the allowed calls execute concurrently. See Agent core.

Session Pi's durable record of a conversation: a header plus append-only entries that form a session tree. The selected branch determines the context reconstructed for the model. See Sessions and durable context.

Session file / session format A `.jsonl` file containing one session header followed by JSON entries. Each entry has an ID and parent ID for tree reconstruction; the format also records model changes, compaction, extension state, and messages. See Sessions and durable context.

SessionManager The persistence component that stores a session as an append-only JSONL tree, tracks the selected leaf, and reconstructs active context. See Sessions and durable context.

Skill Reusable task instructions loaded as model-facing resources. See Resources, settings, and trust.

Steering A queued message injected after the current assistant response and tool work finish but before the next provider request. See Prompt lifecycle.

`nextTurn` / `triggerTurn` Two delivery options on `pi.sendMessage()`. `deliverAs: "nextTurn"` queues a custom message to travel with the *next* user-originated prompt: it never interrupts the active run and never starts one by itself. `triggerTurn: true` starts an AI-model turn immediately when the agent is idle; without it, a `"steer"` or `"followUp"` message sent while idle is only appended to the session. `triggerTurn` is ignored for `"nextTurn"`. See Prompt lifecycle and Extension patterns.

System prompt The model-facing instruction context assembled from defaults, resources, settings, and extension changes. See Resources and Agent core.

Theme A named set of semantic colors and UI styling values used by the TUI. See Terminal UI.

Tool A model-callable operation with a parameter schema, description, execute function, and optional renderers. See Built-in tools and Extensions.

Tool call One model request to invoke a named tool with arguments. It is distinct from the tool definition and from the resulting tool message. See Built-in tools.

Tool result / `tool_result` A runtime result produced by tool execution. A tool-result message is sent to the model; a `tool_execution_end` or related event announces lifecycle state to handlers and the UI. See Agent core and Extension event reference.

Turn One model request and the tool work that follows it. A run can contain many turns. In compaction, turn boundaries are delimited by user messages: a turn begins at a user message and ends just before the next one. See Prompt lifecycle and Sessions and durable context.

Thinking level How much extended reasoning the model performs per response, selectable in the TUI (the editor border color indicates it) and recorded in the session as a `thinking_level_change` entry. See Sessions and durable context.

TUI Terminal user interface. Generic primitives live in `packages/tui`; coding-agent composes them into the interactive application. See Terminal UI.

TTY / terminal A terminal device or terminal-emulator connection that carries interactive input and output. A TTY provides an interactive surface; print, JSON, and RPC hosts can run without one. See Terminal UI and Modes, SDK, and operations.

Token A unit used to estimate model input and output size. Tokenization depends on the provider and model; Pi uses token counts for context limits, compaction, and cost metadata. See Sessions and durable context and Models, providers, and authentication.

A TypeScript foundations workbook

This optional appendix keeps the long-form TypeScript lessons that support the short Pi-oriented route. Start here if you know basic JavaScript and are new to TypeScript, or return to a linked section when a concept in the main route needs a slower explanation. If JavaScript is new to you, read a JavaScript primer first; this workbook assumes objects, branches, loops, callbacks, template strings, and Promises. The JavaScript survival kit refreshes the exact syntax the examples use, with links to full primers.

The material below is preserved as a workbook: definitions, small examples, worked exercises, common mistakes, TypeBox practice, source-reading tours, and reference checks remain available without blocking readers who already know the language.

A.1 How to use this workbook

The preamble builds the mental model first: What TypeScript checks—and what it erases explains the compile-time/runtime boundary that every exercise leans on, and Working through the examples sets up the practice loops. The JavaScript survival kit sits last in the preamble as an optional detour: visit it whenever a snippet's JavaScript is confusing, and skip it otherwise.

The workbook is organized into five parts plus a reference section. Part 1 teaches everyday TypeScript types, Part 2 shows the Pi patterns built from them, Part 3 covers TypeBox, Part 4 covers the async, cancellation, and class patterns that real extension code uses, and Part 5 applies the basics to real Pi source. The reference section at the end is there when you get stuck.

Read the foundations in order if you are coming from JavaScript. Otherwise use the section links from the TypeScript route, do the checkpoints before reading answers, and return to the main Pi route when you are ready for architecture. Checkpoint and exercise answers are collected in an "Answers" subsection at the end of each part, so you can attempt a question without seeing its answer.

The examples assume Pi's strict, erasable-TypeScript posture. Compiler options can change inferred types and diagnostics: `strict` enables several checks, but options such as `noUncheckedIndexedAccess` and `exactOptionalPropertyTypes` add further distinctions. When an exercise asks what the checker sees, note the assumption; when it asks what JavaScript does, reason about runtime behavior. The official TypeScript Handbook introduction is the broader reference. Use the Handbook's Basics, Everyday Types, Narrowing, More on Functions, Object Types, Classes, Modules, and Utility Types for complete chapter coverage. This appendix teaches Everyday Types, narrowing, functions, object shapes, and the Pi patterns built from them; later sections teach selected patterns for recognition rather than reproducing every Handbook chapter. The repository's erasable-syntax rules intentionally exclude enums, namespaces, parameter properties, and `import =`/`export =` syntax.

For each exercise, first predict the checker result, then predict runtime behavior where relevant, and finally explain which boundary supplies the information.

Each part trains a skill that a specific chapter of this guide then uses. If you only need one chapter, the map shows which part to work first.

Workbook part	Where the skill is used
Part 1, everyday types	everywhere, especially Architecture
Part 2, Pi patterns	Agent core and Extensions
Part 3, TypeBox	Extensions, tool parameters
Part 4, async and classes	Prompt lifecycle and Agent core
Part 5, apply the basics	Testing and contribution

A.2 What TypeScript checks—and what it erases

TypeScript is a **static type checker**: it analyzes a program before the JavaScript runs and reports operations that do not match the types it can prove. This catches typos, missing properties, wrong arguments, and some impossible branches. It does not replace running the program or validating data that arrives later.

```ts
const announcement = "Hello";
// announcement.toLocaleLowercase(); // checker catches the misspelling

const parsed: unknown = JSON.parse('{"count":"many"}');
// TypeScript cannot know that parsed is a valid application object.
// A runtime guard or TypeBox schema must inspect it before use.
```

TypeScript is strongest when code constructs a value inside the checked program: it can catch misspelled fields, wrong primitive types, missing required fields, and invalid literal values. It cannot inspect data that arrives later from a file or network unless runtime code checks that data.

The TypeScript compiler, commonly invoked as `tsc`, performs type checking and can transform TypeScript into JavaScript. Type annotations, interfaces, type aliases, and most other type-only syntax are erased; they do not change runtime behavior. A loader such as `tsx` or `jiti` can execute a file without proving that it passes the repository's type check. Conversely, a successful type check cannot prove that a network response has the shape your code expects.

The two tracks run at different times, which is why each answers different questions:

A small configuration controls how much evidence the checker requires:

Option	Main effect in this workbook
`strict`	enables the family of strict checking options
`noImplicitAny`	reports parameters and values that silently become `any`
`strictNullChecks`	keeps `null` and `undefined` separate from ordinary values
`noUncheckedIndexedAccess`	includes `undefined` in unchecked indexed lookups

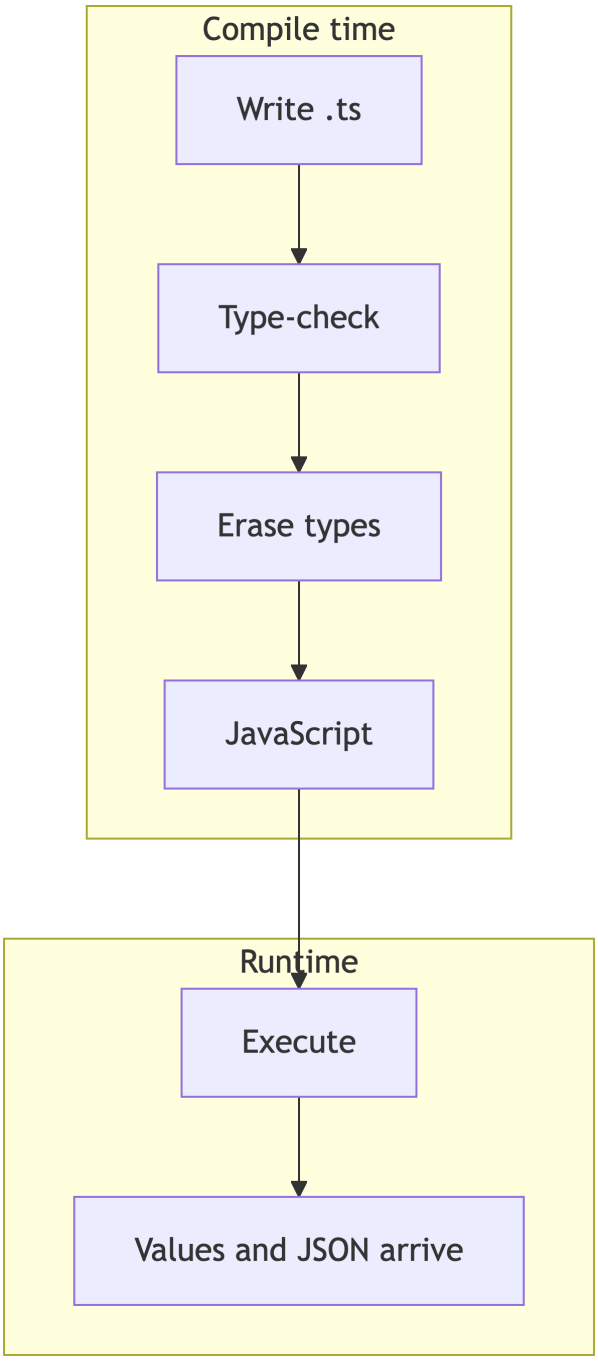

Figure A.1: Compile time checks and erases types; runtime executes values.

Pi enables `strict` but not every stricter option. Always check the active configuration before treating an inferred type as a universal TypeScript rule. The separate Pi TypeScript route explains the repository commands; this section supplies the mental model needed to interpret their results.

Checkpoint: Which of these facts can only be established at runtime: that a value is a string, that a property name is spelled correctly, or that a JSON response has the expected shape? What does type erasure mean? Which option changes the static result of an unchecked array access? (answer: Part 1 answers)

A.3 Working through the examples

Every snippet in this workbook is small enough to try yourself. Use whichever of these three practice loops fits the moment.

Loop 1: the TypeScript Playground. Open the TypeScript Playground, paste a snippet, and hover over names to see inferred types. The right pane shows the erased JavaScript, which makes type erasure visible.

Loop 2: a scratch file you run. Create `scratch.ts` anywhere outside the Pi repository, paste a snippet, and run it with type stripping:

```
$ node --experimental-strip-types scratch.ts
status: running
```

Loop 3: a scratch file you check. Type stripping runs code without checking it. Ask the checker for a second opinion:

```
$ npx tsgo --noEmit scratch.ts
scratch.ts:2:7 - error TS2322: Type 'number' is not assignable to type 'string'.
```

One end-to-end example. Put this in `scratch.ts`:

```
const status: string = "running";
console.log(`status: ${status}`);
```

The run loop prints `status: running`. Now change the value to `42` and keep the annotation. The run loop still prints `status: 42`, because stripping ignores the annotation; the check loop reports the `TS2322` error above. The two loops answer different questions, and you need both.

A.4 JavaScript survival kit

This workbook assumes JavaScript, but if any snippet's JavaScript is confusing, detour here. You can also skip this section now and return on demand whenever an example trips you up; the syntax below appears so often in Pi that a sixty-second refresher pays for itself. Each item shows the pattern and says what it does.

Arrow functions are short function values, common as callbacks:

```ts
const upper = (name: string) => name.toUpperCase();
```

`upper` is a function that takes one string and returns one string; there is no `function` keyword and no `this` of its own.

Destructuring pulls properties or elements into local variables:

```ts
const { name, age = 0 } = user;   // object: name and age, age defaults to 0
const [first, second] = names;    // array: first two elements
```

Spread and rest use the same `...` syntax in two directions:

```ts
const copy = { ...user, active: true };  // spread: copy fields, override one
function log(...args: string[]): void {} // rest: gather arguments into array
```

Spread expands a value into a new object, array, or call; rest gathers the remaining values into one array.

Template literals build strings with backticks and `${...}`:

```ts
const label = `${user.name}: ${count} files`;
```

Array methods do most of Pi's data work. `map` transforms every element, `filter` keeps matching elements, `find` returns the first match or `undefined`, and `some`/`every` answer boolean questions:

```ts
const lengths = names.map((n) => n.length);
const long = names.filter((n) => n.length > 3);
const ada = names.find((n) => n.startsWith("A"));   // string | undefined
const anyReady = states.some((s) => s === "ready"); // boolean
```

`reduce` folds an array into one value; you will recognize it in Pi, but a loop or `map`/`filter` is usually clearer when you write new code.

`Map` and `Set` are the keyed and unique-value collections:

```ts
const counts = new Map<string, number>();
counts.set("read", 3);
counts.get("read");     // 3, or undefined when absent
const seen = new Set<string>();
seen.add("read");
seen.has("read");       // true
```

`Object.entries`, `Object.keys`, and `Object.values` turn plain objects into arrays you can iterate:

```ts
for (const [key, value] of Object.entries(config)) {
  console.log(key, value);
}
```

Logical assignment operators assign only when a condition holds:

```
counts.set("read", (counts.get("read") ?? 0) + 1);
options.retries ??= 3;  // assign 3 only if retries is null or undefined
```

`??=` fills in missing values without overwriting meaningful ones; `||=` and `&&=` are the rarer siblings.

`structuredClone` deep-copies a plain-data value:

```
const snapshot = structuredClone(state);
```

Unlike `{ ...state }`, nested objects and arrays are copied too, so later mutations do not leak into the snapshot.

For a full primer, start with the MDN JavaScript Guide and use the MDN JavaScript reference when you need exact method behavior.

A.5 Part 1 — Everyday TypeScript fundamentals

The sections in this part intentionally follow the beginner progression used by the TypeScript handbook: common values first, then functions and object shapes, then unions, aliases, assertions, literals, and missing values.

In this part you will learn to:

- read `user.last?.toUpperCase() ?? "UNKNOWN"` and explain why both operators are there;
- decide when a type annotation adds information and when inference is enough;
- narrow a union like `number | string` with a check and say what the checker learned;
- tell a safe `satisfies` check from a risky `as` assertion;
- recognize the everyday type patterns that Pi source uses on every page.

A.5.1 Primitives and arrays

A **primitive** is a basic JavaScript value. The three types you will see most often are:

Type	Values	Example
`string`	text	`"working"`
`number`	numeric values	`42`, `3.14`
`boolean`	`true` or `false`	`true`

JavaScript does not have separate runtime `integer` and `float` types. Both are `number` in TypeScript. Use lowercase `string`, `number`, and `boolean`; the capitalized `String`, `Number`, and `Boolean` names are special wrapper types that are almost never what you want.

```ts
const status: string = "working";
const attempts: number = 2;
const finished: boolean = false;

// The checker catches the wrong primitive type.
// const badAttempts: number = "two";
```

An array type says what every item in the array should be:

```ts
const messages: string[] = ["start", "working"];
const counts: Array<number> = [1, 2, 3];
```

`string[]` and `Array<string>` mean the same thing. The `T` inside `Array<T>` is a preview of a generic type parameter; you can read it as "an array of T." We will not design our own generics yet.

Array indexing can produce `undefined` when an index does not exist:

```ts
const first = messages[0];
// At runtime, an empty array makes this undefined.

const missing = messages[99];
// missing may be undefined at runtime.
```

With ordinary `strict` checking, TypeScript may still treat an indexed `string[]` lookup as `string`. The `noUncheckedIndexedAccess` option adds `undefined` to the static result, making the checker reflect the possible missing element. Pi does not enable that option by default, so keep both facts in mind: a static type does not remove the runtime hazard, and compiler options matter when reading an indexed access.

Do not confuse the type of an array with the type of one item. `string[]` means many strings; `string` means one string.

A.5.1.1 Values at runtime and types before runtime

The JavaScript `typeof` operator checks a value while the program is running:

```ts
const statusValue = "working";
const countValue = 2;

console.log(typeof statusValue); // "string"
console.log(typeof countValue);  // "number"
```

The TypeScript type annotation describes what the checker expects before the program runs:

```ts
const statusText: string = "working";
const countNumber: number = 2;
```

The word `string` appears in both examples, but the two uses are different. In `typeof statusValue`, it is a runtime string value. In `statusText: string`, it is type syntax that disappears when the code runs.

Arrays also have runtime behavior that types cannot change:

```
const names: string[] = ["Ada", "Grace"];

names.push("Katherine");
console.log(names.length); // 3
```

The annotation prevents this mistake before execution:

```
// names.push(42); // number is not assignable to string
```

It does not make the array immutable. If you need a read-only view, that is a separate type contract introduced later.

A.5.1.2 `any` is an escape hatch

`any` tells the checker to stop asking questions about a value:

```
let value: any = { name: "Ada" };

value.name.toUpperCase(); // no type error
value.notARealMethod();   // also no type error
const count: number = value; // also allowed
```

The last two lines can fail at runtime. Use `any` only when you are deliberately accepting that risk. Later, `unknown` will provide a safer way to represent data that has not been checked yet.

Pi bridge: message content, tool arguments, and provider data are often arrays or objects containing primitive values. Start by identifying the simple values before trying to understand the larger type.

Checkpoint: What are the types of `"ready"`, `17`, `false`, and `["a", "b"]`? What can `messages[99]` return at runtime? Under ordinary `strict` checking, what may the checker report for that lookup, and what changes when `noUncheckedIndexedAccess` is enabled? Why can `any` hide a bug? (answer: Part 1 answers)

Try it: Before reading the answers, predict the checker's type for each value:

```
let changingStatus = "idle";
const fixedStatus = "idle";
const values = [1, 2, 3];
```

(answer: Part 1 answers)

A.5.2 Annotations and inference

An **annotation** is a type written by the programmer. **Inference** is the checker working out a type from the value or context.

An annotation goes after the name:

```
let displayName: string = "Ada";
```

In most cases, the initializer gives the checker enough information:

```
let inferredName = "Ada";
// inferredName is string.

const inferredCount = 3;
// inferredCount is number.
```

Do not add annotations only because the code is TypeScript. Add one when it documents an important contract, protects a public function, or helps the checker understand a value that cannot be inferred clearly.

A.5.2.1 The same function three ways

Start with JavaScript:

```
function greet(name) {
  return `Hello, ${name}`;
}
```

Add an input annotation:

```
function greet(name: string) {
  return `Hello, ${name}`;
}
```

The return type is inferred as `string`. You may write it explicitly when the return value is an important public contract:

```
function greet(name: string): string {
  return `Hello, ${name}`;
}
```

The explicit return annotation does not convert the result into a string. It asks the checker to reject a future implementation that returns something else.

A.5.2.2 Contextual typing

When a function is used where TypeScript already knows the callback shape, the context can provide the parameter type:

```
const names = ["Ada", "Grace"];

names.forEach((name) => {
  // name is inferred as string from the array and forEach.
  console.log(name.toUpperCase());
});
```

Writing `(name: string)` here is allowed, but usually unnecessary. This is called **contextual typing**: the place where a function is used tells the checker how its parameters will be called.

Checkpoint: Which type is inferred for `greet`'s return value? Why does the `forEach` callback not need a parameter annotation? When would an explicit return annotation make the contract clearer? (answer: Part 1 answers)

Try it: Remove the annotation from this callback and read the error that appears when the callback uses a string-only method:

```
const counts = [1, 2, 3];

counts.map((count) => count.toFixed(2));
// count is inferred as number from the array.
```

(answer: Part 1 answers)

A.5.3 Functions and common parameter forms

A function has inputs, a return value, and possibly side effects. TypeScript can describe the input and output without changing how JavaScript calls it.

```
function add(left: number, right: number): number {
  return left + right;
}

const total = add(2, 3);
// total is inferred as number.

// add("2", 3); // error: the first argument must be number
```

A function is also a value, so its callable shape can be written as a type:

```
type Formatter = (value: string, width: number) => string;

const format: Formatter = (value, width) => value.slice(0, width);
```

The parameter names in a function type are labels for readers; the types and return type describe what callers may provide and receive. An object type can also describe a callable value with a call signature, although the function type expression is usually clearer for a plain callback:

```
type CallableWithLabel = {
  label: string;
  (value: string): string;
};
```

`new (...) => T` is a construct signature for values called with `new`. You will mostly recognize these signatures in library declarations rather than write them in Pi extensions.

`void` means that the caller should not expect a useful return value:

```
function logStatus(status: string): void {
  console.log(status);
}
```

The function still performs work. `void` does not mean "the function does nothing."

A.5.3.1 Optional parameters and default parameters

An optional parameter uses `?` after the parameter name:

```
function connect(host: string, port?: number): string {
  return `${host}:${port ?? 443}`;
}

connect("localhost");
connect("localhost", 8080);
```

Inside the function, `port` may be `undefined`, so the function needs a check or a fallback.

A default parameter supplies a value when the argument is omitted or `undefined`:

```
function connectWithDefault(host: string, port = 443): string {
  return `${host}:${port}`;
}
```

These are related but not identical. `port?: number` describes a possibly missing input. `port = 443` also gives the implementation a runtime value after the default is applied.

A.5.3.2 Rest parameters and destructured parameters

A rest parameter gathers zero or more arguments into an array:

```ts
function joinLabels(separator: string, ...labels: string[]): string {
  return labels.join(separator);
}

joinLabels(", ", "read", "write");
```

The spread syntax passes an array's elements as separate arguments. The type of the rest parameter describes each gathered element, not the whole call:

```ts
const modes = ["read", "write"];
joinLabels(", ", ...modes);
```

A parameter can destructure an object while the annotation describes the whole object pattern:

```ts
function pointLabel({ x, y }: { x: number; y: number }): string {
  return `(${x}, ${y})`;
}
```

The type goes after the destructuring pattern. `x: number` inside the pattern would rename a property to a variable called `number`; it would not annotate it.

A.5.3.3 A short Promise preview

An `async` function returns a `Promise<T>`, where `T` is the eventual value:

```ts
async function loadStatus(): Promise<string> {
  return "working";
}

const status = await loadStatus();
// status is string after await.
```

The detailed Promise and streaming explanation comes later. For now, remember that `Promise<string>` means "a result that will eventually be a string."

Pi bridge: extension commands, event handlers, and tool `execute` methods are functions stored for Pi to call later. The timing of that call matters, but the basic function idea is the same.

Checkpoint: Which parameter forms allow omission? Which one supplies a runtime fallback? What does `Promise<string>` describe? What does `void` mean? (answer: Part 1 answers)

Try it: Compare omission with an explicit `undefined` value:

```ts
function label(prefix?: string): string {
  return prefix ?? "default";
}

label();
label(undefined);
label("status");
```

(answer: Part 1 answers)

A.5.4 Object types and optional properties

An **object type** lists the properties a value must have and the type of each property. Start with an inline object type:

```ts
function printPoint(point: { x: number; y: number }): void {
  console.log(point.x, point.y);
}

printPoint({ x: 3, y: 7 });
```

The function does not require a class named `Point`. It requires a value with numeric `x` and `y` properties.

An optional property has `?` after its name:

```ts
type User = {
  first: string;
  last?: string;
};

const oneName: User = { first: "Ada" };
const fullName: User = { first: "Ada", last: "Lovelace" };
```

When you read `user.last`, the result may be `undefined` because the property may not exist.

An optional property is different from a required property whose value may be `undefined`:

```ts
type MayBeAbsent = { last?: string };
type MustBePresent = { last: string | undefined };

const absent: MayBeAbsent = {};
const present: MustBePresent = { last: undefined };

// const missing: MustBePresent = {}; // required key is missing
```

The first type describes a key that may not be in the object at all. The second describes an object that must contain the key, even if its value is `undefined`. This distinction matters when code checks property presence or serializes configuration.

A.5.4.1 `?`, `?.`, and defaults

These symbols look similar but do different jobs:

Syntax	Meaning
`last?: string`	the property may be absent
`user.last?.toUpperCase()`	access a property only if `last` is not nullish
`port = 443`	default an omitted or `undefined` parameter

The `??` and `||` fallback operators are compared value by value in `null`, `undefined`, and related operators.

For the examples below, use a value that may omit its last name:

```
const user: User = { first: "Ada" };
```

Unsafe property access:

```
function upperLastName(user: User): string {
  // return user.last.toUpperCase();
  // Error: user.last may be undefined.
  return user.last?.toUpperCase() ?? "UNKNOWN";
}
```

Optional chaining is safe access: it performs the property access only when the receiver is not `null` or `undefined`, and produces `undefined` otherwise. The `??` after it chooses a fallback for that result.

An `if` check is often clearer than a compact operator when the missing case needs different behavior:

```
function describeUser(user: User): string {
  if (user.last === undefined) {
    return `${user.first} has no last name`;
  }
  return `${user.first} ${user.last}`;
}
```

The same choices can be written with explicit branches when the fallback is not just a string:

```
function lastNameOrAction(user: User): string {
  if (user.last === undefined) {
    return "ask the user for a last name";
  }
  return user.last.toUpperCase();
}
```

Use the compact operators for simple value selection. Use an `if` when the missing case changes control flow or needs an explanation.

A.5.4.2 Readonly properties and arrays

`readonly` prevents assignment through a particular TypeScript view. It does not freeze the object at runtime, and it is shallow:

```
interface Report {
  readonly id: string;
  tags: readonly string[];
}

const report: Report = { id: "r1", tags: ["build"] };
// report.id = "r2"; // error: readonly property
// report.tags.push("test"); // error: readonly array
```

A readonly view can still refer to mutable data held elsewhere. Treat it as a contract for callers, not as runtime protection. `ReadonlyArray<string>` is another spelling of `readonly string[]`.

A.5.4.3 Index signatures and known keys

An index signature describes an object whose property names are not all known in advance:

```
type Metadata = {
  [key: string]: unknown;
};

const metadata: Metadata = { provider: "local", retries: 2 };
```

Use `Record<K, V>` when the key set is known or is itself a type:

```
type StatusLabels = Record<"idle" | "running", string>;
const labels: StatusLabels = { idle: "Waiting", running: "Working" };
```

An index signature is not an exact-object guarantee. It describes how named properties may be read, while runtime data may still need validation.

A.5.4.4 Intersections and tuples

Optional on first read — come back when Pi source makes you need this.

The `&` operator combines requirements. A value assigned to `DetailedReport` must satisfy both parts:

```
type BasicReport = { title: string };
type DetailedReport = BasicReport & { error?: string };
```

Interfaces can express an additive relationship with `extends`:

```ts
interface NamedReport { title: string }
interface TimedReport extends NamedReport { elapsedMs: number }
```

An intersection is often convenient for combining aliases. Interface extension can make a public inheritance relationship clearer; conflicting intersections can produce an unusable `never` property, so do not combine shapes casually.

A tuple is an array with a fixed position contract:

```ts
type Pair = [string, number];
const entry: Pair = ["attempts", 2];
// const wrong: Pair = [2, "attempts"]; // positions have different types

const fixed = ["read", "write"] as const;
// readonly ["read", "write"]
```

`[string, number]` is not the same as `(string | number)[]`: the tuple is position-aware and has a fixed length. Use `readonly` tuples when callers must not change the positions.

Checkpoint: What is the difference between an optional property and optional chaining? What happens to `0` with `||` and `??`? Why does the checker require a check before calling `toUpperCase()` on `user.last`? What runtime protection does `readonly` provide? Which type describes an open-ended string-keyed object? (answer: Part 1 answers)

A.5.4.5 Recap: values and shapes

- Primitives (`string`, `number`, `boolean`) are single values; arrays and object shapes group values under one type.
- An optional property `last?: string` reads as `string | undefined`, so the checker requires evidence before you use it.
- `user.last?.toUpperCase()` skips the call when the value is nullish; a default or `??` supplies the fallback.
- Structural typing accepts a variable with extra fields, while a fresh object literal gets an extra-property check.

A.5.5 Union types and basic narrowing

A **union type** says that a value may be one of several types. The `|` symbol means "or":

```ts
function printId(id: number | string): void {
  console.log(id);
}

printId(101);
printId("101");
// printId({ value: 101 }); // error
```

Before narrowing, TypeScript only allows operations valid for every member:

```
function uppercaseId(id: number | string): string {
  // return id.toUpperCase();
  // Error: number does not have toUpperCase().

  if (typeof id === "string") {
    return id.toUpperCase();
  }
  return String(id);
}
```

The `typeof` check is evidence. Inside the first branch, `id` is known to be a string. In the second branch, it is known to be a number. Each check shrinks the union for the code it guards:

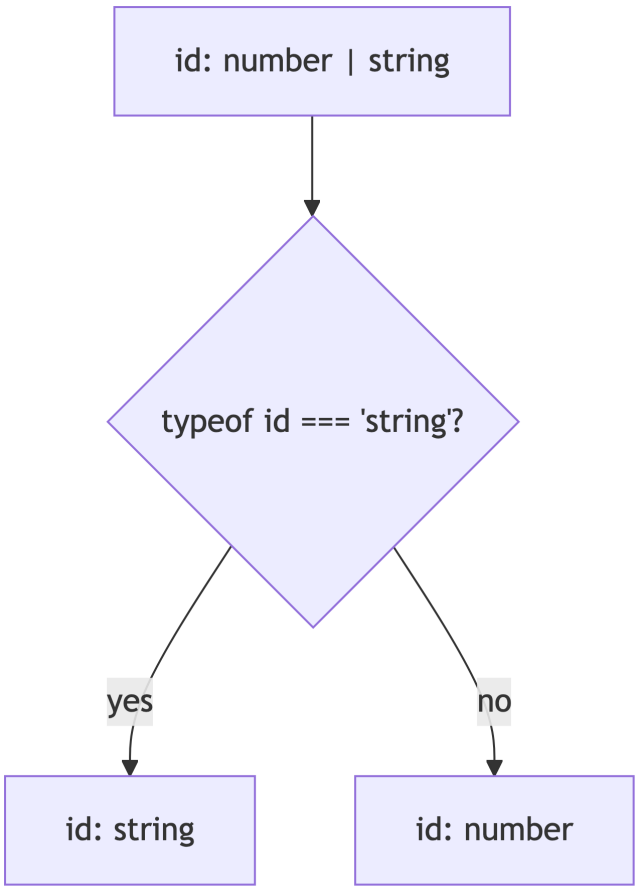

Figure A.2: Narrowing shrinks a union as checks provide evidence.

A.5.5.1 Other everyday narrowing checks

`Array.isArray` is a runtime check that TypeScript understands:

```
function welcome(value: string | string[]): string {
  if (Array.isArray(value)) {
    return `Hello ${value.join(", ")}`;
  }
  return `Hello ${value}`;
}
```

Equality checks can narrow literal unions:

```
type Mode = "read" | "write";

function explain(mode: Mode): string {
  if (mode === "read") return "This operation reads data.";
  return "This operation changes data.";
}
```

A truthiness check also narrows, but it treats `""`, `0`, `false`, `null`, and `undefined` as false. Do not use it when an empty string or zero is meaningful:

```
function label(value: string | undefined): string {
  if (!value) return "missing"; // also treats "" as missing
  return value.toUpperCase();
}
```

The `in` operator narrows object unions by checking property presence:

```
type Fish = { swim: () => void };
type Bird = { fly: () => void };

function move(animal: Fish | Bird): void {
  if ("swim" in animal) animal.swim();
  else animal.fly();
}
```

`instanceof` performs a prototype-based runtime check and narrows class-like values such as `Date | string`. Pi usually prefers structural shapes and literal discriminants, but you may encounter this check in dependencies.

Assignments and reachability affect the observed type too:

```
let value: string | number = "ready";
value.toUpperCase(); // observed as string here
value = 3;
value.toFixed();     // observed as number here
```

An early return can remove a union member from the code that follows. This control-flow analysis is why a check in one branch changes what the checker allows later.

If every union member has a property in common, that property is available without narrowing:

```ts
function firstThree(value: string | number[]): string | number[] {
  return value.slice(0, 3);
}
```

The result is still a union because the input could have been either member.

Pi bridge: Pi uses unions for messages, tool results, and lifecycle events. Read the union definition first, then find the `if` or `switch` that narrows it.

Checkpoint: Why is `id.toUpperCase()` invalid before the `typeof` check? What does `Array.isArray` prove? Which properties can be used before narrowing? (answer: Part 1 answers)

Try it: Identify the common operation before narrowing:

```ts
function preview(value: string | number[]): string | number[] {
  return value.slice(0, 2);
}
```

(answer: Part 1 answers)

A.5.6 Type aliases and interfaces

A **type alias** gives a name to a type:

```ts
type Point = {
  x: number;
  y: number;
};

function distanceFromOrigin(point: Point): number {
  return Math.sqrt(point.x ** 2 + point.y ** 2);
}
```

An **interface** is another way to name an object shape:

```ts
interface User {
  first: string;
  last?: string;
}
```

For ordinary object shapes, both forms are often interchangeable. Pi uses a repository convention: interfaces are common for public object contracts, and type aliases are common for unions and combinations of types.

TypeScript is **structurally typed**. A value is accepted because it has the required structure, not because it was created from a particular named type:

```ts
interface Named {
  name: string;
}

const person = { name: "Ada", age: 36 };
const named: Named = person; // okay: person has name
```

Fresh object literals receive an extra check that catches likely misspellings:

```ts
// const named: Named = { name: "Ada", nmae: "typo" };
// Error: nmae is not a property of Named.
```

The key beginner rule is simple: use the shape that describes the value you have. Do not treat an interface as a class or as a runtime validator.

Type aliases can also name primitives and unions, while interfaces are used for object shapes:

```ts
type UserId = string | number;

interface Account {
  id: UserId;
  active: boolean;
}
```

This does not create a new runtime `UserId` type. It gives the checker a name for a relationship that already exists in JavaScript values.

Checkpoint: What does a type alias name? How is an interface similar? Why can a value with extra fields be accepted through a variable but rejected as a fresh object literal? (answer: Part 1 answers)

A.5.7 Type assertions

A **type assertion** uses `as` to tell the checker, "I have information you do not have." It does not convert the value, validate it, or add a runtime check.

Compare three different operations:

`declare` tells TypeScript that a variable exists without generating runtime code. It is a teaching shortcut here so we can focus on the type operation.

```ts
declare const value: unknown;

const asserted = value as string;
// The checker treats asserted as string. Runtime value is unchanged.

const converted = String(value);
// JavaScript converts the runtime value to a string.
```

```
const checked = typeof value === "string" ? value : undefined;
// JavaScript checks the runtime value before returning it.
```

An assertion can be appropriate when a library gives you a value whose type is too general and you have verified the situation elsewhere. It is not a safe replacement for validation at a JSON or model boundary:

```
const raw = JSON.parse('{"name": 42}') as unknown;
const unsafeUser = raw as { name: string };
// This compiles, but unsafeUser.name is still the number 42 at runtime.
```

Avoid `as unknown as SomeType` in beginner code. It usually means the checker is being silenced instead of the value being made safe.

Pi bridge: use assertions only after identifying the runtime evidence that makes them safe. For external data, prefer `unknown` plus a guard or TypeBox.

Checkpoint: Which example changes the runtime value? Which example checks the runtime value? What does `as` change? (answer: Part 1 answers)

Try it: Write down what each variable contains at runtime:

```
const source: unknown = 42;
const claimed = source as string;
const converted = String(source);
```

(answer: Part 1 answers)

A.5.8 Literal types and literal inference

A **literal type** is one exact value, such as the string literal type `"running"`, the number literal type `200`, or the boolean literal type `true`. This is narrower than the general type `string`, `number`, or `boolean`.

Literal types are useful when a value must belong to a closed set:

```
type Alignment = "left" | "right" | "center";

function align(text: string, alignment: Alignment): string {
  return `${alignment}: ${text}`;
}

align("Hello", "left");
// align("Hello", "centre"); // error: centre is not an allowed literal
```

A.5.8.1 Literal widening

`let` variables may change later, so TypeScript usually widens their type:

```
let changing = "hello";
// changing is string because another string may be assigned later.
changing = "goodbye";
```

`const` variables cannot be reassigned, so a primitive constant can retain its exact literal type:

```
const fixed = "hello";
// fixed is "hello".
```

Object properties usually widen even when the object variable is `const`:

```
const event = { type: "started" };
// event.type is string, not the literal "started".
// The property could be changed later.
```

Use an explicit type when you want a contract:

```
const typedEvent: { type: "started" } = { type: "started" };
```

Use `as const` when the whole literal should retain its exact values and be readonly at the type level:

```
const exactEvent = { type: "started" } as const;
// exactEvent.type is "started".
// exactEvent.type cannot be assigned through this type.
```

Literal types also work for numbers and booleans:

```
type ExitCode = 0 | 1;
type Enabled = true;

const success: ExitCode = 0;
const enabled: Enabled = true;
```

A literal union is a small vocabulary that the checker can enforce. Pi uses these vocabularies for statuses, roles, event names, and modes.

`as const` works for arrays too:

```
const modes = ["read", "write"] as const;
// readonly ["read", "write"]
```

That tuple is more precise than `string[]`, but it is also less mutable. Use it when the exact values and order are part of the contract, not by default.

Use a literal union for a useful closed set rather than a variable that can only ever hold one literal.

Pi bridge: tags such as `"user"`, `"assistant"`, `"toolResult"`, and `"message_update"` are literal values used to select a union branch.

Checkpoint: Why is `changing` a `string` while `fixed` is `"hello"`? Why does `event.type` widen inside a normal object? When is a literal union more useful than one literal type? (answer: Part 1 answers)

A.5.8.2 Checking without widening: `satisfies`

Optional on first read — come back when Pi source makes you need this.

The `satisfies` operator checks that an expression is compatible with a type without replacing the expression's more specific inferred type:

```
type ModeLabels = Record<"read" | "write", string>;

const modeLabels = {
  read: "Reading",
  write: "Writing",
} satisfies ModeLabels;

modeLabels.read.toUpperCase(); // still known as a string
// modeLabels.preview; // error: this key was not declared
```

Compare three different tools:

Syntax	Main effect	Runtime validation?
`const x: T = value`	checks and gives `x` the declared type	no
`value as T`	asserts that the checker should treat it as `T`	no
`value satisfies T`	checks compatibility while preserving useful inference	no

`satisfies` is useful for configuration and literal tables. It does not parse, validate, freeze, or convert runtime data. In Pi source you may see it combined with `as const` and `Record` when exact keys and readonly values are intended.

A Pi-shaped example: tool settings that must cover every known mode while keeping each literal value's type:

```
const toolTimeouts = {
  read: 5_000,
  write: 10_000,
} satisfies Record<"read" | "write", number>;
```

A.5.8.3 Recap: unions and literals

- `A | B` means "or": the value may be either, and only operations valid for every member are allowed up front.
- A check (`typeof`, `Array.isArray`, a discriminant comparison) is evidence that narrows the union in one branch.
- Literal unions like `"idle" | "running"` are closed vocabularies; Pi uses them as tags on events and results.
- `const` keeps a literal type, `let` widens it, and `as const` freezes the type-level view without freezing runtime data.

A.5.9 `null`, `undefined`, and related operators

JavaScript uses both `undefined` and `null` for absent values:

- `undefined` commonly means a value was not provided or was not found;
- `null` commonly means an explicit empty value.

With strict null checking enabled, TypeScript makes you handle these values before using methods or properties on them.

```
function upper(value: string | undefined): string {
  if (value === undefined) return "MISSING";
  return value.toUpperCase();
}
```

The `strictNullChecks` setting is what makes `null` and `undefined` separate from ordinary values. Pi's strict checking means that a possible missing value must be handled before property access:

```
function lengthOf(value: string | null | undefined): number {
  if (value == null) return 0;
  return value.length;
}
```

The `value == null` check is a deliberate JavaScript shorthand that matches both `null` and `undefined`. If you want the two cases to have different meaning, write separate `value === null` and `value === undefined` branches.

Use the following table when choosing an operator:

Value	`value || "fallback"`	`value ?? "fallback"`
`undefined`	fallback	fallback
`null`	fallback	fallback
`0`	fallback	`0`
`false`	fallback	`false`
`""`	fallback	`""`
`"ready"`	`"ready"`	`"ready"`

Optional chaining is about safe access:

```
const length = user.last?.length;
// length is number | undefined.
```

Nullish coalescing is about choosing a fallback:

```
const label = user.last ?? "Unknown";
// label is string.
```

Optional chaining and nullish coalescing do not remove the need to understand the data. They only make two common operations shorter. If missing data should be reported, logged, or repaired, an explicit branch is often easier for a new reader to follow.

The non-null assertion operator `!` removes `null` and `undefined` from the checker's view without checking the runtime value:

```
declare const maybeName: string | undefined;
const name = maybeName!;
// This may crash later if maybeName was actually undefined.
```

Prefer a real check, optional chaining, or a deliberate fallback. Treat `!` as a last-resort assertion that needs a clear explanation.

Checkpoint: What is the difference between `undefined` and `null` in your application? Which values trigger `||` but not `??`? What runtime protection does `!` provide? None: it only changes the checker's view. (answer: Part 1 answers)

A.5.10 Worked example: a small status dashboard

~10 min · core

The following example combines only the concepts from Part 1. Read it in small steps rather than trying to understand the final version immediately.

Start with primitive values:

```
const status: string = "running";
const message: string = "checking files";
const finished: boolean = false;
```

Give the object a shape:

```
type StatusReport = {
  status: string;
  message: string;
  finished: boolean;
};

const report: StatusReport = { status, message, finished };
```

Replace the broad `string` status with a literal union:

```
type Status = "idle" | "running" | "failed";

type BetterStatusReport = {
  status: Status;
  message: string;
  finished: boolean;
};
```

Now the checker catches a misspelled status before the program runs:

```
const good: BetterStatusReport = {
  status: "running",
  message: "checking files",
  finished: false,
};

// const bad: BetterStatusReport = {
//   status: "working", // not in the Status union
//   message: "checking files",
//   finished: false,
// };
```

Add an optional detail:

```
type DetailedStatusReport = BetterStatusReport & {
  errorMessage?: string;
};

const failed: DetailedStatusReport = {
  status: "failed",
  message: "checking stopped",
  finished: true,
  errorMessage: "config file was not found",
};
```

Read the optional detail safely:

```
function summary(report: DetailedStatusReport): string {
  const error = report.errorMessage ?? "no error";
  return `${report.status}: ${report.message} (${error})`;
}
```

Finally, represent changes as a union with a literal tag:

```ts
type StatusEvent =
  | { type: "started"; report: BetterStatusReport }
  | { type: "finished"; report: DetailedStatusReport };

function eventSummary(event: StatusEvent): string {
  if (event.type === "finished") {
    return summary(event.report);
  }
  return `${event.type}: ${event.report.message}`;
}
```

Self-check: Identify the primitive types, object shape, literal union, optional property, and discriminant. Then explain which line prevents `"working"` from becoming a valid status. If you can do that, you are ready to read the Pi-specific sections.

A.5.11 Practice lab

These exercises use only the concepts from Part 1. Try each question before reading its answer in the Part 1 answers.

A.5.11.1 Exercise 1: primitives and arrays

~2 min · warm-up

```ts
const title = "Build";
const attempts = [1, 2, 3];
const ready = false;
```

Answer these questions:

1. What is the type of `title`?
2. What is the type of `attempts`?
3. What is the type of `ready`?
4. What could `attempts[10]` be?

A.5.11.2 Exercise 2: annotation or inference

~2 min · warm-up

```ts
let mode = "read";
const fixedMode = "read";
const names = ["Ada", "Grace"];

names.map((name) => name.length);
```

Which values are inferred as broad types? Which callback parameter receives a contextual type?

A.5.11.3 Exercise 3: optional property and fallback

~5 min · core

```
type Options = {
  retries?: number;
};

function retryCount(options: Options): number {
  return options.retries ?? 3;
}
```

What happens for `{}`, `{ retries: undefined }`, `{ retries: 0 }`, and `{ retries: null }`?

A.5.11.4 Exercise 4: union narrowing

~5 min · core

```
function sizeOf(value: string | string[]): number {
  if (Array.isArray(value)) return value.length;
  return value.length;
}
```

Why is `.length` available in both branches, and why is `value.join` available only in the first branch?

A.5.11.5 Exercise 5: structural typing

~5 min · core

```
type Coordinates = { x: number; y: number };
const point = { x: 3, y: 7, label: "origin" };
const coordinates: Coordinates = point;
```

Why is the assignment allowed? What happens if the extra property appears in a fresh object literal assigned directly to `Coordinates`?

A.5.11.6 Exercise 6: assertion versus validation

~5 min · core

```
const input: unknown = JSON.parse('{"count":"many"}');
const claimed = input as { count: number };
```

Can `claimed.count + 1` be trusted? What would a safe implementation do first?

A.5.11.7 Exercise 7: readonly tuples

~5 min · stretch

Focus: predict a type and distinguish compile-time readonly from runtime mutability.

```
const modes = ["read", "write"] as const;
type Mode = (typeof modes)[number];

function useMode(mode: Mode): void {
  console.log(mode);
}
```

Here, `typeof modes` in type position captures the compile-time type of `modes`, and `[number]` extracts the element types as a union. The full explanation appears later in Types from existing declarations.

What is the type of `modes`? Which values can `Mode` contain? Can code call `modes.push("preview")`? What does this example protect at runtime?

A.5.11.8 Exercise 8: `satisfies` versus assertion

~5 min · stretch

Focus: predict checker diagnostics and runtime behavior at a validation boundary.

```
type Limits = Record<"retries" | "timeoutMs", number>;

const limits = {
  retries: 3,
  timeoutMs: 1000,
} satisfies Limits;
```

Does `satisfies` validate a value received from JSON? Does it change the runtime object? What kind of typo would the checker catch in this example?

A.5.11.9 Exercise 9: exhaustive narrowing

~5 min · stretch

Focus: predict the narrowed type and the compile-time failure caused by an unhandled union variant.

```
function assertNever(value: never): never {
  throw new Error(`Unexpected variant: ${String(value)}`);
}

type Outcome =
  | { type: "ok"; value: string }
```

```
  | { type: "error"; message: string };

function outcomeText(outcome: Outcome): string {
  switch (outcome.type) {
    case "ok": return outcome.value;
    case "error": return outcome.message;
    default: return assertNever(outcome);
  }
}
```

Why does the `default` branch accept `outcome` as `never`? What happens if a `{ type: "cancelled" }` variant is added without adding a new case?

A.5.12 Common beginner mistakes in Everyday Types

These mistakes are worth studying because each one looks reasonable when you are new to TypeScript.

A.5.12.1 Mistake 1: annotating every initializer

```
const name: string = "Ada";
const count: number = 3;
const ready: boolean = false;
```

These annotations are valid, but the initializers already provide the types. Prefer an annotation when it communicates a boundary or catches a meaningful future change. Otherwise, let inference reduce repetition:

```
const name = "Ada";
const count = 3;
const ready = false;
```

A.5.12.2 Mistake 2: treating `const` as deep immutability

```
const settings = { mode: "read" };
settings.mode = "write"; // JavaScript allows this
```

`const` prevents replacing the object variable. It does not freeze the object or preserve a literal type for every property. Use an explicit contract or `as const` when the contents must be treated as fixed by the checker.

A.5.12.3 Mistake 3: using `?` as a fallback

```
type Options = { path?: string };
const options: Options = {};

// `?` describes the property; it does not return a default value.
const path = options.path ?? ".";
```

The question mark in the type declaration and the question mark in optional chaining are syntax in different places. Neither one is the same as `??`.

A.5.12.4 Mistake 4: using `||` for numeric configuration

```
function maxItems(value?: number): number {
  return value || 100;
}
```

This changes an intentional `0` into `100`. If `0` means "do not process any items," use `value ?? 100` instead.

A.5.12.5 Mistake 5: reading a union as though it were one shape

```
type Result =
  | { ok: true; value: string }
  | { ok: false; error: string };

function message(result: Result): string {
  if (result.ok) return result.value;
  return result.error;
}
```

The `ok` check is not decoration. It is the evidence that makes `value` or `error` available in the corresponding branch.

A.5.12.6 Mistake 6: using an assertion to convert data

```
const input: unknown = "42";
const number = input as number;
```

`number` is still the string `"42"` at runtime. Convert with `Number(input)` only when the conversion rules are appropriate, or validate the value before using it as a number.

A.5.12.7 Mistake 7: assuming strict checking makes every array lookup safe

```
function firstLetter(words: string[]): string {
  return words[0][0];
}
```

An empty array makes `words[0]` undefined at runtime. Ordinary `strict` checking does not necessarily add `undefined` to an indexed array result; `noUncheckedIndexedAccess` does. Write code that handles the runtime case regardless of that option:

```
function firstLetter(words: string[]): string | undefined {
  return words[0]?.[0];
}
```

The return type tells callers that they must handle an empty input. Do not change the repository compiler configuration merely to make this one example look safer.

A.5.12.8 Mistake 8: assuming an interface exists at runtime

```
interface Config {
  port: number;
}

// There is no Config constructor and no Config object to call at runtime.
```

Use a schema or a manual guard when a value arrives from JSON. Interfaces help the checker describe code that is already trusted.

A.5.12.9 Mistake 9: widening a discriminant accidentally

```
const event = { type: "started" };

// A consumer expecting { type: "started" } may reject event because
// event.type was widened to string.
```

Give the object an explicit type or use `as const` when the literal tag is part of a closed union.

A.5.12.10 Mistake 10: using `any` to silence a useful error

```
function display(value: any): string {
  return value.name;
}
```

This removes the information that could have told you the value was malformed. Prefer a narrow input type for trusted data or `unknown` plus a check for external data.

A.5.12.11 Mistake 11: assuming `readonly` freezes runtime data

```
const values = ["read", "write"];
const view: readonly string[] = values;

// view.push("preview"); // rejected through the readonly view
values.push("preview");  // allowed through the mutable alias
```

`readonly` protects writes through one static view. It does not freeze the array or make every alias readonly. Use runtime copying or `Object.freeze` when runtime immutability is actually required.

A.5.12.12 Mistake 12: using an assertion when `satisfies` or validation is needed

```
type Settings = { retries: number };
const settings = { retries: "three" } as Settings;
```

The assertion suppresses the useful mismatch and does not convert the string. For a checked source literal, use `satisfies Settings` so the checker reports the error while preserving useful inference. For external data, use `unknown` plus a guard or runtime schema; neither assertion nor `satisfies` validates JSON.

A.5.13 Read one complete function line by line

Use this small function as a final Part 1 reading exercise:

```
type DisplayOptions = {
  title: string;
  width?: number;
};

type DisplayResult = {
  text: string;
  truncated: boolean;
```

```
};

function display(options: DisplayOptions): DisplayResult {
  const width = options.width ?? 80;
  const text = options.title.slice(0, width);

  return {
    text,
    truncated: options.title.length > width,
  };
}
```

Read it in the order a beginner should use when opening unfamiliar code:

1. `DisplayOptions` describes the input object.
2. `title` is required and must be a string.
3. `width?` may be absent, so reading it produces `number | undefined`.
4. `DisplayResult` describes the returned object.
5. `options.width ?? 80` makes the local `width` a number.
6. `slice` is safe because `title` was declared as a string.
7. `truncated` is inferred as boolean because the comparison returns boolean.
8. The explicit `: DisplayResult` return annotation checks that the returned object has the documented shape and rejects an incompatible future change. It does not convert the object at runtime.

Try to predict the result of these calls:

```
display({ title: "Pi" });
display({ title: "TypeScript", width: 4 });
// display({ title: "Pi", width: "wide" }); // error
```

The first call uses width `80` and is not truncated. The second uses width `4` and returns `"Type"` with `truncated: true`. The third is rejected because the optional property, when present, must still be a number.

This is the same method you will use in Pi: read the input shape, identify optional values, follow the narrowing or fallback, then inspect the returned shape.

A.5.14 Everyday Types at a glance

Use this table as a reading aid, not as a list to memorize:

Code	What the checker sees	What to ask next
`const name = "Ada"`	literal string value	Can this value change?
`let name = "Ada"`	broad `string`	Could another string be assigned?
`const ids: number[] = []`	array of numbers	Can an index be missing?

Code	What the checker sees	What to ask next
`last?: string`	optional property	What happens when it is absent?
`string | number`	one of two alternatives	Which check narrows it?
`"idle" | "running"`	closed literal vocabulary	Which values are allowed?
`value as User`	asserted belief	What runtime evidence supports it?
`Static<typeof Schema>`	derived compile-time type	Where does the runtime check happen?

Read this small function using the table:

```
type Summary = {
  title: string;
  count?: number;
};

function format(summary: Summary): string {
  const count = summary.count ?? 0;
  return `${summary.title}: ${count}`;
}
```

The input is an object shape. `count` is optional, so the local fallback makes it a number. The function returns a string. No advanced TypeScript feature is required to understand the data flow.

Now compare a union input:

```
function formatInput(input: string | Summary): string {
  if (typeof input === "string") return input;
  return format(input);
}
```

The `typeof` check selects the string branch. The remaining branch is the object shape. This "identify the value, then find the check" habit is more useful than memorizing the syntax in isolation.

Keep the compile-time/runtime boundary in mind as this table's rows move from everyday types toward TypeBox: the checker is strongest on values constructed inside the checked program and cannot inspect data that arrives later. What TypeScript checks—and what it erases in the preamble develops that mental model in full.

Moving on. You can move on when you can read an optional-chaining fallback like `user.last?.toUpperCase() ?? "UNKNOWN"` out loud, narrow a small union with a check, and explain why `as` is a claim rather than a conversion. The worked example's self-check and the practice lab above are the rehearsal; the answers below grade it. Part 2 puts these everyday types to work on Pi's own patterns.

A.5.15 Answers

What TypeScript checks—and what it erases, Checkpoint. A checker can catch a misspelled property in checked source, but it cannot establish the shape of later JSON without runtime validation. Type syntax is removed or transformed before JavaScript runs. `noUncheckedIndexedAccess` adds `undefined` to the static result of an unchecked indexed access.

Primitives and arrays, Checkpoint. `"ready"` is `string` (the literal `"ready"` for a `const`), `17` is `number`, `false` is `boolean`, and `["a", "b"]` is `string[]`. `messages[99]` can return `undefined` at runtime. Under ordinary `strict` checking the checker may still report the lookup as `string`; `noUncheckedIndexedAccess` adds `undefined` to the static result. `any` disables checking, so a misspelled or missing property compiles and then fails at runtime.

Primitives and arrays, Try it. `changingStatus` is generally `string`, `fixedStatus` is the literal `"idle"`, and `values` is `number[]`. The reason for the difference between the first two values is explained fully in the literal-types section.

Annotations and inference, Checkpoint. `greet`'s return value is inferred as `string`. The `forEach` callback needs no annotation because the array's element type provides it through contextual typing. An explicit return annotation helps when the return value is a public contract that a future edit must not silently change.

Annotations and inference, Try it. The callback parameter is inferred from the array element type. If you instead write `const counts: unknown[]`, the callback cannot call `toFixed` until you narrow each item. Context provides useful information, but it cannot invent facts that the surrounding value does not have.

Functions and common parameter forms, Checkpoint. Optional parameters and default parameters allow omission; the default parameter supplies a runtime fallback. `Promise<string>` describes a result that will eventually be a string. `void` means the caller should not expect a useful return value.

Functions and common parameter forms, Try it. The first two calls both use the fallback. A required parameter does not allow omission merely because its implementation could handle it; the function signature communicates what callers are allowed to pass.

Object types and optional properties, Checkpoint. An optional property (`last?: string`) describes a key that may be absent; optional chaining (`user.last?.`) safely accesses a value that may be missing. `||` turns `0` into the fallback while `??` keeps it. `user.last` may be `undefined`, so the checker requires evidence before a string method is called. `readonly` provides no runtime protection: none. An index signature such as `{ [key: string]: unknown }` describes an open-ended string-keyed object.

Union types and basic narrowing, Checkpoint. Before narrowing, only operations valid for every union member are allowed, and `number` has no `toUpperCase`. `Array.isArray` proves the value is an array in the true branch. Properties shared by every member, such as `length` on `string | number[]`, can be used before narrowing.

Union types and basic narrowing, Try it. Both strings and arrays have `slice`, so the call is safe. The return type still contains both possibilities because the function did not decide which member it received.

Type aliases and interfaces, Checkpoint. A type alias names a type that already exists; an interface similarly names an object shape and can be extended. A variable is compared structurally,

so extra fields are allowed; a fresh object literal also receives an excess-property check, which rejects unexpected keys.

Type assertions, Checkpoint. `String(value)` changes the runtime value; the ternary with `typeof` checks the runtime value; `as` changes only what the checker permits.

Type assertions, Try it. Both `source` and `claimed` contain the number `42`. Only `converted` contains the string `"42"`. The assertion changes what the checker permits; it does not perform the conversion.

Literal types and literal inference, Checkpoint. `changing` is declared with `let`, so it widens to `string` to allow reassignment; `fixed` is a `const`, so it keeps the literal `"hello"`. A normal object's properties are mutable, so `event.type` widens to `string` unless the object is annotated or checked with `as const`. A literal union is more useful when a value has several legal alternatives rather than exactly one.

`null`, `undefined`, and related operators, Checkpoint. `undefined` means "no value was provided" and `null` is usually a deliberate "empty" marker; code that treats them differently can use that distinction, code that does not should use `??`. `||` also triggers on `0`, `""`, and `false`, while `??` only triggers on `null` and `undefined`. `!` provides no runtime protection; it only changes the checker's view.

Exercise 1. `title` is inferred as the literal type `"Build"` because it is a `const` primitive. `attempts` is `number[]`. `ready` is the literal type `false`. Indexing may produce `undefined` when no element exists at runtime; with `noUncheckedIndexedAccess`, the checker also includes `undefined` in the indexed result. Without that option, ordinary `strict` checking does not by itself guarantee that the static result includes `undefined`.

Exercise 2. `mode` is generally `string` because it can be reassigned. `fixedMode` preserves the literal `"read"`. `name` is inferred as `string` from the array passed to `map`.

Exercise 3. The first two use `3`. The third returns `0`, because zero is a meaningful number and `??` preserves it. The last call is rejected by the type checker because `null` is not assignable to `number | undefined` unless the type explicitly includes `null`.

Exercise 4. Both strings and arrays have `length`. Only arrays have `join`, so the `Array.isArray` check is needed before using it.

Exercise 5. The variable has the required `x` and `y` fields, so structural typing accepts it. A fresh object literal receives an extra-property check and may reject an unexpected `label` field in that direct assignment.

Exercise 6. No. The assertion does not change the runtime string `"many"`. A safe implementation checks that the input is an object and that `count` is a number, or validates it with a runtime schema.

Exercise 7. `modes` is `readonly ["read", "write"]`, and `Mode` is `"read" | "write"`. `push` is unavailable through the readonly type. Nothing freezes the runtime array; another mutable reference could still change it.

Exercise 8. No. `satisfies` checks this source expression at compile time and has no runtime effect. It would catch a missing required key, an unexpected key in this object literal, or a non-number value, but it does not inspect later JSON data.

Exercise 9. `assertNever` accepts only `never`, so when every union member is handled the remaining type is `never` and the call is valid. Adding `cancelled` makes the `assertNever` call fail the type check until the new variant is handled.

A.6 Part 2 — Pi patterns built from everyday types

You now have the everyday TypeScript foundation. The next sections add only the patterns needed to read Pi's source. They are not prerequisites for Part 1.

In this part you will learn to:

- follow an `import` across Pi files and know what a package's `exports` allows;
- read a discriminated union like Pi's event types and narrow it by its tag;
- write a callback without annotating its parameters by hand;
- treat `unknown` external data as unchecked until a runtime check proves it;
- read a generic signature by naming each type parameter's job, without designing one yourself.

A.6.1 Modules and imports

A **module** is one file that imports names from other files and exports names for other files to use.

```ts
// math.ts
export function add(left: number, right: number): number {
  return left + right;
}

export type Pair = [number, number];
```

```ts
// main.ts
import { add, type Pair } from "./math.ts";

const pair: Pair = [2, 3];
console.log(add(...pair));
```

`add` is a runtime value, so it is a normal import. `Pair` is only used by the checker, so `type Pair` makes that intent explicit.

Within one Pi package, use relative imports with the `.ts` suffix. Across packages, import through the published package name:

```ts
import { Type } from "@earendil-works/pi-ai";
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";
```

Prefer public entry points. A deep import into another package's internal source bypasses the boundary that published consumers use.

A module can also export a default value or re-export a public name:

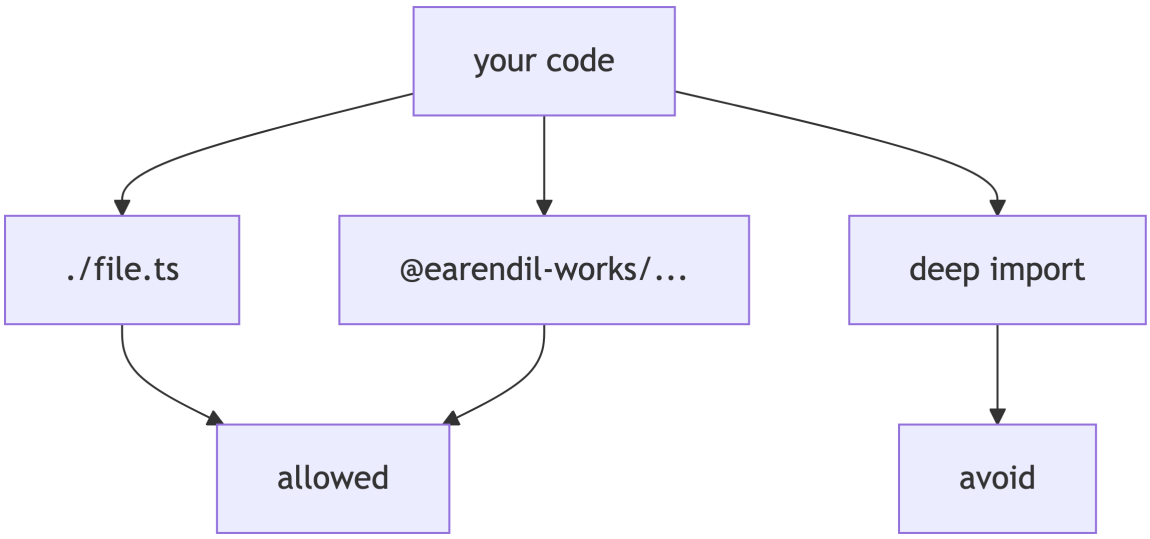

Figure A.3: Three import paths: relative and package-name imports are allowed; deep imports into another package's source are not.

```
// register.ts
export default function register(): void {
  console.log("registered");
}

// index.ts
export { default as register } from "./register.ts";
export type { Pair } from "./math.ts";
```

A default import names the exported value locally:

```
import register from "./register.ts";
import { register as namedRegister } from "./index.ts";
```

Top-level `import` or `export` makes a file a module. TypeScript then resolves relative paths and package names using the configured module-resolution mode. When a module error appears, check the relative path, `.ts` suffix convention, package `exports`, and public entry point before reaching for a deep import. CommonJS interop, namespaces, and `import =`/`export =` syntax are outside Pi's erasable ESM conventions.

Recognition note: type-only cycles can sometimes be broken with `import type`; runtime cycles need a design review. You do not need to solve module cycles to read ordinary imports.

A.6.2 Discriminated unions in Pi

A **discriminant** is a shared literal property that identifies a union variant. Start with the status example:

```ts
type StatusReport = {
  status: "idle" | "running" | "failed";
  message: string;
};

type StatusEvent =
  | { type: "started"; report: StatusReport }
  | { type: "updated"; report: StatusReport }
  | { type: "failed"; message: string };
```

The tag selects which fields are safe:

```ts
function eventLabel(event: StatusEvent): string {
  if (event.type === "failed") {
    return `failed: ${event.message}`;
  }
  return `${event.type}: ${event.report.message}`;
}
```

The same logic can use a `switch`:

```ts
function eventLabelWithSwitch(event: StatusEvent): string {
  switch (event.type) {
    case "started":
      return `started: ${event.report.message}`;
    case "updated":
      return `updated: ${event.report.message}`;
    case "failed":
      return `failed: ${event.message}`;
  }
}
```

Pi uses this pattern for `AgentEvent`, message roles, content blocks, and session entries. Find the literal `type` or `role` values first, then read the branch that handles the value you care about.

Here is the real `AgentEvent` union, excerpted from `packages/agent/src/types.ts` (around line 415; line numbers shift as the repository changes, so search for `export type AgentEvent`):

```ts
export type AgentEvent =
    // Agent lifecycle
    | { type: "agent_start" }
    | { type: "agent_end"; messages: AgentMessage[] }
    // Turn lifecycle - a turn is one assistant response + any tool calls/results
    | { type: "turn_start" }
    | { type: "turn_end"; message: AgentMessage; toolResults: ToolResultMessage[] }
    // Message lifecycle - emitted for user, assistant, and toolResult messages
    | { type: "message_start"; message: AgentMessage }
    // Only emitted for assistant messages during streaming
```

```
    | {
        type: "message_update";
        message: AgentMessage;
        assistantMessageEvent: AssistantMessageEvent;
      }
    | { type: "message_end"; message: AgentMessage }
    // Tool execution lifecycle
    | {
        type: "tool_execution_start";
        toolCallId: string;
        toolName: string;
        args: any;
      }
    | {
        type: "tool_execution_end";
        toolCallId: string;
        toolName: string;
        result: any;
        isError: boolean;
      };
```

(One variant, `tool_execution_update`, is omitted here to keep the excerpt short.) Reading notes:

1. Every member shares the literal `type` property, so `event.type` is the discriminant and every consumer can narrow on it.
2. Variants carry different payloads: `agent_end` has `messages`, while `agent_start` has nothing extra. Accessing `event.messages` is only legal after narrowing to a member that has it.
3. Comments group the variants into lifecycles (agent, turn, message, tool), which is the reading order a newcomer should follow.
4. Some payloads use `any` (for example `args` and `result`). That is a deliberate escape hatch at a UI boundary, not a license to copy the habit.
5. The payload types (`AgentMessage`, `ToolResultMessage`) are imported names; follow them with go-to-definition when you need their shapes.

For the producer side, open `packages/agent/src/agent-loop.ts` and look at the `emit({ type: "agent_start" })` calls and the provider-event switch that emits `message_start`/`message_update`/`message_end`. For an `AgentEvent` consumer, open `packages/agent/src/agent.ts` and inspect `Agent.processEvents()`. The `emit` calls construct union members; `processEvents()` and UI subscribers consume and narrow them. The guided tour in Part 5 builds on this excerpt.

A.6.2.1 User-defined predicates and exhaustive unions

When a runtime check is reusable, a function can report its narrowing result with a type predicate. The `value is User` return type means that callers may treat `value` as `User` in the true branch:

```
type User = { name: string };

function isUser(value: unknown): value is User {
```

```
  return !!value && typeof value === "object" &&
    "name" in value && typeof value.name === "string";
}

function displayUser(value: unknown): string {
  if (!isUser(value)) return "invalid user";
  return value.name.toUpperCase();
}
```

The predicate must agree with the runtime test. It changes the checker's view; it does not validate anything by magic. TypeBox uses the same pattern with `Value.Check` later.

An exhaustive `never` check asks the checker to report a newly added variant:

```
function assertNever(value: never): never {
  throw new Error(`Unexpected variant: ${String(value)}`);
}

type Result =
  | { type: "ok"; value: string }
  | { type: "error"; message: string };

function resultLabel(result: Result): string {
  switch (result.type) {
    case "ok": return result.value;
    case "error": return result.message;
    default: return assertNever(result);
  }
}
```

If a new variant is added to `Result`, the `assertNever(result)` line stops being valid until the new case is handled. This is useful recognition-level knowledge for Pi event consumers.

A.6.3 Functions as callbacks and closures

A **callback** is a function passed to another function to be called now or later. Registration stores a function; invocation calls it.

```
type Listener = (message: string) => void;

let listener: Listener | undefined;

function register(next: Listener): void {
  listener = next; // store the function; do not call it yet
}

function emit(message: string): void {
```

```
  listener?.(message); // call it later if one was registered
}

register((message) => console.log(message));
emit("ready");
```

The common mistake is `register(next())`, which calls the function now and passes its return value instead of passing the function.

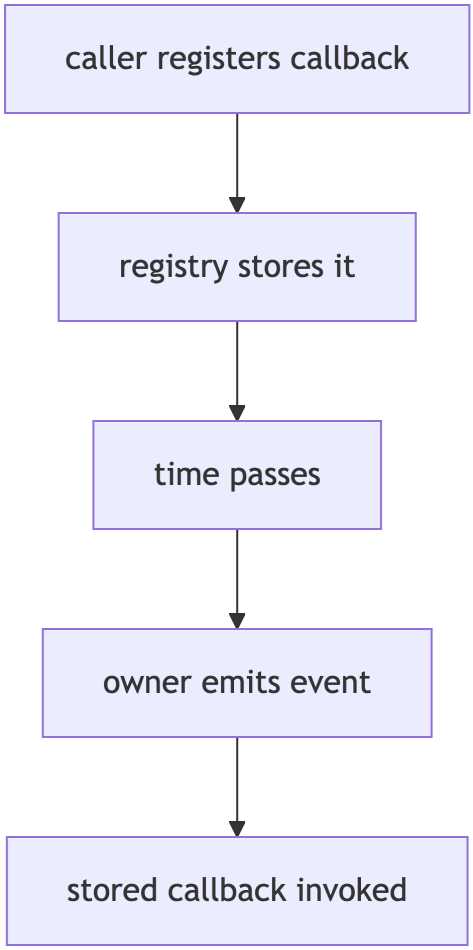

Figure A.4: Registration stores the callback; the owner invokes it later.

A **closure** is a function that remembers variables from where it was created:

```
function makeCounter(): () => number {
  let count = 0;
  return () => {
    count += 1;
    return count;
  };
}
```

Closures are useful for private state, but a long-lived registry can keep large objects alive. Use the cleanup API when one exists.

A.6.3.1 Pi-style callbacks

Pi handlers receive a full signature even when a handler needs only one value:

```
interface SessionStartEvent {
  type: "session_start";
}

interface SessionContext {
  log(message: string): void;
}

function onSessionStart(
  handler: (event: SessionStartEvent, context: SessionContext) => void,
): void {
  handler({ type: "session_start" }, { log: console.log });
}

onSessionStart((_event, context) => {
  // _event is intentionally unused; context is the value we need.
  context.log("started");
});
```

In real Pi code, `_event`, `_signal`, and `_ctx` commonly mean "this parameter is part of the contract, but this handler does not use it."

A callback parameter should be optional only when the implementation may really call the callback without it:

```
function visit(
  values: string[],
  callback: (value: string, index?: number) => void,
): void {
  for (let index = 0; index < values.length; index += 1) {
    callback(values[index], index);
  }
}
```

This signature promises that `callback` might receive no index, so a callback that uses `index` must handle `undefined`. If the implementation always passes an index, make it required instead:

```
function visitWithIndex(
  values: string[],
  callback: (value: string, index: number) => void,
): void {
```

```
  values.forEach(callback);
}
```

A.6.3.2 The real registration in Pi

The synthetic `onSessionStart` above mirrors the real extension contract in `core/extensions/types.ts` within the coding-agent package (line numbers shift as the file evolves). Condensed excerpt:

```
/** Fired when a session is started, loaded, or reloaded */
export interface SessionStartEvent {
    type: "session_start";
    reason: "startup" | "reload" | "new" | "resume" | "fork";
    previousSessionFile?: string;
}

export type ExtensionHandler<E, R = undefined> =
    (event: E, ctx: ExtensionContext) => Promise<R | void> | R | void;

// inside interface ExtensionAPI:
on(event: "session_start", handler: ExtensionHandler<SessionStartEvent>): void;
```

1. `SessionStartEvent` is a discriminated-union member: the literal `"session_start"` tag ties the event data to the registration string.
2. `ExtensionHandler` is the generic callback shape from this section: the event arrives first and the context second, and the handler may be sync or async.
3. `on` is the registration call: you pass a function, and Pi invokes it later — registration stores, invocation calls.

A.6.3.3 Overload recognition

An overloaded function exposes several allowed call shapes, followed by one implementation that handles them:

```
function formatValue(value: string): string;
function formatValue(value: number): string;
function formatValue(value: string | number): string {
  return typeof value === "string" ? value : value.toFixed(2);
}

formatValue("ready");
formatValue(3.14);
```

The implementation signature is not directly visible to callers. Prefer a union parameter instead of overloads when the alternatives have the same return shape and can be expressed clearly with one signature.

Recognition note: a detached ordinary method can lose its `this` receiver. An arrow-function property captures the surrounding `this`, while a normal method receives its `this` from the call site. This matters when registering a class method as a callback; bind it or use an intentional wrapper when the receiver must be preserved.

A.6.4 `unknown`, `any`, and external data

An external boundary is where data enters from JSON, a config file, a network, a model response, or an extension. The checker cannot know that the incoming value has the shape you expect.

Use `any` only when you accept no checking:

```
function printNameBad(value: any): void {
  console.log(value.name.toUpperCase());
}
```

Use `unknown` when you want to require evidence before use:

```
function printName(value: unknown): void {
  if (!value || typeof value !== "object") return;
  if (!("name" in value)) return;

  const name = value.name;
  if (typeof name !== "string") return;
  console.log(name.toUpperCase());
}
```

Take one JSON value through four treatments:

```
const raw: unknown = JSON.parse('{"name":"Ada","age":"unknown"}');

const unsafe = raw as { name: string; age: number };
// No runtime validation happened. age is still a string at runtime.

function readUser(value: unknown): { name: string; age: number } | undefined {
  if (!value || typeof value !== "object") return undefined;
  if (!("name" in value) || !("age" in value)) return undefined;

  const { name, age } = value;
  if (typeof name !== "string" || typeof age !== "number") return undefined;
  return { name, age };
}
```

The manual guard builds a new trusted value from fields that were checked. The TypeBox section shows how to avoid repeating those checks when the shape is shared or exposed as a tool contract.

A.6.4.1 JSONL: line-delimited external data

Pi session files are JSONL: one JSON value per line, not one JSON document. Parse each line separately so one bad line does not destroy the whole file:

```
for (const line of text.split("\n")) {
  if (line.trim() === "") continue;

  let entry: unknown;
  try {
    entry = JSON.parse(line);
  } catch {
    continue; // or record the bad line number and report it
  }

  if (!Value.Check(SessionEntrySchema, entry)) continue;
  // entry is now trusted
}
```

`JSON.parse` returns `unknown` (through the `entry` annotation), so every line goes through the same guard-or-schema step as any other external value. The loop chooses whether to skip or report malformed lines; the one thing it never does is trust them.

The round trip works the same way in the other direction. `JSON.stringify` accepts any value and erases its type at the boundary:

```
const text = JSON.stringify(session); // just a string
```

The string carries no type information. Whoever parses it on the receiving side — another tool, a later session, another process — starts from `unknown` again and must validate for itself. Types do not travel through JSON in either direction; every boundary re-opens the question.

A.6.4.2 Node globals to recognize

Pi runs on Node, so its source uses a few Node-specific names that are not part of the JavaScript language. Recognition only: `process.env` reads environment variables such as API keys; imports prefixed with `node:` such as `node:fs` and `node:path` are Node built-in modules (the prefix makes "this is a built-in, not a package" explicit); `Buffer` is Node's byte container, used when data is not text yet. Look these up in the Node documentation when you need their exact behavior.

Checkpoint: Which value is untrusted? What does `as` do? Why is the object returned by `readUser` safer than the original parsed value? (answer: Part 2 answers)

A.6.5 Errors and exceptions

Exceptions are the other way operations fail: instead of returning a value, code throws. `try/catch` intercepts a thrown value:

```
function readConfig(text: string): unknown {
  try {
    return JSON.parse(text);
  } catch {
    return undefined;
  }
}
```

The `catch` variable is `unknown` under Pi's strict checking, because any value can be thrown. Narrow it before reading properties; `instanceof Error` is the standard check:

```
try {
  await loadSession(id);
} catch (error) {
  if (error instanceof Error) {
    console.error(error.message);
  } else {
    console.error(String(error));
  }
}
```

Define a custom error when callers need to distinguish failure kinds. Extend `Error`, set `name`, and add the fields a handler will read:

```
class ConfigError extends Error {
  readonly path: string;

  constructor(path: string, message: string) {
    super(message);
    this.name = "ConfigError";
    this.path = path;
  }
}
```

`instanceof ConfigError` then narrows a caught value enough to read `path`. When re-throwing a failure you caught, pass `cause` so the original error is not lost:

```
try {
  return parseSession(text);
} catch (error) {
  throw new Error(`Could not load session ${id}`, { cause: error });
}
```

Exceptions and the `Result`-style unions from the discriminated-unions section solve the same problem at different distances. Pi tools may return a structured error result or throw from `execute`; thrown execution errors are converted by the loop, while validation, not-found, block, and other preflight errors can be generated directly as tool results. Pi's own control flow often prefers returned unions such as `{ ok: true } | { ok: false }` where a caller must narrow. When you read Pi code, ask which convention the function chose: a thrown exception crosses layers, while a returned union or result is handled by the immediate caller. The failure and recovery reference covers the runtime side: retries, cancellation during failure, and user-facing errors.

Checkpoint: Why is the `catch` variable typed `unknown` instead of `Error`? What does `instanceof Error` prove about a caught value? What does `cause` preserve when you wrap an error? When does Pi prefer a returned `Result`-style union over a thrown exception? (answer: Part 2 answers)

A.6.6 Generics and utility types

A.6.6.1 Generic relationships

A **generic** preserves a relationship between types selected when code is used. `T` is a type placeholder, not a runtime value:

```
function first<T>(items: readonly T[]): T | undefined {
  return items[0];
}

const firstName = first(["Ada", "Grace"]);
// string | undefined
```

The caller usually supplies `T` through inference. A constraint limits which values are allowed while preserving the relationship:

```
function longer<T extends { length: number }>(left: T, right: T): T {
  return left.length >= right.length ? left : right;
}

const name = longer("Ada", "Grace"); // string
const numbers = longer([1], [1, 2]);  // number[]
// longer(1, 2); // error: numbers do not have length
```

Use a type parameter when it relates two or more positions, such as an input and its result. Do not add a generic that appears only once; a named interface or union is often clearer for a beginner-facing contract.

A.6.6.2 Types from existing declarations

TypeScript also lets one type refer to a runtime declaration without executing it. This is a different meaning of `typeof` from the runtime operator:

```ts
const defaultConfig = { retries: 3, mode: "read" } as const;
type Config = typeof defaultConfig;
// { readonly retries: 3; readonly mode: "read" }
```

`keyof` produces a union of the known property keys. Indexed access selects a property type using the same bracket shape used for JavaScript access:

```ts
type ConfigKey = keyof Config;       // "retries" | "mode"
type RetryCount = Config["retries"]; // 3
```

For an array, `[number]` means the type of any element:

```ts
type Modes = ["read", "write"];
type Mode = Modes[number]; // "read" | "write"
```

Read `Static<typeof UserSchema>` the same way: `typeof UserSchema` obtains the compile-time type of the runtime schema value, and `Static<...>` derives the schema's static value type. Neither operation validates data by itself.

A.6.6.3 Selected utility types

Utility types transform compile-time descriptions. They do not mutate, copy, freeze, validate, or convert runtime values. Three worked examples cover the shapes you will meet most; the reference table lists the rest.

`Partial<T>` makes every property optional, which is how "update only some fields" inputs are described:

```ts
type User = { name: string; retries: number };

function applyPatch(user: User, patch: Partial<User>): User {
  return { ...user, ...patch };
}
```

`Pick<T, K>` keeps only the named keys:

```ts
type UserName = Pick<User, "name">; // { name: string }
```

`Awaited<ReturnType<typeof fn>>` reads what an async function eventually delivers, without calling it:

```ts
async function loadCount(): Promise<number> {
  return 3;
}

type Delivered = Awaited<ReturnType<typeof loadCount>>; // number
```

For example:

```ts
async function loadReport(): Promise<{ title: string; count: number }> {
  return { title: "Build", count: 3 };
}

type Report = Awaited<ReturnType<typeof loadReport>>;
type TitleOnly = Pick<Report, "title">;
type Present = NonNullable<string | undefined>;
```

The same pattern works on a real Pi declaration. `generatePKCE` in `packages/ai/src/auth/oauth/pkce.ts` returns `Promise<{ verifier: string; challenge: string }>`, so:

```ts
type PkcePair = Awaited<ReturnType<typeof generatePKCE>>;
// { verifier: string; challenge: string }
```

These names help you read Pi declarations. They do not add runtime checks to `loadReport` or create a `TitleOnly` object.

`Extract` and `Exclude` select or remove members of a union. The most common extension typing task uses them: picking one variant of Pi's `AgentEvent` union (from `packages/agent/src/types.ts`) to type a handler, without repeating the variant's fields:

```ts
import type { AgentEvent } from "@earendil-works/pi-agent-core";

type MessageUpdateEvent = Extract<AgentEvent, { type: "message_update" }>;

function onMessageUpdate(event: MessageUpdateEvent): void {
  // event.message and event.assistantMessageEvent are fully typed here.
  console.log(event.assistantMessageEvent.type);
}
```

`Extract<AgentEvent, { type: "message_update" }>` keeps only the variant whose `type` is `"message_update"`, so the handler sees exactly that shape. `Exclude` is the mirror image: it removes a variant from a union, which is useful when listing "every event except this one."

A.6.6.4 Advanced type manipulation: recognize, do not design yet

Conditional types act like type-level `if` expressions. Mapped types transform each property of an object type, and template-literal types build string literal types from other literals. They underpin many utility types and may appear in Pi aliases, but do not turn this foundations workbook into a type metaprogramming course. Follow the alias back to its inputs, then consult the Handbook pages for conditional types, mapped types, and template literal types.

Moving on. You can move on when you can narrow a discriminated union by its tag, write a Pi-style callback without annotating its parameters, explain why caught errors arrive as `unknown`, and name the job of each type parameter in a generic signature. The answers below close out these patterns. Part 3 adds the missing half of the story: checking data the checker cannot see, with TypeBox.

A.6.7 Answers

`unknown`, `any`, and external data, Checkpoint. `raw`, the parsed JSON, is untrusted: the checker cannot know its shape. `as` only tells the checker to treat it as `{ name: string; age: number }`; at runtime `age` is still a string. `readUser` returns a new object built from fields that were individually checked, so its type matches the runtime value.

Errors and exceptions, Checkpoint. Any value can be thrown in JavaScript, not just `Error` objects, so the checker honestly reports the caught value as `unknown`. `instanceof Error` proves the value has the `Error` prototype, so `message` and `name` are safe to read. `cause` keeps the original error attached to the wrapping error so logs can show the full failure chain. Pi prefers a returned `Result`-style union when the immediate caller must handle the outcome, and throws when a failure should cross layers to the runtime.

A.7 Part 3 — TypeBox as Pi's runtime validation bridge

The examples below use normal TypeScript module imports. An `import` statement loads a value or type from another module; the package name identifies a published dependency such as `@earendil-works/pi-ai`.

In this part you will learn to:

- describe a shape once as a TypeBox schema and get both a runtime check and a static type from it;
- read `Type.Object`, `Type.Optional`, and literal-union builders as a small vocabulary;
- validate an `unknown` value before trusting its fields;
- expose the same schema as a tool's parameter contract to a model.

A.7.1 What TypeBox does

An interface helps TypeScript check code, but it disappears when the program runs:

```
interface User {
  name: string;
}
```

If the program receives JSON, the interface cannot inspect the JSON. TypeBox lets Pi describe the same shape as a runtime schema:

```
import { Type, type Static } from "@earendil-works/pi-ai";
import { Value } from "typebox/value";

const UserSchema = Type.Object({
  name: Type.String(),
});

type User = Static<typeof UserSchema>;
```

There are two separate things here:

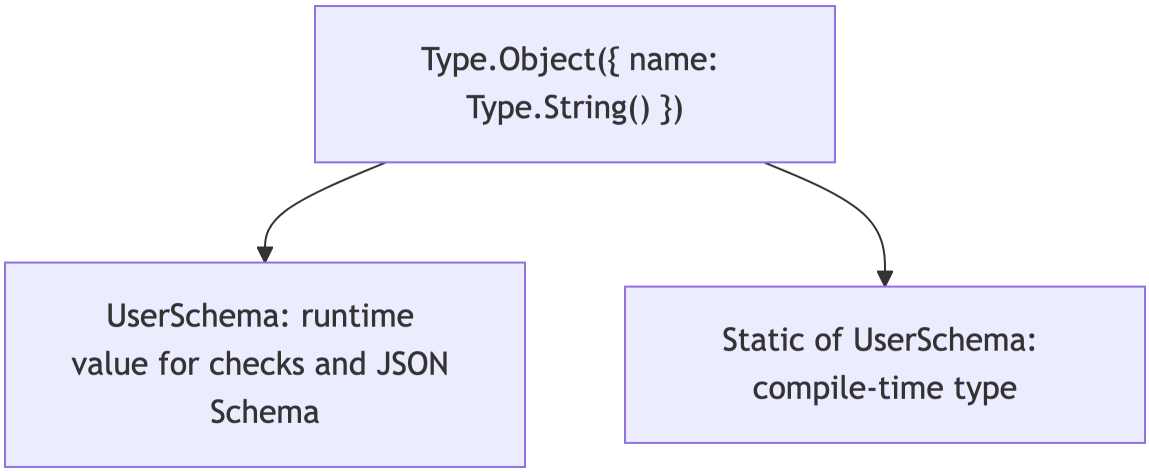

Figure A.5: One schema, two views: a runtime value and a compile-time type.

`UserSchema` exists while the program runs. `User` is only a TypeScript type. `Static<typeof UserSchema>` derives a type from the schema; it does not create a user object and does not validate a value by itself.

It can help to imagine the schema as ordinary runtime data:

```
console.log(UserSchema);
// A schema object describing an object with a string "name" property.
```

The exact internal object is a library detail. The important point is that the program can pass `UserSchema` to a validation function. An interface cannot be passed to a function because it is gone after type checking.

Check an actual value at runtime:

The return type `value is User` is a **type predicate**: after this function returns `true`, TypeScript narrows the value to `User` in the surrounding code.

```
function isUser(value: unknown): value is User {
  return Value.Check(UserSchema, value);
}

const valid: unknown = { name: "Ada" };
const invalid: unknown = { name: 42 };

if (isUser(valid)) {
  console.log(valid.name.toUpperCase());
}

console.log(isUser(invalid)); // false
```

The important operation is `Value.Check`. The schema and the derived type make the contract reusable, but the check is what inspects the runtime value.

A.7.2 TypeBox builders

Each builder creates a runtime schema and contributes to the derived static type:

Builder	Accepted runtime value	Static type
`Type.String()`	text	`string`
`Type.Number()`	number	`number`
`Type.Boolean()`	`true` or `false`	`boolean`
`Type.Array(Type.String())`	array of strings	`string[]`
`Type.Literal("read")`	exactly `"read"`	`"read"`
`Type.Union([...])`	one member schema	union of members

Build the pieces separately before putting them inside an object:

```
const NameSchema = Type.String();
const NamesSchema = Type.Array(NameSchema);
const ReadModeSchema = Type.Literal("read");
const ModesSchema = Type.Union([
  Type.Literal("read"),
  Type.Literal("preview"),
]);
```

Each constant is an ordinary runtime variable. The names ending in `Schema` are not special TypeScript syntax; the suffix is a naming convention that helps readers remember that these values are used for runtime checks.

Build a small search contract one field at a time:

```
const SearchSchema = Type.Object({
  pattern: Type.String({ description: "Text to find" }),
  path: Type.Optional(Type.String()),
  caseSensitive: Type.Boolean(),
  tags: Type.Array(Type.String()),
  mode: Type.Union([
    Type.Literal("read"),
    Type.Literal("preview"),
  ]),
});

type SearchParams = Static<typeof SearchSchema>;
```

The static result is approximately:

```
type SearchParams = {
  pattern: string;
  path?: string;
  caseSensitive: boolean;
```

```
  tags: string[];
  mode: "read" | "preview";
};
```

`path` may be absent because it uses `Type.Optional`. The other keys are required. Validation does not fill in a missing `path`, change `mode`, or create the `tags` array for you.

Compare valid and invalid values:

```
const good = {
  pattern: "TODO",
  caseSensitive: false,
  tags: ["source"],
  mode: "read",
};

const bad = {
  pattern: "TODO",
  caseSensitive: "no",
  tags: [],
  mode: "write",
};

Value.Check(SearchSchema, good); // true
Value.Check(SearchSchema, bad);  // false
```

`Value.Check` does not repair `bad`. It only answers whether the value matches. If an optional field should receive a default, do that in application code:

```
function withDefaults(value: SearchParams): SearchParams & { path: string } {
  return {
    ...value,
    path: value.path ?? ".",
  };
}
```

Validation and defaulting are two separate steps. Keeping them separate makes configuration behavior easier to test and explain.

TypeBox does not parse JSON text, add defaults, migrate old configuration, perform side effects, or prove application rules such as “the path exists.” Those are separate application responsibilities.

A.7.2.1 Richer schema shapes

The builders compose, so real tool contracts can nest objects, maps, and arrays of objects:

```ts
const ToolConfigSchema = Type.Object({
  name: Type.String(),
  limits: Type.Object({
    timeoutMs: Type.Number(),
    retries: Type.Number(),
  }),
  labels: Type.Record(Type.String(), Type.Number()),
  tags: Type.Array(
    Type.Object({
      key: Type.String(),
      weight: Type.Number(),
    }),
  ),
});

type ToolConfig = Static<typeof ToolConfigSchema>;
```

Read each builder by the value it accepts:

- `Type.Object` inside `Type.Object` checks a nested object, and the static type nests the same way.
- `Type.Record(Type.String(), Type.Number())` checks an object with arbitrary string keys whose values are numbers, like the index signatures from Part 1.
- `Type.Array` of `Type.Object` checks a list where every element matches the object schema.

The optional-with-default pattern from Part 1 still applies at any depth: mark the field `Type.Optional`, validate first, then supply the default in application code as `withDefaults` did above. The schema states what valid data looks like; it never fills in missing values for you.

For string literals inside a tool contract, prefer the helper Pi ships for provider compatibility: `StringEnum([...])` instead of hand-written `Type.Union` of literals. Extension patterns explains the difference and the schema policy.

A.7.2.2 Reporting why a value failed: `Value.Errors`

`Value.Check` answers yes or no. When you must report *why* a value failed — for example when a model's tool arguments or a user's config entry is rejected — iterate `Value.Errors` instead. It runs an exhaustive check and returns an array of errors; each error carries an `instancePath` pointing at the offending field and a readable `message`:

```ts
import { Value } from "typebox/value";

const errors = [...Value.Errors(SearchSchema, bad)];
if (errors.length > 0) {
  const first = errors[0];
  console.log(`Invalid config at ${first.instancePath || "/"}: ${first.message}`);
}
// Invalid config at /caseSensitive: must be boolean
```

Unlike `Value.Check`, `Value.Errors` does not stop at the first problem, so the usual pattern is to check first and collect errors only when the check fails. Use check-and-report when a human or a model should see what to fix; use check-and-reject when the caller can only try again with corrected data.

Checkpoint: A user's config fails `Value.Check`. Which function tells you which field is wrong, and what two pieces of information does each error give you?

A.7.3 TypeBox versus other approaches

Approach	Runtime validation?	Static type?	Main limitation
`interface`	no	yes	disappears at runtime
`as User`	no	yes	can make a false claim
manual guard	yes	only if maintained carefully	can drift from the type
TypeBox	yes	yes, derived from schema	does not enforce business rules

Use an interface when you only need a compile-time object shape. Use TypeBox when the same shape must be checked at runtime or described to another system.

A.7.4 TypeBox in a Pi tool

The actual extension pattern has two stages: define the tool and register it with an extension factory.

```
import { Type } from "@earendil-works/pi-ai";
import {
  defineTool,
  type ExtensionAPI,
} from "@earendil-works/pi-coding-agent";

const helloTool = defineTool({
  name: "hello",
  label: "Hello",
  description: "Greet one person.",
  parameters: Type.Object({
    name: Type.String({ description: "Person to greet" }),
  }),

  async execute(_toolCallId, params, signal, _onUpdate, _ctx) {
    if (signal?.aborted) {
      return { content: [{ type: "text", text: "cancelled" }] };
    }
```

```
    return {
      content: [{ type: "text", text: `Hello, ${params.name}!` }],
    };
  },
});

export default function register(pi: ExtensionAPI): void {
  pi.registerTool(helloTool);
}
```

Read the example in this order:

1. `Type.Object` is a runtime parameter schema.
2. `defineTool` preserves the relationship between the schema and `params`.
3. Pi validates raw model arguments before `execute` runs.
4. `params.name` is available as a string after validation.
5. `signal` may be absent and should be forwarded to cancellable work.
6. The returned `content` becomes the tool result sent through Pi's runtime.

For string-literal fields in a tool contract, use `StringEnum` as described in Richer schema shapes rather than hand-written unions of literals.

A.7.5 TypeBox practice: from external data to a tool parameter

Use this four-step sequence when reading a real Pi tool.

A.7.5.1 Step 1: describe the expected value

```
const TodoSchema = Type.Object({
  title: Type.String(),
  completed: Type.Boolean(),
});

type Todo = Static<typeof TodoSchema>;
```

The type describes what valid data looks like. The schema is the runtime value that can inspect an actual object.

A.7.5.2 Step 2: receive an unknown value

```
function readTodo(value: unknown): Todo | undefined {
  if (!Value.Check(TodoSchema, value)) return undefined;
  return value;
}
```

The input is `unknown` because it came from outside the trusted code. The function returns either a value that matches the schema or `undefined`.

A.7.5.3 Step 3: use the checked fields

```
function todoLabel(value: unknown): string {
  const todo = readTodo(value);
  if (!todo) return "invalid todo";
  return todo.completed ? `[done] ${todo.title}` : `[open] ${todo.title}`;
}
```

After the check, the implementation can use `todo.title` as a string and `todo.completed` as a boolean. It does not need to repeat those field checks.

A.7.5.4 Step 4: expose the same contract to a model

```
const todoTool = defineTool({
  name: "show_todo",
  label: "Show todo",
  description: "Display one todo item.",
  parameters: TodoSchema,
  async execute(_toolCallId, params) {
    return {
      content: [{ type: "text", text: todoLabel(params) }],
    };
  },
});
```

The schema is reused as the tool parameter contract. The model supplies raw arguments, Pi validates them, and `params` receives the schema-derived shape when execution begins. The tool still needs an extension factory to register it, as shown in the previous section.

Common mistake: treating `Todo` as if it were a constructor or validator. It is only a compile-time type. `TodoSchema` and `Value.Check` are the runtime pieces.

A.7.5.5 TypeBox self-check

Read this schema without looking at the answer:

```
const ConfigSchema = Type.Object({
  port: Type.Number(),
  host: Type.Optional(Type.String()),
  mode: Type.Union([
    Type.Literal("local"),
```

```
    Type.Literal("remote"),
  ]),
});

type Config = Static<typeof ConfigSchema>;
```

Answer these questions:

1. Which keys are required?
2. What values can `mode` contain?
3. What is the static type of `host`?
4. Is `{ port: 3000, mode: "local" }` valid?
5. Is `{ port: "3000", mode: "local" }` valid?
6. Does the schema turn the string `"3000"` into the number `3000`?

(answer: Part 3 answers)

Moving on. You can move on when you can build a small schema with the TypeBox builders, say why an interface alone cannot check JSON, and run an `unknown` value through a check before reading its fields. The answer below closes out the self-check. Part 4 adds the async and lifetime patterns real Pi extension code needs.

A.7.6 Answers

TypeBox builders, `Value.Errors` Checkpoint. `Value.Errors(schema, value)` runs an exhaustive check and returns an array of errors. Each error carries an `instancePath` naming the offending field and a readable `message`, which is what you format into a report for the user or the model.

TypeBox self-check. `port` and `mode` are required. `mode` is the literal union `"local" | "remote"`. `host` is optional and may be absent. The first object is valid; the second is invalid because `port` must be a number. TypeBox does not automatically convert the string to a number.

A.8 Part 4 — Async, cancellation, and classes

These patterns are needed to read and write real Pi extension code: async tool execution, cancellation through `AbortSignal`, and classes that own a lifetime. They build on the Everyday Types foundation. Still, do not design class hierarchies or type-level machinery from this part alone; reach for them when the runtime lifetime and contract require one.

In this part you will learn to:

- read an `async` function and say where its rejection surfaces;
- choose between `Promise.all` and `Promise.allSettled` for concurrent work;
- follow an `AbortSignal` from the caller into a tool's `execute` and respond to cancellation;
- recognize when a class owns a lifetime and who must stop or dispose it.

A.8.1 Promises, streams, and `AbortSignal`

An `async` function returns a `Promise<T>`:

```
async function loadMessage(): Promise<string> {
  return "ready";
}
```

A Promise settles once: it either fulfills with one value or rejects with an error. Handle rejection with `try`/`catch` around the `await`:

```
try {
  const message = await loadMessage();
  console.log(message);
} catch (error) {
  console.error(error instanceof Error ? error.message : String(error));
}
```

The same rejection can be handled with `.catch` on the Promise itself, which you will recognize in code that cannot use `await`:

```
loadMessage()
  .then((message) => console.log(message))
  .catch((error) => console.error(String(error)));
```

To wait for several independent Promises, `Promise.all` fulfills with an array of results and rejects as soon as any one fails. `Promise.allSettled` always fulfills and reports each outcome, which is what you want when partial success is acceptable:

```
const [config, session] = await Promise.all([loadConfig(), loadSession()]);

const outcomes = await Promise.allSettled([pingA(), pingB()]);
for (const outcome of outcomes) {
  if (outcome.status === "rejected") console.error(outcome.reason);
}
```

An async iterable produces multiple values over time instead of one:

```
async function* tokens(): AsyncIterable<string> {
  yield "Hel";
  yield "lo";
}

for await (const token of tokens()) {
  console.log(token);
}
```

If the generator throws (or a rejected Promise is awaited inside it), the `for await` loop rejects like any other `await`: wrap the loop in `try/catch` when partial output is better than a crash. Note the two loop forms: plain `for...of` iterates an `Iterable<T>` of values that already exist, while `for await...of` iterates an `AsyncIterable<T>` whose values arrive over time. An `AsyncIterable` is not awaitable itself; you await each element as the loop pulls it.

An `AbortSignal` carries a cancellation request from the owner to the work. The owner creates the controller, starts the work with its signal, and is the only party that aborts:

```
function startFetch(url: string): { cancel: () => void; done: Promise<string> } {
  const controller = new AbortController();          // owner creates
  const done = fetch(url, { signal: controller.signal })
    .then((response) => response.text());            // worker receives
  return { cancel: () => controller.abort(), done }; // owner aborts
}
```

The reading questions are always the same three: who creates the controller, who calls `abort`, and who cleans up listeners and timers when the work ends. For a simple deadline you do not need a controller at all:

```
const response = await fetch(url, { signal: AbortSignal.timeout(5000) });
```

Workers that listen for cancellation with `signal.addEventListener("abort", ...)` should remove that listener when they finish, because a long-lived signal otherwise keeps the worker alive. The three shapes side by side:

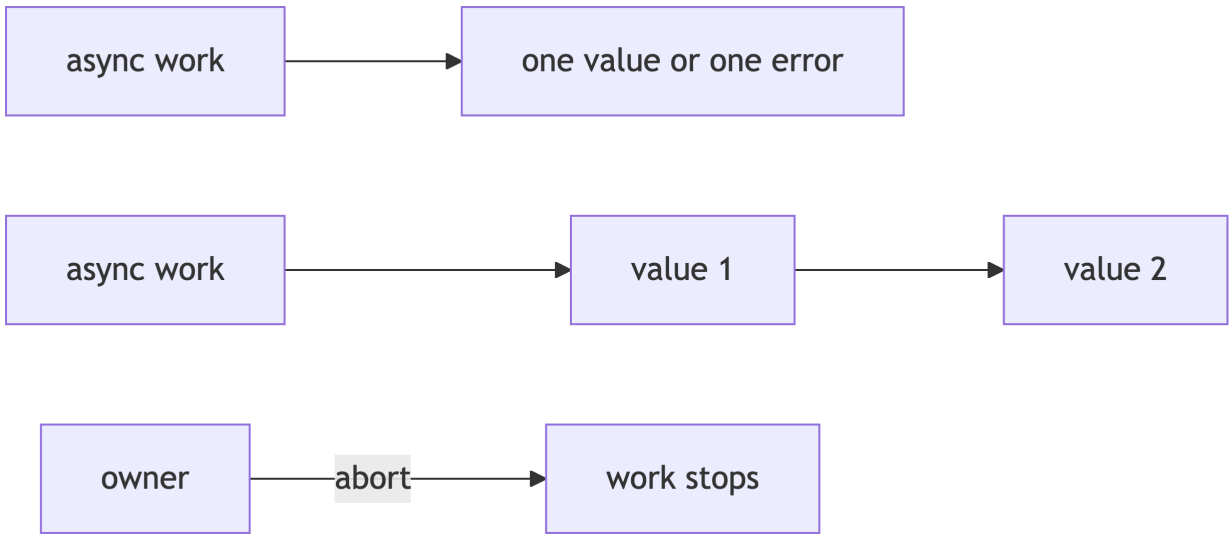

Figure A.6: Promise: one value. Async iterable: many values. AbortSignal: one-way cancellation notice.

The important reading habit is to follow the signal into the operation. Detailed retry, timeout, and parallel Promise behavior belongs in the agent and tool chapters.

A.8.2 Classes and cleanup

Use a class when several methods share state or the object owns a lifetime:

```
interface Stoppable {
  stop(): void;
}

class Runner implements Stoppable {
  private readonly controller: AbortController;
  private readonly name: string;

  constructor(name: string) {
    this.name = name;
    this.controller = new AbortController();
  }

  get label(): string {
    return this.name;
  }

  stop(): void {
    this.controller.abort();
  }
}
```

The explicit fields and constructor assignment are intentional Pi style. A class field can be `public` (the default), `protected` (the class and its subclasses), or `private` (the declaring class). `readonly` prevents assignment through the type after initialization; it does not freeze the runtime object. A getter can expose a computed read-only view.

`implements` checks that a class has a required shape; it does not add members to the class or change how its body is inferred. `extends` creates runtime inheritance, and a derived class must preserve the base class contract:

```
class TimedRunner extends Runner {
  readonly timeoutMs: number;

  constructor(name: string, timeoutMs: number) {
    super(name);
    this.timeoutMs = timeoutMs;
  }
}
```

The class that creates a controller, listener, timer, or process owns stopping or disposing it. Look for `stop()` and `dispose()` when reading lifecycle code. Pi uses explicit fields rather than constructor parameter properties because its source must use erasable TypeScript syntax. Abstract classes, static members, accessors with separate read/write types, and generic classes are topics to recognize rather than design; do not build a class hierarchy until the runtime lifetime and contract require one.

Moving on. You can move on when you can read an `async` tool `execute` and say where rejection surfaces, trace an `AbortSignal` to the cleanup it triggers, and point to the class that owns a listener or timer and must dispose it. Part 5 puts the whole foundation to work on real Pi files.

A.9 Part 5 — Apply the basics to Pi

Work through this part as an arc: orient yourself in Pi's runtime and developer workflow, read real Pi files and the guided source tours, make a small safe change, then verify it by reading the tests. Each section builds a habit the next one uses.

In this part you will learn to:

- say which of Pi's run mechanisms type-check and which only strip types;
- use editor hover and go-to-definition to read an unfamiliar Pi type in place;
- use the guided source tours as TypeScript reading exercises;
- make a small safe change and verify it by reading the tests.

A.9.1 Pi's runtime and developer workflow

Pi uses several ways to run and check TypeScript:

Situation	Mechanism	Type-checks?
Supported Node source execution	type stripping	no
Development launcher	`tsx`	no
Extension loading	`jiti`	no
Package build	`tsgo` emit	yes as part of the build
Repository quality gate	`npm run check`	yes, plus other checks

Runtime success and type-check success are separate evidence. A loader can run code that later fails `erasableSyntaxOnly` or the repository type check.

A.9.2 Reading real Pi files

Beyond ordinary `.ts` source, three artifacts shape what you can import and how the checker sees it.

`.d.ts` declaration files describe types for code that lives elsewhere, such as a published package. They contain no implementations, only shapes, and `declare` marks "this exists at runtime; trust me":

```
// What you might see inside a dependency's .d.ts file
export declare function createClient(options: ClientOptions): Client;
```

You can call `createClient` after importing it, but the body is in compiled JavaScript. A `.d.ts` file is a promise made to the checker, which is why it can also lie: the same caution as `as` applies at package boundaries.

`package.json exports` maps the public entry points of a package. When you write `import { Type } from "@earendil-works/pi-ai"`, the `exports` field of that package's `package.json` decides which file (and which `.d.ts`) resolves. A path not listed there is private, which is why the modules section warns against deep imports.

Editor hover and go-to-definition are the reading tools built on the checker. In VS Code (or any editor using the TypeScript language server), hovering a name shows its inferred type, and go-to-definition (F12) jumps to the declaration, including into `.d.ts` files. When a type in Pi source is unfamiliar, hover it before opening documentation; the answer is usually the declaration one jump away.

Multi-parameter generic signatures become readable when you name each parameter's job. Here is the real `defineTool` and the head of the interface it returns, excerpted from `packages/coding-agent/src/core/extensions/types.ts` (around line 439; line numbers shift, so search for `export function defineTool`):

```
export interface ToolDefinition<
    TParams extends TSchema = TSchema,
    TDetails = unknown,
    TState = any,
> {
    name: string;
    label: string;
    description: string;
    parameters: TParams;
    // ...
    execute(
        toolCallId: string,
        params: Static<TParams>,
        signal: AbortSignal | undefined,
        onUpdate: AgentToolUpdateCallback<TDetails> | undefined,
        ctx: ExtensionContext,
    ): Promise<AgentToolResult<TDetails>>;
    // ...
}

export function defineTool<TParams extends TSchema, TDetails = unknown, TState = any>(
    tool: ToolDefinition<TParams, TDetails, TState>,
): ToolDefinition<TParams, TDetails, TState> & AnyToolDefinition {
    return tool as ToolDefinition<TParams, TDetails, TState> & AnyToolDefinition;
}
```

Read it with the generic tools from Part 2:

1. `<TParams extends TSchema = TSchema, ...>` declares three type parameters. `extends TSchema` is a constraint; `= TSchema`, `= unknown`, `= any` are defaults used when the caller does not pin them.
2. `parameters: TParams` stores whatever schema you pass, so inference captures your exact schema type instead of widening it.

3. `params: Static<TParams>` in `execute` is the payoff: the callback receives the static type derived from your schema. This is why `params.name` is a string in the hello tool without any annotation.
4. The function returns the same interface intersected with `AnyToolDefinition` so a heterogeneous tool array can accept it.

The annotated `AgentEvent` excerpt in Discriminated unions in Pi is the same exercise applied to a union: real path, real code, numbered commentary.

A.9.3 Guided source tours as TypeScript exercises

Guided source tours is the full tour collection. Treat each tour as a reading exercise by naming the TypeScript concepts in the files you open; three starters:

- **A custom tool.** Open `packages/coding-agent/examples/extensions/hello.ts` and identify the Everyday Types concepts in the file: object types in the tool definition, a TypeBox runtime schema, a callback stored as `execute`, an optional cancellation signal, and a structured runtime result. Then follow `defineTool` to `packages/coding-agent/src/core/extensions/types.ts` and read the signature only after understanding the small example above.
- **An agent event.** Start with the `AgentEvent` excerpt in Discriminated unions in Pi, then open the real union in `packages/agent/src/types.ts`, the producer in `packages/agent/src/agent-loop.ts`, and the consumer in `packages/agent/src/agent.ts`. For one event, answer: which literal selects the branch, which fields are available after narrowing, is the event emitted immediately or stored, and what happens if a new variant is added.
- **State and lifecycle.** When reading agent state, trace ownership: which object owns the array, is a shallow snapshot enough, which callback is stored for later, which signal reaches the I/O operation, and which method releases listeners or cancels work.

A.9.4 Make a small safe change

Testing and contribution owns the change workflow: find the declaration and its callers, make the smallest change that preserves the contract, update every union consumer when a variant changes, then run a focused check and `npm run check` when code changes. For a tool change, test valid input, invalid input, and cancellation.

A.9.5 Reading a Pi test file

Pi's tests are the best worked examples of the valid/invalid pattern. Here is an excerpt from `packages/coding-agent/test/git-ssh-url.test.ts`, which tests a small URL parser (line numbers shift; search for `parseGitUrl`):

```
import { describe, expect, it } from "vitest";
import { parseGitUrl } from "../src/utils/git.ts";

describe("Git URL Parsing", () => {
    it("should parse HTTPS URL", () => {
        const result = parseGitUrl("https://github.com/user/repo");
        expect(result).toMatchObject({
```

```
            host: "github.com",
            path: "user/repo",
            repo: "https://github.com/user/repo",
        });
    });

    it("should reject unsafe git install path inputs", () => {
        for (const source of [
            "git:git@evil.example:../../victim/repo",
            "https://evil.example/..%2F..%2Fvictim/repo",
        ]) {
            expect(parseGitUrl(source)).toBeNull();
        }
    });
});
```

Reading notes:

1. `import { describe, expect, it } from "vitest"` brings the three building blocks: `describe` groups tests, `it` is one test, `expect` asserts.
2. The second import is the unit under test, pulled from source with the same relative `.ts` convention as any Pi module.
3. The first `it` is the valid-input case: a well-formed URL must produce a structured result, checked field by field with `toMatchObject`.
4. The second `it` is the invalid-input case: a table of hostile inputs, each expected to return `null` rather than throw or half-parse.

Tool tests add the third case from the advice above — cancellation — by passing an aborted `AbortSignal` into `execute` and asserting the result. When you change a parser, schema, or tool, mirror this shape: one `it` for valid input, one for invalid input, and one for cancellation where a signal reaches the code.

A.9.6 Writing your first test

Turn the reading habit around: write a valid/invalid `it` pair for the `readTodo` guard from Part 3 — one test where matching input comes back whole, one where a missing field returns `undefined` instead of throwing. Test the guard, not the schema: TypeBox is already tested by its authors, so what you own is the `unknown` to `Todo | undefined` decision in `readTodo`. Pi tests import TypeBox straight from the `typebox` package (older code and tutorials use `@sinclair/typebox`). Testing and contribution owns the test workflow: where test files live, how to run one file, and the faux-provider harness for session-level tests.

Checkpoint: A third `it` belongs in that test file. What input does it use, and what does it assert? (answer: Part 5 answers)

A.9.7 Capstone: one todo extension end to end

~30 min · core

Assemble the workbook's pieces into one small extension: a `TodoSchema` with its derived static type (TypeBox builders), a `readTodo` guard that accepts `unknown` (`unknown`, `any`, and external data), a tool that reuses the schema as `parameters` and checks `signal?.aborted` inside `execute` (TypeBox in a Pi tool, Promises, streams, and `AbortSignal`), registration from an extension factory (The real registration in Pi), and a valid/invalid test pair for the guard. For the extension skeleton, the test and verification workflow, and a guided walkthrough, follow A complete first extension, Testing and contribution, and Guided source tours. Reread the linked workbook section whenever a step is not routine; if every step felt routine, you are done with this workbook.

A.9.7.1 Ready for your first extension?

You are ready for Your first extension when you can do each of these without looking at an answer:

- write a TypeBox schema for a tool's parameters and derive its static type (What TypeBox does);
- explain who validates tool arguments and when — Pi validates against the schema before `execute` runs (TypeBox in a Pi tool);
- write a guard that accepts `unknown` and narrows it before any field is read (`unknown`, `any`, and external data);
- type a handler for one variant of the `AgentEvent` union and say which literal selects the branch (Discriminated unions in Pi);
- add a new variant to a union and let the checker find every consumer that must change (User-defined predicates and exhaustive unions);
- register a tool from an extension factory callback and explain when the callback runs (The real registration in Pi);
- follow an `AbortSignal` from owner to worker and say who may call `abort` (Promises, streams, and `AbortSignal`);
- write a valid/invalid test pair and run just that file (Testing and contribution).

When all of these are comfortable, continue to A complete first extension.

Moving on. You can move on when you can open an unfamiliar Pi file and identify its imports, types, and runtime contracts, make a small change behind an existing contract, and find the test that proves it. The reference section below is there whenever a specific error or type stops you.

A.9.8 Answers

Writing your first test Checkpoint. The third case is an edge input that is not an object at all: `readTodo(null)` must also return `undefined`. `Value.Check` rejects `null`, numbers, and strings against an object schema, so the guard holds without extra code. For a tool test rather than a guard test, the third case is cancellation: pass an aborted `AbortSignal` into `execute` and assert the cancelled result.

A.10 Reference: use when you get stuck

A.10.1 Common TypeScript errors

Read a compiler error from left to right: identify the value you have, the type the code requires, and the operation that created the mismatch.

A.10.1.1 Type is not assignable

```
const count: number = "3";
// Type 'string' is not assignable to type 'number'.
```

The value is a string, but the annotation requires a number. Convert it at runtime or fix the source value; do not silence it with an assertion.

A.10.1.2 Object is possibly undefined

```
function lengthOf(value?: string): number {
  return value?.length ?? 0;
}
```

The parameter may be omitted, so check it, use optional chaining, or choose an explicit fallback.

A.10.1.3 Property does not exist

```
const user = { name: "Ada" };
// user.age; // Property 'age' does not exist.
```

Check for a typo or narrow a union before accessing a variant-specific field.

A.10.1.4 Cannot find module

Check the package installation, relative path, `.ts` suffix, and public export. Do not deep-import another package's internal source.

A.10.1.5 Literal is not assignable

```
type Mode = "read" | "write";
// const mode: Mode = "preview";
```

The value is outside the closed set. Check spelling or add a deliberate new variant and update its consumers.

A.10.1.6 Erasable syntax

Pi source avoids constructor parameter properties, enums, and namespaces. Use explicit fields, literal unions, and ESM modules instead.

A.10.1.7 Implicit `any`

```
function announce(message) {
  return message.toUpperCase();
}
```

With `noImplicitAny`, the unannotated parameter is an error. Add a meaningful parameter type or let contextual typing provide one at the call site; do not silence the error with `any`.

A.10.1.8 Readonly assignment

```
const names: readonly string[] = ["Ada"];
// names.push("Grace"); // Property 'push' does not exist
```

The value may be a normal mutable array at runtime. The error describes the readonly view available to this function.

A.10.1.9 No overload matches

When an overloaded function reports "No overload matches this call," compare the arguments with each exposed overload signature. The hidden implementation signature does not automatically make every combination legal; narrow the value, choose a supported call shape, or revise the public overloads.

A.10.2 Utility types to recognize

These transform compile-time descriptions. They do not mutate, copy, freeze, or validate runtime values.

Type	Meaning
`Partial<T>`	all properties optional
`Required<T>`	all properties required
`Readonly<T>`	top-level properties cannot be reassigned through the type
`Pick<T, K>`	keep selected keys
`Omit<T, K>`	remove selected keys
`Record<K, V>`	map keys to values
`ReturnType<typeof fn>`	function result type

Type	Meaning
`Parameters<typeof fn>`	function argument tuple
`Awaited<T>`	unwrap a Promise's eventual value
`Exclude<T, U>`	remove selected union members
`Extract<T, U>`	keep selected union members
`NonNullable<T>`	remove `null` and `undefined`

A.10.3 Symbol cheat sheet

TypeScript punctuation decoded. Each row links to the section that teaches it.

Symbol	Meaning	Section
`?` (type position)	property may be absent	Object types
`?.`	safe access, may short-circuit	`null` and operators
`??`	fallback for null or undefined	`null` and operators
`??=`	assign only if nullish	JavaScript survival kit
`!`	non-null assertion, no check	`null` and operators
`as`	claim to the checker	Type assertions
`as const`	freeze literal inference	Literal types
`satisfies`	check without widening	`satisfies`
`|`	union: one of these	Union types
`&`	intersection: all of these	Intersections
`<T>`	generic type parameter	Generics
`keyof`	union of known keys	Types from declarations
`typeof` (type position)	type of an existing value	Types from declarations
`[number]`	element type of an array	Types from declarations
`value is T`	narrowing type predicate	Predicates
`never`	no possible values	Exhaustive unions
`declare`	exists at runtime, trust me	Type assertions
`...`	rest or spread	Rest parameters

A.10.4 Short glossary

Term	Plain-language meaning
Annotation	A type written by the programmer
Inference	A type worked out from values or context
Literal type	One exact value such as `"running"`
Union	A value that may be one of several alternatives
Narrowing	Evidence that removes alternatives from a union
Optional property	A property that may be absent, written `name?: string`
Optional chaining	Safe access written `value?.property`
Nullish coalescing	Fallback for `null` or `undefined`, written `value ?? fallback`

Term	Plain-language meaning
Assertion	An `as` claim to the checker; not validation
Structural typing	Compatibility based on required fields
`unknown`	A value that must be checked before use
`any`	A type that disables checking
Schema	A runtime description of allowed data
TypeBox	A library for building runtime schemas and derived static types
`Static<typeof S>`	The compile-time type derived from schema `S`
Tuple	An array type with fixed, position-specific elements
Readonly	A compile-time restriction on writes through one type view
Type predicate	A return type such as `value is User` that narrows a value
`never`	A type with no possible values, useful for exhaustiveness
`satisfies`	A compile-time compatibility check that preserves inference
`keyof`	A type-level union of an object's known keys
Type-position `typeof`	The compile-time type of an existing value
Callback	A function passed to be called now or later
Promise	One future fulfilled value or rejected error
`AbortSignal`	A cancellation notification passed to work

A.10.5 Staged self-check

A.10.5.1 Check 1: everyday types

```
type User = { name: string; nickname?: string };
const user: User = { name: "Ada" };
```

What can `user.nickname` be? Which operator gives a fallback without replacing an empty string?

A.10.5.2 Check 2: union and callback

```
type Input = { type: "message"; text: string } | { type: "count"; value: number };
function handle(input: Input, emit: (text: string) => void): void {
  if (input.type === "message") emit(input.text);
}
```

What narrows `input`? When is `emit` called?

A.10.5.3 Check 3: schema and cancellation

```ts
import { Type, type Static } from "@earendil-works/pi-ai";
import { Value } from "typebox/value";

const ParamsSchema = Type.Object({ name: Type.String() });
type Params = Static<typeof ParamsSchema>;

const raw: unknown = { name: "Ada" };
const valid = Value.Check(ParamsSchema, raw);
```

Which values are runtime? Which name is compile-time? What would `Value.Check` inspect?

A.10.5.4 Answers

Check 1. It can be `string | undefined`; `??` preserves an empty string.

Check 2. `input.type` narrows the union; `emit` is called immediately in the branch, although the function itself was passed in by the caller.

Check 3. `ParamsSchema`, `raw`, and `valid` are runtime values; `Params` is a compile-time type; `Value.Check` inspects `raw` against `ParamsSchema`.

A.10.6 Type-checking a standalone extension

Use this appendix only for an extension outside the monorepo. A minimal `tsconfig.json` needs a module target, module resolution, strict checking, and the files to include:

```json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "skipLibCheck": true,
    "noEmit": true,
    "erasableSyntaxOnly": true,
    "allowImportingTsExtensions": true
  },
  "include": ["src/**/*.ts"]
}
```

What each option does and why an extension project wants it:

Option	Effect	Why Pi/extensions need it
`target: ESNext`	compile against the newest JavaScript features	Pi runs on current Node; no down-leveling needed
`module: ESNext`	emit/read standard ESM `import`/`export`	Pi packages and extensions are ESM-only
`moduleResolution: bundler`	resolve imports the way bundlers and loaders do	matches how `jiti`/`tsx` load extension files
`strict: true`	enable the full family of strict checks	the same posture the Pi repository uses
`skipLibCheck: true`	do not type-check dependencies' `.d.ts` files	keeps dependency declaration quirks out of your build
`noEmit: true`	check only, write no JavaScript	`jiti` loads the `.ts` directly; emitting is unused
`erasableSyntaxOnly: true`	reject syntax that needs code generation (enums, namespaces, parameter properties)	Node strips types when running extensions; non-erasable syntax would fail at load time
`allowImportingTsExtensions: true`	permit explicit `.ts` suffixes in imports	Pi's convention is relative imports with the `.ts` suffix
`include: ["src/**/*.ts"]`	the files this project checks	scopes the check to your extension source

Loading through `jiti` and checking through `tsc` or `tsgo` are separate. A loader proving that a file can start does not prove that the file is type-safe.

A.11 Return to the Pi route

The workbook is intentionally optional. Continue with Reading and changing Pi's TypeScript, then Architecture in the main book.